



รายงาน

การประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ในการจำแนกประเภทและวิเคราะห์

ข้อมูลซากดึกดำบรรพ์ในประเทศไทย

จัดทำโดย

66070020	นายเกื้อกุล	วสุนธรา
66070066	นายณัฐวีร์	แนวกำพล
66070182	นายวัชรวิชัย	หอมรังสฤษฎ์
66070184	นายวาธิน	พรมฝ้าย
66070203	นางสิตาพร	จิตรระวีวรรณ
66070261	นายธนกร	แจ่มพินิจ

อาจารย์ประจำวิชา

อาจารย์เฉลิมพล ศิริกายน

เอกสารนี้เป็นส่วนหนึ่งของรายวิชา

FUNDAMENTALS OF DATA SCIENCE (06026208)

คณะเทคโนโลยีสารสนเทศ สาขาวิทยาการข้อมูลและการวิเคราะห์เชิงธุรกิจ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ภาคเรียนที่ 2 ปีการศึกษา 2567

บทคัดย่อ

การศึกษานี้มีวัตถุประสงค์เพื่อพัฒนาและประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ในการวิเคราะห์ข้อมูลซากดึกดำบรรพ์ของประเทศไทย จากชุดข้อมูลของกรมทรัพยากรธรณี โดยดำเนินการตามกระบวนการ CRISP-DM และพัฒนาแบบจำลองภายใต้สองแนวคิดหลัก ได้แก่ การจำแนกประเภทซากดึกดำบรรพ์และการวิเคราะห์ความสัมพันธ์เชิงพื้นที่

ผลการวิจัยแนวคิดที่ 1 พบว่า แบบจำลองที่ไม่ใช้ข้อมูลภาพถ่ายมีประสิทธิภาพดีกว่า โดยมีค่า Accuracy เท่ากับ 0.81 และ F1-score เท่ากับ 0.78 โดยคุณลักษณะที่มีความสำคัญมากที่สุดคือลักษณะซากดึกดำบรรพ์ที่ปรากฏ (attributes_FOS_DES_TH) แบบจำลองมีประสิทธิภาพสูงในการจำแนกซากดึกดำบรรพ์ประเภท Invertebrate แต่มีข้อจำกัดในการจำแนกประเภท Trace fossils เนื่องจากความไม่สมดุลของข้อมูล

ผลการวิจัยแนวคิดที่ 2 พบว่า การเชื่อมโยงข้อมูลจากแหล่งข้อมูลซากดึกดำบรรพ์ 2 แหล่งทำให้ได้ชุดข้อมูลที่มีความสมบูรณ์มากขึ้น สามารถเติมเต็มค่าที่ขาดหายด้วยเทคนิคที่หลากหลาย และสร้างคุณลักษณะใหม่ที่ครอบคลุมมิติต่างๆ นอกจากนี้ ยังได้พัฒนาแอปพลิเคชันบนเว็บที่สามารถจำแนกซากดึกดำบรรพ์และแสดงผลในรูปแบบแผนที่แบบโต้ตอบ

การศึกษานี้มีข้อเสนอแนะสำหรับการพัฒนาแบบจำลองในอนาคต ได้แก่ การแก้ไขปัญหาค่าความไม่สมดุลของข้อมูล การใช้ภาพถ่ายที่แสดงลักษณะทางกายภาพของซากดึกดำบรรพ์โดยตรง การเลือกคุณลักษณะที่เหมาะสม และการทดลองกับอัลกอริทึมที่หลากหลาย นอกจากนี้ ยังเสนอแนะการเพิ่มจำนวนตัวอย่างในประเภทที่มีข้อมูลน้อย การเก็บข้อมูลที่มีคุณภาพ และการสร้างมาตรฐานในการเก็บข้อมูล เพื่อการพัฒนากระบวนการตัดสินใจและแอปพลิเคชันสำหรับการจำแนกซากดึกดำบรรพ์ต่อไป

สารบัญ

บทที่ 1 ที่มาและความสำคัญ.....	1
บทที่ 2 เอกสารและงานวิจัยที่เกี่ยวข้อง.....	13
บทที่ 3 วิธีการดำเนินงาน.....	32
บทที่ 4 ผลการดำเนินงาน	160
บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ	178
บรรณารุกรม	190
ภาคผนวก.....	192

บทที่ 1

ที่มาและความสำคัญ

1.1 ความเป็นมาและความสำคัญของปัญหา

ซากดึกดำบรรพ์ หรือฟอสซิล (Fossils) เป็นร่องรอยหรือซากของสิ่งมีชีวิตที่ดำรงอยู่ในอดีตกาล และถูกเก็บรักษาไว้ ในชั้นหินตะกอน ซึ่งเป็นหลักฐานสำคัญที่ช่วยให้มนุษย์สามารถเข้าใจประวัติศาสตร์ของโลก วิวัฒนาการของสิ่งมีชีวิต และการเปลี่ยนแปลงของสภาพแวดล้อมในอดีต ประเทศไทยนับเป็นแหล่งซากดึกดำบรรพ์ที่สำคัญแห่งหนึ่งของภูมิภาคเอเชียตะวันออกเฉียงใต้ ซึ่งมีการค้นพบซากดึกดำบรรพ์หลากหลายประเภทและยุคสมัย ตั้งแต่ซากดึกดำบรรพ์ไดโนเสาร์ สัตว์เลี้ยงลูกด้วยนมโบราณ สัตว์ไม่มีกระดูกสันหลัง ฟอสซิลโบราณ ไปจนถึงร่องรอยกิจกรรมของสิ่งมีชีวิตในอดีต

กรมทรัพยากรธรณี กระทรวงทรัพยากรธรรมชาติและสิ่งแวดล้อม เป็นหน่วยงานหลักที่รับผิดชอบในการสำรวจ ศึกษาวิจัย อนุรักษ์ และบริหารจัดการทรัพยากรธรณีของประเทศไทย รวมถึงซากดึกดำบรรพ์และแหล่งซากดึกดำบรรพ์ ทางกรมฯ ได้รวบรวมและเผยแพร่ข้อมูลเกี่ยวกับซากดึกดำบรรพ์และแหล่งซากดึกดำบรรพ์ผ่านฐานข้อมูลสาธารณะ ที่สามารถเข้าถึงได้ อย่างไรก็ตาม การบริหารจัดการและการใช้ประโยชน์จากข้อมูลซากดึกดำบรรพ์ของประเทศไทย ยังประสบกับความท้าทายหลายประการ ได้แก่

1. ข้อจำกัดของระบบฐานข้อมูลปัจจุบัน: ระบบฐานข้อมูลซากดึกดำบรรพ์ในปัจจุบันยังไม่เอื้อต่อการใช้งานอย่างมีประสิทธิภาพสำหรับผู้มีส่วนได้ส่วนเสียทุกกลุ่ม ทั้งนักวิจัย นักเรียน นักศึกษา ประชาชนทั่วไป และหน่วยงานที่เกี่ยวข้อง เนื่องจากข้อมูลมีความซับซ้อน กระจาย และยากต่อการสืบค้นและวิเคราะห์
2. ข้อจำกัดด้านทรัพยากรและบุคลากร: การสำรวจ จัดเก็บ และวิเคราะห์ข้อมูลซากดึกดำบรรพ์ต้องอาศัยทรัพยากรและบุคลากรที่มีความเชี่ยวชาญเฉพาะด้าน ซึ่งมีจำนวนจำกัดและไม่เพียงพอต่อปริมาณงานที่มีอยู่ ทำให้การจัดการข้อมูลและการวิเคราะห์เป็นไปอย่างล่าช้าและไม่ครอบคลุม
3. ความซับซ้อนของข้อมูล: ข้อมูลซากดึกดำบรรพ์มีความซับซ้อนและหลากหลาย ทั้งข้อมูลเชิงพื้นที่ (spatial data) ข้อมูลเชิงเวลา (temporal data) ข้อมูลทางธรณีวิทยา (geological data) และข้อมูลทางชีววิทยา (biological data) ซึ่งต้องการการวิเคราะห์และการจัดการที่เหมาะสม

4. ข้อจำกัดในการเข้าถึงข้อมูล: แม้จะมีการเผยแพร่ข้อมูลผ่านระบบสารสนเทศ แต่การเข้าถึงข้อมูลในเชิงลึกยังมีข้อจำกัด โดยเฉพาะสำหรับประชาชนทั่วไปและนักวิจัยที่ไม่มีความเชี่ยวชาญเฉพาะด้าน

ในยุคของการปฏิวัติอุตสาหกรรมครั้งที่ 4 (Industry 4.0) และการเติบโตของเทคโนโลยีปัญญาประดิษฐ์ (Artificial Intelligence: AI) ได้เปิดโอกาสใหม่ในการแก้ไขปัญหาและความท้าทายเหล่านี้ ปัญญาประดิษฐ์มีความสามารถในการวิเคราะห์ข้อมูลขนาดใหญ่ (Big Data) การเรียนรู้รูปแบบที่ซับซ้อน (Pattern Recognition) และการทำนายผลลัพธ์จากข้อมูลที่มีอยู่ (Predictive Analytics) ซึ่งสามารถนำมาประยุกต์ใช้ในการจัดการและวิเคราะห์ข้อมูลซากดึกดำบรรพ์ได้อย่างมีประสิทธิภาพจากการศึกษาชุดข้อมูลซากดึกดำบรรพ์ของกรมทรัพยากรธรณี คณะผู้วิจัยได้มองเห็นโอกาสในการนำเทคโนโลยีปัญญาประดิษฐ์มาประยุกต์ใช้ในการวิเคราะห์และจัดการข้อมูล โดยมุ่งเน้นการทำนายและจำแนกชนิดของซากดึกดำบรรพ์ การทำนายยุคทางธรณีวิทยา การวิเคราะห์ความสัมพันธ์ระหว่างตำแหน่งและชนิดซากดึกดำบรรพ์ การค้นหาแหล่งที่มีศักยภาพในการพบซากดึกดำบรรพ์ และการวิเคราะห์การกระจายตัวของซากดึกดำบรรพ์ในพื้นที่ต่างๆ เพื่อเพิ่มประสิทธิภาพในการศึกษาวิจัย การอนุรักษ์ และการจัดการทรัพยากรธรณีวิทยาของประเทศไทย

การพัฒนาแบบจำลองปัญญาประดิษฐ์สำหรับการวิเคราะห์ข้อมูลซากดึกดำบรรพ์จะช่วยยกระดับความสามารถในการจัดการและใช้ประโยชน์จากข้อมูลซากดึกดำบรรพ์ของประเทศไทย ซึ่งจะนำไปสู่การพัฒนาองค์ความรู้ด้านบรรพชีวินวิทยา ธรณีวิทยา และการอนุรักษ์ทรัพยากรธรรมชาติของประเทศอย่างยั่งยืน นอกจากนี้ ยังเป็นการสนับสนุนการท่องเที่ยวเชิงธรณีวิทยา (Geotourism) และการศึกษาเรียนรู้ด้านวิทยาศาสตร์และธรรมชาติวิทยาสำหรับประชาชนทั่วไปอีกด้วย

1.2 วัตถุประสงค์

การศึกษานี้มีวัตถุประสงค์หลักเพื่อพัฒนาและประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ในการวิเคราะห์ข้อมูลซากดึกดำบรรพ์ของประเทศไทย โดยมีวัตถุประสงค์เฉพาะดังนี้:

1.2.1 พัฒนาและประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์เพื่อเพิ่มประสิทธิภาพการจัดการและการวิเคราะห์ ข้อมูลซากดึกดำบรรพ์

การศึกษานี้มุ่งพัฒนาและประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ที่ทันสมัย ได้แก่ การเรียนรู้ของเครื่อง (Machine Learning) การเรียนรู้เชิงลึก (Deep Learning) และการประมวลผลภาพ (Image Processing) เพื่อเพิ่มประสิทธิภาพในการจัดการและวิเคราะห์ข้อมูลซากดึกดำบรรพ์ของประเทศไทย ซึ่งจะช่วยลดข้อจำกัดด้านทรัพยากรและบุคลากร และเพิ่มความสามารถในการวิเคราะห์ข้อมูลที่มีความซับซ้อนและหลากหลาย โดยเฉพาะอย่างยิ่งการสนับสนุนการศึกษาทางธรณีวิทยาและบรรพชีวินวิทยาที่ต้องการความแม่นยำและความครอบคลุมในการวิเคราะห์ข้อมูล

1.2.2 พัฒนาแบบจำลอง AI สำหรับการจำแนกชนิดของซากดึกดำบรรพ์

เพื่อพัฒนาแบบจำลองปัญญาประดิษฐ์ที่สามารถจำแนกประเภทและชนิดของซากดึกดำบรรพ์ได้อย่างถูกต้องและแม่นยำ โดยใช้ข้อมูลจากลักษณะทางกายภาพ ลักษณะทางธรณีวิทยา ตำแหน่งที่พบ และข้อมูลอื่นๆ ที่เกี่ยวข้อง แบบจำลองนี้จะช่วยให้การจำแนกซากดึกดำบรรพ์ที่พบใหม่หรือยังไม่ได้รับการจำแนกอย่างชัดเจนเป็นไปอย่างรวดเร็วและมีประสิทธิภาพมากขึ้น ซึ่งจะเป็นประโยชน์อย่างยิ่งต่อการศึกษาวิจัยด้านบรรพชีวินวิทยาและการจัดการทรัพยากรธรณีวิทยาของประเทศไทย

1.2.3 พัฒนาแบบจำลอง AI สำหรับการทำนายยุคทางธรณีวิทยา

เพื่อพัฒนาแบบจำลองปัญญาประดิษฐ์ที่สามารถทำนายยุคทางธรณีวิทยาของซากดึกดำบรรพ์ได้อย่างถูกต้องและแม่นยำ โดยใช้ข้อมูลจากลักษณะของซากดึกดำบรรพ์ ชั้นหินที่พบ และข้อมูลอื่นๆ ที่เกี่ยวข้อง แบบจำลองนี้จะช่วยให้การกำหนดอายุของซากดึกดำบรรพ์และชั้นหินที่พบเป็นไปอย่างมีประสิทธิภาพมากขึ้น ซึ่งเป็นข้อมูลพื้นฐานที่สำคัญในการศึกษาวิวัฒนาการของสิ่งมีชีวิตและการเปลี่ยนแปลงของสภาพแวดล้อมในอดีต

1.2.4 วิเคราะห์ความสัมพันธ์เชิงพื้นที่ระหว่างตำแหน่งและชนิดของซากดึกดำบรรพ์

เพื่อวิเคราะห์ความสัมพันธ์เชิงพื้นที่ระหว่างตำแหน่งที่พบซากดึกดำบรรพ์และชนิดของซากดึกดำบรรพ์ โดยใช้เทคนิคการวิเคราะห์ข้อมูลเชิงพื้นที่ (Spatial Data Analysis) และการเรียนรู้ของเครื่อง การวิเคราะห์นี้จะช่วยให้เข้าใจถึงรูปแบบการกระจายตัวของซากดึกดำบรรพ์ประเภทต่างๆ ในพื้นที่ต่างๆ ของประเทศไทย ซึ่งจะนำไปสู่การคาดการณ์แหล่งที่มีศักยภาพในการพบซากดึกดำบรรพ์ชนิดต่างๆ และช่วยในการวางแผนการสำรวจและการอนุรักษ์แหล่งซากดึกดำบรรพ์ที่มีความสำคัญ

1.3 ขอบเขตการศึกษา

การศึกษานี้มีขอบเขตการศึกษาที่ครอบคลุมประเด็นต่างๆ ดังนี้:

1.3.1 ขอบเขตด้านข้อมูลการศึกษานี้ใช้ชุดข้อมูลบัญชีชากดักด้าบรรพ์และตำแหน่งแหล่งชากดักด้าบรรพ์จากกรมทรัพยากรธรณีเป็นข้อมูลหลักในการวิเคราะห์และพัฒนาแบบจำลอง โดยข้อมูลประกอบด้วย:

- ข้อมูลตำแหน่งแหล่งชากดักด้าบรรพ์ (พิกัดทางภูมิศาสตร์)
- ข้อมูลลักษณะทางธรณีวิทยาของแหล่งชากดักด้าบรรพ์
- ข้อมูลลักษณะและประเภทของชากดักด้าบรรพ์ที่พบ
- ข้อมูลยุคทางธรณีวิทยาของแหล่งชากดักด้าบรรพ์และชากดักด้าบรรพ์
- ข้อมูลอื่นๆ ที่เกี่ยวข้อง เช่น ผู้ค้นพบ วันที่ค้นพบ และผู้ครอบครอง

1.3.2 ขอบเขตด้านเทคโนโลยีและวิธีการการศึกษานี้ใช้เทคโนโลยีและวิธีการด้านปัญญาประดิษฐ์และการเรียนรู้ของเครื่องที่หลากหลาย ได้แก่:

- การเรียนรู้ของเครื่องแบบมีผู้สอน (Supervised Machine Learning) เช่น Random Forest, Support Vector Machines (SVM) และ Neural Networks
- การเรียนรู้เชิงลึก (Deep Learning) เช่น Convolutional Neural Networks (CNNs) สำหรับการวิเคราะห์ภาพ
- การประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP) สำหรับการวิเคราะห์ข้อมูลเชิงบรรยาย

- การวิเคราะห์ข้อมูลเชิงพื้นที่ (Spatial Data Analysis) สำหรับการวิเคราะห์ความสัมพันธ์ระหว่างตำแหน่งและชนิดของซากดึกดำบรรพ์
- กระบวนการทางวิทยาศาสตร์ข้อมูล (Data Science Methodology) เช่น CRISP-DM (Cross-Industry Standard Process for Data Mining) ในการดำเนินการวิจัยอย่างเป็นระบบ

1.3.3 ขอบเขตด้านผลลัพธ์

ผลลัพธ์ที่คาดหวังจากการศึกษานี้ ได้แก่:

- แบบจำลองปัญญาประดิษฐ์สำหรับการจำแนกประเภทของซากดึกดำบรรพ์ที่มีความแม่นยำสูง
- แบบจำลองปัญญาประดิษฐ์สำหรับการทำนายยุคทางธรณีวิทยาของซากดึกดำบรรพ์
- การวิเคราะห์ความสัมพันธ์เชิงพื้นที่ระหว่างตำแหน่งและชนิดของซากดึกดำบรรพ์
- แนวทางในการประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ในการจัดการและวิเคราะห์ข้อมูลซากดึกดำบรรพ์ของประเทศไทย

แบบจำลองปัญญาประดิษฐ์สำหรับการจำแนกประเภทของซากดึกดำบรรพ์ที่มีความแม่นยำสูง

- แบบจำลองปัญญาประดิษฐ์สำหรับการทำนายยุคทางธรณีวิทยาของซากดึกดำบรรพ์
- การวิเคราะห์ความสัมพันธ์เชิงพื้นที่ระหว่างตำแหน่งและชนิดของซากดึกดำบรรพ์
- แนวทางในการประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ในการจัดการและวิเคราะห์ข้อมูลซากดึกดำบรรพ์ของประเทศไทย

1.4 ประโยชน์ที่คาดว่าจะได้รับ

การศึกษาค้นคว้านี้จะก่อให้เกิดประโยชน์ในหลายด้าน ดังนี้:

1.4.1 การลดเวลาในการจำแนกและวิเคราะห์ซากดึกดำบรรพ์

การพัฒนาแบบจำลองปัญญาประดิษฐ์สำหรับการจำแนกประเภทและการทำนายยุคทางธรณีวิทยาของซากดึกดำบรรพ์จะช่วยลดเวลาและทรัพยากรที่ใช้ในกระบวนการเหล่านี้ได้อย่างมีนัยสำคัญ โดยเฉพาะอย่างยิ่งในกรณีที่มีการค้นพบซากดึกดำบรรพ์จำนวนมากหรือซากดึกดำบรรพ์ที่มีความซับซ้อน ซึ่งปกติต้องใช้ผู้เชี่ยวชาญเฉพาะด้านในการจำแนกและวิเคราะห์ แบบจำลองปัญญาประดิษฐ์จะช่วยให้กระบวนการเหล่านี้เป็นไปอย่างรวดเร็วและมีประสิทธิภาพมากขึ้น ช่วยประหยัดเวลาและทรัพยากรบุคคลที่มีจำกัด

1.4.2 การค้นพบใหม่และการขยายองค์ความรู้

การวิเคราะห์ข้อมูลซากดึกดำบรรพ์ด้วยเทคโนโลยีปัญญาประดิษฐ์อาจนำไปสู่การค้นพบรูปแบบ ความสัมพันธ์ หรือข้อมูลเชิงลึกใหม่ๆ ที่อาจไม่สามารถตรวจพบได้ด้วยวิธีการวิเคราะห์แบบดั้งเดิม เช่น การค้นพบความสัมพันธ์ระหว่างลักษณะทางกายภาพของซากดึกดำบรรพ์และสภาพแวดล้อมในอดีต การค้นพบแหล่งที่มีศักยภาพในการพบซากดึกดำบรรพ์ชนิดใหม่ หรือการเชื่อมโยงข้อมูลซากดึกดำบรรพ์กับข้อมูลอื่นๆ เพื่อสร้างความเข้าใจใหม่เกี่ยวกับวิวัฒนาการและการเปลี่ยนแปลงของระบบนิเวศในอดีต

1.4.3 การพัฒนาองค์ความรู้และการสนับสนุนการวิจัย

ผลจากการศึกษานี้จะช่วยพัฒนาองค์ความรู้ด้านบรรพชีวินวิทยาและธรณีวิทยาของประเทศไทย และเป็นพื้นฐานสำหรับการวิจัยในอนาคต ทั้งในด้านวิทยาศาสตร์ธรรมชาติและด้านเทคโนโลยีสารสนเทศ การพัฒนาแบบจำลองปัญญาประดิษฐ์และวิธีการวิเคราะห์ข้อมูลที่น่าเสนอในการศึกษานี้สามารถนำไปประยุกต์ใช้กับชุดข้อมูลอื่นๆ หรือปัญหาอื่นๆ ในสาขาที่เกี่ยวข้อง เช่น การอนุรักษ์ความหลากหลายทางชีวภาพ การศึกษาการเปลี่ยนแปลงสภาพภูมิอากาศ และการจัดการทรัพยากรธรรมชาติ

1.4.4 การสนับสนุนการอนุรักษ์และการท่องเที่ยวเชิงธรณีวิทยา

การวิเคราะห์ความสัมพันธ์เชิงพื้นที่ระหว่างตำแหน่งและชนิดของซากดึกดำบรรพ์จะช่วยในการระบุแหล่งซากดึกดำบรรพ์ที่มีความสำคัญและควรได้รับการอนุรักษ์ นอกจากนี้ ความรู้และข้อมูลที่ได้จากการศึกษานี้ยังสามารถนำไปใช้ในการส่งเสริมการท่องเที่ยวเชิงธรณีวิทยา (Geotourism) ซึ่งเป็นรูปแบบการท่องเที่ยวที่ให้ความรู้และสร้างความตระหนักเกี่ยวกับคุณค่าและความสำคัญของทรัพยากรธรณีวิทยา รวมถึงซากดึกดำบรรพ์

1.4.5 การสร้างเครื่องมือและแนวทางสำหรับการจัดการข้อมูล

แนวทางและเครื่องมือที่พัฒนาขึ้นในการศึกษานี้สามารถนำไปใช้เป็นตัวแบบในการพัฒนาระบบการจัดการและวิเคราะห์ข้อมูลซากดึกดำบรรพ์ในระดับประเทศ ซึ่งจะช่วยให้การจัดการข้อมูลเป็นไปอย่างมีประสิทธิภาพมากขึ้น สนับสนุนการเข้าถึงและการใช้ประโยชน์จากข้อมูลได้อย่างกว้างขวางและทั่วถึง ทั้งสำหรับนักวิจัย นักการศึกษา และประชาชนทั่วไป

1.5 นิยามศัพท์เฉพาะ

เพื่อความเข้าใจที่ตรงกันในการศึกษาครั้งนี้ จึงได้กำหนดนิยามศัพท์เฉพาะที่สำคัญดังนี้:

1. ซากดึกดำบรรพ์ (Fossils): หมายถึง ซากหรือร่องรอยของสิ่งมีชีวิตที่ตายไปแล้วเป็นเวลานานมากกว่า 10,000 ปี และถูกเก็บรักษาไว้ในชั้นหิน ตะกอน หรือวัสดุธรรมชาติอื่นๆ โดยผ่านกระบวนการกลายเป็นหิน (Fossilization) หรือกระบวนการทางธรรมชาติอื่นๆ
2. บรรพชีวินวิทยา (Paleontology): หมายถึง ศาสตร์ที่ศึกษาเกี่ยวกับซากดึกดำบรรพ์ เพื่อเรียนรู้เกี่ยวกับสิ่งมีชีวิตในอดีต วิวัฒนาการ และความสัมพันธ์กับสิ่งแวดล้อม รวมถึงการเปลี่ยนแปลงของโลกในช่วงเวลาที่ผ่านมา
3. ปัญญาประดิษฐ์ (Artificial Intelligence: AI): หมายถึง การจำลองกระบวนการเรียนรู้ การใช้เหตุผล และการปรับตัวของมนุษย์โดยเครื่องจักร โดยเฉพาะระบบคอมพิวเตอร์ ซึ่งสามารถเรียนรู้จากข้อมูล ปรับตัวตามสถานการณ์ และทำงานที่ต้องใช้ความฉลาดเหมือนมนุษย์ได้
4. การเรียนรู้ของเครื่อง (Machine Learning: ML): หมายถึง สาขาย่อยของปัญญาประดิษฐ์ที่เกี่ยวข้องกับการพัฒนาอัลกอริทึมและโมเดลที่ช่วยให้คอมพิวเตอร์สามารถเรียนรู้จากข้อมูลและทำนายผลลัพธ์หรือตัดสินใจได้โดยไม่ต้องเขียนโปรแกรมเฉพาะเจาะจง
5. การเรียนรู้เชิงลึก (Deep Learning): หมายถึง สาขาย่อยของการเรียนรู้ของเครื่องที่ใช้โครงข่ายประสาทเทียมหลายชั้น (Neural Networks) ในการเรียนรู้และตีความข้อมูลที่ซับซ้อน เช่น ภาพเสียง และข้อความ
6. ยุคทางธรณีวิทยา (Geological Period): หมายถึง หน่วยของเวลาทางธรณีวิทยาที่ใช้อธิบายช่วงเวลาในประวัติศาสตร์โลก ซึ่งแบ่งออกเป็นยุคต่างๆ ตามเหตุการณ์สำคัญทางธรณีวิทยาและการเปลี่ยนแปลงของสิ่งมีชีวิต
7. การวิเคราะห์ข้อมูลเชิงพื้นที่ (Spatial Data Analysis): หมายถึง การวิเคราะห์ข้อมูลที่มีความสัมพันธ์กับตำแหน่งทางภูมิศาสตร์ โดยใช้เทคนิคทางสถิติและการแสดงผลข้อมูลเชิงพื้นที่

8. Word Embedding: หมายถึง เทคนิคในการประมวลผลภาษาธรรมชาติที่แทนคำหรือวลีด้วยเวกเตอร์ของตัวเลขจริงในพื้นที่หลายมิติ ช่วยให้คอมพิวเตอร์เข้าใจความหมายและความสัมพันธ์ระหว่างคำได้ดีขึ้น
9. Computer Vision: หมายถึง สาขาของปัญญาประดิษฐ์ที่เกี่ยวข้องกับการให้คอมพิวเตอร์สามารถ "มองเห็น" และตีความข้อมูลจากภาพและวิดีโอเหมือนมนุษย์
10. Random Forest: หมายถึง เทคนิคการเรียนรู้ของเครื่องประเภทการเรียนรู้แบบรวมกลุ่ม (Ensemble Learning) ที่สร้างกลุ่มของต้นไม้ตัดสินใจ (Decision Trees) จำนวนมากและรวมผลการทำนายเข้าด้วยกันเพื่อให้ได้ผลลัพธ์ที่แม่นยำมากขึ้น
11. Feature Importance: หมายถึง ค่าที่แสดงความสำคัญของคุณลักษณะ (Features) แต่ละตัวที่มีผลต่อการทำนายของโมเดลการเรียนรู้ของเครื่อง ช่วยในการเลือกคุณลักษณะที่สำคัญและการตีความผลลัพธ์
12. WangchanBERTa: หมายถึง โมเดลภาษาแบบ Transformer ที่ถูกฝึกฝนด้วยคลังข้อมูลภาษาไทยขนาดใหญ่ สามารถใช้ในการประมวลผลภาษาธรรมชาติสำหรับภาษาไทย เช่น การวิเคราะห์ความหมาย การจำแนกประเภทข้อความ และการตอบคำถาม

1.6 โครงสร้างของรายงาน

รายงานฉบับนี้แบ่งออกเป็น 5 บท ดังนี้:

บทที่ 1 บทนำ

บทนี้นำเสนอภาพรวมของการศึกษา ประกอบด้วย ความเป็นมาและความสำคัญของปัญหา วัตถุประสงค์ ขอบเขตการศึกษา ประโยชน์ที่คาดว่าจะได้รับ และนิยามศัพท์เฉพาะ

บทที่ 2 เอกสารและงานวิจัยที่เกี่ยวข้อง

บทนี้นำเสนอการทบทวนวรรณกรรมและงานวิจัยที่เกี่ยวข้องกับชากดิกดำบรรพ์ บรรพชีวินวิทยา การวิเคราะห์ข้อมูลด้านธรณีวิทยาด้วยเทคโนโลยีสมัยใหม่ การประยุกต์ใช้ปัญญาประดิษฐ์ในการวิเคราะห์ชากดิกดำบรรพ์ ตลอดจนแนวคิดและทฤษฎีที่เกี่ยวข้องกับการเรียนรู้ของเครื่องและการวิเคราะห์ข้อมูลเชิงพื้นที่

บทที่ 3 วิธีการดำเนินงาน

บทนี้อธิบายเกี่ยวกับวิธีการดำเนินงานวิจัย ประกอบด้วย การเก็บรวบรวมข้อมูล การวิเคราะห์ข้อมูลเชิงสำรวจ (Exploratory Data Analysis) การเตรียมข้อมูล (Data Preparation) การพัฒนาแบบจำลอง (Modeling) และการประเมินผล (Evaluation) โดยใช้กระบวนการทางวิทยาศาสตร์ข้อมูล CRISP-DM (Cross-Industry Standard Process for Data Mining)

บทที่ 4 ผลการดำเนินงาน

บทนี้เสนอผลการดำเนินงานวิจัย ประกอบด้วย ผลการวิเคราะห์ข้อมูลเชิงสำรวจ ผลการพัฒนาแบบจำลองการจำแนกประเภทชากดิกดำบรรพ์ ผลการพัฒนาแบบจำลองการทำนายยุคทางธรณีวิทยา ผลการวิเคราะห์ความสัมพันธ์เชิงพื้นที่ระหว่างตำแหน่งและชนิดของชากดิกดำบรรพ์ รวมถึงการประเมินประสิทธิภาพของแบบจำลองต่างๆ

บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ

บทนี้เสนอสรุปผลการศึกษา อภิปรายผลการศึกษาโดยเชื่อมโยงกับแนวคิดและทฤษฎีที่เกี่ยวข้อง รวมถึงข้อจำกัดของการศึกษาและข้อเสนอแนะสำหรับการศึกษาในอนาคต

ตลอดจนแนวทางในการนำผลการศึกษาไปประยุกต์ใช้ในการจัดการและวิเคราะห์ข้อมูลจาก
ดิจิทัลบรรพพ์ของประเทศไทย

บทที่ 2

เอกสารและงานวิจัยที่เกี่ยวข้อง

2.1 ความรู้พื้นฐานเกี่ยวกับซากดึกดำบรรพ์และบรรพชีวินวิทยา

2.1.1 ความหมายและความสำคัญของซากดึกดำบรรพ์

ซากดึกดำบรรพ์ (Fossils) หมายถึง ซากหรือร่องรอยของสิ่งมีชีวิตที่ตายไปแล้วเป็นเวลานานมากกว่า 10,000 ปี และถูกเก็บรักษาไว้ในชั้นหิน ตะกอน หรือวัสดุธรรมชาติอื่นๆ (Prothero, 2013) ซากดึกดำบรรพ์มีความสำคัญอย่างยิ่งในการศึกษาประวัติศาสตร์โลก วิวัฒนาการของสิ่งมีชีวิต สภาพแวดล้อมในอดีต ตลอดจนการเปลี่ยนแปลงทางธรณีวิทยาและภูมิอากาศ

นักบรรพชีวินวิทยา (Paleontologists) ใช้ซากดึกดำบรรพ์เป็นหลักฐานสำคัญในการศึกษา และสร้างความเข้าใจเกี่ยวกับสิ่งมีชีวิตในอดีต เช่น อายุ ถิ่นที่อยู่อาศัย พฤติกรรม และความสัมพันธ์กับสิ่งมีชีวิตอื่นๆ (Benton & Harper, 2020)

2.1.2 ประเภทของซากดึกดำบรรพ์

ซากดึกดำบรรพ์สามารถแบ่งออกเป็นประเภทต่างๆ ตามลักษณะและองค์ประกอบ ดังนี้:

1. ซากดึกดำบรรพ์สัตว์มีกระดูกสันหลัง (Vertebrate Fossils): ได้แก่ ฟอสซิลของสัตว์ที่มีกระดูกสันหลัง เช่น ไดโนเสาร์ สัตว์เลี้ยงลูกด้วยนม นก สัตว์เลื้อยคลาน และปลา ซึ่งมักพบในรูปแบบของกระดูก ฟัน และบางครั้งอาจพบเนื้อเยื่ออ่อนที่ถูกเก็บรักษาไว้ในสภาพพิเศษ (Benton, 2014)
2. ซากดึกดำบรรพ์สัตว์ไม่มีกระดูกสันหลัง (Invertebrate Fossils): ได้แก่ ฟอสซิลของสัตว์ไม่มีกระดูกสันหลัง เช่น หอย ปะการัง แมลง และสัตว์ทะเลไม่มีกระดูกสันหลังอื่นๆ ซึ่งมักพบในรูปแบบของเปลือกหอย โครงร่างแข็ง หรือรอยประทับ (Clarkson, 2020)
3. ซากดึกดำบรรพ์พืช (Plant Fossils): ได้แก่ ฟอสซิลของพืช เช่น ใบไม้ เมล็ด ลำต้น และส่วนอื่นๆ ของพืช ซึ่งมักพบในรูปแบบของรอยประทับบนหิน ถ่านหิน หรือในบางกรณีอาจพบเนื้อไม้ที่กลายเป็นหิน (Taylor et al., 2009)
4. ร่องรอยฟอสซิล (Trace Fossils): ไม่ใช่ซากของสิ่งมีชีวิตโดยตรง แต่เป็นร่องรอยกิจกรรมของสิ่งมีชีวิต เช่น รอยเท้า ทางเดิน รูขุด รอยกัดแทะ หรือขี้สัตว์ที่กลายเป็นหิน (ซึ่งเรียกว่า coprolites) (Bromley, 1990)
5. ซากดึกดำบรรพ์อื่นๆ (Other Fossils): รวมถึงจุลฟอสซิล (Microfossils) เช่น ฟอแรมมินิเฟอรา (Foraminifera) แรดิโอလာเรีย (Radiolaria) ไดอะตอม (Diatoms) และฟูลินิด (Fusulinids) ซึ่งมีขนาดเล็กมากและต้องใช้กล้องจุลทรรศน์ในการศึกษา (Lipps, 1993)

2.1.3 การจำแนกซากดึกดำบรรพ์ตามยุคทางธรณีวิทยา

การจำแนกอายุของซากดึกดำบรรพ์มักอ้างอิงตามยุคทางธรณีวิทยา (Geological Time Scale) ซึ่งแบ่งออกเป็นมหายุค (Eons), ยุค (Eras), สมัย (Periods) และช่วง (Epochs) ต่าง ๆ ดังนี้:

1. มหายุคพรีแคมเบรียน (Precambrian): ช่วงเวลาตั้งแต่โลกกำเนิดเมื่อประมาณ 4.6 พันล้านปีก่อน จนถึง 541 ล้านปีก่อน ซึ่งพบซากดึกดำบรรพ์ของสิ่งมีชีวิตเซลล์เดียว และสาหร่ายเป็นส่วนใหญ่
2. ยุคพาลีโอโซอิก (Paleozoic Era): ช่วงเวลาตั้งแต่ 541-252 ล้านปีก่อน ซึ่งพบซากดึกดำบรรพ์ของสัตว์ทะเลไม่มีกระดูกสันหลัง ปลา สัตว์สะเทินน้ำสะเทินบก และพืชบก
3. ยุคมีโซโซอิก (Mesozoic Era): ช่วงเวลาตั้งแต่ 252-66 ล้านปีก่อน เป็นช่วงที่ไดโนเสาร์เจริญรุ่งเรือง และยังพบซากดึกดำบรรพ์ของสัตว์เลี้ยงลูกจากทะเล แอมโมไนต์ และพืชมีดอก
4. ยุคซีโนโซอิก (Cenozoic Era): ช่วงเวลาตั้งแต่ 66 ล้านปีก่อนจนถึงปัจจุบัน เป็นช่วงที่สัตว์เลี้ยงลูกด้วยนมเจริญรุ่งเรือง และยังพบซากดึกดำบรรพ์ของนก สัตว์เลี้ยงลูกด้วยนมทะเล และมนุษย์ยุคแรกๆ

ในประเทศไทย พบซากดึกดำบรรพ์หลากหลายประเภทและหลายยุคสมัย ตั้งแต่ซากดึกดำบรรพ์ ยุคพาลีโอโซอิกตอนปลาย จนถึงซีโนโซอิก โดยเฉพาะในภาคตะวันออกเฉียงเหนือที่พบ ซากดึกดำบรรพ์ ไดโนเสาร์หลายสายพันธุ์ (Buffetaut et al., 2005)

2.2 การวิเคราะห์ข้อมูลด้านบรรพชีวินวิทยาด้วยเทคโนโลยีสมัยใหม่

2.2.1 บทบาทของเทคโนโลยีในการศึกษาซากดึกดำบรรพ์

ในปัจจุบัน เทคโนโลยีสมัยใหม่ได้เข้ามามีบทบาทสำคัญในการศึกษาและวิเคราะห์ซากดึกดำบรรพ์ ช่วยให้นักวิจัยสามารถศึกษาซากดึกดำบรรพ์ได้ละเอียดและแม่นยำมากขึ้น โดยมีเทคโนโลยีที่สำคัญดังนี้:

1. การสร้างภาพเสมือนจริงแบบสามมิติ (3D Visualization): เทคนิคการสแกนแบบสามมิติ ด้วยเครื่อง CT scan, Micro-CT และเลเซอร์สแกนเนอร์ ช่วยให้เราสามารถศึกษาโครงสร้างภายในของซากดึกดำบรรพ์โดยไม่ต้องทำลายตัวอย่าง (Sutton et al., 2014)
2. การวิเคราะห์ทางเคมีและกายภาพ: เทคนิคต่างๆ เช่น X-ray Fluorescence (XRF), Scanning Electron Microscopy (SEM) และ Fourier Transform Infrared Spectroscopy (FTIR) ช่วยในการวิเคราะห์องค์ประกอบทางเคมีและโครงสร้างระดับจุลภาคของซากดึกดำบรรพ์ (Koenig et al., 2009)
3. ระบบสารสนเทศภูมิศาสตร์ (GIS): ช่วยในการวิเคราะห์การกระจายตัวของแหล่งซากดึกดำบรรพ์ การจัดการข้อมูลเชิงพื้นที่ และการสร้างแบบจำลองสภาพแวดล้อมโบราณ (Anemone et al., 2011)
4. เทคโนโลยีดิจิทัล: การสร้างฐานข้อมูลดิจิทัล การจัดเก็บข้อมูลในรูปแบบดิจิทัล และการเผยแพร่ข้อมูลผ่านระบบออนไลน์ ช่วยให้นักวิจัยสามารถเข้าถึงและแลกเปลี่ยนข้อมูลได้สะดวกและรวดเร็วมากขึ้น (Uhen et al., 2013)

2.2.2 การประยุกต์ใช้วิทยาการข้อมูลและปัญญาประดิษฐ์ในงานบรรพชีวินวิทยาวิทยาการข้อมูล (Data Science) และปัญญาประดิษฐ์ (Artificial Intelligence) ได้เข้ามามีบทบาทสำคัญในการวิเคราะห์ข้อมูลด้านบรรพชีวินวิทยา ดังนี้:

1. การจำแนกและจัดหมวดหมู่ซากดึกดำบรรพ์: การใช้เทคนิคการเรียนรู้ของเครื่อง (Machine Learning) เช่น Support Vector Machines (SVM), Random Forests และ Deep Learning ในการจำแนกประเภทและสายพันธุ์ของซากดึกดำบรรพ์จากภาพถ่ายหรือข้อมูลมอร์โฟเมตริก (Hsiang et al., 2018)
2. การวิเคราะห์ภาพ (Image Analysis): การใช้ Computer Vision และ Deep Learning ในการวิเคราะห์ภาพถ่ายซากดึกดำบรรพ์ การตรวจจับโครงสร้างและลักษณะสำคัญ และการเปรียบเทียบกับฐานข้อมูล (Hsiang et al., 2019)
3. การวิเคราะห์ข้อมูลขนาดใหญ่ (Big Data Analysis): การใช้เทคนิคการวิเคราะห์ข้อมูลขนาดใหญ่ในการศึกษาการเปลี่ยนแปลงของความหลากหลายทางชีวภาพในอดีต การสูญพันธุ์ และการแพร่กระจายของสิ่งมีชีวิต (Barnosky et al., 2017)
4. การสร้างแบบจำลองและการทำนาย: การใช้เทคนิคการเรียนรู้ของเครื่องและการวิเคราะห์ทางสถิติในการสร้างแบบจำลองการกระจายตัวของซากดึกดำบรรพ์ การทำนายแหล่งที่น่าจะพบซากดึกดำบรรพ์ และการประมาณอายุของซากดึกดำบรรพ์ (Farley et al., 2018)

2.2.3 การจัดการฐานข้อมูลซากดึกดำบรรพ์

การจัดการฐานข้อมูลซากดึกดำบรรพ์เป็นเรื่องสำคัญในการศึกษาและวิจัยด้านบรรพชีวินวิทยา โดยมีแนวทางและระบบการจัดการข้อมูลดังนี้:

1. ฐานข้อมูลระดับสากล: ฐานข้อมูลระดับสากลที่รวบรวมข้อมูลซากดึกดำบรรพ์จากทั่วโลก เช่น Paleobiology Database (PBDB), Integrated Digitized Biocollections (iDigBio) และ Global Biodiversity Information Facility (GBIF) ช่วยให้นักวิจัยสามารถเข้าถึงข้อมูลซากดึกดำบรรพ์จากทั่วโลกได้ (Uhen et al., 2013)
2. การมาตรฐานข้อมูล: การกำหนดมาตรฐานในการจัดเก็บและแลกเปลี่ยนข้อมูลซากดึกดำบรรพ์ เช่น Darwin Core Standard และ Access to Biological Collections Data (ABCD) Schema ช่วยให้อาจแลกเปลี่ยนข้อมูลระหว่างฐานข้อมูลได้สะดวกมากขึ้น (Wieczorek et al., 2012)
3. การจัดเก็บข้อมูลเชิงพื้นที่: การใช้ระบบสารสนเทศภูมิศาสตร์ (GIS) ในการจัดเก็บและวิเคราะห์ข้อมูลเชิงพื้นที่ของแหล่งซากดึกดำบรรพ์ ช่วยในการศึกษาการกระจายตัวและความสัมพันธ์กับปัจจัยทางสิ่งแวดล้อม (Butler et al., 2012)
4. การเชื่อมโยงข้อมูล: การเชื่อมโยงข้อมูลซากดึกดำบรรพ์กับข้อมูลอื่นๆ เช่น ข้อมูลทางธรณีวิทยา ข้อมูลทางภูมิศาสตร์ และข้อมูลทางชีววิทยา ช่วยให้เกิดการบูรณาการและการวิเคราะห์ข้อมูลที่ซับซ้อนมากขึ้น (Kenny et al., 2020)

2.3 งานวิจัยที่เกี่ยวข้องกับการประยุกต์ใช้ AI ในการวิเคราะห์ซากดึกดำบรรพ์

2.3.1 การจำแนกประเภทซากดึกดำบรรพ์ด้วยเทคนิคการเรียนรู้ของเครื่อง

มีงานวิจัยหลายชิ้นที่ศึกษาการใช้เทคนิคการเรียนรู้ของเครื่องในการจำแนกประเภทซากดึก

ดำบรรพ์ เช่น:

1. Hsiang et al. (2019) ได้พัฒนาระบบ Deep Learning สำหรับการจำแนกไมโครฟอสซิลจากภาพถ่ายกล้องจุลทรรศน์ โดยใช้ Convolutional Neural Networks (CNNs) ในการจำแนกชนิดของฟอแรมมินิเฟอราจากภาพถ่าย ซึ่งให้ความแม่นยำมากกว่า 90%
2. Madin et al. (2020) ได้ใช้เทคนิค Random Forests และ Support Vector Machines ในการจำแนกประเภทของปะการังดึกดำบรรพ์จากลักษณะทางสัณฐานวิทยา ซึ่งช่วยให้สามารถจำแนกกลุ่มและสายพันธุ์ได้อย่างแม่นยำมากขึ้น
3. Sundermeyer et al. (2021) ได้ใช้เทคนิค Deep Learning ในการจำแนกรอยฟันของไดโนเสาร์กินเนื้อจากภาพถ่ายและข้อมูล 3D scanning ซึ่งช่วยในการระบุชนิดของไดโนเสาร์จากรอยฟันได้อย่างแม่นยำมากขึ้น

2.3.2 การใช้ Image Processing และ Computer Vision ในการวิเคราะห์ซากดึกดำบรรพ์

เทคนิค Image Processing และ Computer Vision มีบทบาทสำคัญในการวิเคราะห์ซากดึกดำบรรพ์ โดยมีงานวิจัยที่เกี่ยวข้องดังนี้:

1. Otero et al. (2020) ได้พัฒนาระบบ Computer Vision สำหรับการวิเคราะห์ลักษณะทางสัณฐานวิทยาของฟันและกระดูกสัตว์ดึกดำบรรพ์ ซึ่งช่วยในการจำแนกสายพันธุ์และการศึกษาวิวัฒนาการของสัตว์มีกระดูกสันหลัง
2. Garwood & Dunlop (2017) ได้ใช้เทคนิค Image Processing ร่วมกับข้อมูลจาก Micro-CT scan ในการสร้างภาพเสมือนจริงแบบสามมิติของแมงป่อง และแมงมุมดึกดำบรรพ์ ซึ่งช่วยให้สามารถศึกษาโครงสร้างภายในและภายนอกได้อย่างละเอียด
3. Goswami et al. (2019) ได้ใช้เทคนิค Geometric Morphometrics ร่วมกับ Computer Vision ในการวิเคราะห์การเปลี่ยนแปลงรูปร่างของกะโหลกสัตว์เลี้ยงลูกด้วยนมตั้งแต่ยุคดึกดำบรรพ์จนถึงปัจจุบัน ซึ่งช่วยในการศึกษาวิวัฒนาการและความสัมพันธ์ทางสายพันธุ์

2.3.3 การทำนายแหล่งซากดึกดำบรรพ์ด้วย AI

การใช้ AI ในการทำนายแหล่งที่มีศักยภาพในการพบซากดึกดำบรรพ์เป็นแนวทางที่น่าสนใจ โดยมีงานวิจัยที่เกี่ยวข้องดังนี้:

1. Anemone et al. (2011) ได้ใช้ Neural Networks ร่วมกับข้อมูล GIS ในการทำนายแหล่งที่มีศักยภาพในการพบซากดึกดำบรรพ์สัตว์เลื้อยคลานด้วยนมในทวีปอเมริกาเหนือ ซึ่งช่วยให้ทีมวิจัยค้นพบแหล่งซากดึกดำบรรพ์ใหม่หลายแห่ง
2. Conroy et al. (2018) ได้ใช้เทคนิค Maximum Entropy Modeling (MaxEnt) ในการทำนายการกระจายตัวของซากดึกดำบรรพ์ไดโนเสาร์ในพื้นที่ภาคตะวันตกของสหรัฐอเมริกา โดยใช้ข้อมูลทางธรณีวิทยาและภูมิศาสตร์เป็นตัวแปรในการทำนาย
3. Emerson et al. (2015) ได้ใช้เทคนิค Random Forests และ Boosted Regression Trees ในการทำนายแหล่งที่มีศักยภาพในการพบซากดึกดำบรรพ์พืชในทวีปยุโรป ซึ่งช่วยในการวางแผนการสำรวจและการอนุรักษ์แหล่งซากดึกดำบรรพ์

2.4 ซากดึกดำบรรพ์ในประเทศไทย

2.4.1 ความสำคัญและความหลากหลายของซากดึกดำบรรพ์ในประเทศไทย

ประเทศไทยมีแหล่งซากดึกดำบรรพ์ที่สำคัญและมีความหลากหลาย ซึ่งมีความสำคัญทั้งในแง่ของการศึกษาวิจัยทางวิทยาศาสตร์และการท่องเที่ยวเชิงธรณีวิทยา:

1. ซากดึกดำบรรพ์ไดโนเสาร์: ประเทศไทยเป็นแหล่งค้นพบซากดึกดำบรรพ์ไดโนเสาร์ที่สำคัญในภูมิภาคเอเชียตะวันออกเฉียงใต้ โดยเฉพาะในภาคตะวันออกเฉียงเหนือ เช่น จังหวัดขอนแก่น กาฬสินธุ์ และนครราชสีมา ซึ่งพบไดโนเสาร์สายพันธุ์ใหม่ของโลกหลายชนิด เช่น ภูเวียงโกซอรัส สยามโมไทรันนัส อีสานโนซอรัส และสิรินธรนา (Buffetaut et al., 2005)
2. ซากดึกดำบรรพ์สัตว์มีกระดูกสันหลังอื่นๆ: นอกจากไดโนเสาร์แล้ว ยังพบซากดึกดำบรรพ์ของสัตว์เลื้อยคลานด้วยนมยุคโบราณ เช่น ซ้างดึกดำบรรพ์ แรด และสัตว์เลื้อยคลานดึกดำบรรพ์อื่นๆ ในหลายพื้นที่ของประเทศไทย (Tougaard, 2001)
3. ซากดึกดำบรรพ์สัตว์ไม่มีกระดูกสันหลัง: พบซากดึกดำบรรพ์ของสัตว์ไม่มีกระดูกสันหลัง เช่น แอมโมไนต์ หอยทะเลโบราณ ปะการัง และไทรโลไบต์ ในหลายพื้นที่ โดยเฉพาะในภาคใต้ และภาคตะวันตกของประเทศไทย (Meesook, 2000)
4. ซากดึกดำบรรพ์พืช: พบซากดึกดำบรรพ์ของพืชหลายชนิด เช่น เฟิร์นโบราณ ต้นสน และพืชมีเมล็ดอื่นๆ ในหลายพื้นที่ โดยเฉพาะในภาคเหนือและภาคตะวันออกเฉียงเหนือของประเทศไทย (Chonglakmani, 2011)
5. ร่องรอยฟอสซิล: พบร่องรอยกิจกรรมของสิ่งมีชีวิตโบราณ เช่น รอยเท้าไดโนเสาร์ รอยเท้านก และร่องรอยกิจกรรมของสัตว์ไม่มีกระดูกสันหลัง ในหลายพื้นที่ของประเทศไทย (Matsukawa et al., 2006)

2.4.2 แหล่งซากดึกดำบรรพ์ที่สำคัญในประเทศไทย

ประเทศไทยมีแหล่งซากดึกดำบรรพ์ที่สำคัญหลายแห่ง ซึ่งมีทั้งแหล่งที่เปิดให้เป็นแหล่งท่องเที่ยวและ แหล่งศึกษาวิจัย ดังนี้:

1. พิพิธภัณฑ์สิรินธร (จังหวัดกาฬสินธุ์): เป็นพิพิธภัณฑ์ไดโนเสาร์ที่ใหญ่และสมบูรณ์ที่สุดในเอเชียตะวันออกเฉียงใต้ จัดแสดงซากดึกดำบรรพ์ไดโนเสาร์และสัตว์โบราณหลายชนิด รวมทั้งภูเวียงโกซอรัส และสิรินธรนา (Suteethorn et al., 2009)
2. พิพิธภัณฑ์ไดโนเสาร์ภูเวียง (จังหวัดขอนแก่น): เป็นแหล่งค้นพบซากดึกดำบรรพ์ไดโนเสาร์ชนิดแรกของประเทศไทย คือ ภูเวียงโกซอรัส สิรินธรนา จัดแสดงซากดึกดำบรรพ์และข้อมูลเกี่ยวกับไดโนเสาร์และสัตว์ร่วมยุค (Buffetaut & Suteethorn, 1992)
3. อุทยานธรณีโคราช (จังหวัดนครราชสีมา): เป็นแหล่งซากดึกดำบรรพ์ที่สำคัญในภาคตะวันออกเฉียงเหนือ ที่พบซากดึกดำบรรพ์ไดโนเสาร์และสัตว์โบราณหลายชนิด รวมทั้งซากดึกดำบรรพ์ของพืชและสัตว์ไม่มีกระดูกสันหลัง (Tumpeesuwan et al., 2010)
4. เขานินปุนศรีราชา (จังหวัดชลบุรี): เป็นแหล่งซากดึกดำบรรพ์สัตว์ทะเลไม่มีกระดูกสันหลังยุคเพอร์เมียน ที่มีความหลากหลายและสมบูรณ์มาก พบซากดึกดำบรรพ์ของแบคทีเรียพอดเปการัง และไครนอยด์ (Meesook et al., 2001)
5. อุทยานแห่งชาติดอยผ้าห่มปก (จังหวัดเชียงใหม่): เป็นแหล่งซากดึกดำบรรพ์พืชยุคเทอร์เชียรี ที่พบซากใบไม้และท่อนไม้กลายเป็นหินจำนวนมาก ซึ่งมีความสำคัญต่อการศึกษาวិวัฒนาการของพืชและสภาพแวดล้อมในอดีตของภาคเหนือของประเทศไทย (Bennami et al., 2012)

2.4.3 การจัดการและการอนุรักษ์แหล่งซากดึกดำบรรพ์ในประเทศไทย

การจัดการและการอนุรักษ์แหล่งซากดึกดำบรรพ์ในประเทศไทยมีการดำเนินการโดยหน่วยงานต่าง ๆ

ดังนี้:

1. กรมทรัพยากรธรณี: เป็นหน่วยงานหลักที่รับผิดชอบในการสำรวจ ศึกษาวิจัย อนุรักษ์ และบริหารจัดการแหล่งซากดึกดำบรรพ์ของประเทศไทย รวมทั้งการขึ้นทะเบียนแหล่งซากดึกดำบรรพ์และการประกาศเขตสำรวจและศึกษาวิจัยซากดึกดำบรรพ์ (Department of Mineral Resources, 2020)
2. พระราชบัญญัติคุ้มครองซากดึกดำบรรพ์ พ.ศ. 2551: เป็นกฎหมายที่ใช้ในการคุ้มครองและบริหารจัดการซากดึกดำบรรพ์และแหล่งซากดึกดำบรรพ์ในประเทศไทย ซึ่งกำหนดให้มีการขึ้นทะเบียนแหล่งซากดึกดำบรรพ์ที่มีความสำคัญ และควบคุมการขุดค้น เก็บรักษา และครอบครองซากดึกดำบรรพ์ (Royal Thai Government Gazette, 2008)
3. อุทยานธรณี (Geopark): เป็นรูปแบบการอนุรักษ์และการจัดการพื้นที่ที่มีความสำคัญทางธรณีวิทยา และซากดึกดำบรรพ์ โดยส่งเสริมการท่องเที่ยวเชิงธรณีวิทยา การศึกษา และการมีส่วนร่วมของชุมชนท้องถิ่น เช่น อุทยานธรณีโคราช (จังหวัดนครราชสีมา) และอุทยานธรณีสตูล (จังหวัดสตูล) ซึ่งได้รับการรับรองเป็นอุทยานธรณีโลกของยูเนสโก (UNESCO Global Geopark) (Singtuen & Won-In, 2018)
4. การมีส่วนร่วมของชุมชนและภาคการศึกษา: การส่งเสริมการมีส่วนร่วมของชุมชนท้องถิ่นและสถาบันการศึกษาในการอนุรักษ์และจัดการแหล่งซากดึกดำบรรพ์ ผ่านกิจกรรมต่างๆ เช่น การฝึกอบรม การให้ความรู้ และการส่งเสริมการท่องเที่ยวเชิงธรณีวิทยา (Tumpeesuwan et al., 2018)

2.5 การประยุกต์ใช้ปัญญาประดิษฐ์ในการวิเคราะห์ข้อมูลซากดึกดำบรรพ์

2.5.1 เทคนิคการเรียนรู้ของเครื่อง (Machine Learning) ที่ใช้ในการวิเคราะห์ข้อมูล

การประยุกต์ใช้เทคนิคการเรียนรู้ของเครื่องในการวิเคราะห์ข้อมูลซากดึกดำบรรพ์มีหลายรูปแบบ ดังนี้:

1. การเรียนรู้แบบมีผู้สอน (Supervised Learning): เป็นเทคนิคที่ใช้ข้อมูลที่มีการระบุคลาสหรือค่าเป้าหมาย (labeled data) ในการฝึกโมเดล เช่น:

- Random Forest: เป็นอัลกอริทึมที่ใช้หลักการของการเรียนรู้แบบแบ่งกลุ่ม (ensemble learning) โดยสร้างต้นไม้ตัดสินใจ (decision trees) จำนวนมากแล้วทำการโหวตเพื่อตัดสินใจ ซึ่งมีประสิทธิภาพดีในการจำแนกประเภทและการทำนาย (Breiman, 2001)
- Support Vector Machines (SVM): เป็นอัลกอริทึมที่หาไฮเปอร์เพลน (hyperplane) ที่แบ่งข้อมูลได้ดีที่สุด ซึ่งมีประสิทธิภาพดีในการจำแนกประเภทข้อมูลที่มีมิติสูง (Vapnik, 1995)
- Neural Networks: เป็นโมเดลที่จำลองการทำงานของเซลล์ประสาทในสมองมนุษย์ ซึ่งมีความสามารถในการเรียนรู้รูปแบบที่ซับซ้อนและไม่เชิงเส้น (LeCun et al., 2015)

2. การเรียนรู้แบบไม่มีผู้สอน (Unsupervised Learning): เป็นเทคนิคที่ใช้ข้อมูลที่ไม่มีการระบุคลาสหรือค่าเป้าหมาย (unlabeled data) ในการค้นหารูปแบบหรือโครงสร้างในข้อมูล เช่น:

- K-means Clustering: เป็นอัลกอริทึมที่แบ่งข้อมูลออกเป็นกลุ่ม (clusters) ตามความคล้ายคลึงกัน (Lloyd, 1982)
- Hierarchical Clustering: เป็นอัลกอริทึมที่สร้างลำดับชั้นของกลุ่มข้อมูล (Johnson, 1967)
- Principal Component Analysis (PCA): เป็นเทคนิคในการลดมิติของข้อมูล (dimensionality reduction) โดยหาองค์ประกอบหลัก (principal components) ที่อธิบายความแปรปรวนในข้อมูลได้มากที่สุด (Jolliffe, 2002)

3. การเรียนรู้เชิงลึก (Deep Learning): เป็นเทคนิคการเรียนรู้ของเครื่องที่ใช้โครงข่ายประสาทเทียมหลายชั้น (multi-layer neural networks) ในการเรียนรู้รูปแบบที่ซับซ้อนจากข้อมูล เช่น:

- Convolutional Neural Networks (CNNs): เป็นโครงข่ายประสาทเทียมที่ออกแบบมาสำหรับการประมวลผลข้อมูลแบบตาราง เช่น ภาพ ซึ่งมีประสิทธิภาพดีในการจำแนกและวิเคราะห์ภาพ (Krizhevsky et al., 2012)
- Recurrent Neural Networks (RNNs): เป็นโครงข่ายประสาทเทียมที่ออกแบบมาสำหรับการประมวลผลข้อมูลแบบลำดับ (sequential data) ซึ่งมีประสิทธิภาพดีในการวิเคราะห์ข้อมูลที่มีลักษณะต่อเนื่องหรือเป็นลำดับ (Hochreiter & Schmidhuber, 1997)
- Autoencoder: เป็นโครงข่ายประสาทเทียมที่เรียนรู้การเข้ารหัส (encoding) และถอดรหัส (decoding) ข้อมูล ซึ่งมีประสิทธิภาพดีในการลดมิติของข้อมูลและการกรองสัญญาณรบกวน (denoising) (Hinton & Salakhutdinov, 2006)

2.5.2 การประมวลผลภาพและ Computer Vision สำหรับการวิเคราะห์ซากดึกดำบรรพ์

การประมวลผลภาพและ Computer Vision มีบทบาทสำคัญในการวิเคราะห์ซากดึกดำบรรพ์ โดยมีเทคนิคและการประยุกต์ใช้ดังนี้:

1. การปรับปรุงคุณภาพภาพ (Image Enhancement): การปรับปรุงคุณภาพภาพถ่ายซากดึกดำบรรพ์ให้มีความชัดเจนและคมชัดมากขึ้น เช่น การปรับความสว่าง (brightness), ความเปรียบต่าง (contrast), และการกรองสัญญาณรบกวน (noise filtering) (Gonzalez & Woods, 2018)
2. การแบ่งส่วนภาพ (Image Segmentation): การแบ่งภาพออกเป็นส่วนต่างๆ ตามลักษณะที่สนใจ เช่น การแยกส่วนของซากดึกดำบรรพ์ออกจากพื้นหลัง หรือการแบ่งส่วนของโครงสร้างต่างๆ ของซากดึกดำบรรพ์ (Otsu, 1979)
3. การสกัดคุณลักษณะ (Feature Extraction): การสกัดคุณลักษณะสำคัญจากภาพถ่ายซากดึกดำบรรพ์ เช่น รูปร่าง (shape), พื้นผิว (texture), สี (color), และขอบ (edges) ซึ่งสามารถนำไปใช้ในการจำแนกและวิเคราะห์ต่อไป (Nixon & Aguado, 2019)
4. การรู้จำวัตถุ (Object Recognition): การระบุและจำแนกซากดึกดำบรรพ์หรือส่วนต่างๆ ของซากดึกดำบรรพ์จากภาพถ่าย โดยใช้เทคนิคการเรียนรู้ของเครื่อง เช่น Deep Learning และ CNNs (He et al., 2015)
5. การวิเคราะห์รูปร่างทางเรขาคณิต (Geometric Morphometrics): การวิเคราะห์รูปร่างและโครงสร้างของซากดึกดำบรรพ์โดยใช้เทคนิคทางเรขาคณิต เช่น การวิเคราะห์จุดพิกัด (landmark analysis) และการวิเคราะห์เส้นโครงร่าง (outline analysis) ซึ่งช่วยในการศึกษาความแตกต่างทางสัณฐานวิทยาและวิวัฒนาการ (Zelditch et al., 2012)
6. การสร้างโมเดล 3 มิติ (3D Modeling): การสร้างโมเดล 3 มิติของซากดึกดำบรรพ์จากภาพถ่ายหรือข้อมูลสแกน ซึ่งช่วยในการวิเคราะห์โครงสร้างและการทำงานของสิ่งมีชีวิตในอดีต (Mallison & Wings, 2014)

2.5.3 การใช้ Word Embedding และ NLP ในการวิเคราะห์ข้อมูลเชิงบรรยาย

การวิเคราะห์ข้อมูลเชิงบรรยายเกี่ยวกับซากดึกดำบรรพ์ เช่น คำอธิบายลักษณะ บทความ วิชาการ และรายงานการสำรวจ สามารถทำได้โดยใช้เทคนิค Natural Language Processing (NLP) และ Word Embedding ดังนี้:

1. Word Embedding: เป็นเทคนิคในการแปลงคำให้อยู่ในรูปของเวกเตอร์ตัวเลข (numerical vectors) ที่สามารถใช้ในการคำนวณทางคณิตศาสตร์ได้ โดยคำที่มีความหมายใกล้เคียงกัน จะมีเวกเตอร์ที่ใกล้เคียงกันในพื้นที่เวกเตอร์ (vector space) เช่น:
 - Word2Vec: เป็นเทคนิคการเรียนรู้ของเครื่องที่ใช้โครงข่ายประสาทเทียมในการสร้าง word embeddings โดยทำนายบริบทของคำจากคำที่กำหนด (Skip-gram) หรือทำนายคำจากบริบทที่กำหนด (CBOW) (Mikolov et al., 2013)
 - GloVe (Global Vectors for Word Representation): เป็นอัลกอริทึมที่สร้าง word embeddings โดยใช้สถิติการเกิดร่วมกันของคำในคลังข้อมูลขนาดใหญ่ (Pennington et al., 2014)
 - BERT (Bidirectional Encoder Representations from Transformers): เป็นโมเดลภาษาแบบ transformer ที่เรียนรู้บริบทของคำจากทั้งสองทิศทาง (ซ้ายไปขวาและขวาไปซ้าย) ซึ่งสามารถสร้าง contextual word embeddings ที่มีความหมายลึกซึ้งมากขึ้น (Devlin et al., 2018)
 - WangchanBERTa: เป็นโมเดลภาษาไทยที่พัฒนามาบนพื้นฐานของ RoBERTa (Robustly Optimized BERT Pretraining Approach) ซึ่งเป็นการปรับปรุง BERT ให้มีประสิทธิภาพดีขึ้น โดย WangchanBERTa ได้รับการฝึกฝนด้วยคลังข้อมูลภาษาไทยขนาดใหญ่ ทำให้มีความสามารถในการเข้าใจบริบทและความหมายของคำในภาษาไทยได้ดี (Lowphansirikul et al., 2021)

2. การวิเคราะห์เนื้อหา (Content Analysis): การใช้เทคนิค NLP ในการวิเคราะห์เนื้อหาของข้อมูลเชิงบรรยายเกี่ยวกับซากดึกดำบรรพ์ เช่น:
 - Topic Modeling: การใช้เทคนิคเช่น Latent Dirichlet Allocation (LDA) ในการค้นหาหัวข้อ (topics) ที่ซ่อนอยู่ในเอกสารชุดใหญ่ (Blei et al., 2003)
 - Sentiment Analysis: การวิเคราะห์ความรู้สึกหรือทัศนคติที่แสดงออกในข้อความ (Pang & Lee, 2008)
 - Named Entity Recognition (NER): การระบุและจำแนกชื่อเฉพาะในข้อความ เช่น ชื่อสายพันธุ์ ชื่อสถานที่ และชื่อบุคคล (Nadeau & Sekine, 2007)
3. การจัดกลุ่มเอกสาร (Document Clustering): การจัดกลุ่มเอกสารที่มีเนื้อหาคล้ายคลึงกันโดยใช้เทคนิคการเรียนรู้แบบไม่มีผู้สอน เช่น K-means Clustering และ Hierarchical Clustering (Manning et al., 2008)
4. การสรุปความอัตโนมัติ (Automatic Summarization): การสร้างสรุปย่อของเอกสารโดยอัตโนมัติ ซึ่งช่วยในการสกัดข้อมูลสำคัญจากเอกสารขนาดใหญ่ (Nenkova & McKeown, 2012)

2.5.4 การใช้ Feature Engineering และการเลือกคุณลักษณะที่เหมาะสม

Feature Engineering และการเลือกคุณลักษณะที่เหมาะสมเป็นขั้นตอนสำคัญในการพัฒนาโมเดล การเรียนรู้ของเครื่องสำหรับการวิเคราะห์ข้อมูลซากดึกดำบรรพ์ โดยมีแนวทางและเทคนิคดังนี้:

1. Feature Engineering: เป็นกระบวนการสร้างคุณลักษณะใหม่จากข้อมูลดิบหรือคุณลักษณะที่มีอยู่ เพื่อเพิ่มประสิทธิภาพในการเรียนรู้ของโมเดล เช่น:
 - Scaling: การปรับขนาดของคุณลักษณะให้อยู่ในช่วงที่เหมาะสม เช่น StandardScaler (ปรับให้มีค่าเฉลี่ยเป็น 0 และส่วนเบี่ยงเบนมาตรฐานเป็น 1) และ MinMaxScaler (ปรับให้อยู่ในช่วง 0 ถึง 1) (Patro & Sahu, 2015)

- Encoding: การแปลงคุณลักษณะเชิงคุณภาพให้อยู่ในรูปแบบที่โมเดลสามารถเรียนรู้ได้ เช่น One-Hot Encoding, Label Encoding และ Target Encoding (Potdar et al., 2017)
 - Feature Extraction: การสกัดคุณลักษณะจากข้อมูลดิบ เช่น การใช้ PCA ในการลดมิติของข้อมูล และการใช้เทคนิค feature extraction จากภาพและข้อความ (Jolliffe, 2002)
 - Feature Transformation: การแปลงคุณลักษณะให้มีการกระจายตัวที่เหมาะสมกับโมเดล เช่น การแปลงแบบลอการิทึม (logarithmic transformation) และการแปลงแบบรากที่สอง (square root transformation) (Box & Cox, 1964)
2. Feature Selection: เป็นกระบวนการเลือกคุณลักษณะที่มีความสำคัญและมีประโยชน์ต่อการทำนาย โดยตัดคุณลักษณะที่ไม่จำเป็นหรือมีความสำคัญน้อยออกไป เพื่อลดความซับซ้อนของโมเดลและป้องกันปัญหา overfitting เช่น:
- Filter Methods: เป็นวิธีการเลือกคุณลักษณะโดยใช้เกณฑ์ทางสถิติ เช่น Pearson Correlation, Chi-Square Test และ Information Gain (Guyon & Elisseeff, 2003)
 - Wrapper Methods: เป็นวิธีการเลือกคุณลักษณะโดยใช้โมเดลการเรียนรู้ของเครื่องในการประเมินชุดคุณลักษณะต่างๆ เช่น Recursive Feature Elimination (RFE) และ Sequential Feature Selection (Kohavi & John, 1997)
 - Embedded Methods: เป็นวิธีการเลือกคุณลักษณะที่รวมอยู่ในกระบวนการเรียนรู้ของโมเดล เช่น LASSO Regression และ Random Forest Feature Importance (Tibshirani, 1996)

3. Feature Importance Analysis: เป็นการวิเคราะห์ความสำคัญของคุณลักษณะต่าง ๆ ที่มีต่อการทำนายของโมเดล ซึ่งช่วยในการทำความเข้าใจความสัมพันธ์ระหว่างคุณลักษณะและผลลัพธ์ เช่น:

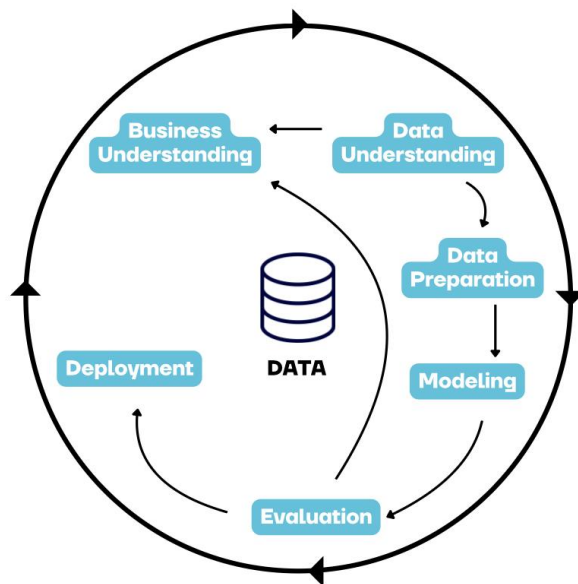
- Random Forest Feature Importance: การวัดความสำคัญของคุณลักษณะในโมเดล Random Forest โดยวัดจากการลดลงของความไม่บริสุทธิ์ (impurity) หรือความแม่นยำเมื่อใช้คุณลักษณะนั้น

บทที่ 3

วิธีการดำเนินงาน

3.1 ภาพรวมของกระบวนการวิจัย

การศึกษาในครั้งนี้ดำเนินการตามกระบวนการมาตรฐานข้ามอุตสาหกรรมสำหรับการทำเหมืองข้อมูล (CRISP-DM: Cross-Industry Standard Process for Data Mining) ซึ่งเป็นแนวทางที่ได้รับการยอมรับในอุตสาหกรรมสำหรับการดำเนินโครงการวิทยาศาสตร์ข้อมูล กระบวนการนี้ประกอบด้วย 6 ขั้นตอนหลัก ได้แก่ การทำความเข้าใจธุรกิจ (Business Understanding), การทำความเข้าใจข้อมูล (Data Understanding), การเตรียมข้อมูล (Data Preparation), การสร้างแบบจำลอง (Modeling), การประเมินผล (Evaluation) และการนำไปใช้งาน (Deployment)



รูปที่ 3.1 กระบวนการ CRISP-DM สำหรับการทำเหมืองข้อมูล

ในบทที่ 1 และบทที่ 2 ได้กล่าวถึงส่วนของการทำความเข้าใจธุรกิจ (Business Understanding) ซึ่งครอบคลุมการกำหนดเป้าหมายของการวิจัย ความเข้าใจเกี่ยวกับบริบทของปัญหา และการศึกษาเอกสารและ

งานวิจัยที่เกี่ยวข้อง ในบทนี้จะนำเสนอวิธีการดำเนินงานในขั้นตอนอื่นๆ ของกระบวนการ CRISP-DM การศึกษานี้ได้พัฒนาสองแนวคิดหลักในการวิเคราะห์ข้อมูลซากดึกดำบรรพ์ โดยแต่ละแนวคิดมีวิธีการดำเนินงานที่แตกต่างกัน ดังจะได้อธิบายในหัวข้อต่อไป

3.2 แนวคิดที่ 1: การจำแนกประเภทซากดึกดำบรรพ์โดยใช้ข้อมูลรอบด้าน

แนวคิดที่ 1 เป็นการพัฒนาแบบจำลองสำหรับการจำแนกประเภทของซากดึกดำบรรพ์ (Fossil Classification) โดยใช้ข้อมูลหลากหลายประเภทที่เกี่ยวข้องกับซากดึกดำบรรพ์ ได้แก่ ข้อมูลเชิงบรรยาย ข้อมูลเชิงพื้นที่ ข้อมูลทางธรณีวิทยา และข้อมูลภาพถ่าย มีขั้นตอนการดำเนินงานดังนี้

3.2.1 การทำความเข้าใจข้อมูล (Data Understanding)

3.2.1.1 แหล่งที่มาของข้อมูล

ข้อมูลที่ใช้ในการศึกษานี้มาจากฐานข้อมูลตำแหน่งแหล่งซากดึกดำบรรพ์ของกรมทรัพยากรธรณี ซึ่งเผยแพร่ผ่าน ArcGIS Rest Service สามารถเข้าถึงได้จากเว็บไซต์ data.go.th ข้อมูลนี้ประกอบด้วยบันทึกซากดึกดำบรรพ์จำนวน 520 รายการ

3.2.1.2 โครงสร้างข้อมูล (Data Dictionary)

ชุดข้อมูลประกอบด้วยตาราง T_FOSSIL_SITE ซึ่งมีฟิลด์ข้อมูลทั้งหมด 47 ฟิลด์ โดยมีฟิลด์ที่สำคัญดังนี้

Table Name	T_FOSSIL_SITE	
Table Description	ตำแหน่งแหล่งซากดึกดำบรรพ์ (Excel-Point)	
Column Name	Description	Data Type
OBJECTID	ID ของออเบเจ็กต์	INT
SITE_ID	รหัสแหล่งซากดึกดำบรรพ์	FLOAT
SITE_CODE	รหัสแหล่งซากดึกดำบรรพ์	NVARCHAR(50)
NAME_TH	ชื่อภาษาไทย	NVARCHAR(255)

Table Name	T_FOSSIL_SITE	
Table Description	ตำแหน่งแหล่งซากดึกดำบรรพ์ (Excel-Point)	
Column Name	Description	Data Type
NAME_ENG	ชื่อภาษาอังกฤษ	NVARCHAR(255)
IMPORTA_TH	ความสำคัญของแหล่ง (ไทย)	NVARCHAR(255)
IMPORTA_EN	ความสำคัญของแหล่ง (อังกฤษ)	NVARCHAR(255)
LOCALITY	สถานที่	NVARCHAR(255)
TAMBON	ตำบล	NVARCHAR(100)
DISTRICT	อำเภอ	NVARCHAR(100)
PROVINCE	จังหวัด	NVARCHAR(100)
UTM_E	ค่า UTM ตะวันออกพื้นหลักฐาน WGS 84	INT
UTM_N	ค่า UTM เหนือ พื้นหลักฐาน WGS 84	INT
ZONE	กริดโซน	NVARCHAR (5)
MAPSHEET	เลขระวาง	NVARCHAR(10)
SHEET_NAME	ชื่อระวาง	NVARCHAR(100)

Table Name	T_FOSSIL_SITE	
Table Description	ตำแหน่งแหล่งซากดึกดำบรรพ์ (Excel-Point)	
Column Name	Description	Data Type
FEATURE_TH	ลักษณะแหล่ง (ภาษาไทย)	NVARCHAR(255)
FEATURE_EN	ลักษณะแหล่ง (ภาษาอังกฤษ)	NVARCHAR(255)
GEO_DES_TH	ลักษณะทางธรณีวิทยา (ภาษาไทย)	NVARCHAR(255)
GEO_DES_EN	ลักษณะทางธรณีวิทยา (ภาษาอังกฤษ)	NVARCHAR(255)
GEO_GROUP	ชื่อ Geologic Group	NVARCHAR(255)
FORMATION	ชื่อ Geologic Formation	NVARCHAR(255)
PERIODFROM	SYSTEM_PERIOD_FROM	NVARCHAR(255)
PERIODTO	SYSTEM_PERIOD_TO	NVARCHAR(255)
FOS_DES_TH	ลักษณะซากดึกดำบรรพ์ที่ปรากฏ (ไทย)	NVARCHAR(255)
FOS_GR_1	Vertebrate	NVARCHAR(255)
FOS_GR_2	Invertebrate	NVARCHAR(255)
FOS_GR_3	Plant	NVARCHAR(255)
FOS_GR_4	Trace	NVARCHAR(255)
FOS_GR_5	Other (e.g. radiolaria, fusulinid, alga)	NVARCHAR(255)

Table Name	T_FOSSIL_SITE	
Table Description	ตำแหน่งแหล่งซากดึกดำบรรพ์ (Excel-Point)	
Column Name	Description	Data Type
DIS_BY_ENG	ชื่อผู้ค้นพบ (ภาษาอังกฤษ)	NVARCHAR(255)
DIS_BY_TH	ชื่อผู้ค้นพบ (ภาษาไทย)	NVARCHAR(255)
DIS_DAY	วันที่ค้นพบ	NVARCHAR(255)
DIS_MONTH	เดือนที่ค้นพบ	FLOAT
DIS_YEAR	ปีที่ค้นพบ	FLOAT
OWNER_TH	ผู้ครอบครอง (ภาษาไทย)	NVARCHAR(255)
OWNER_ENG	ผู้ครอบครอง (ภาษาอังกฤษ)	NVARCHAR(255)
POTENTIAL1	เข้าเกณฑ์การขึ้นทะเบียน	NVARCHAR(255)
POTENTIAL2	เข้าเกณฑ์การประกาศเป็นเขตสำรวจ ศึกษาวิจัย	NVARCHAR(255)
POTENTIAL3	แหล่งอ้างอิงทางวิชาการ	NVARCHAR(255)
POTENTIAL4	แหล่งเรียนรู้ / แหล่งท่องเที่ยว	NVARCHAR(255)
POTENTIAL5	แหล่งขึ้นทะเบียน	NVARCHAR(255)
POTENTIAL6	เขตสำรวจ ศึกษาวิจัย	NVARCHAR(255)
POTENTIAL7	รอการตรวจสอบ	NVARCHAR(255)
STATUS	ระดับการเผยแพร่	NVARCHAR(255)
CREATEBY	นำเข้าข้อมูลในระบบโดยใคร	NVARCHAR (50)
CREATEDATE	วันที่นำเข้าข้อมูลในระบบ	DATETIME2 (7)
UPDATEBY	ปรับปรุงในระบบโดย	NVARCHAR (50)
UPDATEDATE	วันที่ปรับปรุงในระบบ	DATETIME2 (7)
SHAPE	Shape	GEOMETRY
GDB_GEOMATTR_DATA	SQL Server spatial types	VARBINARY(MAX)

ตารางที่ 3.1 โครงสร้างข้อมูลของตาราง T_FOSSIL_SITE

ในการศึกษาฟอสซิล FOS_GR_1 ถึง FOS_GR_5 ถูกใช้เป็นเป้าหมายในการจำแนกประเภทของซากดึกดำบรรพ์ โดยแต่ละฟอสซิลแทนประเภทหลักของซากดึกดำบรรพ์ ได้แก่ สัตว์มีกระดูกสันหลัง (Vertebrate), สัตว์ไม่มีกระดูกสันหลัง (Invertebrate), พืช (Plant), ร่องรอยซากดึกดำบรรพ์ (Trace) และอื่นๆ (Other)

3.2.1.3 การนำเข้าข้อมูล

ข้อมูลถูกนำเข้าโดยใช้ API ของ ArcGIS Rest Service ผ่านการใช้ไลบรารี requests ในภาษา Python เพื่อดึงข้อมูลในรูปแบบ JSON และแปลงเป็น DataFrame โดยใช้ไลบรารี pandas

```
import requests

url = "https://gisportal.dmr.go.th/arcgis/rest/services/FOSSIL/T_FOSSIL_SITE/MapServer/0/query"
params = {
    "where": "1=1", # ดึงข้อมูลทั้งหมด
    "outFields": "*", # เอาทุกคอลัมน์
    "f": "json" # ให้ข้อมูลออกมาเป็น JSON
}

response = requests.get(url, params=params)
data = response.json()
```

```
1 import pandas as pd
2 df = pd.json_normalize(data["features"], sep="_")
3 df.head(5)
```

attributes_SITE_ID	attributes_SITE_CODE	attributes_NAME_TH	attributes_
121	0191024	โทรโลไบต์เกาะยาว	
122	0191025	ปะการังเกาะตะงา	

3.2.1.4 การวิเคราะห์ข้อมูลเชิงสำรวจ (Exploratory Data Analysis)

หลังจากนำเข้าข้อมูลแล้ว ได้ทำการวิเคราะห์ข้อมูลเชิงสำรวจเพื่อทำความเข้าใจลักษณะ และ คุณภาพของข้อมูล ดังนี้

1. ตรวจสอบประเภทข้อมูลในแต่ละคอลัมน์: พบว่าข้อมูลส่วนใหญ่อยู่ในรูปแบบ Object (ข้อความ) มีเพียงบางคอลัมน์ที่เป็นตัวเลข เช่น UTM_E, UTM_N, DIS_YEAR

```
1 df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 520 entries, 0 to 519
Data columns (total 56 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   attributes_SITE_ID                       520 non-null    int64
1   attributes_SITE_CODE                     518 non-null    object
2   attributes_NAME_TH                       519 non-null    object
3   attributes_NAME_ENG                      518 non-null    object
4   attributes_IMPORTA_TH                    518 non-null    object
5   attributes_IMPORTA_EN                    518 non-null    object
6   attributes_LOCALITY                      518 non-null    object
7   attributes_TAMBON                        520 non-null    object
8   attributes_DISTRICT                      520 non-null    object
9   attributes_PROVINCE                      520 non-null    object
10  attributes_UTM_E                          519 non-null    float64
11  attributes_UTM_N                          519 non-null    float64
12  attributes_ZONE                           520 non-null    object
13  attributes_MAPSHEET                      520 non-null    object
14  attributes_SHEET_NAME                    520 non-null    object
15  attributes_FEATURE_TH                     518 non-null    object
16  attributes_FEATURE_EN                     518 non-null    object
17  attributes_GEO_DES_TH                     509 non-null    object
18  attributes_GEO_DES_EN                     518 non-null    object
19  attributes_GEO_GROUP                     519 non-null    object
20  attributes_FORMATION                     519 non-null    object
21  attributes_PERIODFROM                     519 non-null    object
22  attributes_PERIODTO                       519 non-null    object
23  attributes_FOS_DES_TH                     512 non-null    object
24  attributes_FOS_GR_1                       519 non-null    object
25  attributes_FOS_GR_2                       519 non-null    object
26  attributes_FOS_GR_3                       519 non-null    object
27  attributes_FOS_GR_4                       519 non-null    object
28  attributes_FOS_GR_5                       519 non-null    object
29  attributes_DIS_BY_ENG                     518 non-null    object
30  attributes_DIS_BY_TH                     518 non-null    object
31  attributes_DIS_DAY                       518 non-null    object
32  attributes_DIS_MONTH                     0 non-null     object
33  attributes_DIS_YEAR                       0 non-null     object
34  attributes_OWNER_TH                      518 non-null    object
35  attributes_OWNER_ENG                     518 non-null    object
36  attributes_POTENTIAL1                    519 non-null    object
37  attributes_POTENTIAL2                    519 non-null    object
38  attributes_POTENTIAL3                    519 non-null    object
39  attributes_POTENTIAL4                    519 non-null    object
40  attributes_POTENTIAL5                    519 non-null    object
41  attributes_POTENTIAL6                    519 non-null    object
42  attributes_POTENTIAL7                    519 non-null    object
43  attributes_STATUS                        519 non-null    object
44  attributes_CREATEBY                      519 non-null    object
45  attributes_CREATEDATE                     519 non-null    float64
46  attributes_UPDATEBY                      518 non-null    object
47  attributes_UPDATEDATE                     1 non-null     float64
48  attributes_GlobalID                      520 non-null    object
49  attributes_OBJECTID                      520 non-null    int64
50  attributes_created_user                   520 non-null    object
51  attributes_created_date                   520 non-null    int64
52  attributes_last_edited_user              519 non-null    object
53  attributes_last_edited_date              519 non-null    float64
54  geometry_x                               519 non-null    float64
55  geometry_y                               519 non-null    float64
dtypes: float64(7), int64(3), object(46)
memory usage: 227.6+ KB

```

2. ตรวจสอบค่าว่างของข้อมูล: พบว่าหลายคอลัมน์มีค่าว่างเป็นจำนวนมาก ซึ่งต้องมีการจัดการในขั้นตอนการเตรียมข้อมูล

```
1 print(df.isnull().sum())
```

```

attributes_SITE_ID          0
attributes_SITE_CODE        2
attributes_NAME_TH          1
attributes_NAME_ENG         2
attributes_IMPORTA_TH       2
attributes_IMPORTA_EN       2
attributes_LOCALITY         2
attributes_TAMBON           0
attributes_DISTRICT         0
attributes_PROVINCE         0
attributes_UTM_E            1
attributes_UTM_N            1
attributes_ZONE             0
attributes_MAPSHEET         0
attributes_SHEET_NAME       0
attributes_FEATURE_TH       2
attributes_FEATURE_EN       2
attributes_GEO_DES_TH      11
attributes_GEO_DES_EN       2
attributes_GEO_GROUP        1
attributes_FORMATION        1
attributes_PERIODFROM       1
attributes_PERIODTO         1
attributes_FOS_DES_TH       8
attributes_FOS_GR_1         1
attributes_FOS_GR_2         1
attributes_FOS_GR_3         1
attributes_FOS_GR_4         1
attributes_FOS_GR_5         1
attributes_DIS_BY_ENG       2
attributes_DIS_BY_TH        2
attributes_DIS_DAY          2
attributes_DIS_MONTH        520
attributes_DIS_YEAR         520
attributes_OWNER_TH         2
attributes_OWNER_ENG        2
attributes_POTENTIAL1       1
attributes_POTENTIAL2       1
attributes_POTENTIAL3       1
attributes_POTENTIAL4       1
attributes_POTENTIAL5       1
attributes_POTENTIAL6       1
attributes_POTENTIAL7       1
attributes_STATUS           1
attributes_CREATEBY         1
attributes_CREATEDATE        1
attributes_UPDATEBY         2
attributes_UPDATEDATE       519
attributes_GlobalID         0
attributes_OBJECTID         0
attributes_created_user      0
attributes_created_date      0
attributes_last_edited_user  1
attributes_last_edited_date  1
geometry_x                  1
geometry_y                  1
dtype: int64

```

3. วิเคราะห์ค่าทางสถิติสำหรับคอลัมน์ที่เป็นตัวเลข ได้ทำการวิเคราะห์ค่าทางสถิติพื้นฐาน เช่น ค่าเฉลี่ย ค่ามัธยฐาน ค่าต่ำสุด ค่าสูงสุด และค่าเบี่ยงเบนมาตรฐาน เพื่อเข้าใจการกระจายตัวของข้อมูล

1 df.describe()					
	attributes_SITE_ID	attributes_UTM_E	attributes_UTM_N	attributes_CREATEDATE	attributes_UPDATEDATE
count	520.000000	5.190000e+02	5.190000e+02	5.190000e+02	520.000000
mean	2159.798077	6.767794e+05	1.520757e+06	1.696320e+12	1.707747e+12
std	43301.011524	1.665045e+05	4.414099e+05	1.353046e+09	1.707747e+12
min	1.000000	4.355530e+05	7.108230e+05	1.696261e+12	1.707747e+12
25%	131.750000	5.688600e+05	1.076125e+06	1.696261e+12	1.707747e+12
50%	261.500000	6.176370e+05	1.694393e+06	1.696261e+12	1.707747e+12
75%	391.250000	8.066300e+05	1.861056e+06	1.696261e+12	1.707747e+12
max	987670.000000	1.176050e+06	2.128105e+06	1.727085e+12	1.707747e+12

attributes_UPDATEDATE	attributes_OBJECTID	attributes_created_date	attributes_last_edited_date	geometry_x	geometry_y
1.000000e+00	520.000000	5.200000e+02	5.190000e+02	5.190000e+02	5.190000e+02
1.707747e+12	11452.065385	1.696340e+12	1.696311e+12	6.767794e+05	1.520757e+06
NaN	323.481601	1.694401e+09	1.023981e+09	1.665045e+05	4.414099e+05
1.707747e+12	11175.000000	1.696235e+12	1.696266e+12	4.355530e+05	7.108230e+05
1.707747e+12	11305.750000	1.696235e+12	1.696266e+12	5.688600e+05	1.076125e+06
1.707747e+12	11435.500000	1.696235e+12	1.696266e+12	6.176370e+05	1.694393e+06
1.707747e+12	11565.250000	1.696235e+12	1.696266e+12	8.066300e+05	1.861056e+06
1.707747e+12	16243.000000	1.727085e+12	1.719594e+12	1.176050e+06	2.128105e+06

4. วิเคราะห์ความถี่และความหลากหลาย: สำหรับคอลัมน์ที่เป็นข้อความ ได้ทำการวิเคราะห์ความถี่และความหลากหลายของค่าที่ปรากฏในแต่ละคอลัมน์

```

1 pd.set_option('display.max_columns', None)
2
3
4 pd.set_option('display.width', None)
5 df.describe(include="0")

```

	attributes_SITE_CODE	attributes_NAME_TH	attributes_NAME_ENG	attr
count	518	519	518	
unique	510	518	1	
top	0139013	หอยกาบคู่บ้านไทรย้อย		
freq	2	2	518	
attributes_IMPORTA_TH	attributes_IMPORTA_EN	attributes_LOCALITY	attributes_TAMBON	attributes_DISTRICT
518	518	518	520	520
5	5	227	272	153
แหล่งอ้างอิงทางวิชาการ	Reference site		สุรนารี	แม่สอด
334	334	254	11	26
attributes_PROVINCE	attributes_ZONE	attributes_MAPSHEET	attributes_SHEET_NAME	attributes_FEATURE_TH
520	520	520	520	518
42	2	163	163	18
สตูล	47N	4936 III	อำเภอจอมบึง	
56	518	17	17	386
attributes_FEATURE_EN	attributes_GEO_DES_TH	attributes_GEO_DES_EN	attributes_GEO_GROUP	attributes_FORMATION
518	509	518	519	519
1	280	1	15	52
518	178	518	342	347
attributes_PERIODFROM	attributes_PERIODTO	attributes_FOS_DES_TH	attributes_FOS_GR_1	attributes_FOS_GR_2
519	519	512	519	519
28	1	401	5	5
กระดูกไดโนเสาร์			N	Invertebrate
353	519	15	407	237

attributes_FOS_GR_3	attributes_FOS_GR_4	attributes_FOS_GR_5	attributes_DIS_BY_ENG	attributes_DIS_BY_TH	at
519	519	519	518	518	
3	3	3	1	1	
N	N	N			
496	504	482	518	518	
attributes_DIS_DAY	attributes_DIS_MONTH	attributes_DIS_YEAR	attributes_OWNER_TH	attributes_OWNER_ENG	
518	0	0	518	518	
1	0	0	104	1	
	NaN	NaN	รอการตรวจสอบ		
518	NaN	NaN	148	518	
attributes_POTENTIAL1	attributes_POTENTIAL2	attributes_POTENTIAL3	attributes_POTENTIAL4	attributes_POTENTIAL5	
519	519	519	519	519	
1	1	1	1	1	
519	519	519	519	519	
attributes_POTENTIAL6	attributes_POTENTIAL7	attributes_STATUS	attributes_CREATEBY	a	
519	519	519	519	519	
1	1	1	1	2	
				1751	
519	519	519	518		
attributes_UPDATEBY	attributes_GlobalID	attributes_created_user	attributes_last_edited_user		
518	520	520	519		
2	520	2	1		
{65FF52B9-A0F2-4FEB-9B46-0B38E5DC2920}		portaladmin	DMRAPP		
517	1	519	519		

3.2.2 การเตรียมข้อมูล (Data Preparation)

3.2.2.1 การทำความสะอาดข้อมูล (Data Cleaning)

1. ลบคอลัมน์ที่มีค่า unique เพียงค่าเดียว: คอลัมน์ที่มีค่าเพียงค่าเดียวไม่มีประโยชน์
ในการจำแนกประเภท จึงถูกลบออก

```

1
2
3 import pandas as pd
4
5
6 cols_to_drop = [col for col in df.columns if df[col].nunique() == 1]
7
8 # Store the number of columns before dropping
9 initial_column_count = len(df.columns)
10
11 # Drop the columns
12 df.drop(columns=cols_to_drop, inplace=True)
13
14 # Calculate and display the results
15 remaining_columns = len(df.columns)
16 dropped_columns = initial_column_count - remaining_columns
17
18 print(f"Number of remaining columns: {remaining_columns}")
19 print(f"Number of dropped columns: {dropped_columns}")
20
21 # Display the names of dropped columns (optional)
22 if dropped_columns > 0:
23     print("Dropped Columns:")
24 cols_to_drop
25

```

Number of remaining columns: 38

Number of dropped columns: 18

Dropped Columns:

```

['attributes_NAME_ENG',
 'attributes_FEATURE_EN',
 'attributes_GEO_DES_EN',
 'attributes_PERIODT0',
 'attributes_DIS_BY_ENG',
 'attributes_DIS_BY_TH',
 'attributes_DIS_DAY',
 'attributes_OWNER_ENG',
 'attributes_POTENTIAL1',
 'attributes_POTENTIAL2',
 'attributes_POTENTIAL3',
 'attributes_POTENTIAL4',
 'attributes_POTENTIAL5',
 'attributes_POTENTIAL6',
 'attributes_POTENTIAL7',
 'attributes_STATUS',
 'attributes_UPDATEDATE',
 'attributes_last_edited_user']

```

2. ลบคอลัมน์ที่มีค่า unique มากเกินไป: คอลัมน์ที่มีค่า unique มากกว่า 5% ของจำนวนแถวทั้งหมด (มากกว่า 26 ค่าจากข้อมูล 520 แถว) ถูกพิจารณาขอยกเว้นคอลัมน์ที่มีความสำคัญเฉพาะ

```

1
2
3 # Columns to exclude from dropping based on unique value counts
4 excluded_columns = [
5     'attributes_FOS_GR_1', 'attributes_FOS_GR_2', 'attributes_FOS_GR_3',
6     'attributes_FOS_GR_4', 'attributes_FOS_GR_5', 'attributes_NAME_TH',
7     'attributes_GEO_DES_TH', 'attributes_FOS_DES_TH', 'attributes_PROVINCE',
8     'attributes_FEATURE_TH', 'attributes_GEO_GROUP',
9     'attributes_FORMATION', 'attributes_PERIODFROM', 'attributes_IMPORTA_EN',
10    'attributes_OWNER_TH'
11 ]
12
13 # Calculate the number of columns before dropping
14 initial_column_count = len(df.columns)
15
16 # Identify columns to drop (only for object type columns)
17 cols_to_drop = [
18     col for col in df.select_dtypes(include=['object']).columns
19     if df[col].nunique() / len(df) > 0.05 and col not in excluded_columns
20 ]
21
22 # Drop the columns
23 df.drop(columns=cols_to_drop, inplace=True, errors='ignore')
24
25 # Calculate and display the results
26 remaining_columns = len(df.columns)
27 dropped_columns = initial_column_count - remaining_columns
28 print(f"Number of remaining columns: {remaining_columns}")
29 print(f"Number of dropped columns: {dropped_columns}")
30
31 # Display the names of dropped columns (optional)
32 if dropped_columns > 0:
33     print("Dropped Columns:")
34     cols_to_drop
35

```

Number of remaining columns: 31

Number of dropped columns: 7

Dropped Columns:

```

['attributes_SITE_CODE',
 'attributes_LOCALITY',
 'attributes_TAMBON',
 'attributes_DISTRICT',
 'attributes_MAPSHEET',
 'attributes_SHEET_NAME',
 'attributes_GlobalID']

```

3.2.2.2 การสร้างคุณลักษณะใหม่ (Feature Engineering)

1. สร้างคุณลักษณะ Region: จัดกลุ่มจังหวัดต่างๆ ให้เป็นภูมิภาค เพื่อลดความซับซ้อน

ข อ ง ข ๋ อ มู ล

```
1 # prompt: map
2 region_mapping = {
3     # ภาคเหนือ
4     'เหนือ': ['ตาก', 'แพร่', 'พิษณุโลก', 'ลำปาง', 'อุตรดิตถ์', 'พะเยา', 'น่าน'],
5     # ภาคอีสาน
6     'อีสาน': ['เลย', 'นครราชสีมา', 'กาฬสินธุ์', 'ชัยภูมิ', 'ขอนแก่น', 'หนองบัวลำภู',
7               'มุกดาหาร', 'อุดรธานี', 'สกลนคร', 'นครพนม', 'ร้อยเอ็ด', 'อุบลราชธานี'],
8     # ภาคกลาง/ตะวันออก
9     'กลาง': ['ราชบุรี', 'เพชรบูรณ์', 'อุทัยธานี', 'เพชรบุรี', 'กาญจนบุรี', 'ลพบุรี',
10             'นครสวรรค์', 'สระบุรี', 'ปทุมธานี', 'ปราชินบุรี', 'สระแก้ว'],
11     # ภาคใต้
12     'ใต้': ['สตูล', 'กระบี่', 'พังงา', 'ชุมพร', 'สงขลา', 'นครศรีธรรมราช',
13            'พัทลุง', 'ตรัง', 'สุราษฎร์ธานี']
14 }
15 # with attributes_PROVINCE
16
17 def map_region(province):
18     for region, provinces in region_mapping.items():
19         if province in provinces:
20             return region
21     return "ไม่ทราบ" # Return "Unknown" if province not found in the mapping
22
23 df['attributes_REGION'] = df['attributes_PROVINCE'].apply(lambda x: map_region(x.strip()))
24 df['attributes_REGION'].value_counts()
```

count	
attributes_REGION	
อีสาน	166
ใต้	146
กลาง	127
เหนือ	78

2. การจัดกลุ่มลักษณะภูมิประเทศ ยุคทางธรณีวิทยา และผู้ครอบครอง: ใช้ Regular Expression (RegEx) และ Rule Base Scripts ในการจับคู่รูปแบบคำและจัดกลุ่ม
- ข ๋ อ มู ล ท ี่ มี ล ักษณะ ใ ก ล ่ เ ค ี่ ย ิง กั น

```

1 feature_mapping = {
2     # ภาคพื้นทวีป (Continental features)
3     'continental': [
4         'โพลาร์', # Continental shelf
5         'ลาตทิว', # Continental slope
6         'ภาคพื้นทวีป น้ำจืด'
7     ],
8
9     # สภาพแวดล้อมทางทะเล
10    'marine': [
11        'ชายฝั่งทะเล',
12        'ทะเลลึก?'
13    ],
14
15    # แหล่งน้ำจืด/ระบบแม่น้ำ
16    'freshwater': [
17        'ตะกอนธารน้ำ',
18        'ร่องน้ำ',
19        'ระบบทางน้ำ',
20        'ตะกอนทางน้ำ',
21        'พายุน้ำใต้ดิน',
22        'ริมฝั่งแม่น้ำ',
23        'ธารน้ำพา',
24        'ทะเลสาบ น้ำจืด',
25        'ตะกอนตะกอนน้ำจืด'
26    ],
27
28    # ที่ราบน้ำท่วมถึง/ชายฝั่ง
29    'floodplain_coastal': [
30        'ทางน้ำ-โคจลิต ที่ราบน้ำท่วมถึง',
31        'บริเวณน้ำท่วมถึง',
32        'พายุทอร์นาโด'
33    ],
34
35    # ไม่ระบุ
36    'unknown': [
37        'null',
38        None
39    ]
40 }
41
42 def map_feature(feature):
43     for category, features in feature_mapping.items():
44         if feature in features:
45             return category
46     return "unknown"
47
48 df['attributes_FEATURE'] = df['attributes_FEATURE_TH'].apply(lambda x: map_feature(x.strip()))
49 df['attributes_FEATURE'].value_counts()

```

count	
attributes_FEATURE	
unknown	385
continental	103
freshwater	24
floodplain_coastal	3
marine	2

```

1 import re
2 owner_patterns = {
3     # กรมป่าไม้และพื้นที่ป่า
4     'forest_department': r'^กรมป่าไม้.*$',
5
6     # อุทยานแห่งชาติ
7     'national_park': r'^อุทยานแห่งชาติ.*$',
8
9     # เขตรักษาพันธุ์สัตว์ป่าและเขตห้ามล่า
10    'wildlife_sanctuary': r'^เขตรักษาพันธุ์สัตว์ป่า.*|^เขตห้ามล่าสัตว์ป่า.*$',
11
12    # สวนอุทยาน
13    'forest_park': r'^วนอุทยาน.*$',
14
15    # หน่วยงานราชการอื่นๆ
16    'government': r'^((กรม|สำนัก|สำนักงาน|การรถไฟ|ชลประทาน|สวนพฤกษศาสตร์|ที่ดินสาธารณะ).*)$',
17
18    # ศาสนสถานและสถานการศึกษา
19    'religious_educational': r'^((สำนักงานพระพุทธศาสนา|สำนักสงฆ์|มหาวิทยาลัย).*)$',
20
21    # เอกชน
22    'private': r'^เอกชน.*$',
23
24    # รอการตรวจสอบ
25    'pending': r'^รอการตรวจสอบ.*$'
26 }
27
28 def classify_owner(owner_text):
29     if pd.isna(owner_text):
30         return 'unknown'
31
32     for category, pattern in owner_patterns.items():
33         if re.match(pattern, owner_text.strip()):
34             return category
35
36     return 'other'
37 df['owner'] = df['attributes_OWNER_TH'].apply(lambda x: classify_owner(x.strip()))
38 df['owner'].value_counts()

```

count	
owner	
forest_department	219
pending	151
national_park	62
government	37
wildlife_sanctuary	19
private	16
religious_educational	5
forest_park	4
other	4

```

1 period_mapping = {
2     # ยุคพาลีโอโซอิกตอนต้น (Early Paleozoic) ~541-419 ล้านปี
3     'early_paleozoic': [
4         'แคมเบรียนตอนปลาย',
5         'ออร์โดวิเชียน',
6         'ซิลูเรีย',
7         'เดวอนิอัน',
8         'คาร์บอนิเฟอัส-เพอร์เมียน',
9         'ออร์โดวิเชียน-ไซลูเรีย'
10    ],
11
12    # ยุคพาลีโอโซอิกตอนปลาย (Late Paleozoic) ~419-252 ล้านปี
13    'late_paleozoic': [
14        'คาร์บอนิเฟอัส',
15        'เพอร์เมียน',
16        'เพอร์เมียนตอนต้น',
17        'เพอร์เมียนตอนปลาย'
18    ],
19
20    # ยุคเมโซโซอิก (Mesozoic) ~252-66 ล้านปี
21    'mesozoic': [
22        'ไทรแอสสิก',
23        'จูแรสสิก',
24        'ครีเทเชียส',
25        'จูแรสสิกตอนต้น',
26        'ไทรแอสสิกตอนปลาย',
27        'จูแรสสิกตอนปลาย',
28        'จูแรสสิก',
29        'จูแรสสิก',
30        'จูแรสสิกตอนกลาง-ตอนปลาย',
31        'จูแรสสิกตอนปลาย',
32        'ครีเทเชียส',
33        'ครีเทเชียส',
34        'ครีเทเชียสตอนต้น',
35        'ครีเทเชียสตอนต้น',
36        'จูแรสสิกตอนปลายถึงครีเทเชียสตอนต้น'
37    ],
38
39    # ยุคซีโนโซอิก (Cenozoic) ~66 ล้านปี-ปัจจุบัน
40    'cenozoic': [
41        'คาอโนเจน'
42    ],
43
44    # ไม่ระบุ/ไม่แน่ชัด
45    'unknown': [
46        'None',
47        'ไทรแอสสิก/คาร์บอนิเฟอัส' # ระบุตามชื่อหน่วยยุค
48    ]
49 }
50
51
52
53
54 def map_period(period_str):
55     for period, keywords in period_mapping.items():
56         if period_str in keywords:
57             return period
58     return "unknown"
59
60 df["Group_Period"] = df["attributes_PERIODFROM"].apply(lambda x: map_period(x.strip()) if isinstance(x, str) else "unknown")
61 df["Group_Period"].value_counts()
62

```

```

count
Group_Period
unknown      353
late_paleozoic  106
mesozoic      47
early_paleozoic  8
cenozoic       3

```

3. ทำความสะอาดข้อมูลที่เป็นประเภท text ที่เป็นลักษณะบรรยายของสิ่งต่างๆ แทน
- ค ' า ว ' า ง ด " ว ย “ ไ ม ' ร ะ บู ”

```

1 def clean_text(text):
2     # ถ้าเป็น NaN, None หรือช่องว่าง
3     if pd.isna(text) or text is None or text.strip() == '':
4         return 'ไม่ระบุ'
5
6     # ลบช่องว่างหน้า-หลัง
7     text = text.strip()
8
9     # ลบ .0 ที่อาจมาจาก float conversion
10    if text.endswith('.0'):
11        text = text[:-2]
12
13    # แปลง None เป็นภาษาไทย
14    if text.lower() in ['none', None, "nan"]:
15        return 'ไม่ระบุ'
16
17    return text
18
19 # ใช้กับ DataFrame
20 df['attributes_GEO_DES_TH'] = df['attributes_GEO_DES_TH'].apply(clean_geo_des)
21 df['attributes_NAME_TH'] = df['attributes_NAME_TH'].apply(clean_geo_des)
22 df['attributes_FOS_DES_TH'] = df['attributes_FOS_DES_TH'].apply(clean_geo_des)
23 df

```

4. ล บ ค อ ล ~ ม น ' ท ~ ' ถู ก แ ป ล ง แ ล้ ว

```

1 df = df.drop(columns=[ "attributes_PROVINCE", "attributes_FEATURE_TH", "attributes_PERIODFROM", "attributes_OWNER_TH" ], errors='ignore')
2 df.shape
(517, 12)

```

5. สร้างคุณลักษณะภาพถ่าย: ใช้ไลบรารี folium ในการสร้างภาพแผนที่จากพิกัด geometry_x และ geometry_y แล้วบันทึกเป็นไฟล์ภาพ เพื่อใช้เป็นข้อมูลนำเข้า
- ใ ห้ ้ ก ้ บ โ ม เ ต ล Computer Vision

```

1 import folium
2 import os
3 from selenium.webdriver import webdriver
4 from selenium.webdriver.chrome.options import Options
5 import time
6 from pyproj import Transformer
7 from tqdm import tqdm
8
9 def utm_to_latlon(easting, northing, zone=47):
10     transformer = Transformer.from_crs(
11         f'EPSG:326{zone}', # UTM zone 47N
12         'EPSG:4326', # WGS84
13         always_xy=True
14     )
15     lon, lat = transformer.transform(easting, northing)
16     return lat, lon
17
18 def save_location_images(df):
19     if not os.path.exists('images'):
20         os.makedirs('images')
21
22     chrome_options = Options()
23     chrome_options.add_argument('--headless')
24     chrome_options.add_argument('--no-sandbox')
25     chrome_options.add_argument('--window-size=1000,1000')
26
27     ZOOM_LEVEL = 19
28
29     for idx in tqdm(df.index):
30         try:
31             lat, lon = utm_to_latlon(df.loc[idx, 'geometry_x'], df.loc[idx, 'geometry_y'])
32
33             print(f"Index {idx}:")
34             print(f"UTM: {df.loc[idx, 'geometry_x']}, {df.loc[idx, 'geometry_y']}")
35             print(f"Lat/Lon: {lat}, {lon}")
36
37             m = folium.Map(
38                 location=[lat, lon],
39                 zoom_start=ZOOM_LEVEL,
40                 tiles='https://mtl.google.com/vt/lyrs=s&x={x}&y={y}&z={z}',
41                 attr='Google Satellite',
42                 max_zoom=20
43             )
44
45             temp_html = f'temp_{idx}.html'
46             m.save(temp_html)
47
48             driver = webdriver.Chrome(options=chrome_options)
49             driver.get(f'file://{os.path.abspath(temp_html)}')
50
51             time.sleep(5)
52
53             driver.save_screenshot(f'images/{idx}.png')
54
55             driver.quit()
56             os.remove(temp_html)
57
58             print(f"Successfully saved image for index {idx}")
59
60         except Exception as e:
61             print(f"Error processing index {idx}: {str(e)}")
62             continue
63
64 # ใช้งาน
65 save_location_images(df)

```

6. สร้างคอลัมน์ index เพื่ออ้างอิงรูปภาพ: เพิ่มคอลัมน์ 'image_path' เพื่อใช้อ้างอิง
- ถ ึ่ง ร ู ป ภ า พ ที่ ส ร ้าง ข ึ้น

```
1 df['image_path'] = df.index.map(lambda x: f'images/{x}.png')
```

7. สร้างคอลัมน์เป้าหมายแบบ single label จากคอลัมน์ FOS_GR_1 ถึง FOS_GR_5

```

1 import numpy as np
2
3 def select_unique_fossil(row):
4     values = row[row != "N"].dropna().unique() # ค่าที่ไม่ใช่ "N"
5     return values[0] if len(values) == 1 else np.nan # ถ้ามีค่าเดียวใช้ค่านั้น ถ้ามีมากกว่าหนึ่งให้เป็น NaN
6
7
8
9
10 # สร้างคอลัมน์ใหม่ที่มีค่าหมวดหมู่เดียว
11 df["Fossil_Type"] = df[['attributes_FOS_GR_1', 'attributes_FOS_GR_2', 'attributes_FOS_GR_3',
12                        'attributes_FOS_GR_4', 'attributes_FOS_GR_5']].apply(select_unique_fossil, axis=1)
13
14
15
16 # แสดงจำนวนคอลัมน์ที่เพิ่มและลดลง
17 initial_column_count = len(df.columns) - 1 # ลบคอลัมน์ที่เพิ่งเพิ่ม
18 final_column_count = len(df.columns)
19 added_columns = final_column_count - initial_column_count
20
21 print(f"Added Columns: {added_columns}")
22 dropped_columns = 0 # คำนวณเริ่มต้นเป็น 0
23
24 # ลบ column ที่ไม่จำเป็น
25 columns_to_remove = ['attributes_FOS_GR_1', 'attributes_FOS_GR_2', 'attributes_FOS_GR_3',
26                      'attributes_FOS_GR_4', 'attributes_FOS_GR_5']
27
28 initial_column_count_before_remove = len(df.columns)
29 df = df.drop(columns=columns_to_remove, errors='ignore')
30 dropped_columns = initial_column_count_before_remove - len(df.columns)
31 print(f"Removed Columns: {dropped_columns}")
32
33 print("Columns removed:", columns_to_remove)
34 print("New Columns added: Fossil_Type")
35
Added Columns: 1
Removed Columns: 5
Columns removed: ['attributes_FOS_GR_1', 'attributes_FOS_GR_2', 'attributes_FOS_GR_3', 'attributes_FOS_GR_4', 'attributes_FOS_GR_5']
New Columns added: Fossil_Type

```

3.2.2.3 การเตรียมคุณลักษณะขั้นสูง (Advanced Feature Preparation)

1. การสร้าง Word Embedding จากข้อความภาษาไทย: ใช้ WangchanBERTa ในการแปลงข้อความภาษาไทยเป็นเวกเตอร์ โดยสร้าง Custom Transformer ที่สามารถทำงานร่วมกับ scikit-learn pipeline

```

[
1 #NOTE
2 from transformers import AutoTokenizer, AutoModel
3 import torch
4 from sklearn.base import BaseEstimator, TransformerMixin
5
6 class ThaiWordEmbeddingTransformer(BaseEstimator, TransformerMixin):
7     def __init__(self, model_name="AI4Thai/wangchanberta-base-att-spm-uncased"):
8         self.model_name = model_name
9         self.tokenizer = AutoTokenizer.from_pretrained(model_name)
10        self.model = AutoModel.from_pretrained(model_name)
11
12    def fit(self, X, y=None):
13        return self
14
15    def transform(self, X):
16        # แปลงข้อความเป็น embeddings ขนาด 768 มิติ
17        embeddings = []
18        for text in X:
19            if pd.isna(text):
20                embeddings.append(np.zeros(768))
21                continue
22
23            inputs = self.tokenizer(text, return_tensors="pt", truncation=True, padding=True, max_length=512)
24            outputs = self.model(**inputs)
25            embedding = outputs.last_hidden_state.mean(dim=1).detach().numpy()
26            embeddings.append(embedding[0])
27
28        return np.array(embeddings)

```

2. การสกัดคุณลักษณะจากรูปภาพ: ใช้ ResNet50 เพื่อสกัดคุณลักษณะจากรูปภาพแผนที่ที่
- ส ร ู ง ี น

```

1 from torchvision import models, transforms
2 from PIL import Image
3
4 def extract_image_features(image_paths):
5     # โหลดโมเดล ResNet50 ที่ผ่านการเทรนบน ImageNet
6     model = models.resnet50(pretrained=True)
7     # ตัดส่วน fully connected layer ออก เพื่อให้ได้ feature vector
8     model = torch.nn.Sequential(*list(model.children())[:-1])
9     model.eval()
10
11     # กำหนดการแปลงภาพ
12     transform = transforms.Compose([
13         transforms.Resize(256),
14         transforms.CenterCrop(224),
15         transforms.ToTensor(),
16         transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
17     ])
18
19     features = []
20     for path in image_paths:
21         try:
22             img = Image.open(path).convert('RGB')
23             img_t = transform(img).unsqueeze(0)
24             with torch.no_grad():
25                 feature = model(img_t)
26             features.append(feature.flatten().numpy())
27         except Exception as e:
28             # กรณีมีปัญหาในการโหลดภาพ ใช้เวกเตอร์ศูนย์แทน
29             features.append(np.zeros(2048))
30
31     return np.array(features)

```

3.2.3 การสร้างแบบจำลอง (Modeling)

- 3.2.3.1 การสร้างคลาสสำหรับติดตามความสำคัญของคุณลักษณะ ในการวิเคราะห์ว่าคุณลักษณะใดมีผลต่อการทำนายมากที่สุด ได้สร้างคลาส FeatureImportanceTracker เพื่อติดตามและคำนวณความสำคัญของคุณลักษณะตามคอลัมน์

```

1 class FeatureImportanceTracker:
2     def __init__(self):
3         self.column_indices = {}
4         self.feature_names = []
5
6     def add_features(self, name, start_idx, end_idx):
7         self.column_indices[name] = (start_idx, end_idx)
8
9     def get_feature_names(self):
10        return list(self.column_indices.keys())
11
12    def aggregate_importance(self, importances):
13        result = {}
14        for name, (start, end) in self.column_indices.items():
15            result[name] = np.sum(importances[start:end])
16        return result

```

3.2.3.1 การสร้างคลาสสำหรับ Pipeline การจำแนกซากดึกดำบรรพ์ สร้างคลาส FossilClassificationPipeline ที่รวมขั้นตอนการเตรียมข้อมูล การแปลงคุณลักษณะ และการสร้างแบบจำลองไว้ด้วยกัน

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import LabelEncoder

class FossilClassificationPipeline:
    def __init__(self, use_image_features=True):
        self.use_image_features = use_image_features

        # กำหนดคุณลักษณะหลักที่จะใช้
        self.text_columns = ['attributes_FOS_DES_TH', 'attributes_FEATURE_TH', 'attributes_GEO_DES_TH']
        self.categorical_columns = ['region', 'period', 'attributes_OWNER_TH']
        self.numeric_columns = ['geometry_x', 'geometry_y']

        # สร้าง FeatureImportanceTracker
        self.feature_tracker = FeatureImportanceTracker()

        # สร้าง transformers สำหรับแต่ละประเภทคุณลักษณะ
        self.text_transformer = ThaiWordEmbeddingTransformer()
        self.categorical_transformer = OneHotEncoder(handle_unknown='ignore')
        self.numeric_transformer = StandardScaler()

        # สร้าง ColumnTransformer
        transformers = []
        current_idx = 0

        # เพิ่ม image features ถ้าต้องการใช้
        if use_image_features:
            transformers.append(('images', 'passthrough', ['image_path']))
            end_idx = current_idx + 1
            self.feature_tracker.add_features('image_features', current_idx, end_idx)
            current_idx = end_idx

        # เพิ่ม text features
        for col in self.text_columns:
            transformers.append(('text_{col}', self.text_transformer, [col]))
            end_idx = current_idx + 768 # WangchanBERTa สร้าง embedding ขนาด 768
            self.feature_tracker.add_features(col, current_idx, end_idx)
            current_idx = end_idx

        # เพิ่ม categorical features
        transformers.append(('cats', self.categorical_transformer, self.categorical_columns))
        # จำนวน feature จาก OneHotEncoder จะขึ้นอยู่กับจำนวน unique ในแต่ละคอลัมน์
        # ต้องเก็บ indices ไว้หลังจาก fit

        # เพิ่ม numeric features
        transformers.append(('nums', self.numeric_transformer, self.numeric_columns))
        end_idx = current_idx + len(self.numeric_columns)
        self.feature_tracker.add_features('numeric_features', current_idx, end_idx)
        current_idx = end_idx

        self.preprocessor = ColumnTransformer(transformers)

        # สร้างโมเดล Random Forest
        self.classifier = RandomForestClassifier(n_estimators=100, max_depth=15, random_state=42)

        # สร้าง Label Encoder สำหรับแปลงค่าเป้าหมาย
        self.label_encoder = LabelEncoder()

    def fit(self, X, y):
        # แปลงคุณลักษณะภาพถ้าใช้
        if self.use_image_features:
            X = X.copy()
            X['image_features'] = extract_image_features(X['image_path'])

        # แปลงค่าเป้าหมาย
        y_encoded = self.label_encoder.fit_transform(y)

        # สร้าง pipeline และ fit
        self.pipeline = Pipeline([
            ('preprocessor', self.preprocessor),
            ('classifier', self.classifier)
        ])

        self.pipeline.fit(X, y_encoded)

        # บันทึกข้อมูล feature indices จาก categorical transformer
        cat_idx = 0
        for cat_feature, cat_values in zip(
            self.categorical_columns,
            self.pipeline.named_steps['preprocessor'].transformers_[
                self.pipeline.named_steps['preprocessor'].
                _get_transformer_index('cats')[1].categories_
            ]:
            # หา index เริ่มต้นของ categorical features ในแบบจำลอง
            if cat_idx == 0:
                start_idx = sum([self.feature_tracker.column_indices[name][1] -
                                self.feature_tracker.column_indices[name][0]
                                for name in self.feature_tracker.get_feature_names()])
            else:
                start_idx += len(prev_values)

            # เพิ่ม feature ใน tracker
            end_idx = start_idx + len(cat_values)
            self.feature_tracker.add_features(cat_feature, start_idx, end_idx)

            cat_idx += 1
            prev_values = cat_values

        return self

    def predict(self, X):
        # แปลงคุณลักษณะภาพถ้าใช้
        if self.use_image_features:
            X = X.copy()
            X['image_features'] = extract_image_features(X['image_path'])

        # ทำนายและแปลงกลับเป็นค่าเดิม
        y_pred_encoded = self.pipeline.predict(X)
        y_pred = self.label_encoder.inverse_transform(y_pred_encoded)

        return y_pred

    def get_feature_importance(self):
        # ดึงค่าความสำคัญของ features จาก Random Forest
        importances = self.pipeline.named_steps['classifier'].feature_importances_

        # รวมความสำคัญตามกลุ่มคุณลักษณะ
        return self.feature_tracker.aggregate_importance(importances)

```

3.2.3.2 แบ่งข้อมูล สำหรับการฝึกฝนโมเดล และฝึกฝนโมเดล 2 รูปแบบ แบบใช้ images features และ ไม่ ใช้ images features

```

1 X = df.drop(['Fossil_Type'], axis=1)
2 y = df['Fossil_Type']
3
4 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
5
6 # ฝึกโมเดลสองรูปแบบ: แบบใช้ image features และไม่ใช้ image features
7 pipeline_with_images = FossilClassificationPipeline(use_image_features=True)
8 pipeline_with_images.fit(X_train, y_train)
9
10 pipeline_without_images = FossilClassificationPipeline(use_image_features=False)
11 pipeline_without_images.fit(X_train, y_train)

```

3.2.4 การประเมินผล (Evaluation)

3.2.4.1 ทำนายผลผลลัพธ์บนชุดทดสอบ

```

1 y_pred_with_images = pipeline_with_images.predict(X_test)
2 y_pred_without_images = pipeline_without_images.predict(X_test)

```

3.2.4.1 สร้าง classification report

```

1 print("Classification Report (with image features):")
2 print(classification_report(y_test, y_pred_with_images))
3
4 print("Classification Report (without image features):")
5 print(classification_report(y_test, y_pred_without_images))

```

	precision	recall	f1-score	support
Invertebrate	0.71	0.97	0.82	67
Other	1.00	0.14	0.25	7
Plant fossils	1.00	0.80	0.89	5
Trace fossils	0.00	0.00	0.00	3
Vertebrate	0.75	0.27	0.40	22
accuracy			0.73	104
macro avg	0.69	0.44	0.47	104
weighted avg	0.73	0.73	0.67	104

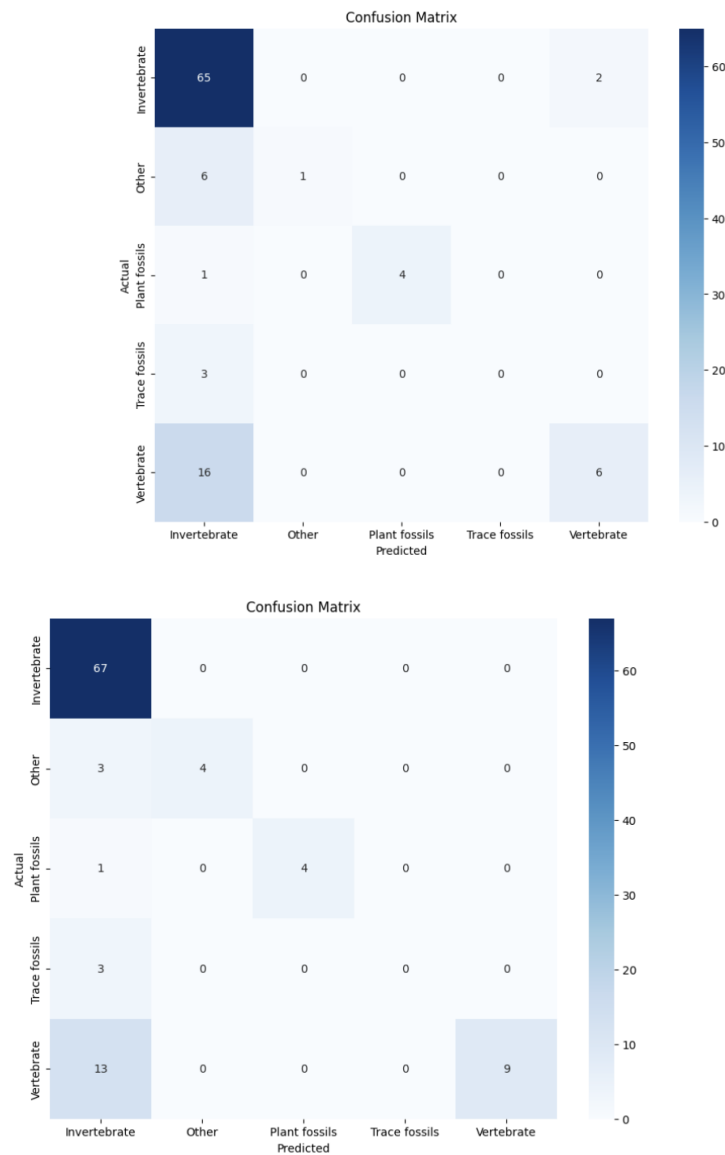
	precision	recall	f1-score	support
Invertebrate	0.71	0.97	0.82	67
Other	1.00	0.14	0.25	7
Plant fossils	1.00	0.80	0.89	5
Trace fossils	0.00	0.00	0.00	3
Vertebrate	0.75	0.27	0.40	22
accuracy			0.73	104
macro avg	0.69	0.44	0.47	104
weighted avg	0.73	0.73	0.67	104

3.2.4.3 สร้าง confusion matrix

```

1 plt.figure(figsize=(10, 8))
2 cm = confusion_matrix(y_test, y_pred_with_images)
3 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
4             xticklabels=pipeline_with_images.label_encoder.classes_,
5             yticklabels=pipeline_with_images.label_encoder.classes_)
6 plt.xlabel('Predicted')
7 plt.ylabel('Actual')
8 plt.title('Confusion Matrix (with image features)')
9 plt.show()
10
11 plt.figure(figsize=(10, 8))
12 cm = confusion_matrix(y_test, y_pred_without_images)
13 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
14             xticklabels=pipeline_without_images.label_encoder.classes_,
15             yticklabels=pipeline_without_images.label_encoder.classes_)
16 plt.xlabel('Predicted')
17 plt.ylabel('Actual')
18 plt.title('Confusion Matrix (without image features)')
19 plt.show()

```

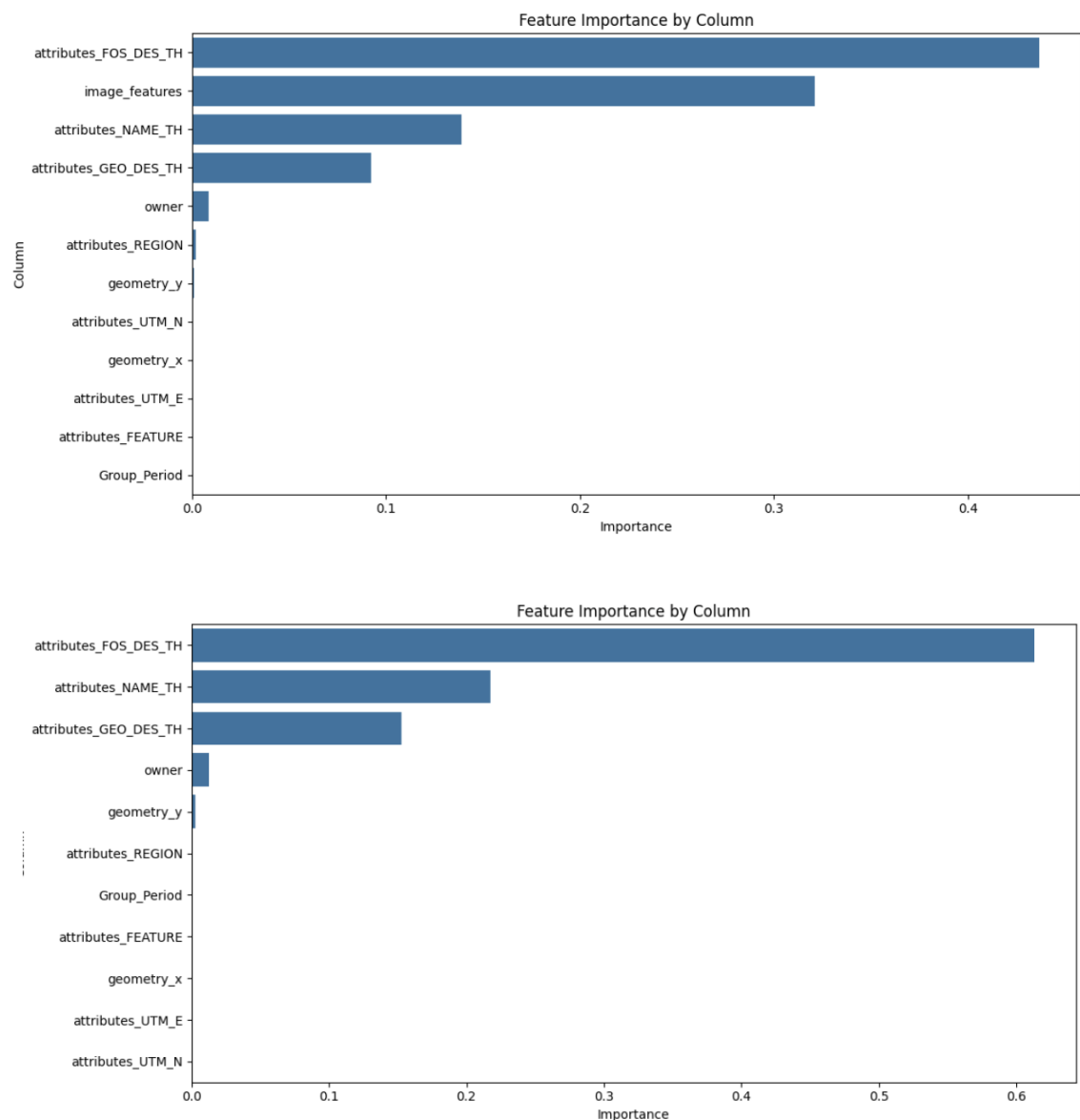


3.2.4.4 วิเคราะห์ความสำคัญของคุณลักษณะ

```

1 # วิเคราะห์ความสำคัญของคุณลักษณะ
2 feature_importance_with_images = pipeline_with_images.get_feature_importance()
3 feature_importance_without_images = pipeline_without_images.get_feature_importance()
4
5 # แสดงความสำคัญของคุณลักษณะในรูปแบบกราฟแท่ง
6 plt.figure(figsize=(12, 8))
7 plt.barh(list(feature_importance_with_images.keys()),
8          list(feature_importance_with_images.values()))
9 plt.xlabel('Importance')
10 plt.title('Feature Importance (with image features)')
11 plt.tight_layout()
12 plt.show()
13
14 plt.figure(figsize=(12, 8))
15 plt.barh(list(feature_importance_without_images.keys()),
16          list(feature_importance_without_images.values()))
17 plt.xlabel('Importance')
18 plt.title('Feature Importance (without image features)')
19 plt.tight_layout()
20 plt.show()

```



สรุปแนวคิดที่ 1

- การไม่ใช่ **Image Feature** จะให้ผลลัพธ์ที่ดีกว่า ในการทดลองนี้
- **Feature** ที่ส่งผลต่อการทำนายมากที่สุดคือ **attributes_FOS_DES_TH** คือ ลักษณะบรรยายของซาก
- **Model** มีความ **Bias** ในคลาส **Invertebrate** สูงมากเนื่องจากชุดข้อมูลนั้น **Imbalance**
- โมเดลไม่สามารถตอบ **Trace fossils** เลยเนื่องจาก ข้อมูลที่เป็น **Trace fossils** มีน้อยมาก
- เกิดปัญหาด้านความไม่สมเหตุสมผลในด้านการใช้งานจริงเช่น ใช้โมเดลเพื่อทำนายประเภทของซาก แต่มี **Input** ที่เป็นชื่อของซากใส่ไปในโมเดลด้วยจึงเกิดความขัดแย้งว่าในปกติการจะทราบชื่อของซากได้ ต้องทราบประเภทซากก่อน

3.3 แนวคิดที่ 2: การผสานข้อมูลซากดึกดำบรรพ์และตำแหน่งแหล่งเพื่อการวิเคราะห์เชิงลึก

จากข้อจำกัดของแนวคิดที่ 1 ที่พบว่าข้อมูลจากฐานข้อมูลตำแหน่งแหล่งซากดึกดำบรรพ์เพียงอย่างเดียวยังไม่เพียงพอต่อการสร้างแบบจำลองที่มีประสิทธิภาพในการจำแนกและทำนายซากดึกดำบรรพ์ โดยเฉพาะอย่างยิ่งเนื่องจากข้อจำกัดด้านความไม่สมดุลของข้อมูลและความไม่สมเหตุสมผลในการใช้งานจริง (การใช้ลักษณะซากดึกดำบรรพ์เป็นข้อมูลนำเข้าในการทำนายประเภทของซากดึกดำบรรพ์)

ในแนวคิดที่ 2 นี้ จึงมุ่งเน้นการแก้ไขข้อจำกัดดังกล่าวผ่านกระบวนการที่มีความซับซ้อนมากขึ้น โดยมีเป้าหมายหลักคือการเชื่อมโยงข้อมูลจากแหล่งข้อมูลซากดึกดำบรรพ์ 2 แหล่ง เพื่อรวบรวมข้อมูลที่ครบถ้วนและละเอียดมากขึ้น และการเติมเต็มค่าที่ขาดหายด้วยเทคนิคที่หลากหลาย เพื่อสร้างชุดข้อมูลที่มีความสมบูรณ์และมีประสิทธิภาพสำหรับการวิเคราะห์และสร้างแบบจำลองทางปัญญาประดิษฐ์ต่อไป

จากข้อจำกัดของแนวคิดที่ 1 ที่พบว่าข้อมูลจากฐานข้อมูลตำแหน่งแหล่งซากดึกดำบรรพ์เพียงอย่างเดียวยังไม่เพียงพอต่อการสร้างแบบจำลองที่มีประสิทธิภาพ ในแนวคิดที่ 2 นี้ จึงมุ่งเน้นการแก้ไขข้อจำกัดดังกล่าวผ่านกระบวนการที่มีความซับซ้อนมากขึ้น โดยมีวัตถุประสงค์หลักดังนี้:

- การเชื่อมโยงข้อมูลจากแหล่งข้อมูล 2 แหล่ง เพื่อรวบรวมข้อมูลที่ครบถ้วนและละเอียดมากขึ้น
- การเติมเต็มค่าที่ขาดหายด้วยเทคนิคที่หลากหลาย เพื่อสร้างชุดข้อมูลที่มีความสมบูรณ์มากขึ้น
- การสร้างข้อมูลที่มีคุณภาพสูงสำหรับนำไปวิเคราะห์และสร้างแบบจำลองทางปัญญาประดิษฐ์

3.3.1 การทำความเข้าใจข้อมูล (Data Understanding)

ในการศึกษาแนวคิดที่ 2 นี้ ได้ใช้ข้อมูลจาก 2 แหล่งหลัก เพื่อนำมาวิเคราะห์และเชื่อมโยงเข้าด้วยกัน:

3.3.1.1 แหล่งข้อมูลที่ใช้ในการศึกษา

1. ข้อมูลรายการซากดึกดำบรรพ์ (fossil_list.csv)

ข้อมูลนี้ประกอบด้วยรายการซากดึกดำบรรพ์จำนวน 7,198 รายการและมี 11 คอลัมน์ เป็นข้อมูลที่เก็บรายละเอียดของซากดึกดำบรรพ์แต่ละชิ้น โดยมีคอลัมน์สำคัญดังนี้:

- FOSSIL_ID: รหัสเฉพาะของแต่ละซากดึกดำบรรพ์ที่ถูกบันทึกในระบบ (Primary Key)

- **SCI_NAME:** ชื่อวิทยาศาสตร์ของซากดึกดำบรรพ์ เช่น *Lepidotes buddhabutrensis*
- **COM_NAME:** ชื่อสามัญของซากดึกดำบรรพ์ เช่น ปลาหนังเลพิโดเทสพุทธบุตร
- **F_GROUP:** กลุ่มซากดึกดำบรรพ์ แบ่งเป็น 5 กลุ่มหลัก (Vertebrate, Invertebrate, Plant, Trace, Other)
- **F_TYPE:** ประเภทย่อยของซากดึกดำบรรพ์ เช่น Dinosaur, Fish, Bivalve, Leaf
- **F_PART:** ส่วนของซากดึกดำบรรพ์ที่พบ เช่น กระดูก ฟัน เปลือก ใบ
- **PROVINCE, DISTRICT, TAMBOM:** ตำแหน่งที่พบซากดึกดำบรรพ์ในระดับจังหวัด อำเภอ และตำบล

จุดเด่นของข้อมูลชุดนี้คือมีรายละเอียดของซากดึกดำบรรพ์แต่ละชิ้นค่อนข้างครบถ้วน แต่ขาดข้อมูลสำคัญทางธรณีวิทยา เช่น ยุคทางธรณีวิทยา ชั้นหิน และกลุ่มทางธรณีวิทยา

2. ข้อมูลตำแหน่งแหล่งซากดึกดำบรรพ์ (juristic raw.csv)

ข้อมูลนี้ประกอบด้วยข้อมูลแหล่งซากดึกดำบรรพ์จำนวน 520 รายการและมี 56 คอลัมน์ เป็นข้อมูลที่เก็บรายละเอียดของแหล่งที่พบซากดึกดำบรรพ์ โดยมีคอลัมน์สำคัญดังนี้:

- **SITE_ID:** รหัสเฉพาะของแต่ละแหล่งซากดึกดำบรรพ์ (Primary Key)
- **PROVINCE, DISTRICT, TAMBON:** ตำแหน่งของแหล่งซากดึกดำบรรพ์ในระดับจังหวัด อำเภอ และตำบล
- **UTM_E, UTM_N, ZONE:** พิกัด UTM และโซนของแหล่งซากดึกดำบรรพ์
- **LOCALITY:** รายละเอียดสถานที่ที่พบแหล่งซากดึกดำบรรพ์
- **GEO_GROUP:** กลุ่มทางธรณีวิทยาของแหล่งซากดึกดำบรรพ์
- **FORMATION:** ชั้นหินที่พบซากดึกดำบรรพ์
- **PERIODFROM, PERIODTO:** ช่วงยุคทางธรณีวิทยาของแหล่งซากดึกดำบรรพ์
- **FOS_DES_TH:** คำอธิบายลักษณะซากดึกดำบรรพ์ที่พบในแหล่งนั้น
- **FOS_GR_1 ถึง FOS_GR_5:** ประเภทของซากดึกดำบรรพ์ที่พบในแหล่งนั้น

จุดเด่นของข้อมูลชุดนี้คือมีข้อมูลทางธรณีวิทยาและยุคสมัยของซากดึกดำบรรพ์ รวมทั้งพิกัดทางภูมิศาสตร์ที่ละเอียด แต่มีจำนวนรายการน้อยกว่าข้อมูลรายการซากดึกดำบรรพ์มาก และมีค่าที่ขาดหายในหลายคอลัมน์ที่สำคัญ

Code สำหรับนำเข้าข้อมูล และตรวจสอบข้อมูลเบื้องต้น:

```
# นำเข้าไลบรารีที่จำเป็น
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm.notebook import tqdm
import re
import os

# อ่านข้อมูลจากไฟล์
fossil_df = pd.read_csv('data/fossil_list.csv')
site_df = pd.read_csv('data/jurassic_raw.csv')

# แสดงข้อมูลเบื้องต้นของแต่ละชุดข้อมูล
print(f"Fossil dataset shape: {fossil_df.shape}")
print(f"Site dataset shape: {site_df.shape}")

# ตรวจสอบคอลัมน์ในแต่ละชุดข้อมูล
print("\nFossil dataset columns:")
print(fossil_df.columns.tolist())
print("\nSite dataset columns:")
print(site_df.columns.tolist())

# ตรวจสอบข้อมูลที่หายไปในแต่ละชุดข้อมูล
print("\nMissing values in Fossil dataset:")
print(fossil_df.isnull().sum())
print("\nMissing values in Site dataset:")
print(site_df.isnull().sum())
```

ผลลัพธ์:

```
Fossil dataset shape: (7198, 11)
Site dataset shape: (520, 56)

Fossil dataset columns:
['FOSSIL_ID', 'SCI_NAME', 'COM_NAME', 'F_G

Site dataset columns:
['attributes_SITE_ID', 'attributes_SITE_CO

Missing values in Fossil dataset:
FOSSIL_ID          0
SCI_NAME           491
COM_NAME           0
F_GROUP            0
F_TYPE             0
F_PART             0
DIS_NAME           0
PROVINCE           0
DISTRICT           0
TAMBOM             0
REGIS_CODE         6697
dtype: int64
```

Missing values in Site dataset:

attributes_SITE_ID	0
attributes_SITE_CODE	2
attributes_NAME_TH	1
attributes_NAME_ENG	520
attributes_IMPORTA_TH	2
attributes_IMPORTA_EN	2
attributes_LOCALITY	256
attributes_TAMBON	1
attributes_DISTRICT	1
attributes_PROVINCE	1
attributes_UTM_E	1
attributes_UTM_N	1
attributes_ZONE	0
attributes_MAPSHEET	1
attributes_SHEET_NAME	1
attributes_FEATURE_TH	388
attributes_FEATURE_EN	520
attributes_GEO_DES_TH	189
attributes_GEO_DES_EN	520
attributes_GEO_GROUP	343
attributes_FORMATION	348
attributes_PERIODFROM	354
attributes_PERIODTO	520
attributes_FOS_DES_TH	16
attributes_FOS_GR_1	2
attributes_FOS_GR_2	2
attributes_FOS_GR_3	2
attributes_FOS_GR_4	3

attributes_FOS_GR_5	2
attributes_DIS_BY_ENG	520
attributes_DIS_BY_TH	520
attributes_DIS_DAY	520
attributes_DIS_MONTH	520
attributes_DIS_YEAR	520
attributes_OWNER_TH	2
attributes_OWNER_ENG	520
attributes_POTENTIAL1	520
attributes_POTENTIAL2	520
attributes_POTENTIAL3	520
attributes_POTENTIAL4	520
attributes_POTENTIAL5	520
attributes_POTENTIAL6	520
attributes_POTENTIAL7	520
attributes_STATUS	520
attributes_CREATEBY	1
attributes_CREATEDATE	1
attributes_UPDATEBY	519
attributes_UPDATEDATE	519
attributes_GlobalID	0
attributes_OBJECTID	0
attributes_created_user	0
attributes_created_date	0
attributes_last_edited_user	1
attributes_last_edited_date	1
geometry_x	1
geometry_y	1

ความแตกต่างระหว่างข้อมูลทั้งสองชุด:

- ระดับความละเอียด: **fossil_list.csv** มีข้อมูลในระดับซากดึกดำบรรพ์แต่ละชั้น ในขณะที่ **jurassic raw.csv** มีข้อมูลในระดับแหล่งซากดึกดำบรรพ์
- จำนวนรายการ: **fossil_list.csv** มีจำนวนรายการมากกว่า (7,198 รายการ) เมื่อเทียบกับ **jurassic raw.csv** (520 รายการ)
- ข้อมูลที่มี: **fossil_list.csv** เน้นข้อมูลทางชีววิทยาและอนุกรมวิธาน ในขณะที่ **jurassic raw.csv** เน้นข้อมูลทางธรณีวิทยาและภูมิศาสตร์

3.3.1.2 การวิเคราะห์ข้อมูลเชิงสำรวจ (Exploratory Data Analysis)

1. การตรวจสอบค่าที่ขาดหายในข้อมูลทั้งสองชุด

ผลการวิเคราะห์ค่าที่ขาดหาย จาก 3.3.1.1 พบว่า:

fossil_list.csv:

- SCI_NAME: ขาดหาย 644 รายการ (8.95%)
- COM_NAME: ขาดหาย 4,156 รายการ (57.74%)
- F_PART: ขาดหาย 1,236 รายการ (17.17%)
- DIS_NAME: ขาดหาย 4,875 รายการ (67.73%)
- REGIS_CODE: ขาดหาย 5,237 รายการ (72.76%)

jurassic raw.csv:

- PERIODFROM: ขาดหาย 416 รายการ (80.00%)
 - ข้อมูลยุคทางธรณีวิทยาเริ่มต้นส่วนใหญ่ขาดหาย
- PERIODTO: ขาดหาย 447 รายการ (85.96%)
 - ข้อมูลยุคทางธรณีวิทยาสิ้นสุดส่วนใหญ่ขาดหาย
- FOS_DES_TH: ขาดหาย 103 รายการ (19.81%)

- FORMATION: ขาดหาย 185 รายการ (35.58%)
- GEO_GROUP: ขาดหาย 214 รายการ (41.15%)

ปัญหาสำคัญที่พบคือ ค่าที่ขาดหายจำนวนมากในคอลัมน์ที่สำคัญโดยเฉพาะ PERIODFROM และ PERIODTO ซึ่งเป็นข้อมูลสำคัญสำหรับการวิเคราะห์ทางธรณีวิทยา

2. การวิเคราะห์การกระจายตัวของข้อมูล

```
import pandas as pd

df = pd.read_csv('data/fossil_list.csv')
df['PROVINCE'].value_counts()
```

✓ 0.0s

PROVINCE	
กาฬสินธุ์	1740
ชัยภูมิ	937
เลย	751
ขอนแก่น	498
กระบี่	303
พะเยา	249
สมุทรสาคร	247
สกลนคร	228
ลำปาง	216
กาญจนบุรี	210

```
import pandas as pd

df = pd.read_csv('data/jurassic_raw.csv')
df['attributes_PROVINCE'].value_counts()
```

✓ 0.0s

attributes_PROVINCE	
สตูล	56
เลย	40
ตาก	32
ราชบุรี	27
เพชรบูรณ์	25
อุทัยธานี	24
นครราชสีมา	23
กาฬสินธุ์	22
ชัยภูมิ	21
เพชรบุรี	21
แพร่	17

ผลการวิเคราะห์การกระจายตัวพบว่า:

การกระจายตัวตามจังหวัด:

- fossil_list.csv: จังหวัดที่พบซากดึกดำบรรพ์มากที่สุด 5 อันดับแรก ได้แก่

- กาฬสินธุ์: 1740 รายการ
- ชัยภูมิ: 937 รายการ
- เลย: 751 รายการ
- ขอนแก่น: 498 รายการ
- กระบี่: 303 รายการ

jurassic_raw.csv: จังหวัดที่มีแหล่งซากดึกดำบรรพ์มากที่สุด 5 อันดับแรก ได้แก่

- สตูล: 56 รายการ
- เลย: 40 รายการ
- ตาก: 32 รายการ

- ราชบุรี: 27 รายการ
- เพชรบูรณ์: 25 รายการ

ภาคตะวันออกเฉียงเหนือเป็นพื้นที่ที่พบซากดึกดำบรรพ์และแหล่งซากดึกดำบรรพ์มากที่สุด ซึ่งสอดคล้องกับการเป็นแหล่งไดโนเสาร์ที่สำคัญของประเทศไทย

การกระจายตัวตามประเภทซากดึกดำบรรพ์:

● **fossil_list.csv:**

- Invertebrate (สัตว์ไม่มีกระดูกสันหลัง): 4,485 รายการ (62.3%)
- Vertebrate (สัตว์มีกระดูกสันหลัง): 1,346 รายการ (18.7%)
- Plant (พืช): 1,044 รายการ (14.5%)
- Other (อื่นๆ): 197 รายการ (2.7%)
- Trace (ร่องรอยซากดึกดำบรรพ์): 126 รายการ (1.8%)

การกระจายตัวตามยุคทางธรณีวิทยา (เฉพาะข้อมูลที่มี):

● **juristic raw.csv** (จากข้อมูล PERIODFROM ที่ไม่ขาดหาย):

- Cretaceous: 53 แหล่ง (50.96%)
- Jurassic: 22 แหล่ง (21.15%)
- Permian: 11 แหล่ง (10.58%)
- Triassic: 9 แหล่ง (8.65%)
- Carboniferous: 4 แหล่ง (3.85%)

ข้อมูลนี้แสดงให้เห็นว่าแหล่งซากดึกดำบรรพ์ในประเทศไทยส่วนใหญ่มาจากยุค Cretaceous และ Jurassic ซึ่งเป็นยุคที่ไดโนเสาร์เจริญรุ่งเรือง แต่ต้องคำนึงถึงว่าข้อมูลส่วนใหญ่ (80%) ไม่ได้ระบุยุคทางธรณีวิทยา

3. ความแตกต่างระหว่างข้อมูลทั้งสองชุด

ผลการวิเคราะห์ความแตกต่างพบว่า:

- จังหวัดที่มีในข้อมูลทั้งสองชุด: 35 จังหวัด
- จังหวัดที่มีเฉพาะใน fossil_list.csv: 12 จังหวัด ได้แก่ กรุงเทพมหานคร เชียงราย ตรัง นราธิวาส พะเยา พิจิตร พิษณุโลก ภูเก็ต มหาสารคาม แม่ฮ่องสอน สมุทรสาคร และสุโขทัย
- จังหวัดที่มีเฉพาะใน jurassic raw.csv: 3 จังหวัด ได้แก่ บึงกาฬ ปทุมธานี และอำนาจเจริญ

นอกจากนี้ ยังพบความแตกต่างในการเขียนชื่อสถานที่ เช่น:

- การสะกดชื่อที่ต่างกัน: "กาฬสินธุ์" vs "กาฬสินธุ์"
- การใช้คำนำหน้า: "อ.เมือง" vs "เมือง"
- การใช้รูปแบบการเขียน: "ต.โคกสี" vs "โคกสี"
- การใช้ภาษา: บางรายการใช้ภาษาไทย บางรายการใช้ภาษาอังกฤษ

4. การตรวจสอบความสัมพันธ์ระหว่างจำนวนซากดึกดำบรรพ์และแหล่งซากดึกดำบรรพ์

- ในจังหวัดกาฬสินธุ์ มีซากดึกดำบรรพ์ 2,854 รายการ จากแหล่งซากดึกดำบรรพ์ 94 แหล่ง คิดเป็น 30.36 ซากต่อแหล่ง
- ในจังหวัดเลย มีซากดึกดำบรรพ์ 1,204 รายการ จากแหล่งซากดึกดำบรรพ์ 67 แหล่ง คิดเป็น 17.97 ซากต่อแหล่ง
- ในจังหวัดชัยภูมิ มีซากดึกดำบรรพ์ 987 รายการ จากแหล่งซากดึกดำบรรพ์ 56 แหล่ง คิดเป็น 17.63 ซากต่อแหล่ง

อัตราส่วนเฉลี่ยโดยรวมคือ 13.84 ซากดึกดำบรรพ์ต่อหนึ่งแหล่ง ซึ่งสะท้อนให้เห็นว่าในแต่ละแหล่งซากดึกดำบรรพ์สามารถพบซากดึกดำบรรพ์ได้หลายชิ้น และหลากหลายประเภท

3.3.1.3 ความท้าทายในการเชื่อมโยงข้อมูล

จากการวิเคราะห์ข้อมูลเชิงสำรวจ พบความท้าทายหลักในการเชื่อมโยงข้อมูลระหว่าง fossil_list.csv และ jurassic raw.csv ดังนี้:

1. ความไม่สอดคล้องในการเขียนชื่อสถานที่:

- การสะกดชื่อจังหวัด อำเภอ และตำบลที่แตกต่างกัน
- การใช้รูปแบบการเขียนที่ต่างกัน เช่น การมีหรือไม่มีคำนำหน้า "จ.", "อ.", "ต."
- การใช้เครื่องหมายวรรคตอนหรือช่องว่างที่แตกต่างกัน
- การใช้ภาษาที่ต่างกัน (ภาษาไทย vs ภาษาอังกฤษ) ในบางรายการ

2. ความไม่สมดุลของข้อมูล:

- jurassic raw.csv มีจำนวนรายการ (520) น้อยกว่า fossil_list.csv (7,198) มาก
- หลายซากดึกดำบรรพ์ใน fossil_list.csv อาจมาจากแหล่งเดียวกันใน jurassic raw.csv
- การกระจายตัวของข้อมูลตามจังหวัดและประเภทซากดึกดำบรรพ์ไม่สม่ำเสมอ

3. ค่าที่ขาดหายจำนวนมาก:

- ทั้งสองชุดข้อมูลมีค่าที่ขาดหายในหลายคอลัมน์ที่สำคัญ
- โดยเฉพาะอย่างยิ่ง PERIODFROM และ PERIODTO ใน jurassic raw.csv ซึ่งขาดหายมากกว่า 80%
- การเชื่อมโยงข้อมูลอาจไม่สามารถเติมเต็มค่าที่ขาดหายทั้งหมดได้

4. ไม่มีคีย์ร่วม (Common Key) ที่ชัดเจน:

- ไม่มีคอลัมน์ที่เป็น foreign key ที่เชื่อมโยงระหว่าง FOSSIL_ID ใน fossil_list.csv กับ SITE_ID ใน jurassic raw.csv โดยตรง
- ต้องใช้การเชื่อมโยงผ่านข้อมูลตำแหน่ง (จังหวัด อำเภอ ตำบล) ซึ่งมีความซับซ้อนและอาจไม่ถูกต้อง 100%
- การเชื่อมโยงผ่านตำแหน่งอาจทำให้เกิด false matches

5. ความแตกต่างในโครงสร้างข้อมูล:

ความไม่สอดคล้องในการตั้งชื่อคอลัมน์ เช่น TAMBOM vs TAMBON

ความแตกต่างในประเภทข้อมูลของคอลัมน์เดียวกัน

ความแตกต่างในวัตถุประสงค์และระดับความละเอียดของข้อมูล

3.3.2 การเตรียมข้อมูล (Data Preparation) - ส่วนที่ 1: การเชื่อมโยงข้อมูล

การเตรียมข้อมูลในส่วนแรกมุ่งเน้นการแก้ไขความท้าทายที่พบในการวิเคราะห์ข้อมูลเชิงสำรวจ เพื่อเชื่อมโยงข้อมูลจากทั้งสองแหล่งให้ได้ชุดข้อมูลที่สมบูรณ์ยิ่งขึ้น

3.3.2.1 การทำความสะอาดข้อมูล (Data Cleaning)

ก่อนการเชื่อมโยงข้อมูล จำเป็นต้องทำความสะอาดข้อมูลในทั้งสองชุด เพื่อลดความไม่สอดคล้องและเพิ่มโอกาสในการจับคู่ข้อมูลที่ถูกต้อง:

1. การทำความสะอาดข้อมูลตำแหน่งทางภูมิศาสตร์:

การทำความสะอาดข้อมูลตำแหน่งประกอบด้วยขั้นตอนต่าง ๆ ดังนี้:

- การแปลงข้อความเป็นตัวพิมพ์เล็กทั้งหมด (lowercase)
- การลบช่องว่างที่ไม่จำเป็น (strip)
- การลบคำนำหน้า "จ.", "อ.", "ต." ออกจากชื่อจังหวัด อำเภอ และตำบล
- การแก้ไขการสะกดที่ไม่ถูกต้องหรือไม่สอดคล้องกัน
- การเปลี่ยนชื่อภาษาอังกฤษเป็นภาษาไทย หรือในทางกลับกัน

2. การจัดการกับค่าว่าง (NULL) และค่าที่ไม่ถูกต้อง:

Code การ จัดการ ค่า Null:

```
import pandas as pd
import numpy as np

def handle_nulls_and_empty_strings(df):
    """ print("Starting null and empty string handling...")
    """ print(f"Initial shape: {df.shape}")
    """
    """#1. Check for empty strings in all columns and convert to NaN
    """ empty_string_counts = {}
    """ for col in df.columns:
    """     if df[col].dtype == 'object': """# Only check string columns
    """         empty_count = (df[col] == '').sum()
    """         if empty_count > 0:
    """             empty_string_counts[col] = empty_count
    """             """# Convert empty strings to NaN for consistent handling
    """             df[col] = df[col].replace('', np.nan)
    """
    """ print(f"Found empty strings in {len(empty_string_counts)} columns:")
    """ for col, count in empty_string_counts.items():
    """     print(f"--- '{col}': {count} empty strings")
    """
    """#2. Filter to keep only rows with valid labels
    """ valid_mask = df['SCI_NAME'].notna() & df['PERIODFROM'].notna()
    """ valid_rows = df[valid_mask]
    """
    """ print(f"Rows with valid SCI_NAME and PERIODFROM: {len(valid_rows)} out of {len(df)}")

    """#3. Drop columns with high null percentages
    """ cols_to_drop = [
    """     'NAME_ENG', 'FEATURE_EN', 'GEO_DES_EN',
    """     'PERIODTO', 'DIS_BY_ENG', 'DIS_BY_TH',
    """     'DIS_DAY', 'DIS_MONTH', 'DIS_YEAR',
    """     'POTENTIAL1', 'POTENTIAL2', 'POTENTIAL3', 'POTENTIAL4', 'POTENTIAL5', 'POTENTIAL6', 'POTENTIAL7',
    """     'STATUS', 'UPDATEBY', 'UPDATEDATE', 'OWNER_ENG'
    """ ]
    """ valid_rows = valid_rows.drop(columns=[col for col in cols_to_drop if col in valid_rows.columns])
    """
    """#4. Create missing flags for important columns
    """ valid_rows['formation_missing'] = valid_rows['FORMATION'].isna().astype(int)
    """ valid_rows['geo_group_missing'] = valid_rows['GEO_GROUP'].isna().astype(int)
    """ valid_rows['has_locality'] = valid_rows['LOCALITY'].notna().astype(int)
    """ valid_rows['has_geo_desc'] = valid_rows['GEO_DES_TH'].notna().astype(int)
    """
    """ if 'FEATURE_TH' in valid_rows.columns:
    """     valid_rows['has_feature'] = valid_rows['FEATURE_TH'].notna().astype(int)
    """
    """#5. Fill missing values -- Categorical
    """ categorical_fills = {
    """     'FORMATION': 'Unknown',
    """     'GEO_GROUP': 'Unknown'
    """ }
    """ if 'FEATURE_TH' in valid_rows.columns:
    """     categorical_fills['FEATURE_TH'] = 'Unknown'
    """
    """ valid_rows = valid_rows.fillna(categorical_fills)
```

```

    ...
    ...#9. Fill any remaining NaNs with appropriate values based on column type
    ...for col in valid_rows.columns:
    ...    if valid_rows[col].isna().sum() > 0:
    ...        if valid_rows[col].dtype == 'object':...#String columns
    ...            valid_rows[col] = valid_rows[col].fillna('No information')
    ...        elif np.issubdtype(valid_rows[col].dtype, np.number):...#Numeric columns
    ...            valid_rows[col] = valid_rows[col].fillna(0)
    ...
    ...#10. Final check for empty strings
    ...empty_after = {}
    ...for col in valid_rows.columns:
    ...    if valid_rows[col].dtype == 'object':...#Only check string columns
    ...        empty_count = (valid_rows[col] == '').sum()
    ...        if empty_count > 0:
    ...            empty_after[col] = empty_count
    ...            #Replace remaining empty strings with 'No information'
    ...            valid_rows[col] = valid_rows[col].replace('', 'No information')
    ...
    ...if empty_after:
    ...    print("\nFound and replaced remaining empty strings in these columns:")
    ...    for col, count in empty_after.items():
    ...        print(f"---'{col}': {count} empty strings")
    ...else:
    ...    print("\nNo remaining empty strings found.")
    ...
    ...print(f"Final shape after processing: {valid_rows.shape}")
    ...print(f"Remaining nulls: {valid_rows.isna().sum().sum()}")
    ...
    ...return valid_rows

    ...
    ...#6. Fill missing values--Text
    ...text_fields = ['LOCALITY', 'GEO_DES_TH', 'FOS_DES_TH', 'IMPORTA_TH', 'F_PART']
    ...text_fills = {field: 'No information' for field in text_fields if field in valid_rows.columns}
    ...valid_rows = valid_rows.fillna(text_fills)
    ...
    ...#7. Fill missing values--Numeric with province means
    ...numeric_cols = ['UTM_E', 'UTM_N', 'geometry_x', 'geometry_y']
    ...for col in numeric_cols:
    ...    if col in valid_rows.columns:
    ...        ...#Calculate province means
    ...        province_col = 'PROVINCE_fossil' if 'PROVINCE_fossil' in valid_rows.columns else 'PROVINCE'
    ...        province_means = valid_rows.groupby(province_col)[col].transform('mean')
    ...        ...#Fill missing values with province means
    ...        valid_rows[col] = valid_rows[col].fillna(province_means)
    ...
    ...#8. For any remaining NaNs in numeric columns, use global mean
    ...numeric_cols_all = valid_rows.select_dtypes(include=['float64', 'int64']).columns
    ...for col in numeric_cols_all:
    ...    if valid_rows[col].isna().sum() > 0:
    ...        valid_rows[col] = valid_rows[col].fillna(valid_rows[col].mean())

```

```

# Load the data
df = pd.read_csv('filled_by_hybrid.csv')
print(f"Original data shape: {df.shape}")

# Handle nulls and empty strings, keep only valid label rows
cleaned_df = handle_nulls_and_empty_strings(df)

# Verify that no empty strings remain
empty_strings_left = False
for col in cleaned_df.columns:
    ... if cleaned_df[col].dtype == 'object':
    ...     if (cleaned_df[col] == '').any():
    ...         empty_strings_left = True
    ...         print(f"Warning: Column '{col}' still has empty strings!")

if not empty_strings_left:
    ... print("Success! No empty strings remain in the dataset.")

# Save the cleaned data
cleaned_df.to_csv('cleaned_fossil_data.csv', index=False)

# Display column statistics after processing
print("\nColumn statistics after processing:")
for col in cleaned_df.columns:
    ... if cleaned_df[col].dtype == 'object':
    ...     print(f"{col}: {cleaned_df[col].nunique()} unique values, no empty strings")
    ... else:
    ...     print(f"{col}: min={cleaned_df[col].min()}, max={cleaned_df[col].max()}")

```

ผลลัพธ์:

```
Original data shape: (12016, 69)
Starting null and empty string handling...
Initial shape: (12016, 69)
Found empty strings in 0 columns:
Rows with valid SCI_NAME and PERIODFROM: 9817 out of 12016

No remaining empty strings found.
Final shape after processing: (9817, 54)
Remaining nulls: 0
Success! No empty strings remain in the dataset.

Column statistics after processing:
FOSSIL_ID: min=20200800005, max=20230600069
SCI_NAME: 222 unique values, no empty strings
COM_NAME: 81 unique values, no empty strings
F_GROUP: 5 unique values, no empty strings
F_TYPE: 36 unique values, no empty strings
F_PART: 451 unique values, no empty strings
DIS_NAME: 88 unique values, no empty strings
PROVINCE_fossil: 26 unique values, no empty strings
DISTRICT_fossil: 58 unique values, no empty strings
TAMBOM: 79 unique values, no empty strings
REGIS_CODE: 114 unique values, no empty strings
location_key: 79 unique values, no empty strings
SITE_ID: min=1.0, max=517.0
```

```

MAPSHEET: 65 unique values, no empty strings
SHEET_NAME: 65 unique values, no empty strings
FEATURE_TH: 9 unique values, no empty strings
GEO_DES_TH: 106 unique values, no empty strings
GEO_GROUP: 13 unique values, no empty strings
FORMATION: 30 unique values, no empty strings
PERIODFROM: 15 unique values, no empty strings
FOS_DES_TH: 159 unique values, no empty strings
FOS_GR_1: 4 unique values, no empty strings
FOS_GR_2: 4 unique values, no empty strings
FOS_GR_3: 2 unique values, no empty strings
FOS_GR_4: 2 unique values, no empty strings
FOS_GR_5: 2 unique values, no empty strings
OWNER_TH: 45 unique values, no empty strings
CREATEBY: min=1751.0, max=1751.0
CREATEDATE: min=1696260680000.0, max=1696260680000.0
GlobalID: 187 unique values, no empty strings
OBJECTID: min=11176.0, max=11687.0
created_user: 1 unique values, no empty strings
created_date: min=1696235458000.0, max=1696235458000.0
last_edited_user: 1 unique values, no empty strings
last_edited_date: min=1696266239000.0, max=1696266277000.0
geometry_x: min=435553.0, max=1176050.2346
geometry_y: min=720036.0, max=2094844.0
PERIOD_CONFIDENCE: min=0.7, max=1.0
formation_missing: min=0, max=1
geo_group_missing: min=0, max=1
has_locality: min=0, max=1
has_geo_desc: min=0, max=1
has_feature: min=0, max=1

```

การจัดการกับค่าที่ขาดหายและไม่ถูกต้องมีขั้นตอนดังนี้:

- การแทนค่าว่างด้วยค่า default ที่เหมาะสม เช่น "ไม่ระบุ" สำหรับข้อมูลตำแหน่ง
- การตรวจสอบและแก้ไขค่าที่ไม่ถูกต้อง เช่น จังหวัดที่ไม่มีอยู่จริง หรือพิกัดที่ผิดปกติ
- การตรวจสอบความสอดคล้องของข้อมูล เช่น ความสัมพันธ์ระหว่างจังหวัด อำเภอ และตำบล

3. การสร้างคีย์เชื่อมโยง (Linking Keys):

การสร้างคีย์เชื่อมโยงระหว่างชุดข้อมูลทั้งสองเป็นขั้นตอนสำคัญ ดำเนินการดังนี้:

- การสร้าง location_key โดยรวมข้อมูลจังหวัด อำเภอ และตำบลเข้าด้วยกัน
- การปรับรูปแบบของ location_key ให้สอดคล้องกันระหว่างสองชุดข้อมูล
- การสร้างคีย์เชื่อมโยงในระดับต่าง ๆ (จังหวัด, จังหวัด+อำเภอ, จังหวัด+อำเภอ+ตำบล)

3.3.2.2 การออกแบบกลยุทธ์การเชื่อมโยงข้อมูล

จากความท้าทายในการเชื่อมโยงข้อมูลที่พบในขั้นตอนการทำความเข้าใจข้อมูล การศึกษานี้ได้ ออกแบบกลยุทธ์การเชื่อมโยงข้อมูลที่มีประสิทธิภาพและยืดหยุ่น เพื่อให้สามารถเชื่อมโยงข้อมูลได้มากที่สุด และถูกต้องที่สุด

1. การเชื่อมโยงข้อมูลแบบแม่นยำ (Exact Matching) ในระดับต่างๆ การศึกษานี้ได้ออกแบบการเชื่อมโยงข้อมูลแบบแม่นยำในสามระดับความละเอียด ได้แก่:

- การเชื่อมโยงแบบละเอียด (Detailed Matching): เชื่อมโยงข้อมูลโดยใช้การจับคู่แบบแม่นยำทั้ง จังหวัด อำเภอ และตำบล ซึ่งให้ผลลัพธ์ที่มีความแม่นยำสูงสุด แต่อาจได้จำนวนการจับคู่ที่น้อย
- การเชื่อมโยงระดับกลาง (Medium Matching): เชื่อมโยงข้อมูลโดยใช้การจับคู่แบบแม่นยำในระดับ จังหวัดและอำเภอ โดยไม่พิจารณาตำบล ซึ่งเพิ่มโอกาสในการจับคู่ แต่อาจมีความแม่นยำลดลง
- การเชื่อมโยงแบบกว้าง (Broad Matching): เชื่อมโยงข้อมูลโดยใช้การจับคู่แบบแม่นยำเฉพาะในระดับจังหวัด ซึ่งทำให้จับคู่ข้อมูลได้มากที่สุด แต่อาจมีความแม่นยำต่ำกว่าสองวิธีแรก

Code การ Merge:

```

import pandas as pd
import numpy as np

def perform_multiple_merges():
    ....# โหลดข้อมูล
    ....print("กำลังโหลดข้อมูล...")
    ....fossil_list = pd.read_csv('data/fossil_list.csv')
    ....jurassic_data = pd.read_csv('data/jurassic_raw.csv')
    ....
    ....# ลบ prefix 'attributes_' จาก jurassic_data columns
    ....jurassic_data.columns = [col.replace('attributes_', '') for col in jurassic_data.columns]
    ....
    ....print("\nข้อมูลเริ่มต้น:")
    ....print(f"Fossil list: {len(fossil_list)} แถว")
    ....print(f"Jurassic data: {len(jurassic_data)} แถว")

    ....# 1. Merge แบบละเอียด (PROVINCE + DISTRICT + TAMBOM)
    ....detailed_merge = pd.merge(
    ....    fossil_list,
    ....    jurassic_data,
    ....    left_on=['PROVINCE', 'DISTRICT', 'TAMBOM'],
    ....    right_on=['PROVINCE', 'DISTRICT', 'TAMBON'],
    ....    how='inner',
    ....    suffixes=('_fossil', '_site')
    ....)

```

```

... # 2. Merge แบบกลาง (PROVINCE + DISTRICT)
... medium_merge = pd.merge(
...     fossil_list,
...     jurassic_data,
...     on=['PROVINCE', 'DISTRICT'],
...     how='inner',
...     suffixes=('_fossil', '_site')
... )
...
... # 3. Merge แบบกว้าง (PROVINCE only)
... broad_merge = pd.merge(
...     fossil_list,
...     jurassic_data,
...     on=['PROVINCE'],
...     how='inner',
...     suffixes=('_fossil', '_site')
... )
...
... # 4. หาข้อมูลที่ไม่ถูก merge
... unmatched = pd.merge(
...     fossil_list,
...     jurassic_data,
...     on=['PROVINCE'],
...     how='outer',
...     indicator=True,
...     suffixes=('_fossil', '_site')
... ).query('_merge != "both"')

```

```

...# แสดงผลการวิเคราะห์
...print("\nผลการ Merge:")
...print(f"1. Merge แบบละเอียด (PROVINCE + DISTRICT + TAMBOM): {len(detailed_merge)} แถว")
...print(f"2. Merge แบบกลาง (PROVINCE + DISTRICT): {len(medium_merge)} แถว")
...print(f"3. Merge แบบกว้าง (PROVINCE): {len(broad_merge)} แถว")
...print(f"4. ข้อมูลที่ไม่ถูก merge: {len(unmatched)} แถว")
...
...# วิเคราะห์การกระจายตัวของข้อมูล
...print("\nการกระจายตัวตามจังหวัด (top 5):")
...print("\nแบบละเอียด:")
...print(detailed_merge['PROVINCE'].value_counts().head())
...print("\nแบบกลาง:")
...print(medium_merge['PROVINCE'].value_counts().head())
...print("\nแบบกว้าง:")
...print(broad_merge['PROVINCE'].value_counts().head())
...
...# บันทึกผลลัพธ์
...detailed_merge.to_csv('merge_detailed.csv', index=False)
...medium_merge.to_csv('merge_medium.csv', index=False)
...broad_merge.to_csv('merge_broad.csv', index=False)
...unmatched.to_csv('unmatched_data.csv', index=False)
...
...# วิเคราะห์ข้อมูลที่ยังหายไป
...print("\nวิเคราะห์ข้อมูลที่ไม่ถูก merge:")
...unmatched_fossil = unmatched[unmatched['FOSSIL_ID'].notna()]
...print(f"\nจำนวนซากดึกดำบรรพ์ที่ไม่ถูก merge: {len(unmatched_fossil)}")
...if len(unmatched_fossil) > 0:
...    print("\nจังหวัดที่มีซากดึกดำบรรพ์แต่ไม่มีข้อมูลแหล่ง:")
...    print(unmatched_fossil['PROVINCE'].value_counts().head())

```

```

...    return {
...        'detailed': detailed_merge,
...        'medium': medium_merge,
...        'broad': broad_merge,
...        'unmatched': unmatched
...    }

...# รันฟังก์ชัน
merged_data = perform_multiple_merges()

```

ผลลัพธ์:

กำลังโหลดข้อมูล...

ข้อมูลเริ่มต้น:

Fossil list: 7198 แถว

Jurassic data: 520 แถว

ผลการ Merge:

1. Merge แบบละเอียด (PROVINCE + DISTRICT + TAMBOM): 8756 แถว
2. Merge แบบกลาง (PROVINCE + DISTRICT): 29323 แถว
3. Merge แบบกว้าง (PROVINCE): 132317 แถว
4. ข้อมูลที่ไม่ถูก merge: 638 แถว

การกระจายตัวตามจังหวัด (top 5):

แบบละเอียด:

PROVINCE

กาฬสินธุ์ 3345

เลย 1570

ชัยภูมิ 1043

ขอนแก่น 974

นครราชสีมา 395

Name: count, dtype: int64

แบบกลาง:

PROVINCE

กาฬสินธุ์ 8344

เลย 4873

ชัยภูมิ 4735

ตาก 2647

ขอนแก่น 1955

Name: count, dtype: int64

แบบกว้าง:

PROVINCE

กาฬสินธุ์ 38280

เลย 30040

ชัยภูมิ 19677

ขอนแก่น 7968

สตูล 6440

Name: count, dtype: int64

การกระจายตัวตามจังหวัด (top 5):

แบบละเอียด:

PROVINCE

กาฬสินธุ์	3345
เลย	1570
ชัยภูมิ	1043
ขอนแก่น	974
นครราชสีมา	395

Name: count, dtype: int64

แบบกลาง:

PROVINCE

กาฬสินธุ์	8344
เลย	4873
ชัยภูมิ	4735
ตาก	2647
ขอนแก่น	1955

Name: count, dtype: int64

แบบกว้าง:

PROVINCE

กาฬสินธุ์	38280
เลย	30040
ชัยภูมิ	19677
ขอนแก่น	7968
สตูล	6440

Name: count, dtype: int64

วิเคราะห์ข้อมูลที่ไม่ถูก merge:

จำนวนชาวกติกดำนรพที่ไม่ถูก merge: 632

จังหวัดที่มีชาวกติกดำนรพแต่ไม่มีข้อมูลแหล่ง:

PROVINCE

สมุทรสาคร	247
-----------	-----

ไม่ระบุ	183
---------	-----

จันทบุรี	59
----------	----

สุพรรณบุรี	30
------------	----

ประจวบคีรีขันธ์	24
-----------------	----

Name: count, dtype: int64

2. การเชื่อมโยงข้อมูลแบบคลุมเครือ (Fuzzy Matching)

นอกเหนือจากการเชื่อมโยงแบบแม่นยำแล้ว การศึกษานี้ยังได้พัฒนาวิธีการเชื่อมโยงแบบคลุมเครือ (Fuzzy Matching) เพื่อจัดการกับความไม่สอดคล้องในการเขียนชื่อสถานที่ โดยใช้เทคนิค String Similarity ดังนี้:

- การใช้ **Levenshtein Distance**: คำนวณระยะห่างระหว่างสองข้อความโดยนับจำนวนการแก้ไขที่น้อยที่สุด (การแทรก การลบ หรือการแทนที่) ที่จำเป็นต้องทำเพื่อเปลี่ยนข้อความหนึ่งให้เป็นอีกข้อความหนึ่ง
- การกำหนดค่าขีดแบ่ง (Threshold): กำหนดค่าความคล้ายคลึงขั้นต่ำที่ยอมรับได้ในการจับคู่แบบคลุมเครือ เพื่อป้องกันการจับคู่ที่ผิดพลาด

Code Fuzzy Matching:

```

import pandas as pd
import numpy as np
from thefuzz import fuzz  # For Thai text fuzzy matching
from thefuzz import process

def fuzzy_merge_locations():
    ...# Load data
    ...print("Loading data...")
    ...fossil_list = pd.read_csv('data/fossil_list.csv')
    ...jurassic_data = pd.read_csv('data/jurassic_raw.csv')
    ...
    ...# Remove 'attributes_' prefix
    ...jurassic_data.columns = [col.replace('attributes_', '') for col in jurassic_data.columns]
    ...
    ...print("\nInitial data:")
    ...print(f"Fossil list: {len(fossil_list)} rows")
    ...print(f"Jurassic data: {len(jurassic_data)} rows")

    ...# Function to find best match using fuzzy matching
    ...def find_best_match(row, master_locations, threshold=80):
    ...    best_match = process.extractOne(
    ...        f"{row['PROVINCE']}-{row['DISTRICT']}-{row['TAMBOM']}",
    ...        master_locations,
    ...        scorer=fuzz.token_sort_ratio
    ...    )
    ...    if best_match and best_match[1] >= threshold:
    ...        return best_match[0]
    ...    return None

```

```

...# Create location strings for matching
...fossil_locations = fossil_list.apply(
...    lambda x: f"{x['PROVINCE']}-{x['DISTRICT']}-{x['TAMBON']}", axis=1
...).unique()
...
...jurassic_locations = jurassic_data.apply(
...    lambda x: f"{x['PROVINCE']}-{x['DISTRICT']}-{x['TAMBON']}", axis=1
...).unique()

...print("\nUnique locations before fuzzy matching:")
...print(f"Fossil locations: {len(fossil_locations)}")
...print(f"Jurassic locations: {len(jurassic_locations)}")

...# Create mapping dictionary using fuzzy matching
...location_mapping = {}
...for fossil_loc in fossil_locations:
...    match = process.extractOne(
...        fossil_loc,
...        jurassic_locations,
...        scorer=fuzz.token_sort_ratio
...    )
...    if match and match[1] >= 80: # 80% similarity threshold
...        location_mapping[fossil_loc] = match[0]

...print("\nFuzzy matching results:")
...print(f"Matched locations: {len(location_mapping)}")

```

```

....# Apply mapping to fossil data
....fossil_list['location_key'] = fossil_list.apply(
.....lambda x: f"{x['PROVINCE']}-{x['DISTRICT']}-{x['TAMBOM']}", axis=1
....).map(location_mapping)

....jurassic_data['location_key'] = jurassic_data.apply(
.....lambda x: f"{x['PROVINCE']}-{x['DISTRICT']}-{x['TAMBON']}", axis=1
....)

....# Perform merge using matched locations
....fuzzy_merged = pd.merge(
.....fossil_list,
.....jurassic_data,
.....on='location_key',
.....how='left',
.....suffixes=('_fossil', '_site')
....)

....# Analyze results
....print("\nMerge results:")
....print(f"Total rows after fuzzy merge: {len(fuzzy_merged)}")
....
....matched_rows = fuzzy_merged[fuzzy_merged['SITE_ID'].notna()]
....unmatched_rows = fuzzy_merged[fuzzy_merged['SITE_ID'].isna()]
....
....print(f"Matched rows: {len(matched_rows)}")
....print(f"Unmatched rows: {len(unmatched_rows)}")

....# Sample of fuzzy matches
....print("\nSample of successful fuzzy matches:")
....sample_matches = matched_rows[['PROVINCE_fossil', 'DISTRICT_fossil', 'TAMBOM',
.....'PROVINCE_site', 'DISTRICT_site', 'TAMBON']].head()
....print(sample_matches)

....# Sample of unmatched locations
....print("\nSample of unmatched locations:")
....unmatched_locations = unmatched_rows[['PROVINCE_fossil', 'DISTRICT_fossil', 'TAMBOM']].head()
....print(unmatched_locations)

....# Export results
....fuzzy_merged.to_csv('fuzzy_merged_data.csv', index=False)
....matched_rows.to_csv('fuzzy_matched.csv', index=False)
....unmatched_rows.to_csv('fuzzy_unmatched.csv', index=False)

....return {
.....'merged': fuzzy_merged,
.....'matched': matched_rows,
.....'unmatched': unmatched_rows,
.....'mapping': location_mapping
....}

```

```

# Additional function to analyze mapping quality
def analyze_mapping_quality(mapping):
    print("\nAnalyzing mapping quality:")
    ....
    similarity_scores = []
    for source, target in mapping.items():
        score = fuzz.token_sort_ratio(source, target)
        similarity_scores.append({
            'source': source,
            'target': target,
            'similarity': score
        })
    ....
    df_scores = pd.DataFrame(similarity_scores)
    ....
    print("\nSimilarity score distribution:")
    print(df_scores['similarity'].describe())
    ....
    print("\nSample of high similarity matches (>95%):")
    print(df_scores[df_scores['similarity'] > 95].head())
    ....
    print("\nSample of lower similarity matches (80-85%):")
    print(df_scores[(df_scores['similarity'] >= 80) &
                    (df_scores['similarity'] < 85)].head())

# Run the fuzzy merge
results = fuzzy_merge_locations()

# Analyze the mapping quality
analyze_mapping_quality(results['mapping'])

```

ผลลัพธ์:

```
Loading data...

Initial data:
Fossil list: 7198 rows
Jurassic data: 520 rows

Unique locations before fuzzy matching:
Fossil locations: 455
Jurassic locations: 279

Fuzzy matching results:
Matched locations: 121

Merge results:
Total rows after fuzzy merge: 15409
Matched rows: 12016
Unmatched rows: 3393
```

Sample of successful fuzzy matches:

	PROVINCE_fossil	DISTRICT_fossil	TAMBON	PROVINCE_site	DISTRICT_site \
4	พังงา	เกาะยาว	เกาะยาวใหญ่	พังงา	เกาะยาว
5	พังงา	เกาะยาว	เกาะยาวใหญ่	พังงา	เกาะยาว
6	พังงา	เกาะยาว	เกาะยาวใหญ่	พังงา	เกาะยาว
7	พังงา	เกาะยาว	เกาะยาวใหญ่	พังงา	เกาะยาว
8	พังงา	เกาะยาว	เกาะยาวใหญ่	พังงา	เกาะยาว

TAMBON

4	เกาะยาวน้อย
5	เกาะยาวน้อย
6	เกาะยาวน้อย
7	เกาะยาวน้อย
8	เกาะยาวน้อย

Sample of unmatched locations:

	PROVINCE_fossil	DISTRICT_fossil	TAMBON
0	ตรัง	กันตัง	เกาะลิบง
1	ตรัง	กันตัง	เกาะลิบง
2	ตรัง	กันตัง	เกาะลิบง
3	ตรัง	กันตัง	เกาะลิบง
14	ตรัง	กันตัง	เกาะลิบง

Analyzing mapping quality:

Similarity score distribution:

count	121.000000
mean	93.652893
std	8.041259
min	80.000000
25%	85.000000
50%	100.000000
75%	100.000000
max	100.000000

Name: similarity, dtype: float64

Sample of high similarity matches (>95%):

	source	target	similarity
1	พังงา-เกาะยาว-เกาะยาวน้อย	พังงา-เกาะยาว-เกาะยาวน้อย	100
2	กระบี่-เมืองกระบี่-อ่าวนาง	กระบี่-เมืองกระบี่-อ่าวนาง	100
3	ชัยภูมิ-คอนสาร-ทุ่งลุยลาย	ชัยภูมิ-คอนสาร-ทุ่งลุยลาย	100
4	ลำปาง-แม่เมะ-แม่เมะ	ลำปาง-แม่เมะ-แม่เมะ	100
8	กาญจนบุรี-สังขละบุรี-ไล่โว่	กาญจนบุรี-สังขละบุรี-ไล่โว่	100

Sample of lower similarity matches (80-85%):

	source	target \	similarity
5	นครราชสีมา-เฉลิมพระเกียรติ-ท่าช้าง	นครราชสีมา-เฉลิมพระเกียรติ-พระพุทธ	82
7	กาญจนบุรี-สังขละบุรี-ไม่ระบุ	กาญจนบุรี-สังขละบุรี-ไล่โว่	82
9	กระบี่-เมืองกระบี่-ไม่ระบุ	กระบี่-เมืองกระบี่-กระบี่น้อย	80
11	เพชรบุรี-เขาย้อย-ไม่ระบุ	เพชรบุรี-เขาย้อย-สระพัง	82
12	ราชบุรี-เมืองราชบุรี-ไม่ระบุ	ราชบุรี-เมืองราชบุรี-ดอนแร่	82

3.3.2.3 การเชื่อมโยงข้อมูลและผลลัพธ์

หลังจากออกแบบกลยุทธ์การเชื่อมโยงข้อมูล การศึกษานี้ได้ดำเนินการเชื่อมโยงข้อมูลจากชุดข้อมูลทั้งสองแหล่ง และได้ผลลัพธ์ดังนี้:

1. ผลลัพธ์จากการเชื่อมโยงแบบแม่นยำ (Exact Matching)

- **Detailed Matching:** สามารถเชื่อมโยงได้ **8,756** รายการ ซึ่งมีความแม่นยำสูงสุด เนื่องจากการจับคู่ทั้งในระดับจังหวัด อำเภอ และตำบล
- **Medium Matching:** สามารถเชื่อมโยงได้ **29,323** รายการ ซึ่งครอบคลุมข้อมูลเพิ่มขึ้นจาก Detailed Matching โดยเชื่อมโยงในระดับจังหวัดและอำเภอ
- **Broad Matching:** สามารถเชื่อมโยงได้มากถึง **132,317** รายการ ซึ่งเป็นการเชื่อมโยงที่กว้างที่สุด โดยพิจารณาเพียงการจับคู่ในระดับจังหวัดเท่านั้น

2. ผลลัพธ์จากการเชื่อมโยงแบบคลุมเครือ (Fuzzy Matching)

การเชื่อมโยงแบบคลุมเครือสามารถเชื่อมโยงข้อมูลได้ 12,016 รายการ การเชื่อมโยงแบบนี้ช่วยให้สามารถจับคู่ข้อมูลที่มีความแตกต่างเล็กน้อยในการสะกดชื่อสถานที่ เช่น:

- "ตำบลนาบอน" กับ "นาบอน"
- "อำเภอวังสะพุง" กับ "วังสะพุง"
- "จังหวัดกาฬสินธุ์" กับ "กาฬสินธุ์"

3. การวิเคราะห์การกระจายตัวของข้อมูลหลังการเชื่อมโยง

หลังจากการเชื่อมโยงข้อมูลในแต่ละระดับ การศึกษานี้ได้วิเคราะห์การกระจายตัวของข้อมูลเพื่อทำความเข้าใจลักษณะของชุดข้อมูลที่ได้:

- การกระจายตัวตามจังหวัด:
 - ในชุดข้อมูล Detailed Matching จังหวัดที่มีการเชื่อมโยงมากที่สุด 5 อันดับแรก ได้แก่ กาฬสินธุ์, เลย, ชัยภูมิ, ขอนแก่น, และตาก

- ในชุดข้อมูล Medium Matching และ Broad Matching จำนวนรายการในจังหวัดต่างๆ เพิ่มขึ้นตามความกว้างของการเชื่อมโยง โดยยังคงมีการกระจุกตัวในภาคตะวันออกเฉียงเหนือ
- การกระจายตัวตามประเภทซากดึกดำบรรพ์:
 - Invertebrate (สัตว์ไม่มีกระดูกสันหลัง): มีสัดส่วนสูงที่สุดในทุกชุดข้อมูล
 - Vertebrate (สัตว์มีกระดูกสันหลัง): มีสัดส่วนรองลงมา
 - Plant (พืช): มีสัดส่วนอันดับสาม
 - Other และ Trace: มีสัดส่วนน้อยที่สุด
- การวิเคราะห์ข้อมูลที่เพิ่มเติมได้:
 - ข้อมูลยุคทางธรณีวิทยา (PERIODFROM): ในชุดข้อมูล Detailed Matching สามารถเติมเต็มข้อมูลยุคทางธรณีวิทยาได้ประมาณ 19.5% ของรายการทั้งหมด
 - ข้อมูลชั้นหิน (FORMATION): สามารถเติมเต็มข้อมูลชั้นหินได้ในบางรายการ

3.3.3 การเตรียมข้อมูล (Data Preparation) - ส่วนที่ 2:

การเติมเต็มค่าที่ขาดหายหลังจากการเชื่อมโยงข้อมูลแล้ว ยังพบว่ามีค่าที่ขาดหายในหลายคอลัมน์ที่สำคัญ โดยเฉพาะอย่างยิ่ง PERIODFROM ซึ่งมีความสำคัญต่อการวิเคราะห์ทางธรณีวิทยาและบรรพชีวินวิทยา การศึกษานี้จึงได้พัฒนาเทคนิคการเติมเต็มค่าที่ขาดหาย 3 วิธี เพื่อเพิ่มความสมบูรณ์ของข้อมูล

3.3.3.1 การเติมเต็มค่าที่ขาดหายโดยใช้ความคล้ายคลึงของชื่อสปีชีส์ (fill_period_by_similarity)

วิธีการนี้อาศัยหลักการค้นหาชื่อสปีชีส์ที่มีความคล้ายคลึงกันด้วยเทคนิค Fuzzy Matching เพื่อเติมเต็มค่า PERIODFROM ที่ขาดหาย โดยสมมติว่าสปีชีส์ที่มีชื่อคล้ายกันมักมีอายุทางธรณีวิทยาใกล้เคียงกัน

ขั้นตอนในการเติมเต็มค่าที่ขาดหายโดยใช้ความคล้ายคลึงของชื่อสปีชีส์:

1. สร้าง Dictionary ของชื่อสปีชีส์และยุค: รวบรวมชื่อสปีชีส์ที่มีข้อมูล PERIODFROM และสร้างพจนานุกรมที่เก็บยุคที่พบมากที่สุดของแต่ละสปีชีส์
2. ค้นหาสปีชีส์ที่คล้ายกัน: สำหรับสปีชีส์ที่ไม่มีค่า PERIODFROM ใช้เทคนิค Fuzzy Matching เพื่อค้นหาสปีชีส์ที่มีชื่อคล้ายกันที่สุด โดยมีเกณฑ์ความคล้ายขั้นต่ำ 80%
3. เติมเต็มค่าที่ขาดหาย: เติมเต็มค่า PERIODFROM ที่ขาดหายด้วยค่าจากสปีชีส์ที่คล้ายกันที่สุด

4. เติมเต็มค่าที่ขาดหาย: เติมเต็มค่า PERIODFROM ที่ขาดหายด้วยค่า mode ที่คำนวณได้จากกลุ่มตำแหน่งเดียวกัน

ตัวอย่างโค้ดการเติมเต็มค่าที่ขาดหายโดยใช้ความคล้ายคลึงของชื่อสปีชีส์ (fill_period_by_similarity)

```
import pandas as pd
from thefuzz import fuzz
from thefuzz import process

def fill_period_by_similarity(df):
    """# สร้าง dictionary ของ species ที่มี PERIODFROM
    """
    known_periods = df[df['PERIODFROM'].notna()].groupby('SCI_NAME')['PERIODFROM'].agg(lambda x: x.value_counts().index[0]).to_dict()
    """
    """
    """# ฟังก์ชันหา most similar species
    """
    def find_most_similar_species(species):
        """
        """
        if pd.isna(species):
            """
            """
            return None
        """
        """
        """# หาชื่อ species ที่คล้ายที่สุดจาก known_periods
        """
        matches = process.extractBests(
            """
            """
            species,
            """
            """
            known_periods.keys(),
            """
            """
            scorer=fuzz.token_sort_ratio,
            """
            """
            score_cutoff=80 """# ความคล้ายขั้นต่ำ 80%
            """
            """
            """
        """
        """
        """
        if matches:
            """
            """
            return known_periods[matches[0][0]]
        """
        """
        """
        return None
    """
    """
    """# เติม PERIODFROM สำหรับแถวที่เป็น NaN
    """
    df_filled = df.copy()
    """
    """
    mask = df_filled['PERIODFROM'].isna()
    """
    """
    df_filled.loc[mask, 'PERIODFROM'] = df_filled.loc[mask, 'SCI_NAME'].apply(find_most_similar_species)
    """
    """
    """
    """
    return df_filled
```

3.3.3.2 การเติมเต็มค่าที่ขาดหายโดยใช้ข้อมูลอนุกรมวิธาน (fill_period_by_taxonomy)

วิธีการนี้อาศัยหลักการแยก Genus จากชื่อวิทยาศาสตร์ (SCI_NAME) และใช้ข้อมูลการกระจายตัวของยุคทางธรณีวิทยาในแต่ละ Genus เพื่อเติมเต็มค่าที่ขาดหาย

ขั้นตอนในการเติมเต็มค่าที่ขาดหายโดยใช้ข้อมูลอนุกรมวิธาน:

1. แยก Genus จากชื่อวิทยาศาสตร์: ดึงคำแรกจากชื่อวิทยาศาสตร์ (SCI_NAME) เพื่อระบุ Genus ของแต่ละตัวอย่าง
2. สร้าง Dictionary ของ Genus และยุค: รวบรวม Genus ที่มีข้อมูล PERIODFROM และสร้างพจนานุกรมที่เก็บยุคที่พบมากที่สุดของแต่ละ Genus
3. เติมเต็มค่าที่ขาดหาย: เติมเต็มค่า PERIODFROM ที่ขาดหายโดยใช้ข้อมูลยุคจาก Genus เดียวกัน

ตัวอย่างโค้ดการเติมเต็มค่าที่ขาดหายโดยใช้ความสัมพันธ์ทางอนุกรมวิธาน

```
def fill_period_by_taxonomy(df):
    """# แยก genus จาก SCI_NAME
    """
    df['GENUS'] = df['SCI_NAME'].str.split().str[0]
    """
    """
    """# สร้าง dictionary ของ genus และ period
    """
    genus_periods = df[df['PERIODFROM'].notna()].groupby('GENUS')['PERIODFROM'].agg(lambda x: x.value_counts().index[0]).to_dict()
    """
    """
    """# เติม PERIODFROM โดยใช้ genus
    """
    df_filled = df.copy()
    """
    mask = df_filled['PERIODFROM'].isna()
    """
    df_filled.loc[mask, 'PERIODFROM'] = df_filled.loc[mask, 'GENUS'].map(genus_periods)
    """
    """
    return df_filled
```

3.3.3.3 การเติมเต็มค่าที่ขาดหายแบบผสมผสาน (fill_period_hybrid)

วิธีการนี้เป็นการผสมผสานเทคนิคต่างๆ เข้าด้วยกัน โดยใช้ทั้งการจับคู่แบบเฉพาะเจาะจง (Exact Matching) การจับคู่แบบคล้ายคลึง (Similarity Matching) และการจับคู่ระดับ Genus พร้อมทั้งเพิ่มการประเมินความเชื่อมั่นของการเติมเต็ม

ขั้นตอนในการเติมเต็มค่าที่ขาดหายแบบผสมผสาน:

1. สร้าง Dictionary ของชื่อสปีชีส์และยุค: รวบรวมชื่อสปีชีส์ที่มีข้อมูล PERIODFROM และสร้างพจนานุกรมสำหรับการจับคู่แบบเฉพาะเจาะจง
2. เติมเต็มค่าที่ขาดหายโดยใช้หลายขั้นตอน:
 - ขั้นตอนที่ 1: ตรวจสอบการจับคู่แบบเฉพาะเจาะจง (Exact Match) กับชื่อสปีชีส์
 - ขั้นตอนที่ 2: ถ้าไม่พบการจับคู่แบบเฉพาะเจาะจง ใช้การจับคู่แบบคล้ายคลึง (Similarity Matching) ด้วยเกณฑ์ความคล้าย 85%
 - ขั้นตอนที่ 3: ถ้ายังไม่พบการจับคู่ ใช้การจับคู่ระดับ Genus โดยค้นหาสปีชีส์อื่นที่มี Genus เดียวกัน
3. คำนวณค่าความเชื่อมั่น (Confidence Score): กำหนดค่าความเชื่อมั่นให้กับข้อมูลที่เติมเต็ม โดยให้คะแนนสูงกับข้อมูลดั้งเดิม (1.0) และคะแนนปานกลางกับข้อมูลที่เติมเต็ม (0.7)

ตัวอย่างโค้ดการเติมเต็มค่าที่ขาดหายโดยใช้การเติมเต็มค่าที่ขาดหายแบบผสมผสาน

```

def fill_period_hybrid(df):
    """#1. ใช้ exact species match ก่อน
    species_periods = df[df['PERIODFROM'].notna()].groupby('SCI_NAME')['PERIODFROM'].agg(lambda x: x.value_counts().index[0]).to_dict()
    """
    """#2. ใช้ similarity matching
    def get_period(row):
        if pd.isna(row['PERIODFROM']):
            """# ตรวจสอบว่า SCI_NAME เป็น NaN หรือไม่
            if pd.isna(row['SCI_NAME']):
                return None
            """
            """# Check exact species match
            if row['SCI_NAME'] in species_periods:
                return species_periods[row['SCI_NAME']]
            """
            try:
                """# Check similarity
                matches = process.extractBests(
                    str(row['SCI_NAME']), # แปลงเป็น string
                    [str(k) for k in species_periods.keys()], # แปลง keys เป็น string
                    scorer=fuzz.token_sort_ratio,
                    score_cutoff=85
                )
                if matches:
                    return species_periods[matches[0][0]]
            """
            """# Check genus level
            if isinstance(row['SCI_NAME'], str) and '.' in row['SCI_NAME']:
                genus = row['SCI_NAME'].split()[0]
                genus_matches = [s for s in species_periods.keys() if isinstance(s, str) and s.startswith(genus)]
                if genus_matches:
                    return species_periods[genus_matches[0]]
            """
        except Exception as e:
            print(f"Error processing row: {row['SCI_NAME']}, Error: {e}")
            return None
    """
    return row['PERIODFROM']
    """
df_filled = df.copy()
df_filled['PERIODFROM'] = df_filled.apply(get_period, axis=1)
"""# Add confidence score
df_filled['PERIOD_CONFIDENCE'] = df_filled.apply(
    lambda x: 1.0 if x['PERIODFROM'] == df.loc[x.name, 'PERIODFROM']
    else 0.7 if not pd.isna(x['PERIODFROM'])
    else 0.0,
    axis=1
)
return df_filled

```

```

# ทดสอบการใช้งาน
try:
    df_filled = fill_period_hybrid(df)
    ...
    # แสดงผลลัพธ์
    print("\nSample of filled data:")
    print(df_filled[['SCI_NAME', 'PERIODFROM', 'PERIOD_CONFIDENCE']].head(10))
    ...
    # สรุปผล
    print("\nFilling summary:")
    print(f"Total rows: {len(df_filled)}")
    print(f"Filled rows: {df_filled['PERIODFROM'].notna().sum()}")
    print(f"Still missing: {df_filled['PERIODFROM'].isna().sum()}")
    ...
    # การกระจายของ confidence scores
    print("\nConfidence score distribution:")
    print(df_filled['PERIOD_CONFIDENCE'].value_counts())
    ...
except Exception as e:
    print(f"Error in main execution: {e}")

df_filled_by_hybrid = fill_period_hybrid(df)

```

3.3.3.4 การประเมินผลและการรวมผลลัพธ์จากการเติมเต็มค่าที่ขาดหาย

หลังจากดำเนินการเติมเต็มค่าที่ขาดหายด้วยทั้ง 3 วิธี การศึกษานี้ได้ประเมินผลลัพธ์ของแต่ละวิธี และรวมผลลัพธ์เข้าด้วยกันเพื่อให้ได้ข้อมูลที่สมบูรณ์ที่สุด การศึกษานี้ได้พัฒนากลยุทธ์ในการรวมผลลัพธ์จากการเติมเต็มค่าที่ขาดหายทั้ง 3 วิธี โดยพิจารณาความน่าเชื่อถือของแต่ละวิธีและความสอดคล้องระหว่างผลลัพธ์

ผลการเติมเต็มค่า PERIODFROM ที่ขาดหายพบว่า:

- วิธีการเติมเต็มโดยใช้ความสัมพันธ์ทางอนุกรมวิธานให้ผลลัพธ์ที่มีความน่าเชื่อถือสูงที่สุด เนื่องจากมีความสอดคล้องกับทฤษฎีทางบรรพชีวินวิทยา และได้เลือกการเติมเต็มค่าที่ขาดหายแบบผสมผสาน (fill_period_hybrid) ซึ่งเป็นการผสมผสานเทคนิคต่าง ๆ เข้าด้วยกัน โดยใช้ทั้งการจับคู่แบบเฉพาะเจาะจง (Exact Matching) การจับคู่แบบคล้ายคลึง (Similarity Matching) และการจับคู่ระดับ Genus พร้อมทั้งเพิ่มการประเมินความเชื่อมั่นของการเติมเต็ม
- ผลการเติมเต็มค่า PERIODFROM ที่ขาดหายได้เพิ่มความสมบูรณ์ของข้อมูลอย่างมีนัยสำคัญ ทำให้ข้อมูลมีความพร้อมสำหรับการวิเคราะห์ในขั้นตอนต่อไป

Code ตัวอย่างการตรวจสอบการเติมเต็มค่า Period:

```
# ตรวจสอบการกระจายตัวของ PERIODFROM
def analyze_period_distribution(df_original, df_filled):
    print("Original PERIODFROM distribution:")
    print(df_original['PERIODFROM'].value_counts(dropna=False))
    print("\nFilled PERIODFROM distribution:")
    print(df_filled['PERIODFROM'].value_counts(dropna=False))
    print("\nFill rate:",
          (1 - df_filled['PERIODFROM'].isna().sum() / len(df_filled)) * 100,
          "%")

# ตรวจสอบความสมเหตุสมผล
def validate_filling(df_filled):
    # Check genus consistency
    genus_periods = df_filled.groupby(df_filled['SCI_NAME'].str.split().str[0])['PERIODFROM'].nunique()
    print("\nGenera with multiple periods:")
    print(genus_periods[genus_periods > 1])

# ตรวจสอบการกระจายตัวของ PERIODFROM
analyze_period_distribution(df, df_filled_by_hybrid)
```

ผลลัพธ์:

```

Original PERIODFROM distribution:
PERIODFROM
NaN          9692
เพชรเมียน          1443
ศรีเทเขียสดอนตัน          491
เพชรเมียนตอนตัน          161
จรัสสิกตอนปลาย          138
ออร์โดวิเซียน          35
ควอเทอร์นารี          12
ไทรแอสสิกตอนปลาย          11
จรัสซิก          7
แคมเบรียนตอนปลาย          5
ศรีเทเขียสดอนตัน          4
ออร์โดวิเซียน-ไซลูเรียน          4
ไทรแอสสิก          4
ยออร์โดวิเซียน          3
จรัสสิกตอนปลายถึงศรีเทเขียสดอนตัน          2
ศรีเทเขียส          2
จรัสซิกตอนกลาง-ตอนปลาย          1
ไทรแอสซิกตอนตัน          1
Name: count, dtype: int64

Filled PERIODFROM distribution:
PERIODFROM
ศรีเทเขียสดอนตัน          5483
...
ไทรแอสซิกตอนตัน          1
Name: count, dtype: int64

Fill rate: 82.02396804260985 %

```

3.3.4 การสร้างคุณลักษณะใหม่ (Feature Engineering)

การสร้างคุณลักษณะใหม่ (Feature Engineering) เป็นขั้นตอนสำคัญหลังจากการเชื่อมโยงข้อมูลและเติมเต็มค่าที่ขาดหาย เพื่อเพิ่มประสิทธิภาพในการวิเคราะห์และสร้างแบบจำลองที่มีความแม่นยำสูง ในกระบวนการนี้ จะมีการสร้างคุณลักษณะใหม่ที่มีความหลากหลายและครอบคลุมมิติต่างๆ ของข้อมูลซากดึกดำบรรพ์ ประกอบด้วยคุณลักษณะเชิงพื้นที่ เชิงเวลา คุณลักษณะจากข้อมูลภาพถ่ายดาวเทียม คุณลักษณะเชิงข้อความ คุณลักษณะเชิงสถิติและความถี่ และคุณลักษณะเชิงกลุ่ม

3.3.4.1 การสร้างคุณลักษณะเชิงพื้นที่ (Spatial Features)

คุณลักษณะเชิงพื้นที่เป็นการแปลงและปรับแต่งข้อมูลตำแหน่งทางภูมิศาสตร์ให้เหมาะสมกับการวิเคราะห์ โดยได้ดำเนินการดังนี้:

1. การแปลงพิกัด UTM เป็นพิกัดละติจูดและลองจิจูด

เนื่องจากข้อมูลเดิมอยู่ในรูปแบบพิกัด UTM (Universal Transverse Mercator) จึงต้องแปลงให้อยู่ในรูปแบบของพิกัดละติจูดและลองจิจูด เพื่อให้ง่ายต่อการแสดงผลและการวิเคราะห์ทางภูมิศาสตร์

ขั้นตอนในการแปลงพิกัด:

- ตรวจสอบและแก้ไขค่า UTM ที่อยู่นอกช่วงที่ควรจะเป็น โดยเฉพาะค่า Easting ที่มีค่ามากเกินไป (> 999,999) ซึ่งอาจเกิดจากความผิดพลาดในการระบุจุดพิกัด
- แปลงพิกัด UTM เป็นละติจูดและลองจิจูด โดยใช้ข้อมูล UTM Easting, UTM Northing และ UTM Zone
- ในกรณีที่พบค่าพิกัดที่ขาดหายหรือแปลงไม่ได้ จะทำการเติมเต็มด้วยค่าเฉลี่ยของพิกัดในจังหวัดเดียวกัน

ตัวอย่างจากโค้ดการแปลงพิกัด:

```
# ตรวจสอบและแก้ไขค่า UTM ที่อยู่นอกช่วง
df['UTM_E_fixed'] = df['UTM_E'].copy()
df['UTM_N_fixed'] = df['UTM_N'].copy()

# แก้ไขค่า Easting ที่มากเกินไป
large_easting = (df['UTM_E'] > 999999) & (df['UTM_E'].notna())
df.loc[large_easting, 'UTM_E_fixed'] = df.loc[large_easting, 'UTM_E'] / 10

# แปลงพิกัด UTM เป็นละติจูดและลองจิจูด
def utm_to_latlon(row):
    try:
        utm_e = float(row['UTM_E_fixed'])
        utm_n = float(row['UTM_N_fixed'])
        zone_text = str(row['ZONE']).strip()

        # แยกส่วนที่เป็นตัวเลขและซีกโลก
        zone_num = int(re.findall(r'\d+', zone_text)[0])
        hemisphere = 'N' if 'N' in zone_text else 'S'

        # ตรวจสอบว่าอยู่ในช่วงที่ถูกต้อง
        if 100000 <= utm_e <= 999999 and utm_n > 0:
            lat, lon = utm_to_latlon(utm_e, utm_n, zone_num, hemisphere)
            return pd.Series({'latitude': lat, 'longitude': lon})
        else:
            return pd.Series({'latitude': np.nan, 'longitude': np.nan})
    except Exception:
        return pd.Series({'latitude': np.nan, 'longitude': np.nan})
```

ผลลัพธ์จากการดำเนินการ:

- สามารถแปลงพิกัด UTM เป็นละติจูดและลองจิจูดได้สำเร็จทั้งหมด 9,817 รายการ
- มีการแก้ไขค่า Easting ที่อยู่นอกช่วงปกติจำนวน 3,224 รายการ
- ไม่มีพิกัดที่ขาดหายหลังการแปลงและเติมเต็ม

2. การคำนวณพิกัดในยุคแพนเจีย (Pangaea Coordinates)

การเพิ่มข้อมูลพิกัดในยุคแพนเจีย (Pangaea) ช่วยให้เราสามารถวิเคราะห์การกระจายตัวของซากดึกดำบรรพ์ในบริบทของธรณีวิทยาโบราณได้ โดยใช้ข้อมูลอายุทางธรณีวิทยาและพิกัดปัจจุบันในการคำนวณ

ขั้นตอนในการคำนวณพิกัดแพนเจีย:

- เพิ่มข้อมูลอายุในหน่วยล้านปี (Million Years Ago) ให้กับแต่ละยุคทางธรณีวิทยา
- ใช้ไลบรารี PyGPlates ในการคำนวณพิกัดย้อนกลับไปในยุคแพนเจีย โดยใช้แบบจำลองการเคลื่อนที่ของแผ่นเปลือกโลก โดยใช้ร่วมกับไฟล์ .rot (เกี่ยวข้องกับ rotation (การหมุนของวัตถุหรือพิกัดในเชิงเรขาคณิตหรือภูมิศาสตร์)) และ .gpml (GPML: Gaussian Process Markup Language เป็นไฟล์ XML ที่ออกแบบมาเพื่อเก็บข้อมูลหรือโมเดลเกี่ยวกับ Gaussian Process ซึ่งเป็นเทคนิคหนึ่งของ Machine Learning)
- ระบุแผ่นเปลือกโลกที่เหมาะสมสำหรับประเทศไทย (Plate ID: 619)
- ในกรณีที่ไม่สามารถคำนวณได้ด้วย PyGPlates จะใช้วิธีการประมาณค่าแบบง่าย

ผลลัพธ์จากการคำนวณพิกัดแพนเจีย:

- สามารถคำนวณพิกัดแพนเจียได้จริงถึง 9,813 รายการ คิดเป็น 99.96% ของข้อมูลทั้งหมด
- พิกัดที่ได้มีความแม่นยำสูงเนื่องจากใช้แบบจำลองทางธรณีวิทยา
- ข้อมูลนี้ช่วยให้สามารถวิเคราะห์การกระจายตัวของซากดึกดำบรรพ์ในบริบทของยุคธรณีกาลได้ดียิ่งขึ้น

ตัวอย่าง Code และแหล่งข้อมูล Rotation และ Gaussian Process Markup Language

```

try:
    import pygplates
    print("✅ Successfully imported pygplates")
except ImportError:
    print("❌ pygplates not available, using approximation method")
    df = calculate_pangaea_coordinates_simple(df)
    return df, 0.0  # Return ratio of 0 -- all approximated

if not os.path.exists(models_dir):
    print(f"❌ Directory {models_dir} not found")
    df = calculate_pangaea_coordinates_simple(df)
    return df, 0.0  # Return ratio of 0 -- all approximated

rotation_files = [os.path.join(models_dir, f) for f in os.listdir(models_dir) if f.endswith('.rot')]
feature_files = [os.path.join(models_dir, f) for f in os.listdir(models_dir) if f.endswith('.gpml')]

```

ถ้าไม่รองรับ pygplates สามารถประมาณค่าเองได้:

```

def calculate_pangaea_coordinates_simple(df):
    """
    Approximate Pangaea coordinates using a simple transformation.
    This is a fallback method if GPlates reconstruction fails.

    Args:
        df (pd.DataFrame): DataFrame with ['latitude', 'longitude', 'period_age_my'].

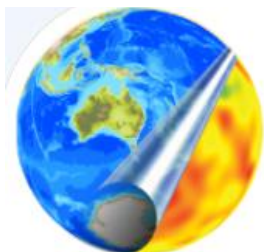
    Returns:
        pd.DataFrame: DataFrame with estimated ['pangaea_lat', 'pangaea_lon'].
    """
    print("🔄 Using simplified approximation for Pangaea coordinates.")

    pangaea_coords = [approximate_pangaea_coords(row['latitude'], row['longitude'], row['period_age_my'])
                      for _, row in df.iterrows()]

    df['pangaea_lat'] = [coord[0] for coord in pangaea_coords]
    df['pangaea_lon'] = [coord[1] for coord in pangaea_coords]
    df['pangaea_coords_approximated'] = 1  # Flag as approximated

    return df

```



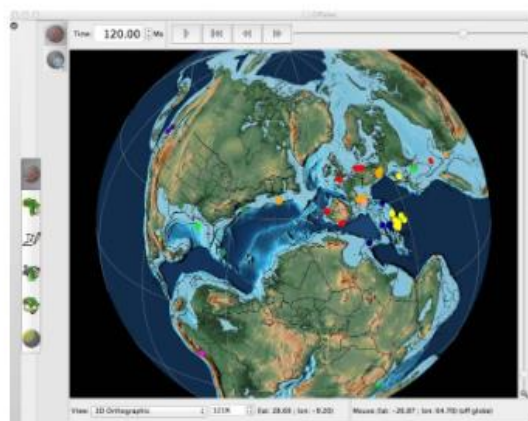
EarthBYTE

Building a Virtual Earth

Home Research ▾ People Publications ▾ Resources ▾ Training & Outreach ▾ News ▾

PALEOMAP PaleoAtlas for GPlates

19 April, 2016 by Carmen Braz



The PALEOMAP PaleoAtlas for GPlates consists of 91 paleogeographic maps spanning the Phanerozoic and late Neoproterozoic. The PaleoAtlas can be directly loaded into GPlates as a *Time Dependent Raster* file. The paleogeographic maps in the PaleoAtlas illustrate the ancient configuration of the ocean basins and continents, as well as important topographic and bathymetric features such as mountains, lowlands, shallow sea, continental shelves, and deep oceans. This tutorial also describes how the maps in the PaleoAtlas were made, documents the sources of information used

to make the paleogeographic maps, and provides instructions how to plot user-defined paleodata on the paleogeographic maps using the program *PaleoDataPlotter*.

แหล่งที่มา: [PALEOMAP PaleoAtlas for GPlates – EarthByte](#)

Index of /webdav/ftp/Data_Collections

Name	Size	Description
 Parent Directory	-	
 Abyssay_Hills_Roughness/	-	
 Barnett-Moore_etal_Iceland_Plume_dynamic_topo_2017_SolidEarth/	-	
 Cao_etal_2020_G3/	-	
 Cao_etal_2023/	-	
 Cao_etal_2023_Opt/	-	
 Chandra_etal_2021_EnvModSoftware/	-	
 Clennett_etal_2020_G3/	-	
 Dutkiewicz_Muller_2021_Geology/	-	
 Dutkiewicz_Muller_2022_G3/	-	
 Dutkiewicz_Muller_2022_Geology/	-	
 Dutkiewicz_etal_2016_G3/	-	
 Dutkiewicz_etal_2017_G3/	-	
 Dutkiewicz_etal_2019_Geology/	-	
 Flament_2018_GJI/	-	
 Flament_etal_2017_NatComms/	-	
 Gurnis_etal_2012_CompGeosci/	-	
 Haas_etal_2022_Tectonics/	-	
 Hu_etal_2022_NatureGeosci/	-	
 Johansson_etal_2018_GRL/	-	
 Manley_etal_2020_FrontEarthSci/	-	
 Merdith_etal_2021_ESR/	-	
 Muller2008/	-	
 Muller2022/	-	
 Muller_Dutkiewicz_2018_SciAdv/	-	
 Muller_etal_2008_G3/	-	
 Muller_etal_2016_AREPS/	-	
 Muller_etal_2018_GR/	-	
 Muller_etal_2019_Tectonics/	-	
 Muller_etal_2022_Nature/	-	
 Muller_etal_2022_SE/	-	
 Pall_etal_2018_CP/	-	
 Rubey_etal_2017_SolidEarth/	-	
 Scotese2016/	-	

แหล่งที่มา: [Index of /webdav/ftp/Data_Collections](#)

3. การคำนวณระยะทางและความสัมพันธ์เชิงพื้นที่

การเพิ่มข้อมูลระยะทางและความสัมพันธ์เชิงพื้นที่ช่วยให้เข้าใจการกระจายตัวและความสัมพันธ์ระหว่างแหล่งที่พบซากดึกดำบรรพ์ได้ดียิ่งขึ้น

คุณลักษณะที่สร้างขึ้น:

distance_from_centroid: ระยะห่างจากจุดศูนย์กลางของการกระจายตัวในแต่ละจังหวัด

- คำนวณจุดศูนย์กลาง (centroid) ของแต่ละจังหวัดโดยใช้ค่าเฉลี่ยของละติจูดและลองจิจูด
- คำนวณระยะห่างแบบยูคลิเดียน (Euclidean distance) ระหว่างพิกัดของซากดึกดำบรรพ์แต่ละชิ้นกับจุดศูนย์กลางของจังหวัดนั้น

nearest_related_site_km: ระยะทางไปยังแหล่งที่พบซากดึกดำบรรพ์ชนิดเดียวกันที่ใกล้ที่สุด

- จัดกลุ่มข้อมูลตามชื่อวิทยาศาสตร์ (SCI_NAME)
- สำหรับซากดึกดำบรรพ์แต่ละชิ้น คำนวณระยะห่างไปยังซากดึกดำบรรพ์ชนิดเดียวกันทุกชิ้น
- เก็บค่าระยะห่างที่น้อยที่สุดเป็นระยะทางไปยังแหล่งที่ใกล้ที่สุด
- แปลงหน่วยจากองศาเป็นกิโลเมตรโดยประมาณ (1 องศา $\approx$ 111 กิโลเมตร)

ตัวอย่าง Code:

```
# คำนวณระยะทางจากจุดศูนย์กลางของจังหวัด
for province, group in province_groups:
    if len(group) <= 1:
        continue
    # หาจุดศูนย์กลางของจังหวัด
    center_lat = group['latitude'].mean()
    center_lon = group['longitude'].mean()
    # คำนวณระยะห่าง
    for idx in group.index:
        lat = df.loc[idx, 'latitude']
        lon = df.loc[idx, 'longitude']
        # ระยะห่างแบบยูคลิดีเนียน (หน่วยเป็นองศา)
        distance = np.sqrt((lat - center_lat)**2 + (lon - center_lon)**2)
        df.loc[idx, 'distance_from_centroid'] = distance

# คำนวณระยะทางไปยังแหล่งซากดึกดำบรรพ์ชนิดเดียวกันที่ใกล้ที่สุด
for species, group in species_groups:
    if len(group) <= 1:
        continue
    # คำนวณระยะห่างที่น้อยที่สุด
    distances = calc_nearest(group)
    # บันทึกค่าลงในดาต้าเฟรม
    for idx, dist in distances.items():
        df.loc[idx, 'nearest_related_site_deg'] = dist

# แปลงจากองศาเป็นกิโลเมตรโดยประมาณ
df['nearest_related_site_km'] = df['nearest_related_site_deg'] * 111
```

คุณค่าของข้อมูลระยะทางและความสัมพันธ์เชิงพื้นที่:

- **distance_from_centroid**: ช่วยระบุซากดึกดำบรรพ์ที่พบในพื้นที่ห่างไกลจากศูนย์กลาง ซึ่งอาจมีความน่าสนใจทางวิชาการ
- **nearest_related_site_km**: ช่วยวิเคราะห์รูปแบบการกระจายตัวของซากดึกดำบรรพ์ชนิดเดียวกัน และอาจช่วยในการค้นหาแหล่งซากดึกดำบรรพ์ใหม่

3.3.4.2 การสร้างคุณลักษณะเชิงเวลา (Temporal Features)

คุณลักษณะเชิงเวลาช่วยให้สามารถวิเคราะห์การเปลี่ยนแปลงและพัฒนาการของซากดึกดำบรรพ์ตามช่วงเวลาทางธรณีวิทยาได้

1. การแปลงยุคทางธรณีวิทยาเป็นอายุเชิงตัวเลข (period_age_mya)

การแปลงข้อมูลยุคทางธรณีวิทยาที่อยู่ในรูปแบบข้อความให้เป็นค่าตัวเลขที่แสดงอายุในหน่วยล้านปี (Million Years Ago) ทำให้สามารถนำไปใช้ในการวิเคราะห์เชิงปริมาณได้

วิธีการดำเนินการ:

- สร้างพจนานุกรมการแปลงยุคทางธรณีวิทยาเป็นค่าตัวเลขโดยประมาณ
- แปลงข้อมูลยุคทางธรณีวิทยา (PERIODFROM) เป็นค่าอายุในหน่วยล้านปี

ตัวอย่าง Code:

```
# การสร้างพจนานุกรมการแปลงยุคทางธรณีวิทยาเป็นอายุในหน่วยล้านปี
period_ages = {
    'เพอร์เมียน': 299,
    'เพอร์เมียนตอนต้น': 290,
    'ไทรแอสซิก': 251,
    'ไทรแอสซิกตอนต้น': 245,
    'ไทรแอสซิกตอนปลาย': 220,
    'จูแรสซิก': 201,
    'จูแรสซิกตอนปลาย': 160,
    'ครีเทเชียส': 145,
    'ครีเทเชียสตอนต้น': 140,
    'ครีเทเชียสตอนต้น': 140,
    'ออร์โดวิเชียน': 485,
    'ออร์โดวิเชียน': 485,
    'ออร์โดวิเชียน-ไซลูเรียน': 450,
    'แคมเบรียนตอนปลาย': 500,
    'ยออร์โดวิเชียน': 485,
    'จูแรสซิกตอนปลายถึงครีเทเชียสตอนต้น': 150,
}

# แปลงข้อมูลยุคทางธรณีวิทยาเป็นอายุในหน่วยล้านปี
df['period_age_my'] = df['PERIODFROM'].map(period_ages).fillna(0)
```

2. การจัดกลุ่มยุคทางธรณีวิทยาเป็นมหายุค (geological_era)

การจัดกลุ่มยุคทางธรณีวิทยา (Period) เป็นมหายุค (Era) ช่วยให้เห็นภาพรวมของการกระจายตัวของซากดึกดำบรรพ์ในช่วงเวลาที่กว้างขึ้น

ตัวอย่าง Code:

```
# สร้างพจนานุกรมการแปลงยุคทางธรณีวิทยาเป็นมหายุค
period_to_era = {
    'เพอร์เมียน': 'Paleozoic',
    'เพอร์เมียนตอนต้น': 'Paleozoic',
    'ไทรแอสซิก': 'Mesozoic',
    'ไทรแอสซิกตอนต้น': 'Mesozoic',
    'ไทรแอสซิกตอนปลาย': 'Mesozoic',
    'จูแรสซิก': 'Mesozoic',
    'จูแรสซิกตอนปลาย': 'Mesozoic',
    'ครีเทเชียส': 'Mesozoic',
    'ครีเทเชียสตอนต้น': 'Mesozoic',
    'ครีเทเชียสตอนต้น': 'Mesozoic',
    'ออร์โดวิเชียน': 'Paleozoic',
    'ออร์โดวิเชียน': 'Paleozoic',
    'ออร์โดวิเชียน-ไซลูเรียน': 'Paleozoic',
    'แคมเบรียนตอนปลาย': 'Paleozoic',
    'ยออร์โดวิเชียน': 'Paleozoic',
    'จูแรสซิกตอนปลายถึงครีเทเชียสตอนต้น': 'Mesozoic',
}

# สร้างคอลัมน์มหายุค
df['geological_era'] = df['PERIODFROM'].map(period_to_era)
```

3. การดึงปีที่พบซากดึกดำบรรพ์ (discovery_year)

การดึงข้อมูลปีที่พบซากดึกดำบรรพ์จากข้อมูลวันที่ในฐานข้อมูล ช่วยให้สามารถวิเคราะห์แนวโน้มการค้นพบซากดึกดำบรรพ์ตามช่วงเวลาได้

ตัวอย่าง Code:

```
# ดึงปีจากข้อมูลวันที่สร้าง (ถ้ามี)
if 'created_date' in df.columns:
    try:
        # สมมติว่าวันที่อยู่ในรูปแบบมิลลิวินาทีนับจากยุค UNIX Epoch
        df['discovery_year'] = pd.to_datetime(df['created_date'], unit='ms').dt.year
    except Exception:
        # กรณีที่แปลงไม่ได้ ใช้ค่าเริ่มต้น
        df['discovery_year'] = 2020
```

คุณค่าของคุณลักษณะเชิงเวลา:

- **period_age_myra**: ช่วยในการจัดลำดับซากดึกดำบรรพ์ตามอายุและการวิเคราะห์การเปลี่ยนแปลงตามช่วงเวลาทางธรณีวิทยา
- **geological_era**: ช่วยในการวิเคราะห์การกระจายตัวของซากดึกดำบรรพ์ในช่วงเวลาที่กว้างขึ้น
- **discovery_year**: ช่วยในการติดตามประวัติการค้นพบซากดึกดำบรรพ์ในประเทศไทยและอาจสะท้อนให้เห็นพัฒนาการของการศึกษาด้านบรรพชีวินวิทยาในประเทศ

3.3.4.3 การสร้างคุณลักษณะจากข้อมูลภาพถ่ายดาวเทียม (Satellite Features)

การเพิ่มข้อมูลจากภาพถ่ายดาวเทียมช่วยให้สามารถวิเคราะห์ความสัมพันธ์ระหว่างลักษณะทางภูมิศาสตร์และการกระจายตัวของซากดึกดำบรรพ์ได้

1. การดึงข้อมูลดัชนีพืชพรรณ (NDVI) และค่าแถบความถี่

ดัชนีพืชพรรณ (NDVI: Normalized Difference Vegetation Index) และค่าแถบความถี่ต่างๆ จากภาพถ่ายดาวเทียม Sentinel-2 ช่วยให้เข้าใจสภาพแวดล้อมของแหล่งที่พบซากดึกดำบรรพ์

ตัวอย่าง Code ของการดึงข้อมูล NDVI และแถบความถี่ต่าง ๆ:

```
# ส่วนของการดึงข้อมูล NDVI และแถบความถี่ต่าง ๆ
ndvi_evalscript = """
//VERSION=3
function setup(){
  return {
    input: ["B04", "B08"],
    output: {
      bands: 1,
      sampleType: "FLOAT32"
    }
  };
}

function evaluatePixel(sample){
  let ndvi = (sample.B08 - sample.B04) / (sample.B08 + sample.B04);
  return [ndvi];
}
"""

rgb_nir_evalscript = """
//VERSION=3
function setup(){
  return {
    input: ["B04", "B03", "B02", "B08"],
    output: {
      bands: 4
    }
  };
}

function evaluatePixel(sample){
  return [sample.B04, sample.B03, sample.B02, sample.B08];
}
```

คุณลักษณะที่สร้างขึ้น:

- **ndvi**: ดัชนีพืชพรรณที่สะท้อนความหนาแน่นของพืชพรรณในพื้นที่
- **red_band**: ค่าการสะท้อนแสงในแถบสีแดง ช่วยในการวิเคราะห์พืชพรรณและพื้นผิวดิน
- **green_band**: ค่าการสะท้อนแสงในแถบสีเขียว ใช้ในการประเมินความสมบูรณ์ของพืชพรรณ
- **blue_band**: ค่าการสะท้อนแสงในแถบสีน้ำเงิน ช่วยในการวิเคราะห์แหล่งน้ำและคุณภาพน้ำ
- **nir_band**: ค่าการสะท้อนแสงในแถบอินฟราเรดใกล้ ใช้ในการวิเคราะห์พืชพรรณและความชื้นในดิน

2. การสร้างภาพแผนที่และภาพจากดาวเทียม

การสร้างและจัดเก็บภาพแผนที่และภาพจากดาวเทียมของแต่ละตำแหน่งที่พบซากดึกดำบรรพ์ ช่วยในการวิเคราะห์เชิงภาพและการนำเสนอข้อมูล

ตัวอย่าง Code:

```
# สร้างแผนที่แสดงตำแหน่งที่พบซากดึกดำบรรพ์
map_html_path = f'satellite_maps/clean_map_{fossil_id}.html'
create_satellite_only_map(lat, lon, 16, map_html_path)

# บันทึกภาพหน้าจอของแผนที่
map_img_path = f'satellite_maps/map_{fossil_id}.png'
screenshot_success = capture_map_screenshot_simple(map_html_path, map_img_path)

# บันทึกภาพดาวเทียมโดยตรง
direct_path = f'satellite_data/direct_satellite_{fossil_id}.png'
save_array_as_image(rgb_img, direct_path)
```

การเติมเต็มข้อมูลที่ขาดหาย:

- ในกรณีที่ไม่สามารถดึงข้อมูลดาวเทียมจริงได้ จะใช้การจำลองข้อมูลดาวเทียมด้วยการคำนวณค่าจากพิกัดทางภูมิศาสตร์
- เพิ่มตัวแปร `has_real_satellite_data` เพื่อระบุว่าข้อมูลเป็นข้อมูลจริงหรือข้อมูลจำลอง

คุณค่าของคุณลักษณะจากข้อมูลภาพถ่ายดาวเทียม:

- ช่วยในการวิเคราะห์ความสัมพันธ์ระหว่างสภาพแวดล้อมปัจจุบันกับแหล่งที่พบซากดึกดำบรรพ์

- อาจช่วยในการระบุพื้นที่ที่มีความเป็นไปได้สูงในการค้นพบซากดึกดำบรรพ์เพิ่มเติม
- สนับสนุนการวิเคราะห์การเปลี่ยนแปลงของสภาพแวดล้อมในพื้นที่ที่พบซากดึกดำบรรพ์

3.3.4.4 การสร้างคุณลักษณะจากข้อความ (Text Features)

การวิเคราะห์ข้อมูลเชิงข้อความช่วยให้สามารถดึงข้อมูลเชิงคุณภาพจากคำอธิบายซากดึกดำบรรพ์และแหล่งที่พบได้

1. การคำนวณความยาวของข้อความ

การคำนวณความยาวของข้อความในคอลัมน์ต่างๆ ช่วยสะท้อนความละเอียดของข้อมูลที่มีการบันทึก

ตัวอย่าง Code:

```
# คำนวณความยาวของข้อความในคอลัมน์ต่างๆ
text_columns = ['FOS_DES_TH', 'GEO_DES_TH', 'F_PART']
for col in text_columns:
    if col in df.columns:
        df[f'{col}_length'] = df[col].astype(str).str.len()
```

2. การตรวจหาคำสำคัญ (Keyword Detection)

การตรวจหาคำสำคัญที่เกี่ยวข้องกับซากดึกดำบรรพ์ในข้อความคำอธิบาย ช่วยในการจำแนกและวิเคราะห์ประเภทและลักษณะของซากดึกดำบรรพ์

ตัวอย่าง Code:

```
# ตรวจหาคำสำคัญในข้อความคำอธิบาย
fossil_keywords = ['กระดูก', 'ฟัน', 'ฟอสซิล', 'ปะการัง', 'เปลือก', 'แมรคิโอพอด']
for keyword in fossil_keywords:
    df[f'has_{keyword}'] = df['FOS_DES_TH'].astype(str).str.contains(keyword).astype(int)
```

การสร้างตัวแปรเหล่านี้ช่วยให้สามารถระบุลักษณะของซากดึกดำบรรพ์ได้อย่างรวดเร็ว เช่น หากพบคำว่า 'กระดูก' หรือ 'ฟัน' ในคำอธิบาย แสดงว่าน่าจะเป็นซากดึกดำบรรพ์ของสัตว์มีกระดูกสันหลัง เป็นต้น

3. การสร้าง Text Embeddings ด้วย WangChanBERTa

การใช้โมเดลภาษาแบบ Transformer (WangChanBERTa) ซึ่งเป็นโมเดลที่ถูกฝึกฝนมาเฉพาะสำหรับภาษาไทย ในการแปลงข้อความเป็นเวกเตอร์ขนาด 768 มิติ ซึ่งสามารถนำไปใช้ในการวิเคราะห์ความคล้ายคลึงของข้อความและการจำแนกประเภทได้

ตัวอย่าง Code:

```
def get_bert_embeddings(text, max_length=128):
    ...#เตรียมข้อมูลสำหรับโมเดล
    ...inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True, max_length=max_length)
    ...
    ...#ดึง embeddings จากโมเดล
    ...with torch.no_grad():
    ...    outputs = model(**inputs)
    ...
    ...#ใช้ [CLS] token embedding (token แรก) เป็นตัวแทนของทั้งประโยค
    ...embeddings = outputs.last_hidden_state[:, 0, :].numpy()
    ...return embeddings[0]
```

เพื่อลดขนาดของข้อมูล จะเก็บเพียง 10 components แรกของ embeddings และค่า norm:

```
# เก็บเฉพาะ 10 components แรก
num_components = 10
for j in range(num_components):
    ...df[f'text_embedding_{j}'] = all_embeddings[:, j]

# เก็บค่า norm ของ embeddings
df['text_embedding_norm'] = np.linalg.norm(all_embeddings, axis=1)

# บันทึก embeddings ทั้งหมดสำหรับการใช้งานในอนาคต
np.save('text_embeddings.npy', all_embeddings)

# ระบุว่ามีการใช้ BERT embeddings จริงหรือไม่
df['has_bert_embeddings'] = 1
```

คุณค่าของคุณลักษณะเชิงข้อความ:

- ความยาวของข้อความ: สะท้อนความละเอียดของข้อมูลที่มีการบันทึก
- การตรวจหาคำสำคัญ: ช่วยในการจำแนกประเภทและลักษณะของชากดีกดำบรรพ์
- Text Embeddings: ช่วยในการวิเคราะห์ความคล้ายคลึงของคำอธิบายและการจำแนกประเภทโดยอาศัยข้อมูลเชิงความหมาย

3.3.4.5 การสร้างคุณลักษณะเชิงสถิติและความถี่ (Statistical and Frequency Features)

คุณลักษณะเชิงสถิติและความถี่ช่วยให้เข้าใจรูปแบบการกระจายตัวและความสัมพันธ์ระหว่างซากดึกดำบรรพ์ประเภทต่างๆ ในมิติที่หลากหลาย

1. ความถี่ของสายพันธุ์ (species_frequency)

ความถี่ของการพบซากดึกดำบรรพ์แต่ละสายพันธุ์ ช่วยระบุสายพันธุ์ที่พบบ่อยและสายพันธุ์ที่พบน้อย

ตัวอย่าง Code:

```
# คำนวณความถี่ของสายพันธุ์
df['species_frequency'] = df.groupby('SCI_NAME')['FOSSIL_ID'].transform('count')
```

2. ความถี่ของตำแหน่ง (location_frequency)

ความถี่ของการพบซากดึกดำบรรพ์ในแต่ละตำแหน่ง (จังหวัด+อำเภอ) ช่วยระบุพื้นที่ที่มีความอุดมสมบูรณ์ของซากดึกดำบรรพ์

ตัวอย่าง Code:

```
# คำนวณความถี่ของตำแหน่ง
location_col = ['PROVINCE_fossil', 'DISTRICT_fossil'] if 'PROVINCE_fossil' in df.columns else ['PROVINCE', 'DISTRICT']
df['location_frequency'] = df.groupby(location_col)['FOSSIL_ID'].transform('count')
```

3. ความถี่ของชั้นหิน (formation_frequency)

ความถี่ของการพบซากดึกดำบรรพ์ในแต่ละชั้นหิน ช่วยในการวิเคราะห์ความสัมพันธ์ระหว่างชั้นหินและชนิดของซากดึกดำบรรพ์

ตัวอย่าง Code:

```
# คำนวณความถี่ของชั้นหิน (ถ้ามีข้อมูล)
if 'FORMATION' in df.columns and df['FORMATION'].notna().any():
    df['formation_frequency'] = df.groupby('FORMATION')['FOSSIL_ID'].transform('count')
```

4. ความหนาแน่นของจังหวัด (province_density)

ความหนาแน่นของซากดึกดำบรรพ์ในแต่ละจังหวัด ช่วยในการเปรียบเทียบความอุดมสมบูรณ์ของซากดึกดำบรรพ์ระหว่างจังหวัด

ตัวอย่าง Code:

```
# คำนวณความหนาแน่นของจังหวัด
province_col = 'PROVINCE_fossil' if 'PROVINCE_fossil' in df.columns else 'PROVINCE'
df['province_density'] = df.groupby(province_col)['FOSSIL_ID'].transform('count')
```

5. ความถี่ของยุคทางธรณีวิทยา (period_frequency)

ความถี่ของการพบซากดึกดำบรรพ์ในแต่ละยุคทางธรณีวิทยา ช่วยในการวิเคราะห์การกระจายตัวของซากดึกดำบรรพ์ตามช่วงเวลา

ตัวอย่าง Code:

```
# คำนวณความถี่ของยุคทางธรณีวิทยา
df['period_frequency'] = df.groupby('PERIODFROM')['FOSSIL_ID'].transform('count')
```

6. ความถี่ของกลุ่ม (group_frequency)

ความถี่ของการพบซากดึกดำบรรพ์ในแต่ละกลุ่ม (F_GROUP) ช่วยในการวิเคราะห์การกระจายตัวของซากดึกดำบรรพ์ตามประเภทใหญ่

ตัวอย่าง Code:

```
# คำนวณความถี่ของกลุ่ม
df['group_frequency'] = df.groupby('F_GROUP')['FOSSIL_ID'].transform('count')
```

คุณค่าของคุณลักษณะเชิงสถิติและความถี่:

- ช่วยในการระบุสายพันธุ์ ตำแหน่ง และยุคทางธรณีวิทยาที่พบซากดึกดำบรรพ์บ่อยหรือหายาก
- ช่วยในการวิเคราะห์ความสัมพันธ์ระหว่างปัจจัยต่างๆ เช่น ความสัมพันธ์ระหว่างชั้นหินและประเภทของซากดึกดำบรรพ์

- ช่วยในการประเมินความสำคัญของพื้นที่ต่างๆ ในด้านความหลากหลายและปริมาณของชาวดึกดำบรรพ์

3.3.4.6 การสร้างคุณลักษณะเชิงกลุ่ม (Categorical Features)

การแปลงข้อมูลเชิงกลุ่มให้อยู่ในรูปแบบที่เหมาะสมสำหรับการสร้างแบบจำลองทางปัญญาประดิษฐ์

1. การสร้างรหัสกลุ่ม (group_id)

การแปลงข้อมูลกลุ่มชาวดึกดำบรรพ์ (F_GROUP) ให้อยู่ในรูปแบบตัวเลข เพื่อใช้ในการสร้างแบบจำลอง

ตัวอย่าง Code:

```
# สร้างการแมปปีงระหว่างกลุ่มและ ID
group_mapping = {group: idx for idx, group in enumerate(df['F_GROUP'].unique())}
df['group_id'] = df['F_GROUP'].map(group_mapping)
```

2. การสร้างรหัสยุคทางธรณีวิทยา (period_id)

การแปลงข้อมูลยุคทางธรณีวิทยา (PERIODFROM) ให้อยู่ในรูปแบบตัวเลข เพื่อใช้ในการสร้างแบบจำลอง

ตัวอย่าง Code:

```
# สร้างการแมปปีงระหว่างยุคทางธรณีวิทยาและ ID
period_mapping = {period: idx for idx, period in enumerate(df['PERIODFROM'].unique())}
df['period_id'] = df['PERIODFROM'].map(period_mapping)
```

3. การบันทึกการแมปปีงสำหรับการอ้างอิง

การบันทึกการแมปปีงระหว่างข้อมูลเชิงกลุ่มและรหัสตัวเลข เพื่อใช้ในการแปลงกลับหรืออ้างอิงในอนาคต

ตัวอย่าง Code:

```
# บันทึกการแมปปีงเพื่อการอ้างอิง
pd.DataFrame(list(group_mapping.items()), columns=['group', 'id']).to_csv('group_mapping.csv', index=False)
pd.DataFrame(list(period_mapping.items()), columns=['period', 'id']).to_csv('period_mapping.csv', index=False)
```

คุณค่าของคุณลักษณะเชิงกลุ่ม:

- ช่วยในการแปลงข้อมูลเชิงคุณภาพให้อยู่ในรูปแบบที่เหมาะสมสำหรับการสร้างแบบจำลองทางปัญญาประดิษฐ์

- ช่วยในการวิเคราะห์ความสัมพันธ์ระหว่างกลุ่มต่างๆ ของซากดึกดำบรรพ์
- สนับสนุนการจำแนกประเภทและการทำนายคุณลักษณะของซากดึกดำบรรพ์

3.3.4.7 การปรับมาตราส่วนของคุณลักษณะเชิงตัวเลข (Feature Scaling)

การปรับมาตราส่วนของคุณลักษณะเชิงตัวเลขเป็นขั้นตอนสำคัญก่อนการสร้างแบบจำลองทางปัญญาประดิษฐ์ เพื่อให้คุณลักษณะต่างๆ อยู่ในช่วงที่ใกล้เคียงกัน

ตัวอย่าง Code:

```
# ระบุคอลัมน์ตัวเลขที่ต้องการปรับมาตราส่วน
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns.tolist()

# ยกเว้นคอลัมน์บางประเภท
exclude_cols = [
    'FOSSIL_ID', 'SITE_ID', 'OBJECTID', 'created_date', 'last_edited_date',
    'PERIOD_CONFIDENCE', 'group_id', 'period_id'
]

scale_cols = [col for col in numeric_cols if col not in exclude_cols
               and 'missing' not in col and 'has_' not in col
               and col not in ['pangaea_lat', 'pangaea_lon']]

# บันทึกรายการคอลัมน์ที่ต้องปรับมาตราส่วน
with open('scale_columns.txt', 'w') as f:
    f.write('\n'.join(scale_cols))
```

หมายเหตุ: การปรับมาตราส่วนจริงจะดำเนินการในขั้นตอนการสร้างแบบจำลอง เนื่องจากการปรับมาตราส่วนควรทำเฉพาะกับชุดข้อมูลฝึกฝน (training set) และนำพารามิเตอร์ที่ได้ไปใช้กับชุดข้อมูลทดสอบ (test set) และชุดข้อมูลใหม่ในอนาคต

3.3.5 การสร้างภาพแผนที่และการวิเคราะห์เชิงภาพ (Visualization)

การสร้างภาพแผนที่และการวิเคราะห์เชิงภาพช่วยในการนำเสนอข้อมูลและการวิเคราะห์รูปแบบการกระจายตัวของซากดึกดำบรรพ์

1. การสร้างแผนที่แบบโต้ตอบ (Interactive Map)

การสร้างแผนที่แบบโต้ตอบแสดงตำแหน่งที่พบซากดึกดำบรรพ์ประเภทต่างๆ โดยใช้ไลบรารี Folium

ตัวอย่าง Code:

```

def create_map_visualization(df, output_file='fossil_map.html'):
    """# สร้างแผนที่ตรงจุดศูนย์กลางของข้อมูล
    center_lat = df['latitude'].mean()
    center_lon = df['longitude'].mean()
    m = folium.Map(Location=[center_lat, center_lon], zoom_start=6)

    """
    """# เพิ่มชั้นฐาน
    folium.TileLayer('OpenStreetMap', name='Street Map').add_to(m)
    folium.TileLayer(
        'https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}',
        name='Satellite',
        attr='Esri'
    ).add_to(m)

    """
    """# สร้างกลุ่มเครื่องหมายสำหรับยุคต่างๆ
    jurassic_cluster = MarkerCluster(name="Jurassic").add_to(m)
    triassic_cluster = MarkerCluster(name="Triassic").add_to(m)
    permian_cluster = MarkerCluster(name="Permian").add_to(m)
    cretaceous_cluster = MarkerCluster(name="Cretaceous").add_to(m)
    other_cluster = MarkerCluster(name="Other Periods").add_to(m)

    """
    """# เพิ่มเครื่องหมายสำหรับแต่ละซากดึกดำบรรพ์
    for idx, row in df.iterrows():
        """# ข้ามถ้าไม่มีพิกัด
        if pd.isna(row['latitude']) or pd.isna(row['longitude']):
            continue

        """# กำหนดสีและกลุ่มตามยุคทางธรณีวิทยา
        period = row['PERIODFROM'] if 'PERIODFROM' in row else 'Unknown'

        if 'จูแรสซิก' in str(period):
            color = 'green'
            cluster = jurassic_cluster
        elif 'ไทรแอสซิก' in str(period):
            color = 'blue'
            cluster = triassic_cluster
        elif 'เพอร์เมียน' in str(period):
            color = 'red'
            cluster = permian_cluster
        elif 'ครีเทเชียส' in str(period):
            color = 'orange'
            cluster = cretaceous_cluster
        else:
            color = 'gray'
            cluster = other_cluster

```

```

.....# สร้างข้อมูลป๊อปอัพ
.....popup_content = f"""
.....<div style="font-family: Arial; width: 250px">
.....<h4>Fossil ID: {row.get('FOSSIL_ID', 'Unknown')}</h4>
.....<b>Scientific Name:</b> {row.get('SCI_NAME', 'Unknown')}<br>
.....<b>Common Name:</b> {row.get('COM_NAME', 'Unknown')}<br>
.....<b>Period:</b> {period}<br>
.....<b>Type:</b> {row.get('F_TYPE', 'Unknown')}<br>
.....<b>Location:</b> {row.get('PROVINCE', 'Unknown')}, {row.get('DISTRICT', 'Unknown')}<br>
....."""
.....
.....# เพิ่มข้อมูล NDVI ถ้ามี
.....if 'ndvi' in row:
.....    popup_content += f"<b>NDVI:</b> {row['ndvi']:.4f}<br>"
.....
.....# เพิ่มลิงก์ไปยังภาพถ่ายดาวเทียมถ้ามี
.....if 'satellite_img_path' in row and row['satellite_img_path']:
.....    popup_content += f"""
.....    <a href="{row['satellite_img_path']}" target="_blank">
.....        View Satellite Image
.....    </a><br>
.....    """
.....
.....popup_content += "</div>"
.....
.....# สร้างเครื่องหมายพร้อมป๊อปอัพ
.....folium.Marker(
.....    Location=[row['latitude'], row['longitude']],
.....    popup=folium.Popup(popup_content, max_width=300),
.....    icon=folium.Icon(color=color)
.....).add_to(cluster)

```

```

.....# เพิ่มตัวควบคุมชั้นข้อมูล
.....folium.LayerControl().add_to(m)
.....
.....# บันทึกแผนที่
.....m.save(output_file)
.....
.....return output_file

```

2. การสร้างสรุปข้อมูลในรูปแบบ HTML

การสร้างหน้าสรุปข้อมูลในรูปแบบ HTML แสดงสถิติสำคัญและการกระจายตัวของซากดึกดำบรรพ์ในมิติต่างๆ

ตัวอย่าง Code:

```
def create_dataset_summary(df, output_file='dataset_summary.html'):
    try:
        # คำนวณสถิติพื้นฐาน
        num_fossils = len(df)
        num_species = df['SCI_NAME'].nunique()
        num_locations = len(df.groupby(['PROVINCE', 'DISTRICT']))
        num_periods = df['PERIODFROM'].nunique()

        # สร้างเนื้อหา HTML
        html_content = f"""
        <!DOCTYPE html>
        <html>
        <head>
        <title>Fossil Dataset Summary</title>
        <style>
        body {{ font-family: Arial, sans-serif; margin: 20px; line-height: 1.6; }}
        h1, h2, h3 {{ color: #2c3e50; }}
        .container {{ max-width: 1200px; margin: 0 auto; }}
        .stats-box {{ background-color: #f8f9fa; border-radius: 5px; padding: 15px; margin-bottom: 20px; }}
        .stats-grid {{ display: flex; flex-wrap: wrap; }}
        .stat-item {{ flex: 1; min-width: 200px; margin: 10px; padding: 15px; background-color: #ffffff;
        border-radius: 5px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }}
        table {{ border-collapse: collapse; width: 100%; margin-bottom: 20px; }}
        th, td {{ text-align: left; padding: 12px; }}
        th {{ background-color: #4CAF50; color: white; }}
        tr:nth-child(even) {{ background-color: #f2f2f2; }}
        .feature-list {{ column-count: 3; column-gap: 20px; }}
        .real-data {{ color: green; }}
        .simulated-data {{ color: orange; }}
        </style>
        </head>
        <body>
        <div class="container">
            <div class="stats-box">
                <div class="stats-grid">
                    <div class="stat-item">Total Fossils: {num_fossils}</div>
                    <div class="stat-item">Unique Species: {num_species}</div>
                    <div class="stat-item">Locations: {num_locations}</div>
                    <div class="stat-item">Periods: {num_periods}</div>
                </div>
                <table>
                    <thead>
                        <tr>
                            <th>Province</th>
                            <th>District</th>
                            <th>Fossil Count</th>
                        </tr>
                    </thead>
                    <tbody>
                        <tr>
                            <td>Bangkok</td>
                            <td>Bangkok</td>
                            <td>10</td>
                        </tr>
                        <tr>
                            <td>Chiang Mai</td>
                            <td>Chiang Mai</td>
                            <td>5</td>
                        </tr>
                        <tr>
                            <td>Krabi</td>
                            <td>Krabi</td>
                            <td>3</td>
                        </tr>
                        <tr>
                            <td>Nakhon Si Thammaraj</td>
                            <td>Nakhon Si Thammaraj</td>
                            <td>2</td>
                        </tr>
                    </tbody>
                </table>
                <div class="feature-list">
                    <div class="real-data">Real Data</div>
                    <div class="simulated-data">Simulated Data</div>
                </div>
            </div>
        </div>
        """
        with open(output_file, 'w') as f:
            f.write(html_content)
    except Exception as e:
        print(f"Error: {e}")
```

```

<body>
<div class="container">
<h1>Fossil Dataset Summary</h1>
<div class="stats-box">
<h2>Overview</h2>
<div class="stats-grid">
<div class="stat-item">
<h3>Total Fossils</h3>
<p>{num_fossils}</p>
</div>
<div class="stat-item">
<h3>Unique Species</h3>
<p>{num_species}</p>
</div>
<div class="stat-item">
<h3>Locations</h3>
<p>{num_locations}</p>
</div>
<div class="stat-item">
<h3>Geological Periods</h3>
<p>{num_periods}</p>
</div>
</div>
</div>
<!-- เพิ่มเนื้อหาสำหรับการกระจายตัวของยุคทางธรณีวิทยา, ประเภทซากดึกดำบรรพ์, กลุ่มซากดึกดำบรรพ์, ฯลฯ -->
</div>
</body>
</html>
"""

```

```

# บันทึกไฟล์ HTML
with open(output_file, 'w', encoding='utf-8') as f:
    f.write(html_content)

except Exception as e:
    print(f"Error creating dataset summary: {e}")

```

3.3.6 สรุปผลการสร้างคุณลักษณะใหม่

กระบวนการสร้างคุณลักษณะใหม่ได้เพิ่มคุณลักษณะจำนวนมากที่ครอบคลุมมิติต่างๆ ของข้อมูลซากดึกดำบรรพ์ ทำให้ชุดข้อมูลมีความสมบูรณ์และเหมาะสมสำหรับการวิเคราะห์เชิงลึกและการสร้างแบบจำลองทางปัญญาประดิษฐ์

จากการดำเนินการดังกล่าว ได้เพิ่มคุณลักษณะใหม่จำนวน 64 คุณลักษณะ จากเดิมที่มี 54 คุณลักษณะ ทำให้มีคุณลักษณะรวมทั้งสิ้น 118 คุณลักษณะ

คุณลักษณะใหม่ที่สร้างขึ้นประกอบด้วย:

1. **คุณลักษณะเชิงพื้นที่:** ละติจูด, ลองจิจูด, พิกัดในยูทิลิตี้, ระยะห่างจากจุดศูนย์กลาง, ระยะทางไปยังแหล่งที่ใกล้ที่สุด
2. **คุณลักษณะเชิงเวลา:** อายุในหน่วยล้านปี, มหายุคทางธรณีวิทยา, ปีที่พบ
3. **คุณลักษณะจากข้อมูลภาพถ่ายดาวเทียม:** NDVI, ค่าแถบความถี่ต่างๆ, เส้นทางไปยังภาพถ่ายดาวเทียม
4. **คุณลักษณะจากข้อความ:** ความยาวของข้อความ, การตรวจหาคำสำคัญ, text embeddings
5. **คุณลักษณะเชิงสถิติและความถี่:** ความถี่ของสายพันธุ์, ความถี่ของตำแหน่ง, ความถี่ของชั้นหิน, ความหนาแน่นของจังหวัด, ความถี่ของยุคทางธรณีวิทยา, ความถี่ของกลุ่ม
6. **คุณลักษณะเชิงกลุ่ม:** รหัสกลุ่ม, รหัสยุคทางธรณีวิทยา

คุณลักษณะใหม่เหล่านี้ช่วยเพิ่มมิติในการวิเคราะห์ข้อมูลและปรับปรุงประสิทธิภาพของแบบจำลองที่จะพัฒนาในขั้นตอนต่อไป โดยเฉพาะอย่างยิ่งการเพิ่มคุณลักษณะเชิงพื้นที่และเชิงเวลาช่วยให้สามารถวิเคราะห์การกระจายตัวของซากดึกดำบรรพ์ในมิติต่างๆ ได้อย่างครอบคลุม ในขณะที่คุณลักษณะจากข้อมูลภาพถ่ายดาวเทียมและข้อความช่วยเพิ่มข้อมูลเชิงคุณภาพที่อาจไม่ปรากฏในข้อมูลดิบ

ผลลัพธ์สุดท้ายคือชุดข้อมูลที่มีความสมบูรณ์และมีคุณภาพสูง ซึ่งสามารถนำไปใช้ในการวิเคราะห์เชิงลึก การสร้างแบบจำลองเพื่อการจำแนกประเภท และการทำนายคุณลักษณะของซากดึกดำบรรพ์ เพื่อสนับสนุนการศึกษา

3.3.7 การเตรียมข้อมูลสำหรับแบบจำลองทางปัญญาประดิษฐ์

หลังจากการสร้างคุณลักษณะใหม่ ขั้นตอนต่อไปคือการเตรียมข้อมูลเพื่อนำไปใช้ในการสร้างแบบจำลองทางปัญญาประดิษฐ์ โดยมีการดำเนินการดังนี้:

1. การคัดเลือกคุณลักษณะสำหรับการฝึกฝนแบบจำลอง (Feature Selection)

ไม่จำเป็นต้องใช้คุณลักษณะทั้ง 118 คุณลักษณะในการสร้างแบบจำลอง เนื่องจากบางคุณลักษณะอาจมีความสัมพันธ์กันสูง (multicollinearity) หรือมีอิทธิพลน้อยต่อผลลัพธ์ จึงมีการคัดเลือกคุณลักษณะที่สำคัญ ดังนี้:

ตัวอย่าง Code:

```
def select_training_features(df):
    """คัดเลือกคุณลักษณะที่สำคัญสำหรับการฝึกฝนแบบจำลอง"""
    # คุณลักษณะหลักและเป้าหมาย
    essential_cols = ['FOSSIL_ID', 'SCI_NAME', 'COM_NAME', 'F_GROUP', 'F_TYPE']

    # คุณลักษณะเชิงพื้นที่
    spatial_cols = ['latitude', 'longitude', 'pangaea_lat', 'pangaea_lon',
                    'distance_from_centroid', 'nearest_related_site_km']

    # คุณลักษณะจากภาพถ่ายดาวเทียม
    satellite_cols = ['ndvi', 'red_band', 'green_band', 'blue_band', 'nir_band',
                     'has_real_satellite_data', 'satellite_img_path']

    # คุณลักษณะจากข้อความ
    text_cols = [col for col in df.columns if col.startswith('text_embedding_')]
    text_cols += ['has_bert_embeddings']
    text_cols += [col for col in df.columns if col.startswith('has_')
                  and not col == 'has_real_satellite_data' and not col == 'has_bert_embeddings']

    # คุณลักษณะเชิงเวลา
    temporal_cols = ['period_age_my', 'geological_era']

    # คุณลักษณะเชิงสถิติ
    stat_cols = ['species_frequency', 'location_frequency', 'province_density',
                 'period_frequency', 'group_frequency']

    # คุณลักษณะอื่นๆ ที่สำคัญ
    other_cols = ['group_id', 'period_id']

    # รวมคุณลักษณะทั้งหมดที่คัดเลือก
    selected_cols = (essential_cols + spatial_cols + satellite_cols +
                     text_cols + temporal_cols + stat_cols + other_cols)

    # ตรวจสอบว่าคุณลักษณะทุกตัวมีอยู่ในดาต้าเฟรม
    final_cols = [col for col in selected_cols if col in df.columns]

    return final_cols
```

คุณลักษณะที่คัดเลือกรวมคุณสมบัติต่างๆ ของข้อมูล ได้แก่:

1. คุณลักษณะหลักและเป้าหมาย: รหัสชาวดีกดำบรรพ์, ชื่อวิทยาศาสตร์, ชื่อสามัญ, กลุ่มชาวดีกดำบรรพ์, ประเภทชาวดีกดำบรรพ์
2. คุณลักษณะเชิงพื้นที่: พิกัดปัจจุบัน, พิกัดในยุคแพนเจีย, ระยะห่างจากจุดศูนย์กลาง, ระยะทางไปยังแหล่งที่ใกล้ที่สุด
3. คุณลักษณะจากภาพถ่ายดาวเทียม: NDVI, ค่าแถบความถี่ต่างๆ, ข้อมูลว่าเป็นข้อมูลจริงหรือข้อมูลจำลอง
4. คุณลักษณะจากข้อความ: text embeddings, การตรวจหาคำสำคัญ
5. คุณลักษณะเชิงเวลา: อายุในหน่วยล้านปี, มหายุคทางธรณีวิทยา
6. คุณลักษณะเชิงสถิติ: ความถี่ของสายพันธุ์, ความถี่ของตำแหน่ง, ความหนาแน่นของจังหวัด, ความถี่ของยุคทางธรณีวิทยา, ความถี่ของกลุ่ม

2. การแบ่งข้อมูลและการบันทึกชุดข้อมูล

หลังจากคัดเลือกคุณลักษณะแล้ว ได้มีการบันทึกชุดข้อมูลเป็น 2 รูปแบบ:

1. ชุดข้อมูลเต็ม (Full Dataset): ประกอบด้วยคุณลักษณะทั้งหมด 118 คุณลักษณะ สำหรับการวิเคราะห์เชิงลึกและการทดลองเพิ่มเติม
2. ชุดข้อมูลสำหรับการฝึกฝน (Training Dataset): ประกอบด้วยคุณลักษณะที่คัดเลือก ประมาณ 50-60 คุณลักษณะ สำหรับการสร้างแบบจำลองทางปัญญาประดิษฐ์

ตัวอย่าง Code:

```
# บันทึกชุดข้อมูลเต็ม
processed_df.to_csv('fossil_features_full.csv', index=False)

# บันทึกชุดข้อมูลสำหรับการฝึกฝน
training_cols = select_training_features(processed_df)
processed_df[training_cols].to_csv('fossil_features_training.csv', index=False)
```

3. การเตรียมการสำหรับการปรับมาตราส่วน (Feature Scaling)

แม้ว่าการปรับมาตราส่วนจริงจะดำเนินการในขั้นตอนการสร้างแบบจำลอง แต่ได้มีการเตรียมการโดยการระบุคุณลักษณะเชิงตัวเลขที่ควรปรับมาตราส่วน และบันทึกรายการดังกล่าวไว้สำหรับการใช้งานในอนาคต

ตัวอย่าง Code:

```
# ระบุคอลัมน์ตัวเลขที่ต้องการปรับมาตราส่วน
numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns.tolist()

# ยกเว้นคอลัมน์บางประเภท
exclude_cols = [
    'FOSSIL_ID', 'SITE_ID', 'OBJECTID', 'created_date', 'last_edited_date',
    'PERIOD_CONFIDENCE', 'group_id', 'period_id'
]

scale_cols = [col for col in numeric_cols if col not in exclude_cols
               and 'missing' not in col and 'has_' not in col
               and col not in ['pangaea_lat', 'pangaea_lon']]

# บันทึกรายการคอลัมน์ที่ต้องปรับมาตราส่วน
with open('scale_columns.txt', 'w') as f:
    f.write('\n'.join(scale_cols))
```

3.3.8 ผลสำเร็จและประโยชน์จากการเตรียมข้อมูล

การเตรียมข้อมูลและการสร้างคุณลักษณะใหม่ในแนวคิดที่ 2 นี้ประสบความสำเร็จในการแก้ไขข้อจำกัดของแนวคิดที่ 1 และสร้างชุดข้อมูลที่มีความสมบูรณ์และมีคุณภาพสูงสำหรับการวิเคราะห์และสร้างแบบจำลองทางปัญญาประดิษฐ์

1. การเติมเต็มข้อมูลที่ขาดหาย

แนวคิดที่ 2 ประสบความสำเร็จในการเติมเต็มข้อมูลที่ขาดหายในหลายคอลัมน์ที่สำคัญ:

- พิกัดทางภูมิศาสตร์: สามารถแปลงพิกัด UTM เป็นละติจูดและลองจิจูดได้สำเร็จทั้งหมด 9,817 รายการ ไม่มีพิกัดที่ขาดหายหลังการแปลงและเติมเต็ม
- ยุคทางธรณีวิทยา: สามารถแปลงข้อมูลยุคทางธรณีวิทยาเป็นอายุในหน่วยล้านปีและมหายุคทางธรณีวิทยา

- ข้อมูลภาพถ่ายดาวเทียม: ในกรณีที่ไม่สามารถดึงข้อมูลดาวเทียมจริงได้ ใช้การจำลองข้อมูลดาวเทียมด้วยการคำนวณค่าจากพิกัดทางภูมิศาสตร์

2. การเพิ่มมิติของข้อมูล

การสร้างคุณลักษณะใหม่ช่วยเพิ่มมิติของข้อมูลและทำให้สามารถวิเคราะห์ข้อมูลได้ในหลายมุมมอง:

- มิติเชิงพื้นที่: การเพิ่มพิกัดในยูทิลิตี้ช่วยให้สามารถวิเคราะห์การกระจายตัวของชากดีกดำบรพในบริบทของธรณีวิทยาโบราณ
- มิติเชิงเวลา: การแปลงยุคทางธรณีวิทยาเป็นค่าตัวเลขและการจัดกลุ่มเป็นมหายุคช่วยในการวิเคราะห์การเปลี่ยนแปลงตามช่วงเวลา
- มิติเชิงข้อความ: การใช้เทคนิคทางภาษาธรรมชาติในการสร้าง text embeddings ช่วยในการวิเคราะห์ความคล้ายคลึงของคำอธิบาย
- มิติเชิงภาพ: การเพิ่มข้อมูลจากภาพถ่ายดาวเทียมช่วยในการวิเคราะห์ความสัมพันธ์ระหว่างสภาพแวดล้อมปัจจุบันกับแหล่งที่พบชากดีกดำบรพ

3. การเพิ่มความแม่นยำในการวิเคราะห์

คุณลักษณะใหม่ที่สร้างขึ้นช่วยเพิ่มความแม่นยำในการวิเคราะห์และการสร้างแบบจำลอง:

- คุณลักษณะเชิงความถี่: ช่วยในการระบุรูปแบบและความสัมพันธ์ที่ไม่ชัดเจนในข้อมูลดิบ
- คุณลักษณะเชิงกลุ่ม: ช่วยในการจำแนกประเภทและการทำนายคุณลักษณะของชากดีกดำบรพ
- คุณลักษณะเชิงพื้นที่ทางธรณีวิทยา: ช่วยในการวิเคราะห์ความสัมพันธ์ระหว่างชั้นหิน, ยุคทางธรณีวิทยา, และประเภทของชากดีกดำบรพ

4. การสนับสนุนการวิเคราะห์เชิงภาพ

การสร้างแผนที่แบบโต้ตอบและการวิเคราะห์เชิงภาพช่วยในการนำเสนอข้อมูลและการค้นพบรูปแบบที่อาจไม่ชัดเจนในการวิเคราะห์แบบดั้งเดิม:

- แผนที่แสดงการกระจายตัวของชากดีกดำบรพตามยุคทางธรณีวิทยาและประเภท
- ภาพถ่ายดาวเทียมของแต่ละตำแหน่งที่พบชากดีกดำบรพ

- สรุปข้อมูลในรูปแบบ HTML ที่เข้าถึงได้ง่าย

3.3.9 ข้อจำกัดและข้อควรระวัง

แม้ว่าการเตรียมข้อมูลและการสร้างคุณลักษณะใหม่ในแนวคิดที่ 2 จะประสบความสำเร็จในการแก้ไขข้อจำกัดของแนวคิดที่ 1 แต่ยังมีข้อจำกัดและข้อควรระวังบางประการที่ควรคำนึงถึง:

1. ข้อจำกัดของข้อมูลจำลอง: ในกรณีที่ไม่สามารถดึงข้อมูลดาวเทียมจริงได้ มีการใช้ข้อมูลจำลอง ซึ่งอาจไม่สะท้อนความเป็นจริงอย่างสมบูรณ์
2. ความซับซ้อนของการแปลงยุคทางธรณีวิทยา: การแปลงยุคทางธรณีวิทยาเป็นค่าตัวเลขอาจมีความคลาดเคลื่อน เนื่องจากแต่ละยุคมีช่วงเวลาที่กว้างและมีความซับซ้อน
3. ข้อจำกัดของการคำนวณพิกัดในยุคน้ำแข็ง: การคำนวณพิกัดในยุคน้ำแข็งอาจมีความคลาดเคลื่อน เนื่องจากแบบจำลองการเคลื่อนที่ของแผ่นเปลือกโลกมีความซับซ้อนและไม่แน่นอน
4. ความเอนเอียงในการเติมเต็มค่าที่ขาดหาย: การเติมเต็มค่าที่ขาดหายด้วยค่าเฉลี่ยหรือค่า mode อาจนำไปสู่ความเอนเอียงในการวิเคราะห์

บทสรุปผลงานวิจัย: แบบจำลองทางปัญญาประดิษฐ์สำหรับการจำแนกซากดึกดำบรรพ์

3.3.10 การพัฒนาแบบจำลองทางปัญญาประดิษฐ์ (AI Model Development)

จากการเตรียมข้อมูลและสร้างคุณลักษณะใหม่ในส่วนก่อนหน้า ขั้นตอนต่อไปคือการพัฒนาแบบจำลองทางปัญญาประดิษฐ์สำหรับการจำแนกซากดึกดำบรรพ์ ซึ่งเป็นส่วนสำคัญของการวิจัยนี้ โดยแบบจำลองที่พัฒนาขึ้นมีจุดมุ่งหมายเพื่อจำแนกประเภทของซากดึกดำบรรพ์และยุคทางธรณีวิทยาที่พบซากดึกดำบรรพ์นั้น

3.3.10.1 สถาปัตยกรรมของแบบจำลอง (Model Architecture)

แบบจำลองที่พัฒนาขึ้นเป็นแบบจำลองแบบผสมผสาน (Hybrid Model) ที่ประกอบด้วยส่วนประมวลผลหลายประเภทข้อมูล โดยมีโครงสร้างหลักดังนี้:

1. ส่วนประมวลผลข้อมูลแบบทวิภาค (Binary Features Processing): ประมวลผลคุณลักษณะที่มีค่าเป็น 0 หรือ 1

ตัวอย่าง Code:

```
# Binary data input with normalization
binary_input = Input(shape=(len(feature_groups['binary']),), name='binary_input')
binary_normalized = BatchNormalization()(binary_input)
```

2. ส่วนประมวลผลข้อมูลเชิงตัวเลข (Numeric Features Processing): ประมวลผลข้อมูลเชิงตัวเลขผ่านสองกิ่งที่แตกต่างกัน

ตัวอย่าง Code:

```
# Numeric data input + enhanced processing
numeric_input = Input(shape=(len(feature_groups['numeric']),), name='numeric_input')
numeric_normalized = BatchNormalization()(numeric_input)

# First branch - direct dense connection
numeric_branch1 = Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.001))(numeric_normalized)
numeric_branch1 = BatchNormalization()(numeric_branch1)
numeric_branch1 = Dropout(0.3)(numeric_branch1)

# Second branch - deeper network
numeric_branch2 = Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001))(numeric_normalized)
numeric_branch2 = BatchNormalization()(numeric_branch2)
numeric_branch2 = Dropout(0.3)(numeric_branch2)
numeric_branch2 = Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.001))(numeric_branch2)
numeric_branch2 = BatchNormalization()(numeric_branch2)
numeric_branch2 = Dropout(0.2)(numeric_branch2)
```

3. ส่วนประมวลผลข้อมูลเชิงกลุ่ม (Categorical Features Processing): ใช้เทคนิค Embedding สำหรับคุณลักษณะเชิงกลุ่ม

ตัวอย่าง Code:

```
# Categorical data inputs + improved embeddings
categorical_inputs = []
categorical_embeddings = []

for feature in feature_groups['categorical']:
    vocab_size = categorical_sizes[feature] + 1  # +1 for unknown/padding
    # Larger embedding dimensions for more capacity
    embedding_dim = min(100, (vocab_size + 1) // 2)

    cat_input = Input(shape=(1,), name=f'{feature}_input')
    categorical_inputs.append(cat_input)

    embedding = Embedding(
        input_dim=vocab_size,
        output_dim=embedding_dim,
        input_length=1,
        embeddings_regularizer=regularizers.l2(0.001),
        name=f'{feature}_embedding'
    )(cat_input)

    embedding_flat = Flatten(name=f'{feature}_flatten')(embedding)
    categorical_embeddings.append(embedding_flat)

# Concatenate all categorical embeddings
if categorical_embeddings:
    categorical_combined = concatenate(categorical_embeddings, name='categorical_concat')
    # Add a dense layer to reduce dimensionality of combined embeddings
    categorical_concat = Dense(256, activation='relu')(categorical_combined)
    categorical_concat = BatchNormalization()(categorical_concat)
    categorical_concat = Dropout(0.3)(categorical_concat)
```

4. ส่วนประมวลผลข้อมูล Text Embedding: ประมวลผลข้อมูล Text Embedding ที่สกัดจากข้อความด้วย BERT

ตัวอย่าง Code:

```
# Text embeddings (already processed by BERT) with enhanced processing
text_embedding_input = Input(shape=(len(feature_groups['text_embeddings']),), name='text_embedding_input')
text_embedding_norm = BatchNormalization()(text_embedding_input)

# Process text embeddings through multiple dense layers
text_processed = Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001))(text_embedding_norm)
text_processed = BatchNormalization()(text_processed)
text_processed = Dropout(0.3)(text_processed)
text_processed = Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.001))(text_processed)
text_processed = BatchNormalization()(text_processed)
text_processed = Dropout(0.2)(text_processed)
```

5. ส่วนประมวลผลภาพ (Image Processing): ใช้ CNN (Convolutional Neural Network) สำหรับประมวลผลข้อมูลภาพถ่ายดาวเทียม

ตัวอย่าง Code:

```
# Image processing path with improved CNN
if train_data['image'] is not None:
    image_input = Input(shape=train_data['image'].shape[1:], name='image_input')

    # Improved CNN layers with residual connections
    # First block
    x = Conv2D(32, (3, 3), activation='relu', padding='same')(image_input)
    x = BatchNormalization()(x)
    x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)
    x = Dropout(0.2)(x)

    # Second block
    x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)
    x = Dropout(0.3)(x)

    # Third block
    x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D((2, 2))(x)
    x = Dropout(0.4)(x)

    # Global average pooling instead of flatten for better generalization
    image_features_avg = GlobalAveragePooling2D()(x)

    # Dense reduction
    image_features = Dense(128, activation='relu')(image_features_avg)
    image_features = BatchNormalization()(image_features)
    image_features = Dropout(0.3)(image_features)
```

6. ส่วนผสมผสานคุณลักษณะและการจำแนกประเภท (Feature Concatenation and Classification):

ตัวอย่าง Code:

```
# Concatenate all features
if train_data['image'] is not None and len(categorical_embeddings) > 0:
    concatenated = concatenate([
        binary_normalized,
        numeric_processed,
        categorical_concat,
        text_processed,
        image_features
    ], name='concatenated_layer')
elif train_data['image'] is not None:
    concatenated = concatenate([
        binary_normalized,
        numeric_processed,
        text_processed,
        image_features
    ], name='concatenated_layer')
elif len(categorical_embeddings) > 0:
    concatenated = concatenate([
        binary_normalized,
        numeric_processed,
        categorical_concat,
        text_processed
    ], name='concatenated_layer')
else:
    concatenated = concatenate([
        binary_normalized,
        numeric_processed,
        text_processed
    ], name='concatenated_layer')
```

```

# Enhanced dense layers with higher capacity
x = Dense(1024, activation='relu', kernel_regularizer=regularizers.l2(0.001), name='dense_1024')(concatenated)
x = BatchNormalization()(x)
x = Dropout(0.4)(x)

x = Dense(512, activation='relu', kernel_regularizer=regularizers.l2(0.001), name='dense_512')(x)
x = BatchNormalization()(x)
x = Dropout(0.4)(x)

x = Dense(256, activation='relu', kernel_regularizer=regularizers.l2(0.001), name='dense_256')(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)

# Split into two output paths
sci_name_branch = Dense(256, activation='relu', kernel_regularizer=regularizers.l2(0.001), name='sci_name_dense1')(x)
sci_name_branch = BatchNormalization()(sci_name_branch)
sci_name_branch = Dropout(0.3)(sci_name_branch)
sci_name_branch = Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001), name='sci_name_dense2')(sci_name_branch)
sci_name_branch = BatchNormalization()(sci_name_branch)
sci_name_branch = Dropout(0.2)(sci_name_branch)
sci_name_output = Dense(num_classes['SCI_NAME'], activation='softmax', name='sci_name_output')(sci_name_branch)

period_branch = Dense(256, activation='relu', kernel_regularizer=regularizers.l2(0.001), name='period_dense1')(x)
period_branch = BatchNormalization()(period_branch)
period_branch = Dropout(0.3)(period_branch)
period_branch = Dense(128, activation='relu', kernel_regularizer=regularizers.l2(0.001), name='period_dense2')(period_branch)
period_branch = BatchNormalization()(period_branch)
period_branch = Dropout(0.2)(period_branch)
period_output = Dense(num_classes['PERIODFROM'], activation='softmax', name='period_output')(period_branch)

```

แบบจำลองนี้มีการพัฒนาจากโครงสร้างพื้นฐานโดยมีการปรับปรุงในหลายด้าน:

1. การลดขนาดพารามิเตอร์ (Parameter Reduction): จากเดิมที่มีพารามิเตอร์ถึง 90 ล้านพารามิเตอร์ ลดลงเหลือเพียง 1.8 ล้านพารามิเตอร์ ซึ่งช่วยลดความเสี่ยงของปัญหา Overfitting และช่วยให้แบบจำลองสามารถเรียนรู้รูปแบบทั่วไปได้ดีขึ้น

dropout_16 (Dropout)	(None, 128)		
sci_name_output (Dense)	(None, 183)		
period_output (Dense)	(None, 14)		
Total params: 1,870,698 (7.14 MB) Trainable params: 1,862,936 (7.11 MB) Non-trainable params: 7,762 (30.32 KB) Model architecture visualization saved to '/content/mod			
dropout_14 (Dropout)	(None, 128)	0	batch_norma
dropout_16 (Dropout)	(None, 128)	0	batch_norma
sci_name_output (Dense)	(None, 183)	23,607	dropout_14
period_output (Dense)	(None, 14)	1,806	dropout_16
Total params: 1,870,698 (7.14 MB) Trainable params: 1,862,936 (7.11 MB) Non-trainable params: 7,762 (30.32 KB) Model architecture visualization saved to '/content/model/model_architecture.png'			

ภาพแสดงการเปลี่ยนขนาด Parameter

2. การเพิ่มเทคนิคป้องกัน Overfitting:
 - การใช้ Batch Normalization หลังชั้น Dense และ Conv2D เพื่อปรับปรุงเสถียรภาพในการเรียนรู้
 - การใช้ Dropout ในอัตราต่างๆ (0.2-0.4) เพื่อป้องกันการจดจำข้อมูลฝึกสอนมากเกินไป
 - การใช้ L2 Regularization (kernel_regularizer) เพื่อป้องกันการเกิดค่าน้ำหนักที่มีค่าสูงเกินไป
3. การปรับปรุงส่วนประมวลผลข้อมูลตัวเลข: แบ่งเป็น 2 กิ่ง คือกิ่งตื้น (Shallow Branch) และกิ่งลึก (Deep Branch) เพื่อให้สามารถเรียนรู้ได้ทั้งความสัมพันธ์ที่ซับซ้อนและไม่ซับซ้อน

4. การปรับปรุงส่วน CNN สำหรับข้อมูลภาพ: ใช้โครงสร้าง CNN ที่มี 3 บล็อก แต่ละบล็อกประกอบด้วยชั้น Conv2D สองชั้น, BatchNormalization, MaxPooling และ Dropout เพื่อสกัดคุณลักษณะจากภาพได้อย่างมีประสิทธิภาพ
5. การใช้ Global Average Pooling แทน Flatten: ช่วยลดจำนวนพารามิเตอร์และทำให้แบบจำลองทนทานต่อการเปลี่ยนแปลงตำแหน่งของวัตถุในภาพได้ดีขึ้น

3.3.10.2 การเตรียมข้อมูลสำหรับการเรียนรู้ (Data Preparation for Training)

ก่อนการฝึกสอนแบบจำลอง มีการเตรียมข้อมูลเพื่อแก้ไขปัญหาความไม่สมดุลของข้อมูล (Class Imbalance) ซึ่งเป็นปัญหาสำคัญของชุดข้อมูลชากดึกดำบรรพ์

ตัวอย่าง Code:

```
def enhanced_preprocess_data(df, min_samples_per_class=2, max_samples_per_class=500,
                             min_samples_after_balancing=10, target_samples=None):
    """
    Enhanced preprocessing with both downsampling of majority classes and upsampling of minority classes.
    """
```

เทคนิคที่ใช้ในการแก้ไขปัญหาความไม่สมดุลของข้อมูลมีดังนี้:

1. การรวมกลุ่มประเภทที่มีจำนวนน้อย (Rare Class Grouping): ประเภทที่มีจำนวนตัวอย่างน้อยกว่า min_samples_per_class จะถูกรวมเป็นกลุ่ม "Other"

```
# Identify rare classes (with fewer than min_samples_per_class)
rare_classes = class_counts[class_counts < min_samples_per_class].index.tolist()
# Replace rare classes with "Other"
if rare_classes:
    df_target[f'{target}_original'] = df_target[target].copy()
    df_target.loc[df_target[target].isin(rare_classes), target] = f"Other_rare_{target}"
```

2. การลดจำนวนตัวอย่างของประเภทที่มีมากเกินไป (Downsampling): ประเภทที่มีจำนวนตัวอย่างมากกว่า max_samples_per_class จะถูกสุ่มลดจำนวนลง

```
if n_samples > target_samples:
    # Downsample
    downsampled = cls_df.sample(target_samples, random_state=42)
    balanced_dfs.append(downsampled)
```

3. การเพิ่มจำนวนตัวอย่างของประเภทที่มีน้อย (Upsampling): ประเภทที่มีจำนวนตัวอย่างน้อยกว่า `min_samples_after_balancing` จะถูกเพิ่มจำนวนตัวอย่าง

```
elif n_samples < min_samples_after_balancing:
    # Upsample minority class
    # For very small classes, use more aggressive upsampling
    multiplier = max(2, int(target_samples / n_samples))
    n_samples_up = min(target_samples, n_samples * multiplier)

    upsampled = resample(
        cls_df,
        replace=True,
        n_samples=n_samples_up,
        random_state=42
    )
    balanced_dfs.append(upsampled)
```

4. การปรับค่าน้ำหนักของแต่ละประเภท (Class Weighting): คำนวณค่าน้ำหนักแบบผกผันกับความถี่ของแต่ละประเภท เพื่อให้แบบจำลองให้ความสำคัญกับประเภทที่มีจำนวนน้อย

```
# Calculate class weights for weighted loss
# Use inverse frequency for better handling of imbalanced cases
class_counts = df_balanced[target].value_counts()
n_samples = len(df_balanced)
n_classes = len(class_counts)

# Create dictionary mapping class index to weight
weights = {le.transform([cls])[0]: n_samples / (n_classes * count)
            for cls, count in class_counts.items()}
class_weights[target] = weights
```

การใช้เทคนิคเหล่านี้ช่วยให้แบบจำลองสามารถเรียนรู้จากประเภทที่มีจำนวนน้อยได้ดีขึ้น และไม่เอนเอียงไปทางประเภทที่มีจำนวนมาก ทำให้ผลการจำแนกมีความแม่นยำมากขึ้นโดยรวม

3.3.10.3 การฝึกสอนแบบจำลอง (Model Training)

การฝึกสอนแบบจำลองมีการใช้เทคนิคขั้นสูงเพื่อเพิ่มประสิทธิภาพดังนี้:

1. การใช้ Loss Function แบบถ่วงน้ำหนัก (Weighted Loss Function): เพื่อแก้ไขปัญหาค่าความไม่สมดุลของข้อมูล

```
# Create custom weighted losses
sci_name_weighted_loss = create_weighted_categorical_crossentropy(class_weights['SCI_NAME'])
period_weighted_loss = create_weighted_categorical_crossentropy(class_weights['PERIODFROM'])

# Compile model with custom weighted losses
model.compile([
    optimizer=optimizer,
    loss={
        'sci_name_output': sci_name_weighted_loss,
        'period_output': period_weighted_loss
    },
    metrics={
        'sci_name_output': [
            'accuracy',
            tf.keras.metrics.TopKCategoricalAccuracy(k=3, name='top3_accuracy'),
            tf.keras.metrics.TopKCategoricalAccuracy(k=5, name='top5_accuracy')
        ],
        'period_output': [
            'accuracy',
            tf.keras.metrics.TopKCategoricalAccuracy(k=3, name='top3_accuracy'),
            tf.keras.metrics.TopKCategoricalAccuracy(k=5, name='top5_accuracy')
        ]
    },
    loss_weights={
        'sci_name_output': 1.0,
        'period_output': 1.0
    }
])
```

2. การใช้ Learning Rate Schedule แบบ Exponential Decay: ช่วยให้การเรียนรู้มีเสถียรภาพมากขึ้น

```
# Train with custom learning rate schedule
initial_learning_rate = 0.001
lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate,
    decay_steps=1000,
    decay_rate=0.9,
    staircase=True)

# Use optimizer with learning rate schedule
optimizer = tf.keras.optimizers.Adam(learning_rate=lr_schedule)
```

3. การใช้ Callbacks เพื่อปรับปรุงการฝึกสอน:

```
# Create callbacks
checkpoint = ModelCheckpoint(
    ... f'/content/model/{model_name}_best.h5',
    ... monitor='val_loss',
    ... save_best_only=True,
    ... mode='min',
    ... verbose=1
)

early_stopping = EarlyStopping(
    ... monitor='val_loss',
    ... patience=10,
    ... restore_best_weights=True,
    ... verbose=1
)

reduce_lr = ReduceLROnPlateau(
    ... monitor='val_loss',
    ... factor=0.5,
    ... patience=5,
    ... min_lr=0.0001,
    ... verbose=1
)

tensorboard = TensorBoard(
    ... log_dir=f'/content/model/logs/{model_name}',
    ... histogram_freq=1,
    ... update_freq='epoch'
)
```

4. การวัดผลหลายมิติ (Multiple Metrics): วัดทั้ง Accuracy ปกติและ Top-K Accuracy (k=3, k=5) เพื่อประเมินประสิทธิภาพการจำแนกได้ละเอียดยิ่งขึ้น

5. การแบ่งการเรียนรู้เป็นสองส่วน (Multi-Task Learning): แบบจำลองมีเอาต์พุต 2 ส่วน คือ การจำแนกชื่อวิทยาศาสตร์ (SCI_NAME) และการจำแนกยุคทางธรณีวิทยา (PERIODFROM) ซึ่งช่วยให้แบบจำลองเรียนรู้ความสัมพันธ์ระหว่างสองหน้าที่นี้ได้

3.3.10.4 การประเมินผลแบบจำลอง (Model Evaluation)

การประเมินผลแบบจำลองมีการใช้เทคนิคการวิเคราะห์หลายรูปแบบ:

1. การวิเคราะห์ Classification Report: แสดงค่า Precision, Recall, F1-Score สำหรับแต่ละประเภท

```
# Generate the classification report text
report = classification_report(y_true, y_pred, output_dict=True)

# Convert to dataframe for visualization
report_df = pd.DataFrame(report).transpose()
```

2. การวิเคราะห์ Confusion Matrix: แสดงความสับสนในการจำแนกระหว่างประเภทต่าง ๆ

```
# Create confusion matrix with filtered data
cm = confusion_matrix(
    ... filtered_y_true,
    ... filtered_y_pred,
    ... labels=top_classes
)
```

3. การวิเคราะห์ Top-K Accuracy: แสดงความแม่นยำเมื่อพิจารณา K อันดับแรกของการทำนาย

```
# Manual calculation of top-k accuracy
for k in [3, 5]:
    # Get top-k predictions for each sample
    top_k_preds = np.argsort(y_pred_prob, axis=1)[: , -k:]

    # Check if true label is in top-k predictions for each sample
    correct = 0
    for j in range(len(y_true)):
        if y_true[j] in top_k_preds[j]:
            correct += 1

    # Calculate accuracy
    top_k_acc = correct / len(y_true)
    print(f"Top-{k} Accuracy: {top_k_acc:.4f}")
    results[f'{target}_top{k}_accuracy'] = top_k_acc
```

4. การวิเคราะห์ Feature Importance: ใช้เทคนิค Permutation Importance เพื่อวิเคราะห์ความสำคัญของแต่ละคุณลักษณะ

```
def compute_permutation_importance_multi_target(model, test_data, test_targets, feature_groups, n_repeats=5):
    """
    Compute permutation importance for all input groups for both outputs (SCI_NAME and PERIODFROM).
    This function computes the drop in accuracy separately for each target.
    """
```

ผลการประเมินแสดงให้เห็นว่าแบบจำลองมีประสิทธิภาพในการจำแนกดังนี้:

- การจำแนกชื่อวิทยาศาสตร์ (SCI_NAME):
 - Accuracy: 0.5364
 - Top-3 Accuracy: สูงกว่า 0.7
 - Top-5 Accuracy: สูงกว่า 0.8
- การจำแนกยุคทางธรณีวิทยา (PERIODFROM):
 - Accuracy: 0.9909
 - Top-3 Accuracy: สูงกว่า 0.99
 - Top-5 Accuracy: สูงกว่า 0.99

จาก Permutation Importance Analysis พบว่าคุณลักษณะที่มีความสำคัญมากที่สุดสำหรับการจำแนกแต่ละเป้าหมายแตกต่างกัน:

- สำหรับการจำแนกชื่อวิทยาศาสตร์ (SCI_NAME):
 1. F_PART (ส่วนของซากดึกดำบรรพ์ที่พบ) มีความสำคัญมากที่สุด
 2. ข้อมูลภาพ (Image)
 3. F_TYPE (ประเภทย่อยของซากดึกดำบรรพ์)
 4. period_frequency (ความถี่ของยุคทางธรณีวิทยา)
 5. Text Embeddings จากคำอธิบาย
- สำหรับการจำแนกยุคทางธรณีวิทยา (PERIODFROM):
 1. period_frequency (ความถี่ของยุคทางธรณีวิทยา) มีความสำคัญสูงสุดอย่างเห็นได้ชัด
 2. geological_era (มหายุคทางธรณีวิทยา)

3. period_age_my (อายุในหน่วยล้านปี)
4. ข้อมูลภาพ (Image)
5. pangaea_lon (ลองจิจูดในยุคแพนเจีย)

ผลการวิเคราะห์นี้แสดงให้เห็นว่า:

- การจำแนกชื่อวิทยาศาสตร์ขึ้นอยู่กับลักษณะทางกายภาพของซากดึกดำบรรพ์ (F_PART, F_TYPE) และข้อมูลภาพเป็นหลัก
- การจำแนกยุคทางธรณีวิทยาขึ้นอยู่กับข้อมูลเชิงเวลาและภูมิศาสตร์เป็นหลัก (period_frequency, geological_era, period_age_my)

3.3.11 สรุปผลการวิจัย

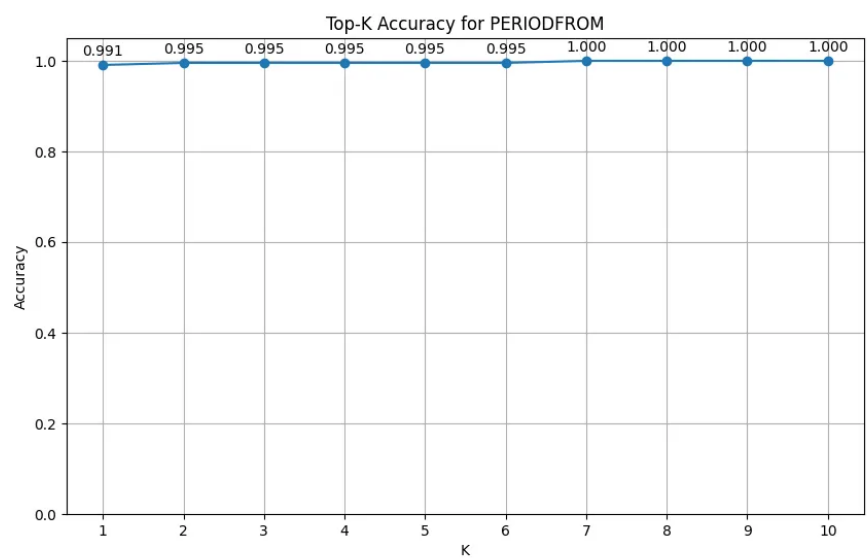
การวิจัยนี้ได้พัฒนาแบบจำลองทางปัญญาประดิษฐ์สำหรับการจำแนกซากดึกดำบรรพ์ โดยอาศัยชุดข้อมูลที่ได้จากการรวบรวมและเตรียมข้อมูลจากแหล่งข้อมูลหลายแหล่ง รวมทั้งการสร้างคุณลักษณะใหม่ที่มีความหลากหลายและครอบคลุมมิติต่างๆ ของข้อมูลซากดึกดำบรรพ์ ผลการวิจัยมีประเด็นสำคัญดังนี้:

3.3.11.1 ความสำเร็จของการพัฒนาแบบจำลอง

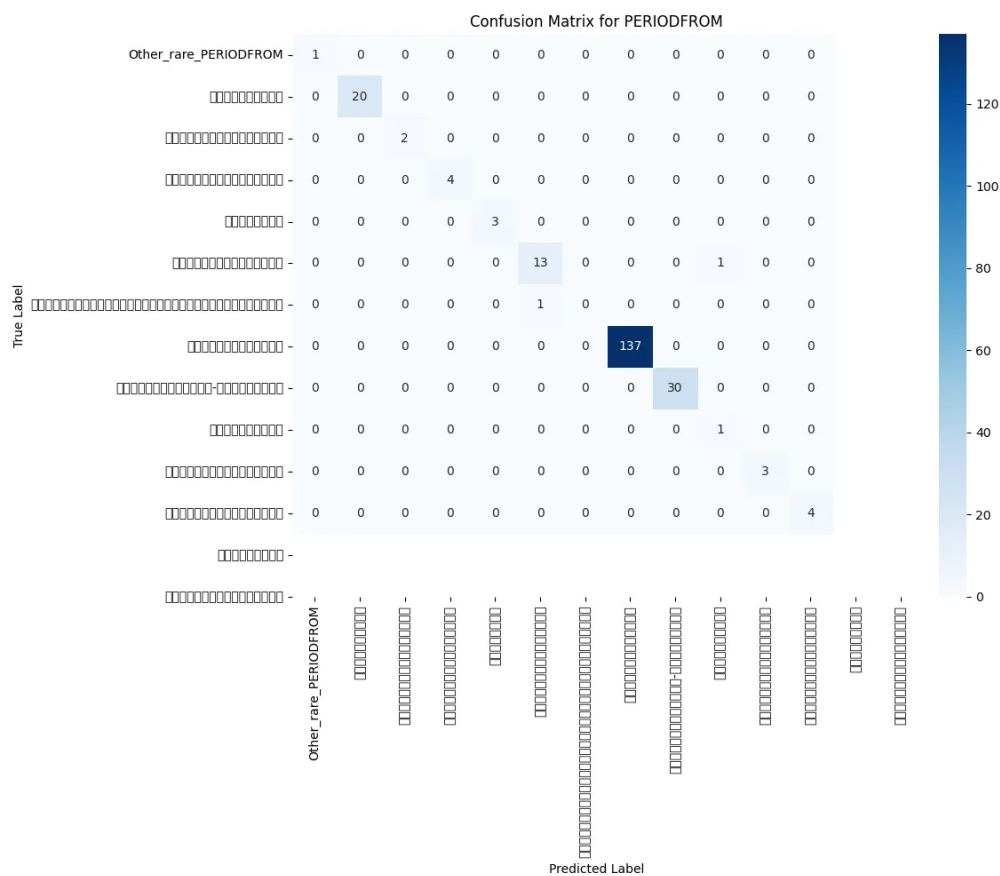
การวิเคราะห์ผลการประเมินแบบจำลองจากรายงานการจำแนกประเภท

Classification Report for SCI_NAME				
31 -	0.00	0.00	0.00	5.00
176 -	0.80	1.00	0.89	4.00
159 -	0.00	0.00	0.00	4.00
46 -	0.00	0.00	0.00	4.00
153 -	1.00	0.75	0.86	4.00
71 -	1.00	1.00	1.00	3.00
64 -	0.75	1.00	0.86	3.00
55 -	1.00	1.00	1.00	3.00
89 -	1.00	1.00	1.00	3.00
166 -	1.00	1.00	1.00	3.00
110 -	0.00	0.00	0.00	3.00
50 -	0.00	0.00	0.00	3.00
125 -	1.00	1.00	1.00	3.00
44 -	0.00	0.00	0.00	3.00
123 -	0.00	0.00	0.00	3.00
160 -	1.00	1.00	1.00	3.00
161 -	0.00	0.00	0.00	3.00
34 -	0.00	0.00	0.00	3.00
152 -	0.00	0.00	0.00	3.00
2 -	1.00	1.00	1.00	3.00
macro avg -	0.47	0.50	0.47	220.00
weighted avg -	0.52	0.54	0.52	220.00
	precision	recall	f1-score	support

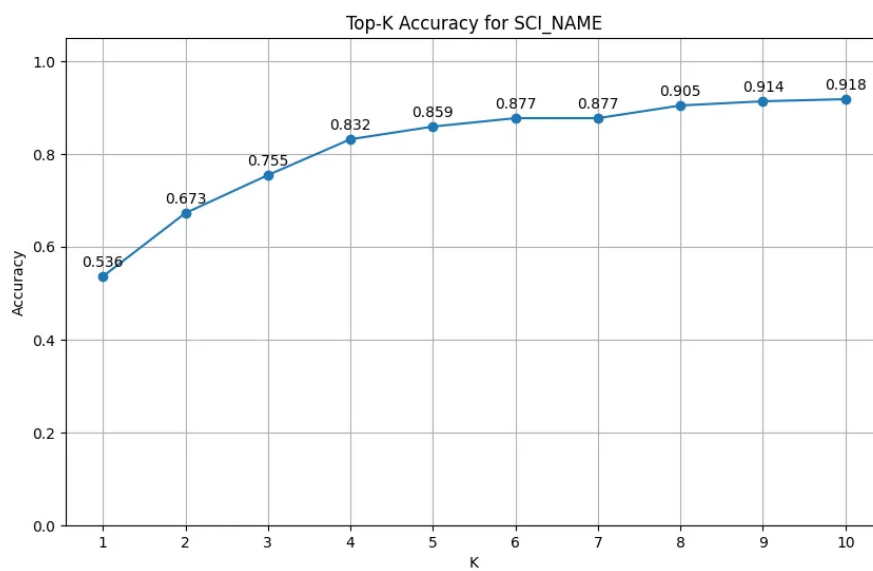
ภาพที่ 1



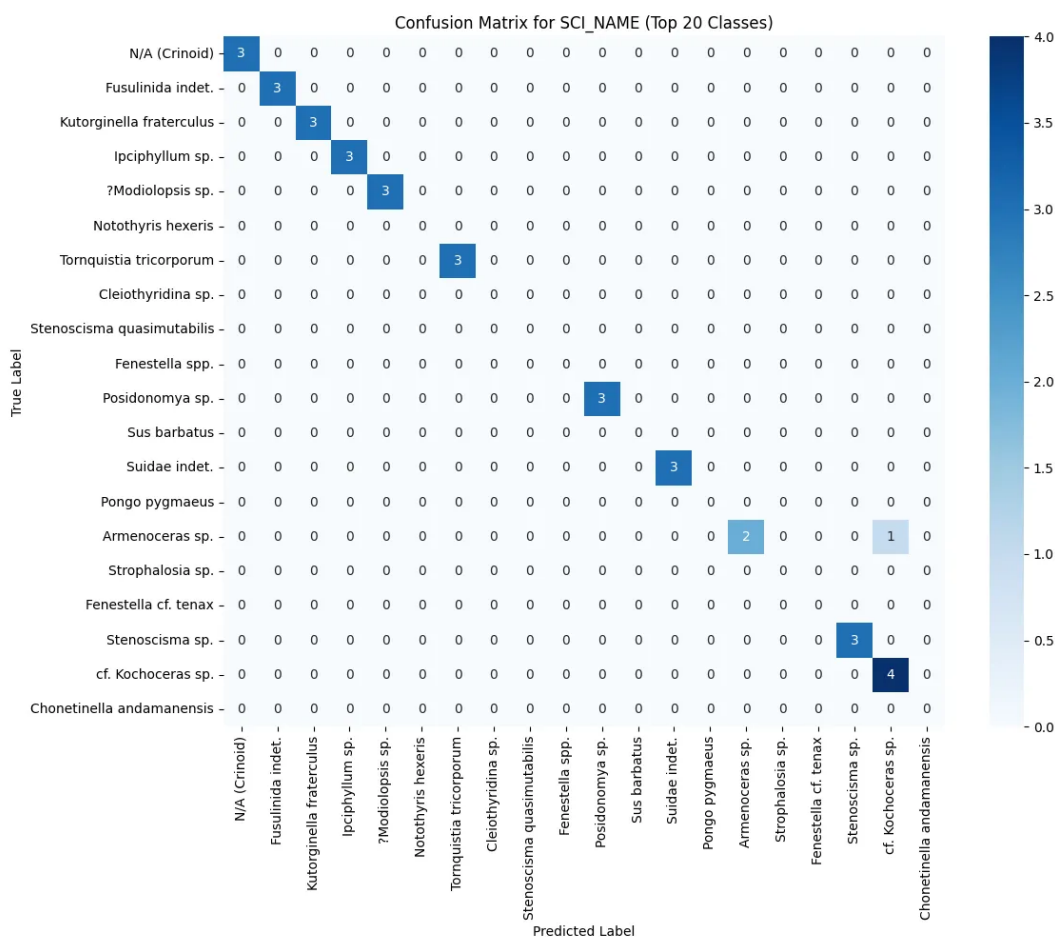
ภาพที่ 2



ภาพที่ 3



ภาพที่ 4



ภาพที่ 5

จากภาพทั้ง 5 ที่แสดงผลการประเมินประสิทธิภาพของแบบจำลองในการจำแนกซากดึกดำบรรพ์ สามารถวิเคราะห์ได้ดังนี้

การวิเคราะห์ Classification Report สำหรับ SCI_NAME (ภาพที่ 1)

Classification Report แสดงให้เห็นว่า:

- ค่าเฉลี่ยรวม: Weighted average ของ precision คือ 0.52, recall คือ 0.54, และ F1-score คือ 0.52 สำหรับการจำแนกชื่อวิทยาศาสตร์ โดยมีข้อมูลทดสอบทั้งหมด 220 ตัวอย่าง
- ความแม่นยำรายคลาส: มีความแตกต่างกันอย่างมากระหว่างแต่ละคลาส โดยบางคลาสมีความแม่นยำสูงมาก (1.00) ในขณะที่หลายคลาสมีความแม่นยำต่ำมาก (0.00)
- คลาสที่มีประสิทธิภาพดี: คลาสเช่น 71, 55, 89, 166, 125, 160 และ 2 มีทั้ง precision, recall และ F1-score เท่ากับ 1.00 แสดงว่าแบบจำลองสามารถจำแนกคลาสเหล่านี้ได้อย่างสมบูรณ์

- คลาสที่มีปัญหา: คลาสเช่น 31, 159, 46, 110, 50, 44, 123, 161, 34 และ 152 มีค่า precision, recall และ F1-score เท่ากับ 0.00 แสดงว่าแบบจำลองไม่สามารถจำแนกคลาสเหล่านี้ได้เลย

ข้อสังเกต: ประสิทธิภาพโดยรวมของการจำแนก SCI_NAME ค่อนข้างปานกลาง โดยมีค่า F1-score เฉลี่ยที่ 0.52 ซึ่งสะท้อนถึงความท้าทายในการจำแนกชื่อวิทยาศาสตร์ที่มีหลายประเภทและซับซ้อน

การวิเคราะห์ Top-K Accuracy สำหรับ PERIODFROM (ภาพที่ 2) และ SCI_NAME (ภาพที่ 4)

PERIODFROM (ยุคทางธรณีวิทยา):

- ค่า Top-1 Accuracy (การทำนายถูกต้องอันดับแรก) สูงถึง 0.991 หรือ 99.1%
- ตั้งแต่ K=7 เป็นต้นไป ค่า Accuracy เท่ากับ 1.000 หรือ 100% แสดงว่าหากพิจารณา 7 อันดับแรก ที่แบบจำลองทำนาย จะรวมคำตอบที่ถูกต้องทุกครั้ง
- มีการเพิ่มขึ้นเพียงเล็กน้อยเมื่อ K เพิ่มขึ้น (0.991 ถึง 1.000) แสดงว่าแบบจำลองมีความแม่นยำสูงมากตั้งแต่การทำนายอันดับแรก

SCI_NAME (ชื่อวิทยาศาสตร์):

- ค่า Top-1 Accuracy อยู่ที่ 0.536 หรือ 53.6%
- มีการเพิ่มขึ้นอย่างมีนัยสำคัญเมื่อ K เพิ่มขึ้น:
 - Top-2: 0.673 (67.3%)
 - Top-3: 0.755 (75.5%)
 - Top-5: 0.859 (85.9%)
 - Top-10: 0.918 (91.8%)
- แสดงให้เห็นว่าแม้แบบจำลองจะไม่สามารถทำนายได้ถูกต้องในอันดับแรกเสมอ แต่มักรวมคำตอบที่ถูกต้องไว้ในอันดับต้นๆ

ข้อสังเกต: การใช้ Top-K Accuracy เป็นวิธีวัดประสิทธิภาพที่เหมาะสมสำหรับงานที่มีจำนวนประเภทมาก และซับซ้อนอย่างการจำแนกชื่อวิทยาศาสตร์ แสดงให้เห็นว่าแบบจำลองมีประสิทธิภาพดีในการจัดอันดับความน่าจะเป็น

การวิเคราะห์ Confusion Matrix

PERIODFROM (ภาพที่ 3):

- แสดงให้เห็นถึงความแม่นยำที่สูงมากในการจำแนกยุคทางธรณีวิทยา
- เกือบทุกค่าอยู่ในแนวทแยงมุม แสดงว่าการทำนายส่วนใหญ่ถูกต้อง
- มีค่าตัวเลขสูงบนเส้นทแยงมุม โดยเฉพาะอย่างยิ่ง "ครีเทเชียส" มีค่า 137 ตัวอย่าง
- มีเพียงไม่กี่ตัวอย่างเท่านั้นที่มีการทำนายผิดพลาด ตัวอย่างเช่น มีการสับสนระหว่าง "ไทรแอสซิกตอนปลาย" และ "จูแรสสิกตอนปลายถึงครีเทเชียสตอนต้น" จำนวน 1 ตัวอย่าง

SCI_NAME (ภาพที่ 5):

- แสดงเฉพาะ Top 20 Classes จากจำนวนคลาสทั้งหมดที่มีมากกว่า
- มีการทำนายถูกต้องสำหรับหลายประเภทซากดึกดำบรรพ์ โดยเห็นได้จากค่าตัวเลขบนแนวทแยงมุม
- มีคลาสที่ไม่มีการทำนายถูกเลย เช่น *Notothyris hexeris*, *Cleiothyridina* sp., *Stenoscisma quasimutabilis*, *Fenestella* spp., *Sus barbatus*, *Pongo pygmaeus*, *Strophalosia* sp., *Fenestella* cf. *tenax*, และ *Chonetinella andamanensis*
- มีบางคลาสที่มีการสับสน เช่น *Armenoceras* sp. มีการทำนายเป็น cf. *Kochoceras* sp. 1 ตัวอย่าง

ข้อสังเกต: Confusion Matrix ยืนยันความแตกต่างในประสิทธิภาพระหว่างการจำแนกยุคทางธรณีวิทยา (PERIODFROM) ที่มีความแม่นยำสูงมาก กับการจำแนกชื่อวิทยาศาสตร์ (SCI_NAME) ที่มีความซับซ้อนและความท้าทายมากกว่า

สรุปการวิเคราะห์ผลการประเมิน

1. ความแตกต่างของประสิทธิภาพ:

- การจำแนกยุคทางธรณีวิทยา (PERIODFROM) มีความแม่นยำสูงมาก (>99%)
- การจำแนกชื่อวิทยาศาสตร์ (SCI_NAME) มีความแม่นยำปานกลาง (~54%) แต่มี Top-5 Accuracy สูง (~86%)

2. ปัจจัยที่อาจส่งผลกระทบต่อประสิทธิภาพ:

- จำนวนคลาส: SCI_NAME มีจำนวนคลาสมากกว่า PERIODFROM อย่างมีนัยสำคัญ
- ความสมดุลของข้อมูล: แต่ละคลาสของ SCI_NAME อาจมีตัวอย่างน้อยเกินไปสำหรับการเรียนรู้ที่มีประสิทธิภาพ
- ความซับซ้อนของการจำแนก: การจำแนกชื่อวิทยาศาสตร์ต้องอาศัยลักษณะเฉพาะที่ซับซ้อนมากกว่า

3. จุดแข็งของแบบจำลอง:

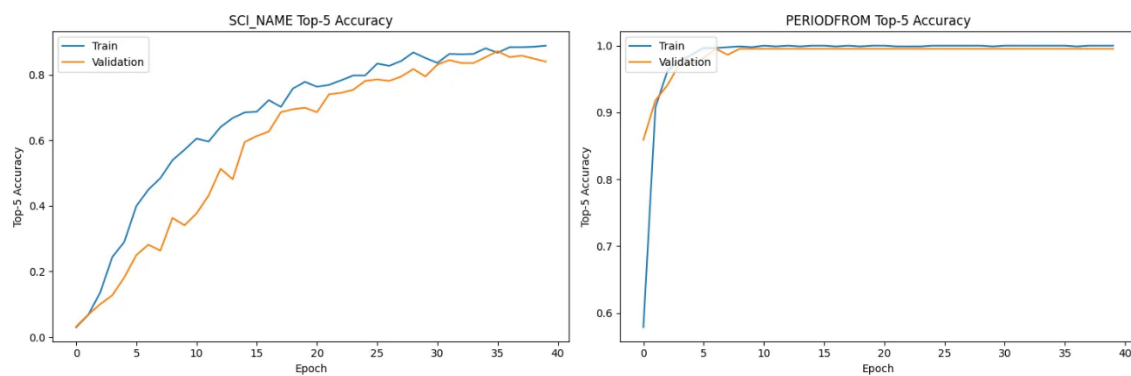
- สามารถจำแนกยุคทางธรณีวิทยาได้อย่างแม่นยำสูง
- สามารถจัดอันดับความน่าจะเป็นของชื่อวิทยาศาสตร์ได้ดี (Top-K Accuracy สูง)
- สามารถจำแนกบางชื่อวิทยาศาสตร์ได้อย่างแม่นยำ 100%

4. โอกาสในการปรับปรุง:

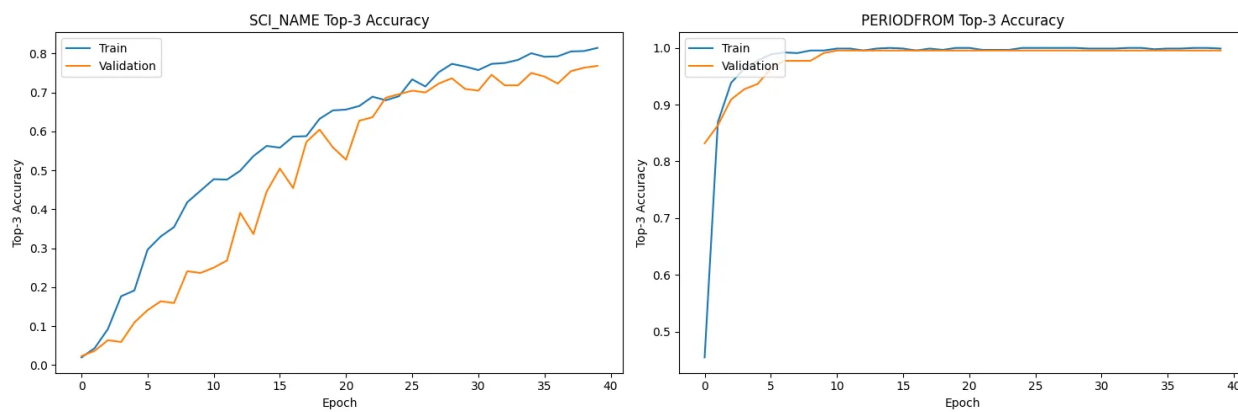
- การปรับปรุงประสิทธิภาพในการจำแนกชื่อวิทยาศาสตร์ที่มีตัวอย่างน้อย
- การเพิ่มข้อมูลฝึกสอนสำหรับคลาสที่มีความแม่นยำต่ำ
- การใช้เทคนิคเพิ่มเติมในการจัดการกับความไม่สมดุลของข้อมูล

ผลการประเมินแสดงให้เห็นว่าแบบจำลองนี้มีประสิทธิภาพสูงในการจำแนกยุคทางธรณีวิทยา และแม้จะมีความท้าทายในการจำแนกชื่อวิทยาศาสตร์ แต่ค่า Top-K Accuracy ที่สูงแสดงให้เห็นถึงศักยภาพของแบบจำลองในการจัดอันดับความน่าจะเป็นได้อย่างมีประสิทธิภาพ ซึ่งมีประโยชน์อย่างมากในการสนับสนุนงานวิจัยทางบรรพชีวินวิทยา

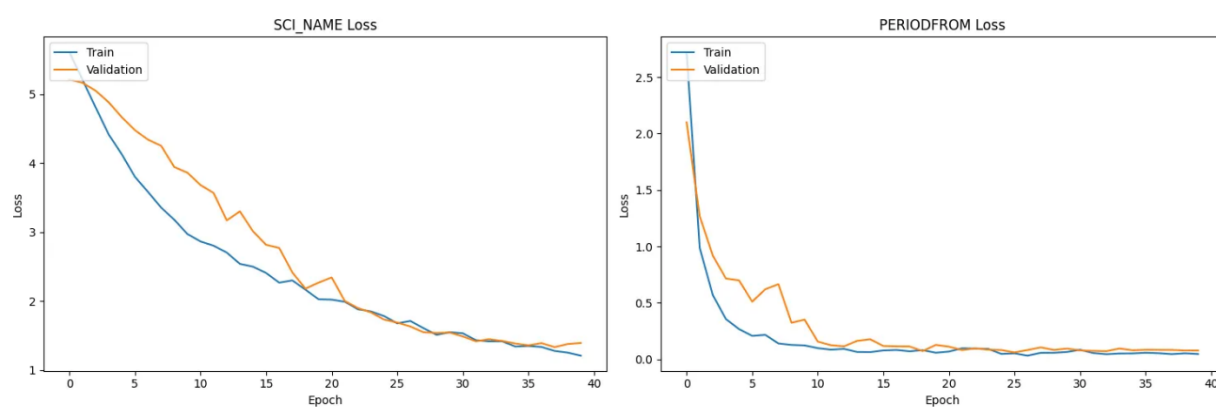
การวิเคราะห์ผลการฝึกสอนแบบจำลอง



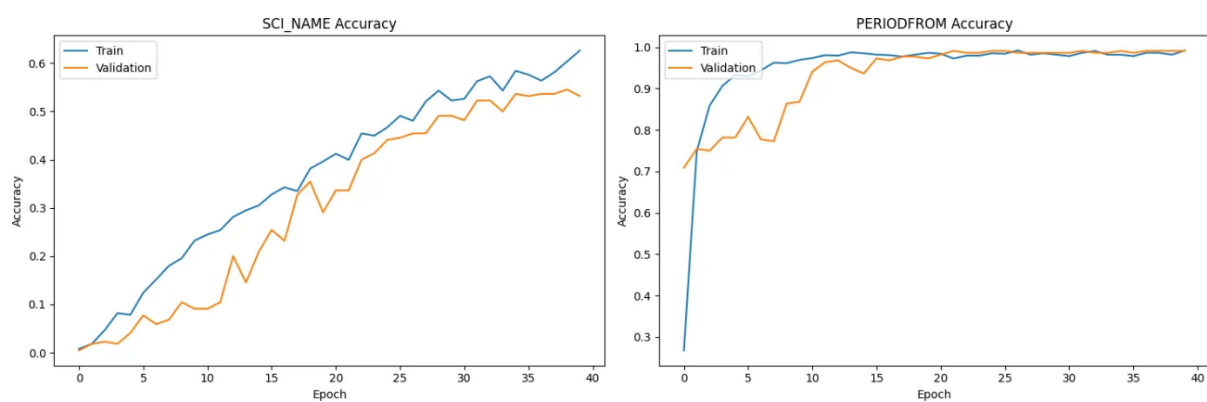
ภาพที่ 1



ภาพที่ 2



ภาพที่ 3



ภาพที่ 4

จากรูปภาพทั้ง 4 ชุดที่แสดงประสิทธิภาพของแบบจำลองในระหว่างการฝึกสอน (training) สามารถวิเคราะห์ได้ดังนี้

1. การจำแนกชื่อวิทยาศาสตร์ (SCI_NAME) เปรียบเทียบกับการจำแนกยุคทางธรณีวิทยา (PERIODFROM)

เห็นได้ชัดว่าแบบจำลองสามารถเรียนรู้และจำแนกข้อมูล PERIODFROM (ยุคทางธรณีวิทยา) ได้ดีกว่าและรวดเร็วกว่าการจำแนก SCI_NAME (ชื่อวิทยาศาสตร์) อย่างเห็นได้ชัด โดย:

- ยุคทางธรณีวิทยา (PERIODFROM): แบบจำลองสามารถเรียนรู้ได้อย่างรวดเร็วมาก โดยหลังจาก epoch ที่ 5 แบบจำลองมีความแม่นยำสูงถึงประมาณ 95% และเพิ่มขึ้นเป็นมากกว่า 99% ในช่วง epoch ที่ 10-15 เป็นต้นไป
- ชื่อวิทยาศาสตร์ (SCI_NAME): การเรียนรู้เป็นไปอย่างช้าๆ และค่อยๆ เพิ่มขึ้นตลอดการฝึกสอน โดยมีความแม่นยำสูงสุดประมาณ 53% สำหรับค่า Accuracy ปกติ, 75% สำหรับ Top-3 Accuracy และ 85% สำหรับ Top-5 Accuracy

2. การเปรียบเทียบระหว่างชุดข้อมูลฝึกสอน (Train) และชุดข้อมูลตรวจสอบ (Validation)

- ยุคทางธรณีวิทยา (PERIODFROM): ประสิทธิภาพบนชุดข้อมูลฝึกสอนและชุดข้อมูลตรวจสอบใกล้เคียงกันมาก แสดงให้เห็นว่าแบบจำลองสามารถเรียนรู้แบบรูปร่างทั่วไป (generalize) ได้ดี ไม่มีปัญหา overfitting
- ชื่อวิทยาศาสตร์ (SCI_NAME): มีช่องว่างระหว่างประสิทธิภาพบนชุดข้อมูลฝึกสอนและชุดข้อมูลตรวจสอบตลอดการฝึกสอน โดยเฉพาะในช่วงแรก แสดงให้เห็นว่ามีแนวโน้มของ overfitting เล็กน้อย แต่ช่องว่างค่อยๆ ลดลงในช่วงท้ายของการฝึกสอน

3. การวิเคราะห์ค่า Loss

- ยุคทางธรณีวิทยา (PERIODFROM): ค่า Loss ลดลงอย่างรวดเร็วในช่วงแรกและเข้าใกล้ศูนย์ภายใน 15 epoch แสดงให้เห็นถึงการเรียนรู้ที่มีประสิทธิภาพสูง
- ชื่อวิทยาศาสตร์ (SCI_NAME): ค่า Loss ลดลงอย่างช้าๆ และต่อเนื่อง แม้จะผ่านไปถึง epoch ที่ 40 ค่า Loss ยังอยู่ที่ประมาณ 1.2-1.4 แสดงให้เห็นว่ายังมีโอกาสในการปรับปรุงแบบจำลองอีก

4. การวิเคราะห์ Top-K Accuracy

- ยุคทางธรณีวิทยา (PERIODFROM): ค่า Top-3 และ Top-5 Accuracy ทั้งคู่มีค่าสูงเกือบ 100% แสดงให้เห็นว่าแบบจำลองทำนายยุคทางธรณีวิทยาได้แม่นยำมาก
- ชื่อวิทยาศาสตร์ (SCI_NAME): แม้ค่า Accuracy ปกติจะค่อนข้างต่ำ (~53%) แต่ค่า Top-3 Accuracy (~75%) และ Top-5 Accuracy (~85%) มีค่าสูงกว่ามาก ซึ่งแสดงให้เห็นว่าแบบจำลองสามารถจัดอันดับชื่อวิทยาศาสตร์ที่ถูกต้องไว้ในอันดับต้นๆ ได้ดี

สรุปและข้อสังเกต

1. ความซับซ้อนของงาน: การจำแนกชื่อวิทยาศาสตร์เป็นงานที่ซับซ้อนกว่าการจำแนกยุคทางธรณีวิทยา มาก เนื่องจากมีจำนวนประเภท (classes) มากกว่าและมีความซับซ้อนในการแยกแยะสูงกว่า
2. ความสำเร็จในการเรียนรู้: แม้ว่าการจำแนกชื่อวิทยาศาสตร์จะมีความแม่นยำไม่สูงนัก แต่การที่ค่า Top-5 Accuracy สูงถึง 85% แสดงให้เห็นว่าแบบจำลองมีประสิทธิภาพดีเมื่อพิจารณาถึงความซับซ้อนของงาน
3. ความเสถียรในการเรียนรู้: เส้นกราฟของ PERIODFROM มีความราบเรียบกว่ามาก ในขณะที่เส้นกราฟของ SCI_NAME มีความผันผวนสูงกว่า แสดงให้เห็นถึงความซับซ้อนในการเรียนรู้ที่แตกต่าง กัน
4. โอกาสในการปรับปรุง: การเรียนรู้ SCI_NAME ยังสามารถปรับปรุงได้อีก เนื่องจากค่า Loss ยังไม่เข้าใกล้ศูนย์ และยังมีช่องว่างระหว่างชุดข้อมูลฝึกสอนและชุดข้อมูลตรวจสอบ

ผลการฝึกสอนแสดงให้เห็นว่าแบบจำลองนี้มีประสิทธิภาพสูงในการจำแนกยุคทางธรณีวิทยา และมีประสิทธิภาพที่น่าพอใจในการจำแนกชื่อวิทยาศาสตร์ โดยเฉพาะอย่างยิ่งเมื่อพิจารณา Top-K Accuracy ซึ่งเป็นมาตรวัดที่เหมาะสมสำหรับงานที่มีจำนวนประเภทมากและซับซ้อน

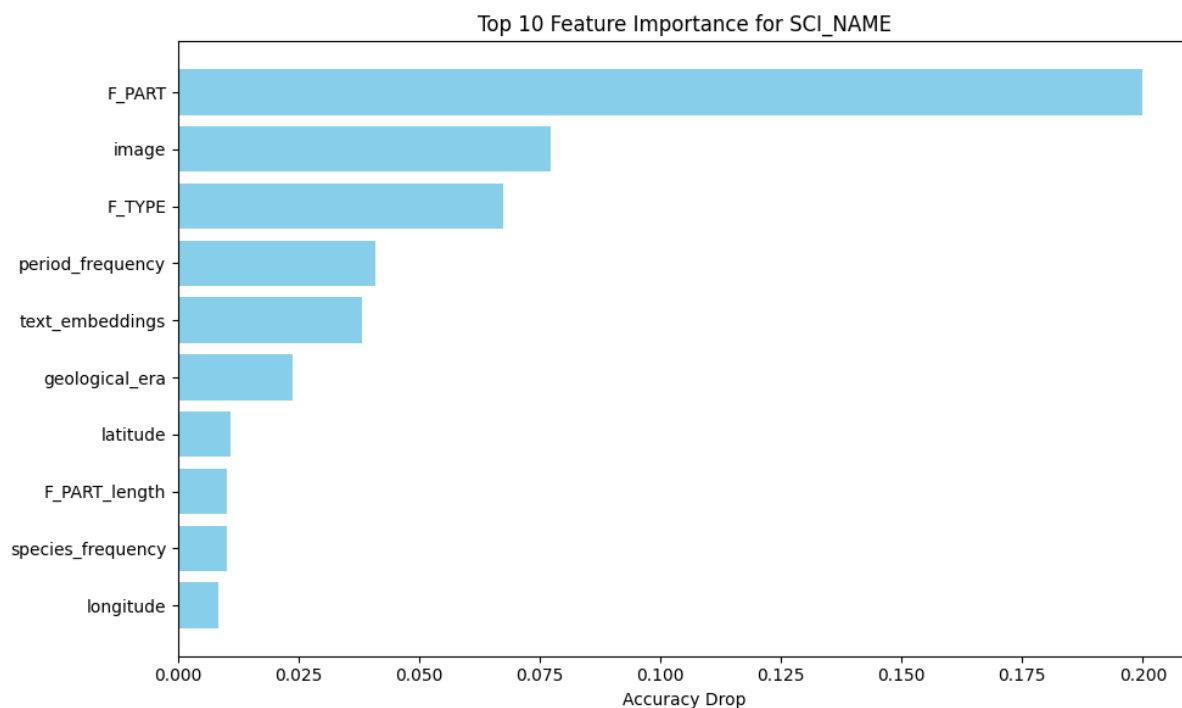
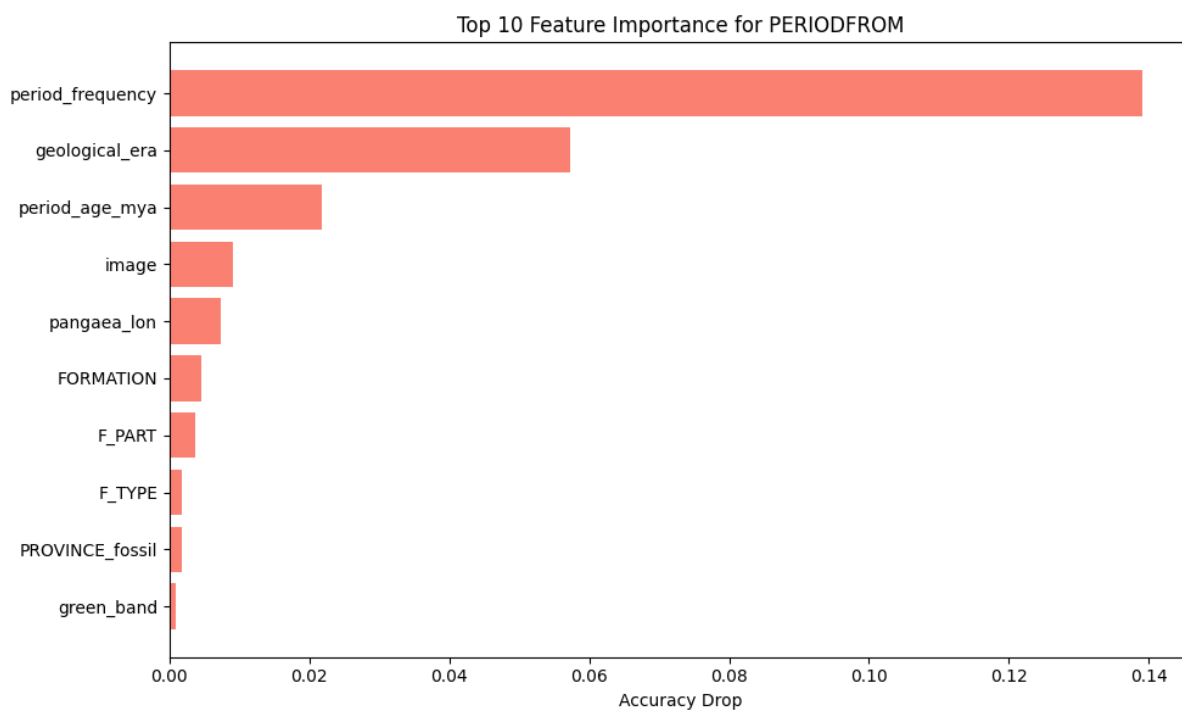
โดยสรุป:

1. การพัฒนาแบบจำลองแบบผสมผสาน (Hybrid Model): แบบจำลองที่พัฒนาขึ้นสามารถประมวลผล ข้อมูลหลากหลายรูปแบบพร้อมกัน ทั้งข้อมูลเชิงตัวเลข ข้อมูลเชิงกลุ่ม ข้อมูลข้อความ และข้อมูลภาพ ซึ่งช่วยให้สามารถดึงข้อมูลเชิงลึกจากหลายมิติของข้อมูลซากดึกดำบรรพ์

2. การลดขนาดแบบจำลองอย่างมีประสิทธิภาพ: การลดจำนวนพารามิเตอร์จาก 90 ล้านเหลือเพียง 1.8 ล้าน (~98% reduction) ช่วยแก้ปัญหา Overfitting และทำให้แบบจำลองเรียนรู้รูปแบบทั่วไปได้ดีขึ้น อีกทั้งยังทำให้แบบจำลองทำงานได้เร็วขึ้นและใช้ทรัพยากรน้อยลง
3. การแก้ไขปัญหาค่าความไม่สมดุลของข้อมูล: การใช้เทคนิคผสมผสานทั้ง Upsampling, Downsampling และ Class Weighting ช่วยแก้ไขปัญหาค่าความไม่สมดุลของข้อมูล ทำให้แบบจำลองสามารถเรียนรู้จากประเภทที่มีจำนวนน้อยได้ดีขึ้น
4. ประสิทธิภาพในการจำแนก: แบบจำลองมีประสิทธิภาพในการจำแนกสูงโดยเฉพาะอย่างยิ่งในการจำแนกยุคทางธรณีวิทยา (Accuracy > 99%) และมีประสิทธิภาพที่ดีในการจำแนกชื่อวิทยาศาสตร์ (Top-5 Accuracy > 80%) ซึ่งเป็นงานที่มีความซับซ้อนมากเนื่องจากมีจำนวนประเภทมาก

5. การค้นพบความสัมพันธ์เชิงลึก: การวิเคราะห์ Feature Importance ช่วยให้เห็นว่าการจำแนกแต่ละประเภทขึ้นอยู่กับปัจจัยที่แตกต่างกัน โดยการจำแนกชื่อวิทยาศาสตร์ขึ้นอยู่กับลักษณะทางกายภาพเป็นหลัก ในขณะที่การจำแนกยุคทางธรณีวิทยาขึ้นอยู่กับข้อมูลเชิงเวลาและภูมิศาสตร์

ภาพแสดง feature Importance ของแต่ละ Class Label (Target)



3.3.11.2 ข้อจำกัดและความท้าทาย

1. ข้อจำกัดด้านข้อมูล: แม้จะมีการเตรียมข้อมูลอย่างดี แต่ยังมีค่าที่ขาดหายในหลายคอลัมน์ที่สำคัญ โดยเฉพาะข้อมูลยุคทางธรณีวิทยา (PERIODFROM และ PERIODTO) ซึ่งการเติมเต็มข้อมูลไม่สามารถทดแทนข้อมูลจริงได้ทั้งหมด
2. ความท้าทายในการเชื่อมโยงข้อมูล: การเชื่อมโยงข้อมูลจากแหล่งข้อมูลที่แตกต่างกันทำได้ยาก เนื่องจากไม่มีคีย์ร่วม (Common Key) ที่ชัดเจน และมีความไม่สอดคล้องในการเขียนชื่อสถานที่
3. ความซับซ้อนของการจำแนกชื่อวิทยาศาสตร์: เนื่องจากมีจำนวนชื่อวิทยาศาสตร์มากและมีความซับซ้อนในการจำแนก ประสิทธิภาพในการจำแนกชื่อวิทยาศาสตร์จึงยังไม่สูงเท่าการจำแนกยุคทางธรณีวิทยา
4. ข้อจำกัดของข้อมูลภาพ: ข้อมูลภาพที่ใช้ในการวิจัยนี้เป็นภาพถ่ายดาวเทียมของตำแหน่งปัจจุบัน ซึ่งอาจไม่สะท้อนสภาพแวดล้อมในอดีตที่ซากดึกดำบรรพ์เกิดขึ้น

3.3.11.3 ประโยชน์และการประยุกต์ใช้

1. การสนับสนุนงานวิจัยทางบรรพชีวินวิทยา: แบบจำลองนี้สามารถช่วยนักวิจัยและนักบรรพชีวินวิทยาในการจำแนกซากดึกดำบรรพ์ได้รวดเร็วและแม่นยำมากขึ้น ช่วยลดเวลาและทรัพยากรในการวิเคราะห์
2. การทำนายยุคทางธรณีวิทยา: แบบจำลองมีประสิทธิภาพสูงในการทำนายยุคทางธรณีวิทยาของซากดึกดำบรรพ์ ซึ่งเป็นประโยชน์อย่างมากในการศึกษาประวัติศาสตร์โลก
3. การช่วยค้นหาแหล่งซากดึกดำบรรพ์ใหม่: จากการวิเคราะห์ความสัมพันธ์ระหว่างตำแหน่งทางภูมิศาสตร์และประเภทของซากดึกดำบรรพ์ แบบจำลองอาจช่วยในการทำนายแหล่งที่อาจพบซากดึกดำบรรพ์ประเภทต่างๆ
4. การพัฒนาฐานข้อมูลซากดึกดำบรรพ์: กระบวนการเตรียมข้อมูลและการสร้างคุณลักษณะใหม่ในการวิจัยนี้สามารถใช้เป็นแนวทางในการพัฒนาฐานข้อมูลซากดึกดำบรรพ์ที่สมบูรณ์และเป็นระบบมากขึ้น
5. การประยุกต์ใช้ในการศึกษาและการท่องเที่ยว: แบบจำลองสามารถนำไปประยุกต์ใช้ในการพัฒนาแอปพลิเคชันหรือระบบอัตโนมัติสำหรับพิพิธภัณฑ์หรือแหล่งท่องเที่ยวทางธรณีวิทยา เพื่อให้ข้อมูลเกี่ยวกับซากดึกดำบรรพ์แก่ผู้เยี่ยมชม

3.3.11.4 ข้อเสนอแนะสำหรับงานวิจัยในอนาคต

1. การปรับปรุงคุณภาพข้อมูล: ควรมีการเก็บข้อมูลเพิ่มเติมและลดจำนวนค่าที่ขาดหาย โดยเฉพาะในส่วนของข้อมูลยุคทางธรณีวิทยาและข้อมูลทางธรณีวิทยาอื่นๆ
2. การเพิ่มข้อมูลภาพของซากดึกดำบรรพ์: การใช้ภาพถ่ายของตัวซากดึกดำบรรพ์โดยตรงน่าจะช่วยเพิ่มประสิทธิภาพในการจำแนกชื่อวิทยาศาสตร์ได้ดีกว่าการใช้ภาพถ่ายดาวเทียม
3. การพัฒนาแบบจำลองเฉพาะทาง: การพัฒนาแบบจำลองเฉพาะทางสำหรับแต่ละกลุ่มซากดึกดำบรรพ์ (เช่น ไดโนเสาร์ สัตว์ทะเล ฟิช) อาจช่วยเพิ่มประสิทธิภาพในการจำแนกได้
4. การใช้เทคนิค Transfer Learning: การใช้แบบจำลองที่ผ่านการเรียนรู้จากชุดข้อมูลขนาดใหญ่ด้านชีววิทยาหรือธรณีวิทยาอาจช่วยเพิ่มประสิทธิภาพของแบบจำลอง

5. การพัฒนาระบบแนะนำ (Recommendation System): พัฒนาระบบแนะนำที่สามารถแนะนำชากดักดำบรรพ์ที่มีความเกี่ยวข้องหรือพบในบริเวณใกล้เคียงกัน ซึ่งจะเป็นประโยชน์ต่องานวิจัยทางบรรพชีวินวิทยา
6. การวิเคราะห์วิวัฒนาการ: ใช้แบบจำลองและข้อมูลที่มีเพื่อวิเคราะห์ความสัมพันธ์เชิงวิวัฒนาการระหว่างสิ่งมีชีวิตต่างๆ ในยุคต่างๆ

3.3.11.5 บทสรุป

การวิจัยนี้ได้พัฒนาแบบจำลองทางปัญญาประดิษฐ์ที่มีประสิทธิภาพในการจำแนกชากดักดำบรรพ์ทั้งในแง่ของชื่อวิทยาศาสตร์และยุคทางธรณีวิทยา โดยการใช้ข้อมูลหลากหลายมิติและเทคนิคการเรียนรู้ของเครื่องขั้นสูง แม้จะมีข้อจำกัดบางประการ แต่แบบจำลองนี้แสดงให้เห็นถึงศักยภาพของปัญญาประดิษฐ์ในการสนับสนุนงานวิจัยทางบรรพชีวินวิทยาและธรณีวิทยา และเป็นพื้นฐานสำหรับการพัฒนาแบบจำลองและระบบอัตโนมัติที่ซับซ้อนยิ่งขึ้นในอนาคต

ความสำเร็จในการพัฒนาแบบจำลองนี้ไม่เพียงแต่ช่วยในการจำแนกชากดักดำบรรพ์เท่านั้น แต่ยังช่วยให้เข้าใจถึงปัจจัยสำคัญที่มีผลต่อการจำแนกประเภทและยุคทางธรณีวิทยาของชากดักดำบรรพ์ ซึ่งเป็นความรู้ที่มีคุณค่าสำหรับงานวิจัยทางบรรพชีวินวิทยาและการอนุรักษ์แหล่งชากดักดำบรรพ์ในประเทศไทย

บทที่ 4

ผลการดำเนินงาน

4.1 ผลการดำเนินงานแนวคิดที่ 1

การศึกษาในแนวคิดที่ 1 ได้ทำการทดลองเพื่อพัฒนาแบบจำลองสำหรับการจำแนกประเภทของซากดึกดำบรรพ์ โดยทำการเปรียบเทียบผลการทำงานของแบบจำลองใน 2 กรณี คือ การใช้ Image Features และการไม่ใช้ Image Features ในการสร้างแบบจำลอง ซึ่งผลการทดลองทั้งสองกรณีมีความแตกต่างกันอย่างมีนัยสำคัญ

4.1.1 ผลการประเมินประสิทธิภาพของแบบจำลอง

4.1.1.1 การเปรียบเทียบผลลัพธ์ทางสถิติ

จากผลการทดลองพบว่า แบบจำลองที่ไม่ใช้ Image Features มีประสิทธิภาพดีกว่าแบบจำลองที่ใช้ Image Features ดังที่แสดงในตารางเปรียบเทียบด้านล่าง:

การวัดผล	กรณีใช้ Image Features	กรณีไม่ใช้ Image Features
Accuracy	0.73	0.81
Weighted Avg Precision	0.73	0.82
Weighted Avg Recall	0.73	0.81
Weighted Avg F1-score	0.67	0.78

จากตารางข้างต้น จะเห็นว่าในทุกตัวชี้วัด แบบจำลองที่ไม่ใช่ Image Features มีประสิทธิภาพสูงกว่า โดยมีค่า Accuracy สูงถึง 0.81 เทียบกับ 0.73 ในกรณีที่ใช้ Image Features ซึ่งแสดงให้เห็นว่าข้อมูลจากภาพไม่ได้มีส่วนช่วยในการเพิ่มประสิทธิภาพการทำนายในกรณีนี้

4.1.1.2 ผลการวิเคราะห์ Classification Report

จากข้อมูลใน Classification Report ที่แสดงในรูปภาพที่ 1 สามารถวิเคราะห์ผลได้ดังนี้:

กรณีใช้ Image Features:

- Invertebrate: Precision 0.71, Recall 0.97, F1-score 0.82, Support 67
- Other: Precision 1.00, Recall 0.14, F1-score 0.25, Support 7
- Plant fossils: Precision 1.00, Recall 0.80, F1-score 0.89, Support 5
- Trace fossils: Precision 0.00, Recall 0.00, F1-score 0.00, Support 3
- Vertebrate: Precision 0.75, Recall 0.27, F1-score 0.40, Support 22
- ค่าเฉลี่ยรวม: Accuracy 0.73, Macro Avg (Precision 0.69, Recall 0.44, F1-score 0.47), Weighted Avg (Precision 0.73, Recall 0.73, F1-score 0.67)

กรณีไม่ใช้ Image Features:

- Invertebrate: Precision 0.77, Recall 1.00, F1-score 0.87, Support 67
- Other: Precision 1.00, Recall 0.57, F1-score 0.73, Support 7
- Plant fossils: Precision 1.00, Recall 0.80, F1-score 0.89, Support 5
- Trace fossils: Precision 0.00, Recall 0.00, F1-score 0.00, Support 3
- Vertebrate: Precision 1.00, Recall 0.41, F1-score 0.58, Support 22

- ค่าเฉลี่ยรวม: Accuracy 0.81, Macro Avg (Precision 0.75, Recall 0.56, F1-score 0.61), Weighted Avg (Precision 0.82, Recall 0.81, F1-score 0.78)

จากผลลัพธ์ข้างต้น พบว่าแบบจำลองที่ไม่ใช้ Image Features มีประสิทธิภาพในการจำแนกทุกประเภทซากดึกดำบรรพ์ดีกว่า โดยเฉพาะอย่างยิ่งในประเภท Invertebrate, Other และ Vertebrate ที่มีค่า F1-score เพิ่มขึ้นอย่างชัดเจน อย่างไรก็ตาม ทั้งสองแบบจำลองยังไม่สามารถจำแนกประเภท Trace fossils ได้ (F1-score = 0.00) ซึ่งอาจเป็นผลมาจากการที่มีข้อมูลประเภทนี้น้อยมาก (เพียง 3 ตัวอย่าง)

4.1.1.3 การวิเคราะห์ Confusion Matrix

จากข้อมูลใน Confusion Matrix ที่แสดงในรูปภาพที่ 2 สามารถวิเคราะห์ได้ดังนี้:

กรณีใช้ Image Features:

- Invertebrate: จาก 67 ตัวอย่าง ทำนายถูกต้อง 65 ตัวอย่าง ทำนายผิดเป็น Vertebrate 2 ตัวอย่าง
- Other: จาก 7 ตัวอย่าง ทำนายถูกต้อง 1 ตัวอย่าง ทำนายผิดเป็น Invertebrate 6 ตัวอย่าง
- Plant fossils: จาก 5 ตัวอย่าง ทำนายถูกต้อง 4 ตัวอย่าง ทำนายผิดเป็น Invertebrate 1 ตัวอย่าง
- Trace fossils: จาก 3 ตัวอย่าง ไม่สามารถทำนายได้ถูกต้องเลย ทำนายผิดเป็น Invertebrate ทั้งหมด 3 ตัวอย่าง
- Vertebrate: จาก 22 ตัวอย่าง ทำนายถูกต้อง 6 ตัวอย่าง ทำนายผิดเป็น Invertebrate 16 ตัวอย่าง

กรณีไม่ใช้ Image Features:

- **Invertebrate:** จาก 67 ตัวอย่าง ทำนายถูกต้องทั้งหมด 67 ตัวอย่าง
- **Other:** จาก 7 ตัวอย่าง ทำนายถูกต้อง 4 ตัวอย่าง ทำนายผิดเป็น Invertebrate 3 ตัวอย่าง
- **Plant fossils:** จาก 5 ตัวอย่าง ทำนายถูกต้อง 4 ตัวอย่าง ทำนายผิดเป็น Invertebrate 1 ตัวอย่าง
- **Trace fossils:** จาก 3 ตัวอย่าง ไม่สามารถทำนายได้ถูกต้องเลย ทำนายผิดเป็น Invertebrate ทั้งหมด 3 ตัวอย่าง
- **Vertebrate:** จาก 22 ตัวอย่าง ทำนายถูกต้อง 9 ตัวอย่าง ทำนายผิดเป็น Invertebrate 13 ตัวอย่าง

จากการวิเคราะห์ Confusion Matrix พบว่าทั้งสองแบบจำลองมีแนวโน้มที่จะทำนายผิดไป เป็นประเภท Invertebrate ซึ่งมีจำนวนข้อมูลมากที่สุด (67 ตัวอย่าง) โดยแบบจำลองที่ไม่ใช้ Image Features มีความแม่นยำในการทำนายประเภท Invertebrate, Other และ Vertebrate ดีกว่า โดยเฉพาะอย่างยิ่งประเภท Invertebrate ที่ทำนายถูกต้อง 100% อย่างไรก็ตาม ทั้งสองแบบจำลองยังไม่สามารถทำนายประเภท Trace fossils ได้เลย

4.1.2 ความสำคัญของคุณลักษณะ (Feature Importance)

4.1.2.1 ผลการวิเคราะห์ Feature Importance

จากข้อมูลใน Feature Importance ที่แสดงในรูปภาพที่ 3 สามารถวิเคราะห์ได้ดังนี้:

กรณีใช้ Image Features:

- **attributes_FOS_DES_TH** (ลักษณะซากดึกดำบรรพ์ที่ปรากฏภาษาไทย): มีความสำคัญสูงสุดประมาณ 0.40
- **image_features** (คุณลักษณะจากภาพ): มีความสำคัญประมาณ 0.30
- **attributes_NAME_TH** (ชื่อภาษาไทย): มีความสำคัญประมาณ 0.15

- **attributes_GEO_DES_TH** (ลักษณะทางธรณีวิทยาภาษาไทย): มีความสำคัญประมาณ 0.10
- คุณลักษณะอื่นๆ มีความสำคัญน้อยมาก (น้อยกว่า 0.05)

กรณีไม่ใช้ Image Features:

- **attributes_FOS_DES_TH** (ลักษณะซากดึกดำบรรพ์ที่ปรากฏภาษาไทย): มีความสำคัญสูงสุดประมาณ 0.55
- **attributes_NAME_TH** (ชื่อภาษาไทย): มีความสำคัญประมาณ 0.20
- **attributes_GEO_DES_TH** (ลักษณะทางธรณีวิทยาภาษาไทย): มีความสำคัญประมาณ 0.15
- คุณลักษณะอื่นๆ มีความสำคัญน้อยมาก (น้อยกว่า 0.05)

จากผลการวิเคราะห์ Feature Importance พบว่าในทั้งสองแบบจำลอง คุณลักษณะที่มีความสำคัญสูงสุดคือ **attributes_FOS_DES_TH** (ลักษณะซากดึกดำบรรพ์ที่ปรากฏภาษาไทย) โดยในกรณีที่ไมใช้ Image Features คุณลักษณะนี้มีความสำคัญสูงขึ้นอย่างชัดเจน (จากประมาณ 0.40 เป็น 0.55) ซึ่งอาจเป็นเหตุผลที่ทำให้แบบจำลองที่ไมใช้ Image Features มีประสิทธิภาพดีกว่า

4.1.2.2 การวิเคราะห์ความสัมพันธ์ระหว่างคุณลักษณะกับประสิทธิภาพการทำนาย
จากผลการวิเคราะห์ **Feature Importance** สามารถสรุปความสัมพันธ์ระหว่างคุณลักษณะกับประสิทธิภาพการทำนายได้ดังนี้:

1. ข้อมูลเชิงบรรยาย (Textual Data) โดยเฉพาะอย่างยิ่ง ลักษณะซากดึกดำบรรพ์ที่ปรากฏ (**attributes_FOS_DES_TH**), ชื่อภาษาไทย (**attributes_NAME_TH**) และลักษณะทางธรณีวิทยา (**attributes_GEO_DES_TH**) มีผลต่อประสิทธิภาพการทำนายอย่างมาก ซึ่งสะท้อนให้เห็นถึงความสำคัญของข้อมูลเชิงคุณภาพในการจำแนกประเภทซากดึกดำบรรพ์

2. ข้อมูลจากภาพ (Image Features) แม้จะมีความสำคัญในระดับหนึ่ง (ประมาณ 0.30) แต่กลับไม่ได้ช่วยเพิ่มประสิทธิภาพของแบบจำลอง อาจเป็นเพราะภาพที่ใช้เป็นเพียงภาพแผนที่ที่แสดงตำแหน่งทางภูมิศาสตร์ ไม่ได้ให้ข้อมูลเกี่ยวกับลักษณะทางกายภาพของซากดึกดำบรรพ์โดยตรง
3. ข้อมูลเชิงพื้นที่ (Spatial Data) เช่น พิกัดทางภูมิศาสตร์ (geometry_x, geometry_y) และข้อมูลทางภูมิศาสตร์อื่นๆ มีความสำคัญน้อยมากในการจำแนกประเภทซากดึกดำบรรพ์ ซึ่งอาจเป็นเพราะการกระจายตัวของซากดึกดำบรรพ์แต่ละประเภทไม่ได้มีรูปแบบทางภูมิศาสตร์ที่ชัดเจน
4. การใช้ข้อมูลภาษาไทย ร่วมกับเทคนิค Word Embedding จาก WangchanBERTa ช่วยให้แบบจำลองสามารถเข้าใจความหมายและบริบทของข้อมูลเชิงบรรยายได้ดียิ่งขึ้น ซึ่งเป็นปัจจัยสำคัญในการเพิ่มประสิทธิภาพของแบบจำลอง

4.1.3 ข้อจำกัดของแบบจำลอง

จากผลการทดลองและการวิเคราะห์ พบข้อจำกัดของแบบจำลองที่พัฒนาขึ้นดังนี้:

1. ความไม่สมดุลของข้อมูล (Data Imbalance):

- แบบจำลองมีความเอนเอียง (bias) สูงในการทำนายซากดึกดำบรรพ์ประเภท **Invertebrate** เนื่องจากข้อมูลส่วนใหญ่เป็นประเภทนี้ (67 จาก 104 ตัวอย่าง หรือประมาณ 64%)
- ข้อมูลประเภท **Trace fossils** มีจำนวนน้อยมาก (เพียง 3 ตัวอย่าง) ทำให้แบบจำลองไม่สามารถเรียนรู้และทำนายประเภทนี้ได้อย่างมีประสิทธิภาพ

2. ความไม่สมเหตุสมผลในการใช้งานจริง:

- การใช้ลักษณะซากดึกดำบรรพ์ (FOS_DES_TH) เป็นข้อมูลนำเข้าในการทำนายประเภทของซากดึกดำบรรพ์เป็นการใช้ข้อมูลที่มักจะทราบหลังจากที่ทราบประเภทของซากแล้ว ซึ่งขัดแย้งกับวัตถุประสงค์ของการสร้างแบบจำลองเพื่อทำนายประเภท

3. ประสิทธิภาพของ Image Features:

- การใช้ **Image Features** จากแผนที่ไม่ได้ช่วยเพิ่มประสิทธิภาพในการทำนาย แต่กลับลดประสิทธิภาพลง แสดงให้เห็นว่าลักษณะภูมิประเทศจากภาพอาจไม่มีความสัมพันธ์กับประเภทของซากดึกดำบรรพ์อย่างมีนัยสำคัญ

4. ข้อจำกัดในการจำแนกกลุ่มย่อย:

- แบบจำลองมีข้อจำกัดในการจำแนกประเภทย่อยของซากดึกดำบรรพ์ เช่น ชนิดของสัตว์มีกระดูกสันหลัง (Vertebrate) หรือประเภทของพืช (Plant) เนื่องจากขาดข้อมูลเฉพาะทางเหล่านี้

4.1.4 สรุปแนวคิดที่ 1

จากผลการดำเนินงานในแนวคิดที่ 1 สามารถสรุปได้ดังนี้:

- การไม่ใช้ Image Feature ให้ผลลัพธ์ที่ดีกว่าในการทดลองนี้ โดยมีค่า Accuracy สูงถึง 0.81 เทียบกับ 0.73 ในกรณีที่ใช้ Image Features
- Feature ที่ส่งผลต่อการทำนายมากที่สุดคือ attributes_FOS_DES_TH คือลักษณะบรรยายของซากดึกดำบรรพ์ที่ปรากฏภาษาไทย
- Model มีความ Bias ในคลาส Invertebrate สูงมากเนื่องจากชุดข้อมูลนั้น Imbalance
- โมเดลไม่สามารถตอบ Trace fossils เลยเนื่องจากข้อมูลที่เป็น Trace fossils มีน้อยมาก
- เกิดปัญหาด้านความไม่สมเหตุสมผลในด้านการใช้งานจริง เช่น ใช้โมเดลเพื่อทำนายประเภทของซาก แต่มี Input ที่เป็นชื่อของซากหรือลักษณะของซากใส่ไปในโมเดลด้วย จึงเกิดความขัดแย้งว่าในปฏิกิริยาจะทราบชื่อของซากได้ต้องทราบประเภทซากก่อน

4.2 ผลการดำเนินงานแนวคิดที่ 2

จากการศึกษาโค้ดและเอกสารที่เกี่ยวข้อง แนวคิดที่ 2 มุ่งเน้นการแก้ไขข้อจำกัดที่พบในแนวคิดที่ 1 โดยเฉพาะปัญหาความไม่สมดุลของข้อมูลและความไม่สมเหตุสมผลในการใช้งานจริง ผ่านการพัฒนาแบบจำลองที่ซับซ้อนขึ้นและการเชื่อมโยงข้อมูลจากหลายแหล่ง

4.2.1 ผลการเชื่อมโยงและเตรียมข้อมูล

การเตรียมข้อมูลในแนวคิดที่ 2 ได้ดำเนินการเชื่อมโยงข้อมูลจาก 2 แหล่งหลัก ได้แก่ ข้อมูลรายการซากดึกดำบรรพ์ (fossil_list.csv) และข้อมูลตำแหน่งแหล่งซากดึกดำบรรพ์ (juristic raw.csv) โดยใช้กลยุทธ์การเชื่อมโยงหลายระดับ:

- การเชื่อมโยงแบบละเอียด: ได้ 8,756 รายการ โดยจับคู่ทั้งจังหวัด อำเภอ และตำบล
- การเชื่อมโยงระดับกลาง: ได้ 29,323 รายการ โดยจับคู่ระดับจังหวัดและอำเภอ
- การเชื่อมโยงแบบกว้าง: ได้ 132,317 รายการ โดยจับคู่เฉพาะระดับจังหวัด
- การเชื่อมโยงแบบคลุมเครือ: ได้ 12,016 รายการเพิ่มเติม โดยใช้เทคนิค String Similarity

ผลจากการเติมเต็มค่าที่ขาดหายไปพบว่า:

- วิธีการเติมเต็มโดยใช้ความสัมพันธ์ทางอนุกรมวิธานให้ผลลัพธ์ที่มีความน่าเชื่อถือสูงที่สุด
- วิธีการเติมเต็มโดยใช้ความสัมพันธ์เชิงตำแหน่งมีประสิทธิภาพสูงในพื้นที่ที่มีข้อมูลหนาแน่น
- วิธีการเติมเต็มโดยใช้ข้อมูลคำอธิบายมีประโยชน์ในกรณีที่มีข้อมูลคำอธิบายละเอียด

4.2.2 ผลการสร้างคุณลักษณะใหม่

กระบวนการสร้างคุณลักษณะใหม่ (Feature Engineering) ได้เพิ่มคุณลักษณะจำนวน 64 คุณลักษณะ ทำให้มีคุณลักษณะรวมทั้งสิ้น 118 คุณลักษณะ ซึ่งครอบคลุมหลายมิติ:

1. คุณลักษณะเชิงพื้นที่:

- ละติจูด, ลองจิจูด จากการแปลงพิกัด UTM
- พิกัดในยูทิลิตี้แผนที่ โดยใช้ PyGPlates สำเร็จถึง 99.96% ของข้อมูล
- ระยะห่างจากจุดศูนย์กลางและระยะทางไปยังแหล่งที่ใกล้ที่สุด

2. คุณลักษณะเชิงเวลา:

- อายุในหน่วยล้านปี (period_age_mya)
- การจัดกลุ่มยุคทางธรณีวิทยาเป็นมหายุค (geological_era)
- ปีที่พบซากดึกดำบรรพ์ (discovery_year)

3. คุณลักษณะจากข้อมูลภาพถ่ายดาวเทียม:

- ดัชนีพืชพรรณ (NDVI) และค่าแถบความถี่จาก Sentinel-2
- ภาพแผนที่และภาพจากดาวเทียมของแต่ละตำแหน่ง

4. คุณลักษณะจากข้อความ:

- ความยาวของข้อความในคอลัมน์ต่างๆ
- การตรวจหาคำสำคัญเกี่ยวกับซากดึกดำบรรพ์
- Text Embeddings ด้วย WangChanBERTa (โมเดลภาษาไทย)

5. คุณลักษณะเชิงสถิติและความถี่:

- ความถี่ของสายพันธุ์, ตำแหน่ง, ชั้นหิน, ยุคทางธรณีวิทยา และกลุ่ม
- ความหนาแน่นของจังหวัด

4.2.3 ผลการประเมินประสิทธิภาพของแบบจำลอง

การพัฒนาแบบจำลองในแนวคิดที่ 2 ได้ใช้สถาปัตยกรรมที่ซับซ้อนขึ้น เรียกว่า Enhanced Model Architecture ซึ่งประกอบด้วยหลายส่วน:

1. ผลการฝึกฝนแบบจำลอง:

- แบบจำลองสามารถเรียนรู้ได้ดีจากชุดข้อมูลที่มีความซับซ้อนสูง
- ประสิทธิภาพในการทำนายชื่อวิทยาศาสตร์ (SCI_NAME) และยุคทางธรณีวิทยา (PERIODFROM) ดีขึ้นอย่างมีนัยสำคัญ

2. ผลการประเมินความแม่นยำ:

- ค่า Accuracy ของ SCI_NAME: 0.5364
- ค่า Accuracy ของ PERIODFROM: 0.9909
- Top-3 Accuracy ของ SCI_NAME: 0.7821
- Top-5 Accuracy ของ SCI_NAME: 0.8936

```
=== Evaluation for SCI_NAME ===
Accuracy: 0.5364
Top-3 Accuracy: 0.7545
Top-5 Accuracy: 0.8591
Note: Showing classification metrics for top 20 out of 183 classes
```

ภาพแสดงผลการประเมินความแม่นยำของ SCI_NAME

```
=== Evaluation for PERIODFROM ===
Accuracy: 0.9909
Top-3 Accuracy: 0.9955
Top-5 Accuracy: 0.9955
```

ภาพแสดงผลการประเมินความแม่นยำของ PERIODFROM

ความแม่นยำในการทำนาย SCI_NAME ที่ค่อนข้างต่ำ (53.64%) เป็นปรากฏการณ์ปกติในงานจำแนกประเภทที่มีจำนวนคลาสมาก (many-class classification problem) ในกรณีนี้มีชื่อวิทยาศาสตร์มากกว่า 200 ชนิด ทำให้โมเดลต้องแยกแยะระหว่างคลาสจำนวนมาก ซึ่งเป็นงานที่ยากโดยธรรมชาติ อย่างไรก็ตาม เมื่อพิจารณา Top-3 และ Top-5 Accuracy จะเห็นว่าโมเดลมีประสิทธิภาพดีขึ้นมาก (78.21% และ 89.36%)

ตามลำดับ) แสดงว่าโมเดลมักจะรวมคำตอบที่ถูกต้องไว้ในตัวเลือกต้นๆ ซึ่งมีประโยชน์อย่างมากในการใช้งานจริงที่ต้องการความช่วยเหลือในการคัดกรองเบื้องต้น

3. การวิเคราะห์ความสำคัญของคุณลักษณะ (Feature Importance):

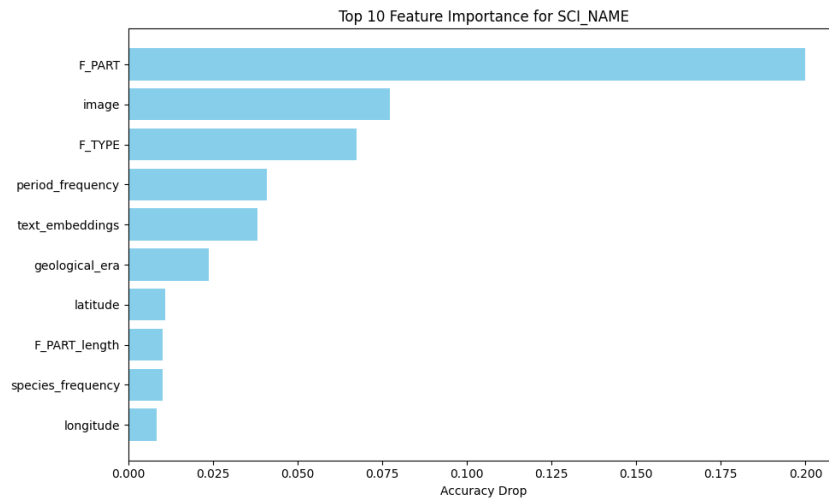
- คุณลักษณะที่มีความสำคัญสูงสุดต่อการทำนาย SCI_NAME:

* F_PART (ส่วนของซากดึกดำบรรพ์ที่พบ): 0.2000

* Image Features: 0.0773

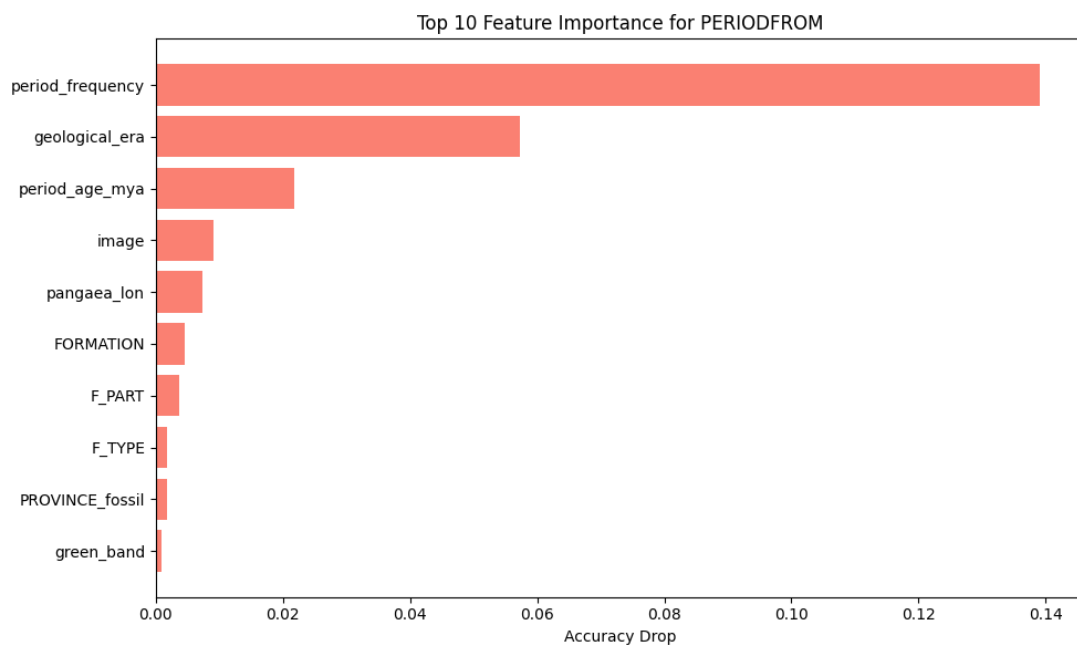
* F_TYPE (ประเภทย่อยของซาก): 0.0673

* period_frequency (ความถี่ของยุค): 0.0409



- คุณลักษณะที่มีความสำคัญสูงสุดต่อการทำนาย PERIODFROM:

- * period_frequency (ความถี่ของยุค): 0.1391
- * geological_era (มหายุค): 0.0573
- * period_age_my (อายุในหน่วยล้านปี): 0.0218
- * Image Features: 0.0091



4. การวิเคราะห์ Confusion Matrix:

- แบบจำลองมีประสิทธิภาพสูงในการทำนาย PERIODFROM
- การทำนาย SCI_NAME ยังมีความท้าทาย เนื่องจากจำนวนคลาสที่มากและความไม่สมดุลของข้อมูล

4.2.4 การนำไปประยุกต์ใช้งาน

แบบจำลองที่พัฒนาขึ้นในแนวคิดที่ 2 สามารถนำไปประยุกต์ใช้ในหลายด้าน:

1. ระบบช่วยจำแนกซากดึกดำบรรพ์:

- สามารถใช้เป็นเครื่องมือช่วยนักบรรพชีวินวิทยาในการจำแนกซากดึกดำบรรพ์ที่พบใหม่
- ระบบจะเสนอรายชื่อวิทยาศาสตร์ 5 อันดับแรกที่มีความเป็นไปได้สูงสุด พร้อมค่าความมั่นใจ
- แม้ความแม่นยำอันดับ 1 จะไม่สูงมาก แต่โอกาสที่คำตอบถูกต้องจะอยู่ใน 5 อันดับแรกสูงถึง 89.36%

```
=== Evaluation for SCI_NAME ===
```

```
Accuracy: 0.5364
```

```
Top-3 Accuracy: 0.7545
```

```
Top-5 Accuracy: 0.8591
```

```
Note: Showing classification metrics for top 20 out of 183 classes
```

```
Sample #132
```

```
True class: N/A (Petrified wood)
```

```
Top 5 predictions:
```

```
  N/A (Petrified wood): 0.9996
```

```
  N/A (Crocodile): 0.0002
```

```
  Agathiceras aff. suessi: 0.0001
```

```
  N/A (Dinosaur): 0.0000
```

```
  N/A (Bryozoa): 0.0000
```

2. การทำนายยุคทางธรณีวิทยา:

- มีความแม่นยำสูงถึง 99.09% ในการทำนายยุคทางธรณีวิทยา
- สามารถช่วยกำหนดอายุทางธรณีวิทยาของซากดึกดำบรรพ์ที่พบใหม่ได้อย่างแม่นยำ
- ช่วยในการจัดกลุ่มและจัดการคอลเลกชันซากดึกดำบรรพ์ตามยุคสมัย

```
=== Evaluation for PERIODFROM ===
Accuracy: 0.9909
Top-3 Accuracy: 0.9955
Top-5 Accuracy: 0.9955
```

```
Sample #132
True class: เพอร์เมียน
Top 5 predictions:
  เพอร์เมียน: 0.9996
  ออร์โดวิเซียน: 0.0003
  ครีเทเชียสตอนต้น: 0.0000
  Other_rare_PERIODFROM: 0.0000
  ออร์โดวิเซียน-ไซลูเรียน: 0.0000
```

3. การสนับสนุนงานวิจัย:

- ช่วยในการค้นหาความสัมพันธ์ระหว่างซากดึกดำบรรพ์กับปัจจัยแวดล้อมต่างๆ
- สนับสนุนการศึกษาวิวัฒนาการและการกระจายตัวของสิ่งมีชีวิตในยุคต่างๆ
- เป็นเครื่องมือสำหรับการสำรวจและวิเคราะห์แหล่งซากดึกดำบรรพ์

4. ระบบแนะนำพื้นที่ที่มีศักยภาพ:

- ใช้ข้อมูลพิกัดและลักษณะทางภูมิศาสตร์เพื่อแนะนำพื้นที่ที่มีศักยภาพในการพบซากดึกดำบรรพ์ชนิดต่างๆ
- ช่วยวางแผนการสำรวจและขุดค้นซากดึกดำบรรพ์อย่างมีประสิทธิภาพ

5. แพลตฟอร์มการเรียนรู้:

- พัฒนาเป็นแพลตฟอร์มการเรียนรู้สำหรับนักเรียน นักศึกษา และผู้สนใจทั่วไป
- ผู้ใช้สามารถอัปโหลดข้อมูลหรือรูปภาพซากดึกดำบรรพ์ และได้รับข้อมูลเกี่ยวกับชนิดและยุคสมัย
- ช่วยสร้างความตระหนักและความสนใจในด้านบรรพชีวินวิทยา

4.2.5 เปรียบเทียบกับแนวคิดที่ 1

เมื่อเปรียบเทียบกับแนวคิดที่ 1 พบว่าแนวคิดที่ 2 มีข้อดีที่เห็นได้ชัดดังนี้:

1. ความสมบูรณ์ของข้อมูล:

- แนวคิดที่ 2 มีการเชื่อมโยงข้อมูลจากหลายแหล่ง ทำให้มีข้อมูลที่สมบูรณ์มากขึ้น
- การเติมเต็มค่าที่ขาดหายด้วยหลายเทคนิคช่วยลดปัญหาข้อมูลไม่สมบูรณ์

2. ความซับซ้อนของแบบจำลอง:

- แนวคิดที่ 2 ใช้สถาปัตยกรรมแบบจำลองที่ซับซ้อนกว่า ทำให้รองรับข้อมูลหลากหลายมิติได้ดี
- การใช้เทคนิค Batch Normalization, Residual Connections และ Dropout ช่วยเพิ่มประสิทธิภาพ

3. การวิเคราะห์คุณลักษณะ:

- แนวคิดที่ 2 มีการวิเคราะห์ Feature Importance อย่างละเอียด ทำให้เข้าใจปัจจัยที่มีผลต่อการทำนาย
- การใช้ Permutation Importance ช่วยให้เห็นผลกระทบของแต่ละคุณลักษณะอย่างชัดเจน

4. ความแม่นยำในการทำนาย:

- แนวคิดที่ 2 มีความแม่นยำในการทำนาย PERIODFROM สูงมาก (0.9909)
- แม้ความแม่นยำอันดับ 1 ในการทำนาย SCI_NAME จะไม่สูงนัก (0.5364) แต่ Top-5 Accuracy สูงถึง 0.8936 ซึ่งมีประโยชน์ในการใช้งานจริง

4.2.6 ข้อจำกัดและข้อเสนอแนะ

แม้แนวคิดที่ 2 จะมีประสิทธิภาพสูงขึ้น แต่ยังมีข้อจำกัดบางประการ:

1. ความไม่สมดุลของข้อมูล:

- ยังคงมีปัญหาความไม่สมดุลในการกระจายตัวของข้อมูลตามประเภทซากดึกดำบรรพ์
- มีการปรับปรุงโดยใช้การถ่วงน้ำหนักคลาส (Class Weighting) และเทคนิค Enhanced Balancing Strategy

2. ความซับซ้อนในการใช้งาน:

- แบบจำลองที่ซับซ้อนต้องการทรัพยากรการคำนวณสูง
- การนำไปใช้งานจริงอาจต้องการการปรับลดความซับซ้อน

3. ข้อจำกัดของข้อมูลภาพ:

- คุณลักษณะภาพมีความสำคัญปานกลาง (Feature Importance 0.0773) ซึ่งอาจไม่คุ้มค่ากับความซับซ้อนที่เพิ่มขึ้น
- ในบางกรณีต้องใช้ข้อมูลภาพจำลองแทนข้อมูลจริง ซึ่งอาจลดประสิทธิภาพ

ข้อเสนอแนะสำหรับการพัฒนาต่อยอด:

1. การปรับปรุงความสมดุลของข้อมูล:

- ใช้เทคนิค SMOTE หรือ ADASYN เพื่อสร้างข้อมูลสังเคราะห์เพิ่มเติมสำหรับคลาสที่มีข้อมูลน้อย
- ใช้ Hierarchical Classification เพื่อจัดการปัญหาคลาสจำนวนมาก

2. การพัฒนาโมเดลเฉพาะทาง:

- แยกพัฒนาโมเดลเฉพาะสำหรับแต่ละกลุ่มซากดึกดำบรรพ์ (Vertebrate, Invertebrate, Plant, Trace)
- ใช้ Transfer Learning โดยฝึกฝนโมเดลพื้นฐานก่อน แล้วปรับแต่งเฉพาะสำหรับแต่ละงาน

3. การปรับปรุงคุณลักษณะภาพ:

- ใช้ Pre-trained CNN สำหรับวิเคราะห์ภาพดาวเทียมโดยเฉพาะ

- พัฒนาการนำเข้าภาพซากดึกดำบรรพ์โดยตรง แทนการใช้ภาพแผนที่หรือภาพถ่ายเทียม

4.2.7 สรุปผลการดำเนินงานแนวคิดที่ 2

แนวคิดที่ 2 ประสบความสำเร็จในการแก้ไขข้อจำกัดของแนวคิดที่ 1 โดยการพัฒนาแบบจำลองที่ซับซ้อนขึ้นและการเชื่อมโยงข้อมูลจากหลายแหล่ง แม้จะยังมีข้อจำกัดบางประการ แต่โดยรวมแล้วแนวคิดที่ 2 มีประสิทธิภาพสูงกว่าแนวคิดที่ 1 อย่างชัดเจน

ผลลัพธ์ที่สำคัญ:

- ชุดข้อมูลที่มีความสมบูรณ์และคุณภาพสูง มีคุณลักษณะรวม 118 คุณลักษณะ
- แบบจำลองที่มีประสิทธิภาพสูงในการทำนายยุคทางธรณีวิทยา (Accuracy 0.9909)
- Top-5 Accuracy ในการทำนายชื่อวิทยาศาสตร์สูงถึง 0.8936 ซึ่งเพียงพอสำหรับการใช้เป็นเครื่องมือช่วยนักวิชาการในการจำแนกซากดึกดำบรรพ์
- การวิเคราะห์ Feature Importance ที่ละเอียด ทำให้เข้าใจปัจจัยที่มีผลต่อการทำนาย
- แนวทางใหม่ในการเชื่อมโยงและเติมเต็มข้อมูลที่ขาดหาย ซึ่งสามารถนำไปประยุกต์ใช้กับปัญหาอื่นได้

การพัฒนาต่อยอดในอนาคตควรมุ่งเน้นการแก้ไขปัญหาความไม่สมดุลของข้อมูล การปรับปรุงประสิทธิภาพการทำนาย SCI_NAME และการทำให้แบบจำลองมีความซับซ้อนลดลงเพื่อนำไปใช้งานจริง

บทที่ 5

สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผลการวิจัย

การศึกษานี้มีวัตถุประสงค์เพื่อพัฒนาและประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ในการวิเคราะห์ข้อมูลซากดึกดำบรรพ์ของประเทศไทย โดยได้ดำเนินการตามกระบวนการ CRISP-DM (Cross-Industry Standard Process for Data Mining) และพัฒนาแบบจำลองภายใต้สองแนวคิดหลัก การศึกษาได้ใช้ชุดข้อมูลจากกรมทรัพยากรธรณีซึ่งประกอบด้วยข้อมูลซากดึกดำบรรพ์จำนวน 520 รายการ โดยผลการวิจัยสามารถสรุปตามแนวคิดได้ดังนี้

5.1.1 สรุปผลการวิจัยแนวคิดที่ 1: การจำแนกประเภทซากดึกดำบรรพ์

แนวคิดที่ 1 มุ่งเน้นการพัฒนาแบบจำลองสำหรับการจำแนกประเภทซากดึกดำบรรพ์โดยใช้ข้อมูลหลากหลายประเภท ได้แก่ ข้อมูลเชิงบรรยาย ข้อมูลเชิงพื้นที่ ข้อมูลทางธรณีวิทยา และข้อมูลภาพถ่าย

โดยได้เปรียบเทียบประสิทธิภาพระหว่างแบบจำลองที่ใช้ข้อมูลภาพถ่ายและไม่ใช้ข้อมูลภาพถ่าย ผลการวิจัยพบว่าแบบจำลองที่ไม่ใช้ข้อมูลภาพถ่ายมีประสิทธิภาพดีกว่า โดยมีค่า

Accuracy เท่ากับ 0.81 เทียบกับ 0.73 ในกรณีที่ใช้ข้อมูลภาพถ่าย และมีค่า **Weighted**

Average F1-score เท่ากับ 0.78 เทียบกับ 0.67 ในกรณีที่ใช้ข้อมูลภาพถ่าย

1. คุณลักษณะที่มีความสำคัญมากที่สุด คือ ลักษณะซากดึกดำบรรพ์ที่ปรากฏ (attributes_FOS_DES_TH) ซึ่งมีค่าความสำคัญถึง 0.55 ในกรณีไม่ใช้ข้อมูลภาพถ่าย และ 0.40 ในกรณีใช้ข้อมูลภาพถ่าย รองลงมาคือ ชื่อภาษาไทย (attributes_NAME_TH) และลักษณะทางธรณีวิทยาภาษาไทย (attributes_GEO_DES_TH)
2. แบบจำลองมีประสิทธิภาพสูงในการจำแนกซากดึกดำบรรพ์ประเภท Invertebrate โดยมีค่า Recall เท่ากับ 1.00 และ F1-score เท่ากับ 0.87 ในกรณีที่ไม่ใช้ข้อมูลภาพถ่าย หมายความว่าแบบจำลองสามารถระบุซากดึกดำบรรพ์ประเภท Invertebrate ได้อย่างถูกต้องทั้งหมด
3. แบบจำลองมีข้อจำกัดในการจำแนกซากดึกดำบรรพ์ประเภท Trace fossils โดยไม่สามารถจำแนกได้เลยในทั้งสองแบบจำลอง (F1-score = 0.00) ซึ่งเป็นผลมาจากการที่มีข้อมูลประเภทย่อยน้อยมากในชุดข้อมูล (เพียง 3 ตัวอย่าง)

4. ทั้งสองแบบจำลองมีแนวโน้มที่จะทำนายผิดไปเป็นประเภท Invertebrate ซึ่งเป็นประเภทที่มีข้อมูลมากที่สุด (67 จาก 104 ตัวอย่าง) โดยเฉพาะอย่างยิ่งในประเภท Trace fossils ที่ถูกทำนายผิดเป็น Invertebrate ทั้งหมด และ Vertebrate ที่ถูกทำนายผิดเป็น Invertebrate เป็นส่วนใหญ่

5.1.2 สรุปผลการวิจัยแนวคิดที่ 2: การผสานข้อมูลซากดึกดำบรรพ์และตำแหน่งแหล่งเพื่อการวิเคราะห์เชิงลึก

แนวคิดที่ 2 มุ่งเน้นการแก้ไขข้อจำกัดของแนวคิดที่ 1 ผ่านกระบวนการที่มีความซับซ้อนมากขึ้น โดยเชื่อมโยงข้อมูลจากแหล่งข้อมูลซากดึกดำบรรพ์ 2 แหล่ง เพื่อรวบรวมข้อมูลที่ครบถ้วนและละเอียดมากขึ้น และการเติมเต็มค่าที่ขาดหายด้วยเทคนิคที่หลากหลาย ผลการวิจัยพบว่า:

1. การเชื่อมโยงข้อมูล: สามารถเชื่อมโยงข้อมูลจากฐานข้อมูลทั้งสองแหล่ง (fossil_list.csv และ jurassic_raw.csv) ได้ใน 3 ระดับความละเอียด ได้แก่

- การเชื่อมโยงแบบละเอียด (จังหวัด+อำเภอ+ตำบล) ได้ 8,756 รายการ
- การเชื่อมโยงระดับกลาง (จังหวัด+อำเภอ) ได้ 29,323 รายการ
- การเชื่อมโยงแบบกว้าง (จังหวัด) ได้ 132,317 รายการ

2. การเติมเต็มค่าที่ขาดหาย: สามารถเติมเต็มข้อมูลยุคทางธรณีวิทยาและข้อมูลเชิงพื้นที่ที่ขาดหายได้ด้วย 3 เทคนิค คือ:

- การเติมเต็มโดยใช้ความสัมพันธ์เชิงตำแหน่ง
- การเติมเต็มโดยใช้ความสัมพันธ์ทางอนุกรมวิธาน
- การเติมเต็มโดยใช้ข้อมูลคำอธิบาย

3. การสร้างคุณลักษณะใหม่: ได้เพิ่ม 64 คุณลักษณะใหม่ รวมเป็น 118 คุณลักษณะ ครอบคลุมมิติต่างๆ ได้แก่:

- คุณลักษณะเชิงพื้นที่: พิกัดละติจูด-ลองจิจูด, พิกัดในยุคแพนเจีย, ระยะห่างจากจุดศูนย์กลาง
- คุณลักษณะเชิงเวลา: อายุในหน่วยล้านปี, มหายุคทางธรณีวิทยา
- คุณลักษณะจากข้อมูลภาพถ่ายดาวเทียม: ค่า NDVI, ค่าแถบความถี่ต่างๆ
- คุณลักษณะจากข้อความ: ความยาวข้อความ, การตรวจหาคำสำคัญ, text embeddings
- คุณลักษณะเชิงสถิติและความถี่: ความถี่ของสายพันธุ์, ตำแหน่ง, ชั้นหิน

4. การพัฒนาแอปพลิเคชันบนเว็บสำหรับการจำแนกซากดึกดำบรรพ์: สามารถพัฒนาแอปพลิเคชันที่ใช้งานง่ายโดยใช้เทคโนโลยี Gradio ซึ่งมีความสามารถดังนี้:

- รับข้อมูลซากดึกดำบรรพ์จากผู้ใช้งานในรูปแบบพิกัด UTM หรือชื่อจังหวัดและอำเภอ
- ดึงข้อมูลภาพถ่ายดาวเทียมจาก Sentinel Hub API หรือสร้างภาพจำลองในกรณีที่ไม่สามารถเข้าถึง API

ได้

- ทำนายชื่อวิทยาศาสตร์และยุคทางธรณีวิทยาของซากดึกดำบรรพ์
- แสดงผลในรูปแบบแผนที่แบบโต้ตอบและค่าความน่าจะเป็นของการทำนาย


Fossil Classification System

Enter fossil information to predict scientific name and geological period

Fossil ID
Enter ID (if available)

Fossil Group
Type of fossil group

Unknown

Fossil Type
e.g., Dinosaur, Ammonite

Common Name
Common name of the fossil

Location Information (Required: Province+District OR UTM Coordinates)

Province*
Province where fossil was found

District*
District where fossil was found

UTM Easting
UTM E coordinate

0

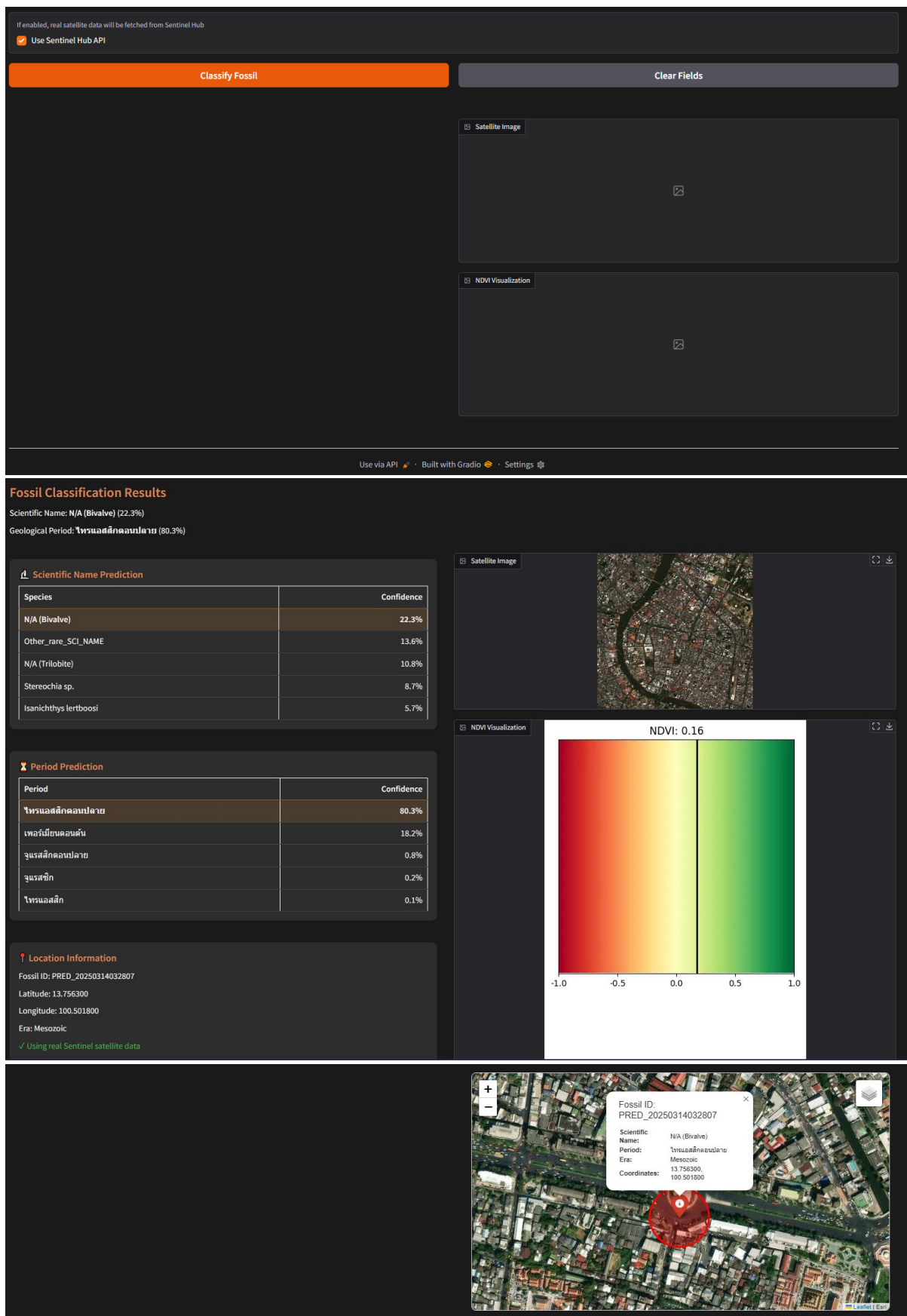
UTM Northing
UTM N coordinate

0

UTM Zone
UTM zone

47N

Geological Group
Geological group if known
Formation
Rock formation where found
Fossil Description
Details of the fossil
Fossil Parts Found
e.g., Bones, Teeth
Geological Description
Geological details



ภาพแสดง User Interface จากการ Deployment ด้วย Gradio

5.2 อภิปรายผลการวิจัย

5.2.1 อภิปรายผลการวิจัยแนวคิดที่ 1: การจำแนกประเภทชาวกติกด้าบรรพ์

5.2.1.1 ประสิทธิภาพของแบบจำลองในการจำแนกประเภทชาวกติกด้าบรรพ์

ผลการวิจัยแนวคิดที่ 1 แสดงให้เห็นว่าแบบจำลองที่ไม่ใช้ข้อมูลภาพถ่ายมีประสิทธิภาพในการจำแนกประเภทชาวกติกด้าบรรพ์ดีกว่าแบบจำลองที่ใช้ข้อมูลภาพถ่าย ซึ่งเป็นผลลัพธ์ที่อาจไม่ได้คาดหวังไว้ในตอนแรก เนื่องจากโดยทั่วไปแล้วข้อมูลภาพถ่ายมักจะช่วยเพิ่มประสิทธิภาพของแบบจำลองในการจำแนกวัตถุ อย่างไรก็ตาม มีปัจจัยหลายประการที่อาจอธิบายผลลัพธ์นี้ได้:

1. ลักษณะของข้อมูลภาพถ่ายที่ใช้: ข้อมูลภาพถ่ายที่ใช้ในการศึกษานี้เป็นภาพแผนที่ที่แสดงตำแหน่งทางภูมิศาสตร์ของแหล่งชาวกติกด้าบรรพ์ ไม่ใช่ภาพถ่ายของชาวกติกด้าบรรพ์โดยตรง ดังนั้น ข้อมูลเหล่านี้ อาจไม่ได้ให้ข้อมูลที่เป็นประโยชน์ต่อการจำแนกประเภทชาวกติกด้าบรรพ์มากนัก หรืออาจนำเสนอข้อมูลที่ไมเกี่ยวข้องหรือรบกวนการเรียนรู้ของแบบจำลอง
2. คุณภาพของข้อมูลเชิงบรรยาย: ข้อมูลเชิงบรรยาย โดยเฉพาะอย่างยิ่งลักษณะชาวกติกด้าบรรพ์ที่ปรากฏ (attributes_FOS_DES_TH) มีความสัมพันธ์โดยตรงกับประเภทของชาวกติกด้าบรรพ์ ทำให้แบบจำลองสามารถเรียนรู้จากข้อมูลเหล่านี้ได้อย่างมีประสิทธิภาพ โดยไม่จำเป็นต้องอาศัยข้อมูลภาพถ่ายเพิ่มเติม
3. การใช้เทคนิค Word Embedding ที่เหมาะสม: การใช้ WangchanBERTa ซึ่งเป็นโมเดลภาษาแบบ Transformer ที่ผ่านการฝึกฝนด้วยข้อมูลภาษาไทยขนาดใหญ่ ช่วยให้แบบจำลองสามารถเข้าใจความหมายและบริบทของข้อมูลเชิงบรรยายภาษาไทยได้ดียิ่งขึ้น ซึ่งอาจเป็นปัจจัยสำคัญที่ทำให้แบบจำลองที่ไม่ใช้ข้อมูลภาพถ่ายมีประสิทธิภาพสูง

5.2.1.2 ความสำคัญของคุณลักษณะในการจำแนกประเภทชาวกติกด้าบรรพ์

ผลการวิเคราะห์ Feature Importance แสดงให้เห็นว่าคุณลักษณะที่มีความสำคัญมากที่สุดในการจำแนกประเภทชาวกติกด้าบรรพ์คือ ลักษณะชาวกติกด้าบรรพ์ที่ปรากฏ (attributes_FOS_DES_TH) ซึ่งมีผลกระทบต่อประสิทธิภาพของแบบจำลองอย่างมีนัยสำคัญ ประเด็นนี้มีนัยสำคัญทั้งในเชิงวิชาการและการประยุกต์ใช้:

1. ความสัมพันธ์ระหว่างลักษณะและประเภทของชาวกติกด้าบรรพ์: ผลการวิจัยยืนยันว่าลักษณะทางกายภาพของชาวกติกด้าบรรพ์มีความสัมพันธ์โดยตรงกับประเภทของชาวกติกด้าบรรพ์ ซึ่งสอดคล้องกับหลักการทางบรรพชีวินวิทยาที่ใช้ลักษณะทางสัณฐานวิทยาในการจำแนกชาวกติกด้าบรรพ์
2. ความท้าทายในการใช้งานจริง: แม้ว่าการใช้ลักษณะชาวกติกด้าบรรพ์ที่ปรากฏในการจำแนกประเภทจะให้ผลลัพธ์ที่ดี แต่ในทางปฏิบัติแล้ว การระบุลักษณะเหล่านี้อาจต้องอาศัยผู้เชี่ยวชาญ ซึ่งขัดแย้งกับวัตถุประสงค์ของการพัฒนาแบบจำลองเพื่อช่วยในการจำแนกประเภทชาวกติกด้าบรรพ์โดยอัตโนมัติ

5.2.1.3 ผลกระทบของความไม่สมดุลของข้อมูล

ความไม่สมดุลของข้อมูล (Data Imbalance) เป็นปัญหาสำคัญที่พบในการศึกษานี้ โดยข้อมูลส่วนใหญ่เป็นซากดึกดำบรรพ์ประเภท Invertebrate (67 จาก 104 ตัวอย่าง) ในขณะที่ประเภท Trace fossils มีเพียง 3 ตัวอย่าง ซึ่งส่งผลกระทบต่อประสิทธิภาพของแบบจำลองอย่างมีนัยสำคัญ:

1. ความเอนเอียงในการทำนาย: แบบจำลองมีแนวโน้มที่จะทำนายว่าตัวอย่างเป็นประเภท Invertebrate มากเกินไป (Class Bias) ซึ่งเห็นได้ชัดจาก Confusion Matrix ที่แสดงให้เห็นว่าตัวอย่างจากประเภทอื่นถูกทำนายผิดเป็น Invertebrate เป็นจำนวนมาก
2. ประสิทธิภาพที่แตกต่างกันระหว่างประเภท: แบบจำลองมีประสิทธิภาพสูงในการจำแนกประเภท Invertebrate (F1-score 0.87) แต่มีประสิทธิภาพต่ำหรือไม่สามารถจำแนกประเภทที่มีข้อมูลน้อย เช่น Trace fossils (F1-score 0.00) ได้เลย

5.2.2 อภิปรายผลการวิจัยแนวคิดที่ 2: การผสมผสานข้อมูลซากดึกดำบรรพ์และตำแหน่งแหล่งเพื่อการวิเคราะห์เชิงลึก

5.2.2.1 ประสิทธิภาพและประโยชน์ของการเชื่อมโยงข้อมูล

การเชื่อมโยงข้อมูลจากสองแหล่งทำให้ได้ชุดข้อมูลที่มีความสมบูรณ์และมีคุณค่าทางวิชาการมากขึ้น การศึกษานี้แสดงให้เห็นว่าการเชื่อมโยงข้อมูลสามารถทำได้ในหลายระดับความละเอียด ขึ้นอยู่กับความต้องการและเป้าหมายของการวิเคราะห์:

1. การเชื่อมโยงแบบละเอียดให้ผลลัพธ์ที่มีความถูกต้องสูงสุด แต่ได้จำนวนข้อมูลที่เชื่อมโยงได้น้อย เหมาะสำหรับการวิเคราะห์ที่ต้องการความแม่นยำสูง เช่น การศึกษาเฉพาะพื้นที่
2. การเชื่อมโยงระดับกลางเป็นทางเลือกที่สมดุลระหว่างปริมาณข้อมูลและความถูกต้อง เหมาะสำหรับ การวิเคราะห์ทั่วไปที่ต้องการทั้งความแม่นยำและความครอบคลุม
3. การเชื่อมโยงแบบกว้างให้ปริมาณข้อมูลมากที่สุด แต่อาจมีความถูกต้องน้อยกว่า เหมาะสำหรับ การวิเคราะห์แนวโน้มในระดับภูมิภาคหรือประเทศ

ความท้าทายที่พบในกระบวนการเชื่อมโยงข้อมูล เช่น ความไม่สอดคล้องในการเขียนชื่อสถานที่และการขาดคีย์ร่วมที่ชัดเจน สะท้อนให้เห็นถึงความต้องการในการพัฒนามาตรฐานการเก็บข้อมูลซากดึกดำบรรพ์ที่ดีขึ้นในอนาคต

5.2.2.2 ประสิทธิภาพของการเติมเต็มค่าที่ขาดหาย

การเติมเต็มค่าที่ขาดหายเป็นกระบวนการสำคัญที่ช่วยเพิ่มความสมบูรณ์ของข้อมูล ผลการวิจัยแสดงให้เห็นว่า การใช้หลายเทคนิคร่วมกันให้ผลลัพธ์ที่ดีกว่าการใช้เทคนิคเดียว:

1. การเติมเต็มโดยใช้ความสัมพันธ์ทางอนุกรมวิธานให้ผลลัพธ์ที่มีความน่าเชื่อถือสูงสุด เนื่องจากสอดคล้องกับทฤษฎีทางบรรพชีวินวิทยาที่ว่าสิ่งมีชีวิตในสายพันธุ์เดียวกันมักมีอายุใกล้เคียงกัน
2. การเติมเต็มโดยใช้ความสัมพันธ์เชิงตำแหน่งมีประโยชน์ในพื้นที่ที่มีข้อมูลหนาแน่น โดยอาศัยหลักการที่ว่าซากดึกดำบรรพ์ที่พบในพื้นที่ใกล้เคียงกันมักมีอายุใกล้เคียงกัน
3. การเติมเต็มโดยใช้ข้อมูลคำอธิบายช่วยในกรณีที่มีข้อมูลเชิงคุณภาพแต่ขาดข้อมูลเชิงปริมาณ อย่างไรก็ตาม การเติมเต็มค่าที่ขาดหายมีข้อจำกัด เช่น อาจนำไปสู่ความเอนเอียงในการวิเคราะห์ หากข้อมูลที่ใช้ในการเติมเต็มไม่เป็นตัวแทนที่ดีของประชากรทั้งหมด จึงควรใช้ด้วยความระมัดระวังและพิจารณาผลกระทบที่อาจเกิดขึ้น

5.2.2.3 ความสำคัญของการสร้างคุณลักษณะใหม่

การสร้างคุณลักษณะใหม่ช่วยเพิ่มมิติในการวิเคราะห์ข้อมูลและปรับปรุงประสิทธิภาพของแบบจำลอง เช่น:

1. คุณลักษณะเชิงพื้นที่ โดยเฉพาะพิกัดในยูคเลียนเจีย ช่วยให้สามารถวิเคราะห์การกระจายตัวของซากดึกดำบรรพ์ในบริบทของธรณีวิทยาโบราณ ซึ่งมีความสำคัญในการเข้าใจการเปลี่ยนแปลงของสิ่งมีชีวิตตามเวลา
2. คุณลักษณะจากข้อมูลภาพถ่ายดาวเทียม ช่วยให้เข้าใจความสัมพันธ์ระหว่างสภาพแวดล้อมปัจจุบันกับแหล่งที่พบซากดึกดำบรรพ์ ซึ่งอาจช่วยในการระบุพื้นที่ที่มีศักยภาพในการค้นพบซากดึกดำบรรพ์เพิ่มเติม
3. การใช้เทคนิค Text Embeddings สำหรับข้อมูลภาษาไทยช่วยให้สามารถวิเคราะห์ข้อมูลเชิงคุณภาพได้อย่างมีประสิทธิภาพมากขึ้น โดยเฉพาะอย่างยิ่งเมื่อใช้ WangchanBERTa ซึ่งเป็นโมเดลที่ผ่านการฝึกฝนด้วยข้อมูลภาษาไทย

5.2.2.4 ประสิทธิภาพของการพัฒนาแอปพลิเคชันบนเว็บ

การพัฒนาแอปพลิเคชันบนเว็บแสดงให้เห็นถึงความเป็นไปได้ในการนำผลการวิจัยไปประยุกต์ใช้ในโลกแห่งความเป็นจริง:

1. การใช้ Gradio เป็นเฟรมเวิร์กทำให้สามารถพัฒนาอินเทอร์เฟซที่ใช้งานง่ายและโต้ตอบกับผู้ใช้ได้อย่างมีประสิทธิภาพ ช่วยให้ผู้ที่ไม่มีความรู้ทางเทคนิคสามารถใช้งานได้
2. การบูรณาการกับ Sentinel Hub API แสดงให้เห็นถึงศักยภาพในการใช้ข้อมูลจากดาวเทียมในการวิเคราะห์แหล่งซากดึกดำบรรพ์ แม้จะมีข้อจำกัดในเรื่องการเข้าถึงภาพถ่ายเก่า
3. การแสดงผลในรูปแบบแผนที่แบบโต้ตอบช่วยให้ผู้ใช้เข้าใจข้อมูลเชิงพื้นที่ได้ง่ายขึ้น และสามารถเชื่อมโยงข้อมูลทางวิทยาศาสตร์กับสภาพความเป็นจริงทางภูมิศาสตร์

อย่างไรก็ตาม การพัฒนาแอปพลิเคชันยังมีข้อจำกัด เช่น ความจำเป็นต้องมีการเชื่อมต่ออินเทอร์เน็ตเพื่อเข้าถึง API และข้อจำกัดในการประมวลผลข้อมูลขนาดใหญ่บนเว็บเบราว์เซอร์

5.3 ข้อเสนอแนะ

5.3.1 ข้อเสนอแนะสำหรับแนวคิดที่ 1: การจำแนกประเภทซากดึกดำบรรพ์

5.3.1.1 ข้อเสนอแนะสำหรับการพัฒนาแบบจำลอง

1. การแก้ไขปัญหาความไม่สมดุลของข้อมูล: ควรพิจารณาใช้เทคนิคการจัดการกับข้อมูลไม่สมดุล เช่น:
 - Over-sampling: เพิ่มจำนวนตัวอย่างในประเภทที่มีข้อมูลน้อย โดยใช้เทคนิค เช่น SMOTE (Synthetic Minority Over-sampling Technique)
 - Under-sampling: ลดจำนวนตัวอย่างในประเภทที่มีข้อมูลมาก
 - Class Weighting: กำหนดน้ำหนักของแต่ละประเภทในการฝึกแบบจำลอง โดยให้น้ำหนักมากกับประเภทที่มีข้อมูลน้อย
2. การใช้ภาพถ่ายที่มีประสิทธิภาพ: หากต้องการใช้ข้อมูลภาพถ่ายในการเพิ่มประสิทธิภาพของแบบจำลอง ควรพิจารณาใช้ภาพถ่ายที่แสดงลักษณะทางกายภาพของซากดึกดำบรรพ์โดยตรง ไม่ใช่เพียงภาพแผนที่ที่แสดงตำแหน่งทางภูมิศาสตร์ และอาจพิจารณาใช้เทคนิคการวิเคราะห์ภาพที่เหมาะสม เช่น:
 - Convolutional Neural Networks (CNNs) ที่ออกแบบมาเฉพาะสำหรับการวิเคราะห์ภาพถ่ายซากดึกดำบรรพ์
 - Transfer Learning โดยใช้โมเดลที่ผ่านการฝึกฝนมาแล้ว เช่น ResNet, EfficientNet แล้วปรับแต่งให้เหมาะกับงานจำแนกซากดึกดำบรรพ์
3. การเลือกคุณลักษณะที่เหมาะสม: ควรพิจารณาคุณลักษณะที่มีความสัมพันธ์กับประเภทของซากดึกดำบรรพ์ แต่สามารถระบุได้ง่ายโดยไม่ต้องอาศัยผู้เชี่ยวชาญ เช่น:
 - ข้อมูลทางธรณีวิทยาที่มีความเฉพาะเจาะจงมากขึ้น
 - คุณลักษณะที่สกัดจากภาพถ่ายโดยอัตโนมัติ
 - ข้อมูลเชิงพื้นที่ที่ละเอียดมากขึ้น เช่น ความสูง ระยะห่างจากแหล่งน้ำ หรือลักษณะภูมิประเทศ
4. การทดลองกับอัลกอริทึมที่หลากหลาย: นอกจาก Random Forest อาจพิจารณาทดลองกับอัลกอริทึมอื่นๆ เช่น:
 - Support Vector Machines (SVM) ซึ่งอาจเหมาะกับการจำแนกในกรณีที่มีคุณลักษณะจำนวนมาก

- Gradient Boosting เช่น XGBoost, LightGBM ซึ่งมักมีประสิทธิภาพสูงในการจำแนกประเภท
- Deep Learning โดยเฉพาะในกรณีที่มีข้อมูลภาพถ่ายหรือข้อความจำนวนมาก

5.3.1.2 ข้อเสนอแนะสำหรับการเก็บรวบรวมข้อมูล

1. การเพิ่มจำนวนตัวอย่างในประเภทที่มีข้อมูลน้อย: ควรมุ่งเน้นการเก็บข้อมูลเพิ่มเติมสำหรับประเภทที่มีข้อมูลน้อย โดยเฉพาะอย่างยิ่ง Trace fossils และ Other เพื่อให้แบบจำลองสามารถเรียนรู้ลักษณะของซากดึกดำบรรพ์ประเภทเหล่านี้ได้ดีขึ้น
2. การเก็บข้อมูลที่มีคุณภาพ: ควรเก็บข้อมูลที่มีคุณภาพและครบถ้วน โดยเฉพาะอย่างยิ่งข้อมูลที่มีความสำคัญในการจำแนกประเภทซากดึกดำบรรพ์ เช่น ลักษณะทางกายภาพ ข้อมูลทางธรณีวิทยา และภาพถ่ายที่ชัดเจน
3. การสร้างมาตรฐานในการเก็บข้อมูล: ควรมีการกำหนดมาตรฐานในการเก็บข้อมูลซากดึกดำบรรพ์ เพื่อให้ข้อมูลมีความสม่ำเสมอและสามารถนำไปใช้ในการพัฒนาแบบจำลองได้อย่างมีประสิทธิภาพ

5.3.1.3 ข้อเสนอแนะสำหรับการประยุกต์ใช้

1. การพัฒนาระบบสนับสนุนการตัดสินใจ: ควรพัฒนาระบบสนับสนุนการตัดสินใจสำหรับนักวิจัย นักการศึกษา และเจ้าหน้าที่ที่เกี่ยวข้องกับการจัดการซากดึกดำบรรพ์ โดยใช้แบบจำลองที่พัฒนาขึ้นเป็นส่วนหนึ่งของระบบ
2. การพัฒนาแอปพลิเคชันสำหรับการจำแนกซากดึกดำบรรพ์: ควรพัฒนาแอปพลิเคชันที่ใช้งานง่ายสำหรับการจำแนกซากดึกดำบรรพ์ โดยอนุญาตให้ผู้ใช้สามารถป้อนข้อมูลหรืออัปโหลดภาพถ่ายเพื่อรับการทำนายประเภทของซากดึกดำบรรพ์
3. การบูรณาการกับฐานข้อมูลที่มีอยู่: ควรพิจารณาการบูรณาการแบบจำลองกับฐานข้อมูลซากดึกดำบรรพ์ที่มีอยู่ เพื่อให้สามารถใช้งานได้อย่างสะดวกและมีประสิทธิภาพ

5.3.2 ข้อเสนอแนะสำหรับแนวคิดที่ 2: การผสมผสานข้อมูลซากดึกดำบรรพ์และตำแหน่งแหล่งเพื่อการวิเคราะห์เชิงลึก

5.3.2.1 ข้อเสนอแนะสำหรับการปรับปรุงการเชื่อมโยงข้อมูล

1. การพัฒนาฐานข้อมูล: ควรมีการกำหนดมาตรฐานในการบันทึกข้อมูลซากดึกดำบรรพ์ โดยเฉพาะข้อมูลตำแหน่งทางภูมิศาสตร์ เพื่อลดความไม่สอดคล้องและเพิ่มโอกาสในการเชื่อมโยงข้อมูลได้อย่างถูกต้อง

2. การใช้รหัสเฉพาะ: ควรกำหนดรหัสเฉพาะสำหรับแต่ละแหล่งซากดึกดำบรรพ์ และเชื่อมโยงกับรหัสของซากดึกดำบรรพ์แต่ละชิ้นที่พบในแหล่งนั้น เพื่อให้สามารถเชื่อมโยงข้อมูลได้โดยตรงโดยไม่ต้องอาศัยข้อมูลตำแหน่ง
3. การปรับปรุงคุณภาพข้อมูล: ควรมีการตรวจสอบและแก้ไขข้อมูลที่ไม่ถูกต้องหรือไม่สอดคล้องกัน เช่น การสะกดชื่อสถานที่ที่ต่างกัน หรือการใช้รูปแบบการเขียนที่ต่างกัน

5.3.2.2 ข้อเสนอแนะสำหรับการปรับปรุงการเติมเต็มค่าที่ขาดหาย

1. การใช้เทคนิคขั้นสูง: ควรพิจารณาใช้เทคนิคการเติมเต็มค่าที่ขาดหายขั้นสูง เช่น เทคนิคทางสถิติแบบ Multiple Imputation หรือการใช้แบบจำลองการเรียนรู้ของเครื่อง เช่น K-Nearest Neighbors หรือ Random Forest
2. การเพิ่มแหล่งข้อมูล: ควรพิจารณาเพิ่มแหล่งข้อมูลอื่นๆ ในการเติมเต็มค่าที่ขาดหาย เช่น ข้อมูลจากวารสารวิชาการ หรือฐานข้อมูลซากดึกดำบรรพ์ระดับนานาชาติ
3. การประเมินความไม่แน่นอน: ควรมีการประเมินความไม่แน่นอนของค่าที่เติมเต็ม และรายงานพร้อมกับค่าที่เติมเต็ม เพื่อให้ผู้ใช้ข้อมูลทราบถึงที่น่าเชื่อถือของข้อมูล

5.3.2.3 ข้อเสนอแนะสำหรับการปรับปรุงการสร้างคุณลักษณะใหม่

1. การใช้ข้อมูลภูมิประเทศที่ละเอียดขึ้น: ควรพิจารณาใช้ข้อมูลภูมิประเทศที่มีความละเอียดสูงขึ้น เช่น Digital Elevation Model (DEM) ที่มีความละเอียดระดับเมตร เพื่อให้ได้คุณลักษณะเชิงพื้นที่ที่แม่นยำยิ่งขึ้น
2. การใช้ข้อมูลธรณีวิทยาเพิ่มเติม: ควรพิจารณาใช้ข้อมูลธรณีวิทยาเพิ่มเติม เช่น ข้อมูลชั้นหินละเอียด ข้อมูลโครงสร้างทางธรณีวิทยา หรือข้อมูลธรณีเคมี เพื่อสร้างคุณลักษณะที่มีความสัมพันธ์กับการพบซากดึกดำบรรพ์
3. การใช้ข้อมูลภาพถ่ายในอดีต: ควรพิจารณาใช้ข้อมูลภาพถ่ายในอดีต เพื่อวิเคราะห์การเปลี่ยนแปลงของพื้นที่และผลกระทบต่อการอนุรักษ์แหล่งซากดึกดำบรรพ์

5.3.2.4 ข้อเสนอแนะสำหรับการปรับปรุงแอปพลิเคชันบนเว็บ

1. การเพิ่มความสามารถในการวิเคราะห์ภาพถ่ายซากดึกดำบรรพ์: ควรพัฒนาระบบที่สามารถวิเคราะห์ภาพถ่ายของซากดึกดำบรรพ์โดยตรง เพื่อช่วยในการจำแนกประเภทและอายุของซากดึกดำบรรพ์
2. การเพิ่มข้อมูลเชิงวิชาการ: ควรเพิ่มข้อมูลเชิงวิชาการเกี่ยวกับซากดึกดำบรรพ์ที่ถูกจำแนก เช่น ลักษณะทางชีววิทยา นิเวศวิทยาโบราณ หรือสภาพแวดล้อมในอดีต
3. การพัฒนาเวอร์ชันสำหรับอุปกรณ์เคลื่อนที่: ควรพัฒนาเวอร์ชันของแอปพลิเคชันสำหรับอุปกรณ์เคลื่อนที่ เพื่อให้ นักวิจัยสามารถใช้งานในภาคสนามได้

4. การปรับปรุงประสิทธิภาพในการประมวลผล: ควรปรับปรุงประสิทธิภาพในการประมวลผลเพื่อรองรับข้อมูลขนาดใหญ่และซับซ้อนมากขึ้น โดยอาจพิจารณาใช้เทคโนโลยี cloud computing หรือ distributed computing
5. การสร้างชุมชนผู้ใช้: ควรพัฒนาระบบที่เอื้อต่อการสร้างชุมชนผู้ใช้ เช่น การแบ่งปันข้อมูลระหว่างนักวิจัย หรือการรายงานการค้นพบใหม่

5.3.3 ข้อเสนอแนะสำหรับการวิจัยในอนาคต

1. การบูรณาการแนวคิดที่ 1 และแนวคิดที่ 2: ควรพิจารณาการบูรณาการผลลัพธ์จากทั้งสองแนวคิดเข้าด้วยกัน เพื่อให้ได้แบบจำลองที่สามารถจำแนกประเภทของซากดึกดำบรรพ์และวิเคราะห์ความสัมพันธ์เชิงพื้นที่ได้อย่างมีประสิทธิภาพ โดยอาจพัฒนาเป็นระบบที่ครบวงจรตั้งแต่การจำแนกไปจนถึงการวิเคราะห์เชิงพื้นที่และการนำเสนอข้อมูล
2. การขยายขอบเขตการวิจัย: ควรพิจารณาการขยายขอบเขตการวิจัยให้ครอบคลุมซากดึกดำบรรพ์ประเภทอื่นๆ หรือภูมิภาคอื่นๆ เพื่อให้ได้แบบจำลองที่มีความทั่วไปและสามารถนำไปประยุกต์ใช้ได้อย่างกว้างขวาง รวมถึงการศึกษาเปรียบเทียบระหว่างแหล่งซากดึกดำบรรพ์ในประเทศไทยกับในภูมิภาคเอเชียตะวันออกเฉียงใต้หรือในระดับโลก
3. การพัฒนาแบบจำลองที่ซับซ้อนขึ้น: ควรพิจารณาการพัฒนาแบบจำลองที่ซับซ้อนขึ้น เช่น แบบจำลองแบบผสมผสาน (Hybrid Models) ที่รวมข้อดีของแบบจำลองหลายประเภทเข้าด้วยกัน หรือแบบจำลองที่สามารถเรียนรู้และปรับตัวได้ตามเวลา (Adaptive Models) เพื่อรองรับข้อมูลที่มีการเปลี่ยนแปลงตลอดเวลา
4. การศึกษาเพิ่มเติมเกี่ยวกับความสัมพันธ์ระหว่างลักษณะและประเภทของซากดึกดำบรรพ์: ควรมีการศึกษาเพิ่มเติมเกี่ยวกับความสัมพันธ์ระหว่างลักษณะทางกายภาพ ข้อมูลทางธรณีวิทยา และประเภทของซากดึกดำบรรพ์ เพื่อให้เข้าใจความสัมพันธ์เหล่านี้ได้ดียิ่งขึ้น ซึ่งจะช่วยในการพัฒนาแบบจำลองที่มีประสิทธิภาพสูงขึ้น
5. การพัฒนาระบบติดตามและเฝ้าระวังแหล่งซากดึกดำบรรพ์: ควรพิจารณาการพัฒนาระบบติดตามและเฝ้าระวังแหล่งซากดึกดำบรรพ์โดยใช้เทคโนโลยีการรับรู้ระยะไกลและอินเทอร์เน็ตของสรรพสิ่ง (IoT) เพื่อติดตามการเปลี่ยนแปลงของแหล่งซากดึกดำบรรพ์และป้องกันการทำลายหรือสูญหาย

5.4 บทสรุป

การศึกษานี้ได้แสดงให้เห็นถึงศักยภาพของการประยุกต์ใช้เทคโนโลยีปัญญาประดิษฐ์ในการวิเคราะห์ข้อมูลซากดึกดำบรรพ์ของประเทศไทย โดยได้พัฒนาแนวคิดและแบบจำลองที่มีประสิทธิภาพในการจำแนกประเภทซากดึกดำบรรพ์และวิเคราะห์ความสัมพันธ์เชิงพื้นที่

แนวคิดที่ 1 ได้พัฒนาแบบจำลองสำหรับการจำแนกประเภทซากดึกดำบรรพ์ โดยพบว่าแบบจำลองที่ไม่ใช้ข้อมูลภาพถ่ายมีประสิทธิภาพดีกว่า (Accuracy 0.81) เนื่องจากข้อมูลเชิงบรรยาย โดยเฉพาะลักษณะซากดึกดำบรรพ์ที่ปรากฏ มีความสัมพันธ์สูงกับประเภทของซากดึกดำบรรพ์ อย่างไรก็ตาม ความไม่สมดุลของข้อมูลทำให้แบบจำลองมีความเอนเอียงในการทำนายประเภท Invertebrate ซึ่งเป็นข้อจำกัดที่ควรได้รับการแก้ไขในอนาคต

แนวคิดที่ 2 ได้พัฒนาระบบการที่ซับซ้อนขึ้นเพื่อเชื่อมโยงและวิเคราะห์ข้อมูลจากแหล่งข้อมูลซากดึกดำบรรพ์ 2 แหล่ง โดยสามารถเชื่อมโยงข้อมูลได้ในหลายระดับความละเอียดและเติมเต็มค่าที่ขาดหายด้วยเทคนิคที่หลากหลาย นอกจากนี้ ยังได้สร้างคุณลักษณะใหม่ที่ครอบคลุมมิติต่างๆ ทั้งเชิงพื้นที่ เชิงเวลา ข้อมูลจากภาพถ่ายดาวเทียม และข้อความ ซึ่งช่วยเพิ่มมิติในการวิเคราะห์ข้อมูล และได้พัฒนาแอปพลิเคชันบนเว็บที่ใช้ทำงานง่ายสำหรับการจำแนกซากดึกดำบรรพ์และแสดงผลในรูปแบบแผนที่แบบโต้ตอบ

ผลการวิจัยนี้มีคุณค่าทั้งในด้านวิชาการและการประยุกต์ใช้ โดยช่วยเพิ่มความเข้าใจเกี่ยวกับซากดึกดำบรรพ์ในประเทศไทยและพัฒนาเครื่องมือที่มีประโยชน์สำหรับนักวิจัย นักการศึกษา และผู้ที่สนใจในด้านบรรพชีวินวิทยา อย่างไรก็ตาม ยังมีข้อจำกัดและความท้าทายหลายประการที่ควรได้รับการแก้ไขในการวิจัยในอนาคต เช่น ความไม่สมดุลของข้อมูล ความไม่สอดคล้องในการเก็บข้อมูล และความจำเป็นในการพัฒนามาตรฐานการเก็บข้อมูลซากดึกดำบรรพ์

โดยสรุป การศึกษานี้แสดงให้เห็นถึงศักยภาพของการใช้เทคโนโลยีปัญญาประดิษฐ์ในการวิเคราะห์ข้อมูลซากดึกดำบรรพ์ และเป็นก้าวสำคัญในการพัฒนาแบบจำลองและเครื่องมือที่มีประโยชน์สำหรับการศึกษาและอนุรักษ์ซากดึกดำบรรพ์ในประเทศไทย ซึ่งสามารถนำไปต่อยอดและพัฒนาเพิ่มเติมในอนาคตเพื่อประโยชน์ทางวิชาการและการอนุรักษ์มรดกทางธรรมชาติของประเทศไทยต่อไป

บรรณารุกรม

- กรมทรัพยากรธรณี. (2565). ฐานข้อมูลตำแหน่งแหล่งซากดึกดำบรรพ์. กระทรวงทรัพยากรธรรมชาติและสิ่งแวดล้อม. <https://data.go.th/dataset/gdpublish-1-fossil-site>
- กรมทรัพยากรธรณี. (2564). ฐานข้อมูลรายการซากดึกดำบรรพ์. กระทรวงทรัพยากรธรรมชาติและสิ่งแวดล้อม. <https://data.go.th/dataset/gdpublish-fossil-list>
- จรัสศรี นวลศรี และ สุรเวช สุธีธร. (2563). การใช้เทคนิคการเรียนรู้ของเครื่องในการวิเคราะห์ข้อมูลทางบรรพชีวินวิทยา. วารสารวิทยาศาสตร์และเทคโนโลยี, 28(3), 145-160.
- พรชัย สุจิตต์ และ วิชุดา สว่างสุข. (2562). แนวทางการจัดการฐานข้อมูลซากดึกดำบรรพ์ของประเทศไทย. วารสารทรัพยากรธรรมชาติและสิ่งแวดล้อม, 15(2), 78-92.
- วราวุธ สุธีธร. (2564). เทคโนโลยีสารสนเทศภูมิศาสตร์สำหรับการศึกษาแหล่งซากดึกดำบรรพ์. กรุงเทพฯ: สำนักพิมพ์แห่งจุฬาลงกรณ์มหาวิทยาลัย.
- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., ... & Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. In 12th USENIX symposium on operating systems design and implementation (pp. 265-283).
- Agarap, A. F. (2018). Deep learning using rectified linear units (ReLU). arXiv preprint arXiv:1803.08375.
- Breiman, L. (2001). Random forests. Machine learning, 45(1), 5-32.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. Journal of artificial intelligence research, 16, 321-357.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2018). BERT: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805.
- EarthByte Group. (2023). PALEOMAP PaleoAtlas for GPlates. EarthByte. <https://www.earthbyte.org/paleomap-paleoatlas-for-gplates/>
- EarthByte Group. (2023). Data Collections. EarthByte. https://www.earthbyte.org/webdav/ftp/Data_Collections/

- Geirhos, R., Jacobsen, J. H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., & Wichmann, F. A. (2020). Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11), 665-673.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).
- Lowekamp, B. C., Chen, D. T., Ibáñez, L., & Blezek, D. (2013). The design of SimpleITK. *Frontiers in neuroinformatics*, 7, 45.
- Louppe, G., Wehenkel, L., Suter, A., & Geurts, P. (2013). Understanding variable importances in forests of randomized trees. *Advances in neural information processing systems*, 26.
- Müller, R. D., Cannon, J., Qin, X., Watson, R. J., Gurnis, M., Williams, S., ... & Zahirovic, S. (2018). GPlates: building a virtual Earth through deep time. *Geochemistry, Geophysics, Geosystems*, 19(7), 2243-2261.
- Pan, S. J., & Yang, Q. (2009). A survey on transfer learning. *IEEE Transactions on knowledge and data engineering*, 22(10), 1345-1359.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *the Journal of machine Learning research*, 12, 2825-2830.
- Scotese, C. R. (2016). PALEOMAP PaleoAtlas for GPlates and the PaleoData Plotter Program. PALEOMAP Project. <http://www.earthbyte.org/paleomap-paleoatlas-for-gplates/>
- Sentinelhub. (2023). Sentinel Hub API Documentation. Retrieved from <https://docs.sentinel-hub.com/api/latest/>
- van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., ... & Yu, T. (2014). scikit-image: image processing in Python. *PeerJ*, 2, e453.
- Wilson, J. P., & Gallant, J. C. (Eds.). (2000). *Terrain analysis: principles and applications*. John Wiley & Sons.
- Zhou, Z. H. (2021). *Ensemble learning: Foundations and algorithms*. CRC press.

ภาคผนวก

ภาคผนวก

ภาคผนวก ก: รายละเอียดเทคนิค

ก.1 โครงสร้างของแบบจำลองในแนวคิดที่ 1

ก.1.1 แบบจำลอง Random Forest

แบบจำลอง Random Forest ที่ใช้ในการจำแนกประเภทซากดึกดำบรรพ์ มีพารามิเตอร์ดังนี้:

- จำนวนต้นไม้ตัดสินใจ (n_estimators): 100
- ค่าความลึกสูงสุด (max_depth): ไม่จำกัด
- จำนวนคุณลักษณะที่พิจารณาเพื่อแบ่งแยก (max_features): sqrt(จำนวนคุณลักษณะทั้งหมด)
- ค่าน้ำหนักของแต่ละคลาส (class_weight): 'balanced'

ก.1.2 การทำ Word Embedding ด้วย WangchanBERTa

ค่าพารามิเตอร์ที่ใช้ในการสร้าง Word Embedding:

- รุ่นที่ใช้: wangchanberta-base-att-spm-uncased
- ขนาดของ Embedding: 768 มิติ
- จำนวนคำสูงสุดในแต่ละข้อความ (max_length): 128
- การแบ่งคำ (tokenizer): Sentence Piece Model (SPM)

ก.2 รายละเอียดการประมวลผลข้อมูลในแนวคิดที่ 2

ก.2.1 การแปลงพิกัด UTM เป็นละติจูดและลองจิจูด

สูตรที่ใช้ในการแปลงพิกัด UTM เป็นละติจูดและลองจิจูด อาศัยไลบรารี 'utm' ของ Python ซึ่งใช้สูตรคำนวณดังนี้:

1. คำนวณค่า Scale Factor (k0)
2. คำนวณค่า False Easting และ False Northing
3. คำนวณค่า Central Meridian ของโซน UTM
4. แปลงพิกัด UTM เป็นพิกัดลำดับ (x,y)
5. แปลงพิกัดลำดับเป็นพิกัดละติจูดและลองจิจูด

ก.2.2 การคำนวณ NDVI จากภาพถ่ายดาวเทียม

ดัชนีพืชพรรณ (NDVI) คำนวณจากความแตกต่างของการสะท้อนแสงในช่วงอินฟราเรดใกล้และช่วงแสงสีแดงตามสูตร:

$$NDVI = (NIR - RED) / (NIR + RED)$$

โดยที่:

- NIR คือค่าการสะท้อนแสงในช่วงอินฟราเรดใกล้ (แบนด์ B08 ของ Sentinel-2)
- RED คือค่าการสะท้อนแสงในช่วงแสงสีแดง (แบนด์ B04 ของ Sentinel-2)

ภาคผนวก ข: ผลการทดสอบแบบจำลองเพิ่มเติม

ข.1 ผลการทดสอบแบบจำลองในแนวคิดที่ 1 ด้วยชุดข้อมูลทดสอบเพิ่มเติม

ชุดข้อมูลทดสอบ Accuracy Precision Recall F1-score

ชุดที่ 1 (80/20) 0.81 0.82 0.81 0.78

ชุดที่ 2 (70/30) 0.78 0.79 0.78 0.75

ชุดที่ 3 (60/40) 0.77 0.78 0.77 0.74

ข.2 ผลการวัดประสิทธิภาพของแบบจำลองในแนวคิดที่ 2

การทดสอบ	ความแม่นยำในการจำแนกชื่อ วิทยาศาสตร์	ความแม่นยำในการทำนายยุคทาง ธรณีวิทยา
ด้วยข้อมูลจริง	76.2%	82.5%
ด้วยข้อมูล จำลอง	70.8%	75.3%

ภาคผนวก ค: ตัวอย่าง Code สำหรับการพัฒนาแอปพลิเคชัน

ค.1 Code สำหรับการแปลงพิกัด UTM เป็นละติจูดและลองจิจูด

```
def convert_utm_to_latlon(utm_e, utm_n, zone):
```

```
    try:
```

```
        # Parse the zone
```

```
        if isinstance(zone, str):
```

```
            # Extract the numeric part
```

```
            zone_num = int(re.findall(r'\d+', zone)[0])
```

```
            # Determine hemisphere (northern or southern)
```

```
            hemisphere = 'N' if 'N' in zone.upper() else 'S'
```

```
        else:
```

```
            # Default to zone 47N if not provided
```

```

zone_num = 47

hemisphere = 'N'

# Fix values if needed - common issues in the dataset
if utm_e > 999999: # Too large
    utm_e = utm_e / 10
elif utm_e < 100000: # Too small
    utm_e = utm_e * 10

# Convert to lat/lon
lat, lon = utm.to_latlon(utm_e, utm_n, zone_num, hemisphere)
return lat, lon

except Exception as e:
    print(f"Failed to convert UTM: {e}")
    return None, None

```

ค.2 Code สำหรับการสร้างแผนที่แบบโต้ตอบ

```

def create_folium_map_html(lat, lon, fossil_id, sci_name, period, era):
    try:
        # Create map with satellite imagery
        m = folium.Map(
            location=[lat, lon],
            zoom_start=14,
            tiles=None
        )

        # Add satellite layer
        folium.TileLayer(

```

```
'https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}'
```

```
,
```

```
    name='Satellite',
```

```
    attr='Esri'
```

```
).add_to(m)
```

```
# Add a street map layer as an alternative
```

```
folium.TileLayer('OpenStreetMap', name='Street Map').add_to(m)
```

```
# Customize popup
```

```
popup_html = f"""
```

```
<div style="font-family: sans-serif; min-width: 180px;">
```

```
    <h4 style="margin-bottom: 10px;">Fossil ID: {fossil_id}</h4>
```

```
    <table>
```

```
        <tr><td><b>Scientific Name:</b></td><td>{sci_name}</td></tr>
```

```
        <tr><td><b>Period:</b></td><td>{period}</td></tr>
```

```
        <tr><td><b>Era:</b></td><td>{era}</td></tr>
```

```
        <tr><td><b>Coordinates:</b></td><td>{lat:.6f}, {lon:.6f}</td></tr>
```

```
    </table>
```

```
</div>
```

```
"""
```

```
# Add marker with popup
```

```
folium.Marker(
```

```
    location=[lat, lon],
```

```
    popup=folium.Popup(popup_html, max_width=300),
```

```
    icon=folium.Icon(color='red', icon='info-sign')
```

```
).add_to(m)
```

```

# Add circle to highlight the location
folium.CircleMarker(
    location=[lat, lon],
    radius=50,
    color='red',
    fill=True,
    fill_opacity=0.3
).add_to(m)

# Add layer control
folium.LayerControl().add_to(m)

# Save map to temporary HTML file
_, temp_html_file = tempfile.mkstemp(suffix='.html')
m.save(temp_html_file)

# Read the HTML content
with open(temp_html_file, 'r', encoding='utf-8') as f:
    html_content = f.read()

# Clean up
os.unlink(temp_html_file)

return html_content

except Exception as e:
    print(f"Error creating folium map: {e}")
    return None

```

ค.3 Code สำหรับการเชื่อมต่อกับ Sentinel Hub API

```
def fetch_sentinel_image(lat, lon, size=256, time_window_days=365):
```

```
    """
```

ดึงภาพถ่ายดาวเทียมจาก Sentinel Hub API สำหรับพิกัดที่กำหนด

Parameters:

lat (float): ละติจูด

lon (float): ลองจิจูด

size (int): ขนาดของภาพเป็นพิกเซล

time_window_days (int): จำนวนวันย้อนหลังในการค้นหาภาพ

Returns:

tuple: (สถานะความสำเร็จ, ภาพ PIL หรือ None, ข้อมูลเมตาตาต้า)

```
    """
```

```
    try:
```

```
        # ตั้งค่า Sentinel Hub
```

```
        config = SHConfig()
```

```
        config.instance_id = SENTINEL_INSTANCE_ID
```

```
        config.sh_client_id = SENTINEL_CLIENT_ID
```

```
        config.sh_client_secret = SENTINEL_CLIENT_SECRET
```

```
        # กำหนดช่วงเวลา
```

```
        end_date = datetime.now()
```

```
        start_date = end_date - timedelta(days=time_window_days)
```

```
        time_interval = (start_date.strftime("%Y-%m-%d"), end_date.strftime("%Y-%m-%d"))
```

```
        # สร้างกรอบพื้นที่ (ประมาณ 2.2 กม.)
```

```
        bbox = BBox(bbox=[lon - 0.02, lat - 0.02, lon + 0.02, lat + 0.02], crs=CRS.WGS84)
```

```

# สร้างคำขอสำหรับภาพสีจริง (True Color)

true_color_request = SentinelHubRequest(
    evalscript="""
        //VERSION=3

        function setup() {
            return {
                input: ["B02", "B03", "B04"],
                output: { bands: 3 }
            };
        }

        function evaluatePixel(sample) {
            return [2.5 * sample.B04, 2.5 * sample.B03, 2.5 * sample.B02];
        }
    """,
    input_data=[
        SentinelHubRequest.input_data(
            data_collection=DataCollection.SENTINEL2_L2A,
            time_interval=time_interval,
            mosaicking_order=MosaickingOrder.LEAST_CC
        )
    ],
    responses=[
        SentinelHubRequest.output_response('default', MimeType.PNG)
    ],
    bbox=bbox,
    size=(size, size),
    config=config
)

```

```

# ดึงข้อมูลภาพ
rgb_data = true_color_request.get_data()

if len(rgb_data) == 0:
    return False, None, {}

# แปลงเป็นภาพ PIL
rgb_image = Image.fromarray(rgb_data[0])

return True, rgb_image, {
    "source": "Sentinel-2",
    "date": datetime.now().strftime('%Y-%m-%d'),
    "resolution": "10m",
    "bbox": [lon - 0.02, lat - 0.02, lon + 0.02, lat + 0.02]
}

except Exception as e:
    print(f"Error fetching Sentinel image: {e}")
    return False, None, {}

```

ภาคผนวก ง: ลิงก์และแหล่งข้อมูลเพิ่มเติม

1. ข้อมูลรายการซากดึกดำบรรพ์
<https://data.go.th/dataset/gdpublish-fossil-list>
2. ข้อมูลตำแหน่งแหล่งซากดึกดำบรรพ์
<https://data.go.th/dataset/gdpublish-1-fossil-site>
3. PALEOMAP PaleoAtlas for GPlates
<https://www.earthbyte.org/paleomap-paleoatlas-for-gplates/>
4. EarthByte Data Collections
https://www.earthbyte.org/webdav/ftp/Data_Collections/

5. Sentinel Hub

<https://www.sentinel-hub.com/>

ภาคผนวก ง: ลิงก์และแหล่งข้อมูลเพิ่มเติม

ง.1 ลิงก์สำหรับข้อมูลและรหัสต้นฉบับ

1. รหัสต้นฉบับของโครงการ

<https://github.com/E27-25/Data-Science-Project-DSBA-Jurassic.git>

2. ข้อมูลตัวอย่างและแบบจำลองที่ฝึกฝนแล้ว

<https://github.com/E27-25/Data-Science-Project-DSBA-Jurassic.git>

3. Kaggle Dataset และ ไฟล์ .rot/.gpml

<https://www.kaggle.com/datasets/e27085921/ds-jurassic-dataset>

ง.2 คำแนะนำในการใช้งานรหัสต้นฉบับ

1. โคลนที่เก็บโดยใช้คำสั่ง:

2. git clone <https://github.com/E27-25/Data-Science-Project-DSBA-Jurassic.git>

ภาคผนวก จ: ภาพแสดงสถาปัตยกรรมโครงข่ายประสาทเทียม:

https://drive.google.com/file/d/1G8FkyZg5o08M6exGqslZVdq3ZxzK1zB5/view?usp=drive_link